Contents

Vector — Project Overview

1. Summary
2. Purpose and scope
3. Feature inventory
4. Tech stack
5. Architecture
6. Data model
7. API
8. Authentication and authorization
9. Frontend
10. Backend
11. Database and infrastructure
12. Testing
13. Performance and scalability
14. Design decisions and trade-offs
15. Known limitations, technical debt and bugs
16. Development history
17. How to run locally and deploy
18. Glossary
19. Appendix: directory tree

Vector — Project Overview

Repository snapshot	branch `main`, HEAD `577b868` (2026-09-10), 289 commits, 944 tracked files
Document date	2026-10-20
Method	Every statement was checked against the source at the snapshot above. Citations in parentheses give the file and, where relevant, the function, class or table. Anything unclear, inconsistent or unverifiable is listed in a **Gaps / to verify** note at the end of its section, and §15 consolidates the defects. Local `.env` files were not read. Environment variables are listed by name only.

1. Summary

Vector is a multi-tenant, Linear-style issue tracker and planning tool. The backend is a single Python 3.12 process: FastAPI as the ASGI shell, Strawberry GraphQL as the primary API (42 queries, 95 mutations, no subscriptions), and a strict resolver → service → repository layering over raw, parameterized asyncpg SQL. The database is PostgreSQL 18, with pgvector, btree_gin and btree_gist. Its schema has 55 tables built by 32 append-only, checksummed SQL migrations, and tenant isolation is enforced both in services and by composite `(workspace_id, id)` foreign keys. The frontend is a React 19 + Apollo Client 4 single-page app with 27 workspace routes (plus public, onboarding and catch-all routes) and GraphQL types generated from a checked-in schema that CI keeps in sync with the backend. Users authenticate by email and password: Argon2id hashes and an opaque, SHA-256-hashed session token in an HttpOnly cookie. Background work (Slack delivery, embedding indexing, due-date reminders, recurring issues, auth cleanup) runs as asyncio tasks inside the API process and claims rows with `FOR UPDATE SKIP LOCKED`, so no broker or scheduler is needed. The project ships Docker images, a Compose stack, plain Kubernetes manifests and a Render staging blueprint, and it is gated by a 10-job GitHub Actions pipeline. The code was written in nine days (2026-09-02 → 2026-09-10).

1.1 Elevator pitch (30 seconds)

Vector is a working clone of Linear's core: workspaces, teams with `ENG-42` issue keys, workflow boards, cycles, projects, initiatives, triage, an inbox, search, and GitHub and Slack integrations. It is built on FastAPI, GraphQL and PostgreSQL without an ORM. What sets it apart is the engineering discipline rather than the feature list:

- Tenant isolation is enforced by the database itself.
- Paginated lists use keyset cursors, and no query uses `OFFSET`.
- Migrations are checksummed and immutable.
- Background jobs are plain PostgreSQL queues, with no extra infrastructure.
- CI fails closed on schema drift, lock drift, leaked secrets and skipped database tests.

The default test suite has 2,674 passing tests, and a further 1,123 run against a real PostgreSQL 18.

1.2 Reading guide

If you want…	Read
What the product does	§2 Purpose and scope, §3 Feature inventory
How it is built	§4 Tech stack, §5 Architecture, §10 Backend, §9 Frontend
Schema and API reference	§6 Data model, §7 API
Security posture	§8 Authentication and authorization, §15.3
Operations	§11 Database and infrastructure, §17 How to run and deploy
Quality and evolution	§12 Testing, §13 Performance, §14 Design decisions, §16 History
What is wrong or missing	§15 Known limitations, technical debt and bugs

2. Purpose and scope

Vector is a multi-tenant issue tracker and planning tool modelled on Linear. A user registers with email and password, creates or joins a **workspace**, and works inside **teams**. Each team has its own issue numbering (`ENG-42`) and its own workflow board. Around issues sit the planning objects Linear users expect: cycles, projects with milestones, initiatives, documents, releases, saved views, triage, an inbox, analytics, and GitHub and Slack integrations. The backend is FastAPI + Strawberry GraphQL + asyncpg over PostgreSQL 18 (`app/`, `migrations/001–032`). The frontend is a React 19 single-page app (`frontend/src/`).

Who it is for. The repository describes its own intent in two ways:

- `README.md` line 3: "A Linear-style issue tracker, built as a backend learning project."
- `CLAUDE.md` ("Vector is a production-oriented Linear-style project management application. This is intended to become a real deployable product, not a tutorial or throwaway prototype."). This file is listed in `.gitignore` (line 27), so it is not part of the tracked repository.

No target customer, pricing or user segment is stated anywhere else in the code or docs.

2.1 Linear concepts replicated

Linear concept	Vector equivalent	Evidence
Workspace (tenant, URL slug)	`workspaces`. Slug `^[a-z0-9]([a-z0-9-]*[a-z0-9])?$`, ≤63 characters. Creator becomes `owner`.	`app/services/memberships.py` (`SLUG_PATTERN`, `create_workspace`); route `/:workspaceSlug` (`frontend/src/app/routes/routes.tsx`)
Member roles	`member` | `admin` | `owner`	`app/domain/tenancy.py` (`WORKSPACE_ROLES`)
Teams with a key	Team key `^[A-Z][A-Z0-9]{0,9}$`, unique per workspace	`app/services/teams.py` (`KEY_PATTERN`)
Issue identifier `ENG-123`	`team_key-number`. The number comes from a per-team counter row (`UPDATE teams SET issue_counter = issue_counter + 1`), so numbering has no gaps.	`app/repositories/teams.py` (`allocate_issue_number`); `app/domain/issues.py` (`IssueEntity.identifier`)
Workflow states and categories	Five categories: `backlog`, `unstarted`, `started`, `completed`, `canceled`. Each team is seeded with Backlog / Todo / In Progress / Done / Canceled.	`app/domain/teams.py` (`WorkflowStateCategory`, `DEFAULT_WORKFLOW_STATES`)
Priority (None / Urgent / High / Medium / Low)	Integer 0–4, where 0 means no priority and 1 means Urgent	`app/services/issues.py` (`PRIORITY_MAX`); `frontend/src/features/issues/lib/priority.ts` (`PRIORITY_NAMES`)
Estimates and team estimate scale	`none` | `points` | `hours` | `tshirt` (XS–XL stored as 1–5)	`app/domain/estimates.py` (`EstimateScale`, `is_valid_estimate`)
Labels and label groups	Workspace labels, with optional groups that can be marked exclusive (one label per group per issue)	`app/services/labels.py`; `migrations/021_triage_label_groups.sql` (`issue_labels_exclusive_group_key`)
Sub-issues and relations	Parent/child links with cycle detection. Relations: `blocks`, `blocked_by`, `related`, `duplicate`.	`app/services/relations.py` (`set_parent`, `create_relation`)
Cycles	Numbered, time-boxed per team. Overlapping cycles are rejected by a GiST exclusion constraint.	`app/services/cycles.py`; `migrations/008_cycles.sql` (`cycles_no_overlap`)
Projects, milestones, project updates and health	Project states; health `on_track` / `at_risk` / `off_track`; milestones; status updates; project dependencies	`app/services/projects.py`; `app/domain/health.py`

Linear concept	Vector equivalent	Evidence
Initiatives	Initiative tree up to 5 tiers (`MAX_INITIATIVE_DEPTH = 4` edges), linked to projects, with updates and health	`app/services/initiatives.py`; `app/domain/initiatives.py`
Roadmap	Initiatives and projects grouped by target month	`frontend/src/features/roadmap/pages/RoadmapPage.tsx`
Documents	ProseMirror-JSON documents with revisions, restore and comments	`app/services/documents.py`; `app/domain/documents.py` (`parse_content`)
Triage	Per-team inbox of incoming issues with accept / decline / mark duplicate / move to another team	`app/services/triage.py`
Custom views and favorites	Saved views (filter, order, layout, grouping, personal or shared) and favorites	`app/services/saved_views.py`
Issue templates and recurring issues	Templates, plus daily / weekly / monthly recurrence schedules	`app/services/templates.py`; `app/domain/recurrence.py`
Inbox and notifications, subscribers	Notification kinds `assigned`, `commented`, `blocked`, `status_changed`, `due_soon`; issue subscribers	`app/domain/notifications.py`; `app/services/activity.py`
Issue history	Per-field activity log (`issue_activity`)	`app/services/activity.py` (`record_changes`)
Bulk edit	Up to 100 issues, all-or-nothing	`app/services/bulk.py`; `app/domain/bulk.py` (`BULK_MAX`)
Search	PostgreSQL full-text search fused with vector search (reciprocal-rank fusion). An exact `ENG-12` query jumps straight to that issue.	`app/services/search.py`; `app/repositories/embeddings.py` (`search_hybrid`)
Similar or duplicate issues	Vector-similarity duplicate suggestions (similarity ≥ 0.6)	`app/services/search.py` (`suggest_duplicates`); `app/domain/semantic_search.py`
Insights	Workspace analytics over a 1–180 day window	`app/services/analytics.py` (`overview`)
Releases	Releases per environment, built from GitHub commit ranges, with generated notes	`app/services/releases.py`
GitHub integration	GitHub App install, PR / branch / commit linking by identifier mention, PR-driven state automation	`app/services/github.py`; `app/rest/github.py`
Slack integration	OAuth install, one default channel, per-event opt-in notifications	`app/services/slack.py`; `app/services/notifications.py` (`SlackNotifier`)
Command menu (Cmd/Ctrl+K) and shortcuts	Command palette, `c` to create an issue, `?` for the shortcut reference	`frontend/src/features/command/CommandPalette.tsx`, `ShortcutsView.tsx`
Board view	Team Kanban board with HTML5 drag between workflow-state columns	`frontend/src/features/board/components/BoardCard.tsx` (`draggable`)

2.2 Linear features not replicated

Each row below was confirmed absent by searching `app/` (`*.py`), `migrations/` and `frontend/src` (`*.ts`, `*.tsx`, `*.graphql`, excluding generated files) for the patterns shown. A row that says "only false positives" means the pattern matched comments or unrelated identifiers and nothing that implements the feature.

Linear feature	Status in Vector	Search
Real-time sync / live updates	Absent. The GraphQL schema has no `type Subscription`. The frontend does not poll and opens no socket.	`type Subscription|graphql_ws|websocket|EventSource`: only a comment in `app/http_cookies.py`. `pollInterval|useSubscription|setInterval`: no matches (`frontend/src`).

Linear feature	Status in Vector	Search
Offline mode / local-first client	Absent: no service worker and no IndexedDB	`serviceWorker|IndexedDB|offline`: only false positives
Mobile / desktop apps	Absent. The only client is the web SPA.	`react-native|capacitor|electron`: no matches
SSO / OAuth sign-in (Google, SAML)	Absent. Sign-in is email and password only (`register`, `login`, `logout` in `app/graphql/mutations/auth.py`). `schema_drafts/033_google_identity.draft.sql` is an untracked draft, not a migration.	`saml|oidc|openid|sso|google`: only CSP font hosts in `app/spa.py`
Email notifications / outbound mail	Absent. Invitation tokens are returned to the inviter instead of being emailed (`InvitationCreatePayload.token`).	`smtp|sendgrid|send_mail|sendmail|aiosmtplib`: no matches
Password reset, email verification, 2FA	Absent	`reset|verify_email|totp|two_factor|change_password` in `frontend/schema.graphql` and `app/graphql/mutations/auth.py`: no matches. The auth mutations are only `register` / `login` / `logout`
@mentions	Absent, and stated as such: "There is no MENTIONED. Mentions do not exist in this product" (`app/domain/notifications.py`, line 39)	`mention`: the other hits are GitHub identifier "mentions" (`app/domain/github.py`)
Comment editing, emoji reactions	No edit operation exists (`CommentService` has only `create`, `delete` and `list_for_issue`), so `Comment.editedAt` is always null. No reactions.	`reaction|emoji`: only false positives
File attachments / uploads	Absent. No upload endpoint (the only REST routes are `healthz`, `readyz`, `github/*` and `slack/*` in `app/rest/`).	`attachment|presign|multipart|UploadFile`: only false positives
Custom fields	Absent	`custom.?field`: no matches
Public API keys / personal access tokens	Absent. GraphQL authenticates by session cookie only.	`api.?key|personal.?access.?token|api_token`: no matches
Time tracking	Absent. The `hours` estimate scale is an estimate, not a log.	`time.?track|time_entr|timesheet`: no matches
SLAs, customer requests	Absent	`\bsla\b|sla_`: no matches. `customer`: only comments and UI prose.
Importers (Jira, CSV)	Absent	`jira|csv`: no matches
Outbound webhooks	Absent. Webhooks are inbound only (GitHub, Slack).	`outbound.?webhook|webhook_url`: no matches
Team membership / private teams	Absent. Every team is visible to every workspace member.	`team_members|team_membership`: no matches
Issue delete / unarchive	Absent. Issues can only be archived (`IssueRepository.archive`). No path clears `archived_at`.	`issue_delete|issueDelete|unarchive|restore_issue`: only comments
Workspace rename, user profile edit	No such mutations. The root `Mutation` field list in `frontend/schema.graphql` (identical to the live schema export) has no `workspaceUpdate` or user update.	`workspace_update|workspaceUpdate|user_update`: only a comment in `frontend/src/features/settings/SettingsPage.tsx` (and its test) noting that no `workspaceUpdate` exists
Audit log	Absent. Only per-issue `issue_activity` exists.	`audit_log`: no matches
AI features calling an LLM	Absent. Embeddings are computed locally (`app/services/embeddings.py`, `Embedder` docstring).	`openai|anthropic|llm|gpt`: only false positives (`restoreAllMocks` in a test)
Light / dark theme choice	One dark palette only. `useTheme` exists but is unused.	`frontend/src/styles/tokens.css` (`color-scheme: dark`); `frontend/src/app/layout/preferences.ts`

2.3 Scope boundaries

- **Deployment status: staging / demo.** `render.yaml` starts: "THIS IS A STAGING / DEMO DEPLOYMENT … Nothing here should be described as production to a user." It defines one free Render web service backed by a hand-created Neon PostgreSQL 18 database. `ENVIRONMENT` is set to `production` only to select the runtime mode (no GraphiQL or introspection, Secure cookies).
- **Other run targets.** These are a local launcher with a seeded demo workspace (`run-vector-local.ps1`, `scripts/seed_local_demo.py`, which refuses non-loopback endpoints), Docker Compose (`docker-compose.yml`) and plain Kubernetes manifests (`k8s/`). `k8s/20-postgres.yaml` is described as development-only (`README.md`, "Deploy to Kubernetes").
- **No billing or plans.** No billing, Stripe, invoice or pricing code exists (searched `billing\|stripe\|invoice\|pricing` and found only comments and test fixture names). No per-workspace limits are enforced beyond list caps, and any signed-in user can create workspaces without limit (`MembershipService.create_workspace`).
- **Semantic search runs on a hashing embedder by default.** `sentence-transformers` is not in `requirements.txt`, so `load_embedder()` falls back to `HashingEmbedder` (`"hashing-v1"`), whose docstring says it is "NOT a semantic model" (`app/services/embeddings.py`). The background embedding worker is off unless `EMBEDDING_WORKER_ENABLED=true` (`app/config.py`; `README.md`, "Configuration").
- **Integrations are optional.** GitHub reports `unconfigured` unless all five App settings are present (`GithubAppConfig.configured`). Slack needs client id, client secret and signing secret (`Settings.slack_configured`). Inbound Slack events are verified and deduplicated but not processed ("Deliberately empty", `app/rest/slack.py`, `_handle_event_callback`).
- **No outbound user messaging** beyond Slack channel posts of six domain-event kinds (`app/domain/slack.py`, `SLACK_NOTIFICATION_EVENTS`).

> **Gaps / to verify:**
>
> - Whether the Render staging service is currently live, and which commit it runs, was not checked. Only the manifest was read.
> - Whether the deployed image installs `sentence-transformers` separately (outside `requirements.txt`) was not verified.

3. Feature inventory

Conventions used in the tables below:

- **Paths.** Backend paths are relative to `app/`. Frontend paths are relative to `frontend/src/features/` unless they start with `app/` (meaning `frontend/src/app/`).
- **Operation names.** GraphQL operations use the camelCase names from `frontend/schema.graphql`. That file is byte-identical to a fresh export of the backend schema (checked during this pass).
- **UI column. Full** means the feature is reachable and operable in the web UI. **Partial** means some of the backend operations have no UI. **Backend only** means no frontend operation document or component calls the operation. These were checked by searching `frontend/src` (`*.graphql`, `*.ts`, `*.tsx`, excluding tests and generated files) for the operation name.
- **Behaviour common to every feature** (so it is not repeated per row):
  - Every workspace operation is scoped through `authorized_scope(info, workspaceSlug)` (`graphql/scope.py`).
  - Actor and author ids come from the session, never from the input.
  - Lists use keyset pagination with `first` between 1 and 100.
  - Validation failures return `errors { field code message }` in the payload.

3.1 Accounts and workspaces

Feature	How it works	Backend	Frontend	UI	Notes / limits
Register	Email (lowercased, ≤320 characters, must match `^[^@\s]+@[^@\s]+$`), password (8–1024 characters, argon2id), optional name. Signs the user in immediately. Rate-limited to 20 per IP per 15 minutes.	`services/auth.py` (`AuthService.register`), `services/passwords.py`	`auth/pages/RegisterPage.tsx`, route `/register`	Full	`EMAIL_TAKEN` is returned for existing emails (an accepted enumeration channel). No email verification.
Sign in / sign out	Opaque 256-bit session token. Only its SHA-256 is stored. Sent as the cookie `vector_session` (HttpOnly, SameSite=Lax, 14-day Max-Age). Login is limited to 10 attempts per email and 100 per IP per 15 minutes, and every failure returns `INVALID_CREDENTIALS`. Logout deletes the session row.	`AuthService.log_in` / `log_out`; `http_cookies.py`	`auth/pages/LoginPage.tsx`; `auth/api/useLogout.ts`; `auth/guards.tsx` (`RequireAuth`, `RequireNoAuth`)	Full	No password reset, 2FA, session list or sign-out-everywhere (no such mutations in `frontend/schema.graphql`; see §8.7).
Session expiry handling	An `UNAUTHENTICATED` GraphQL error writes `me: null` into the Apollo cache, and `RequireAuth` then redirects to `/login`.	`graphql/queries/auth.py` (`me` returns null when signed out)	`auth/api/sessionExpiry.ts` (`sessionExpiryLink`)	Full	
Create workspace	Name ≤200 characters. Slug must match `^[a-z0-9]([a-z0-9-]*[a-z0-9])?$` and be ≤63 characters. A duplicate slug returns `SLUG_TAKEN`. The creator becomes `owner` in the same transaction.	`services/memberships.py` (`MembershipService.create_workspace`)	`onboarding/pages/WorkspaceStep.tsx` (`/onboarding/workspace`)	Partial	The UI creates only the first workspace: `resolveStep` (`onboarding/lib/progress.ts`) redirects away from the workspace step once the viewer has a workspace. The API allows unlimited workspaces.
Workspace switcher	A menu listing the viewer's memberships, shown when there are two or more	`my_workspaces` (`graphql/queries/memberships.py`)	`app/layout/` (`WorkspaceSwitcher`)	Full	

Feature	How it works	Backend	Frontend	UI	Notes / limits
Members and roles	Lists members, including former members. Admins and owners can change roles and remove members. Only an owner can touch another owner or grant owner. Demoting or removing the last owner is refused (`LAST_OWNER`).	`MembershipService.update_member_role`, `remove_member` (`_require_owner_to_touch_owners`, `lock_owner_ids`)	`members/MembersPage.tsx` (`/members`)	Full	Removal is a soft delete (`removed_at`). It is refused while the member leads a project, owns an initiative, created a shared saved view, is a template's default assignee, or connected GitHub or Slack (`_REMOVAL_BLOCKED`). Their issues are unassigned.
Invitations	Admins and owners create invitations with a role. A 256-bit token is returned once. Only its hash is stored. Lifetime is 7 days. Revocable.	`MembershipService.create_invitation`, `revoke_invitation`, `list_invitations`	`members/MembersPage.tsx` (token shown once with a copy control); `onboarding/pages/InviteStep.tsx`	Full	No email delivery: the inviter shares the link. The invitation email is not matched against the accepting account, because the token is the credential.
Accept invitation	Single-use claim: `UPDATE … WHERE accepted_at IS NULL AND expires_at > now()`, then create the membership. This revives a former member. An existing member gets `ALREADY_MEMBER` and the claim rolls back.	`MembershipService.accept_invitation`	`onboarding/pages/AcceptInvitePage.tsx` (`/invite/:token`)	Full	`/invite/:token` and `/onboarding/*` are not wrapped in `RequireAuth` (`onboarding/index.tsx`).
Workspace settings (identity)	Read-only display of the workspace name and slug	none (no `workspaceUpdate` mutation)	`settings/SettingsPage.tsx` (`/settings`)	n/a	No rename or delete exists for workspaces.

3.2 Teams and workflows

Feature	How it works	Backend	Frontend	UI	Notes / limits
Create team	Admin or owner only. Key must match `^[A-Z][A-Z0-9]{0,9}$` and be unique per workspace (`KEY_TAKEN`). The team is seeded with 5 states: Backlog / Todo / In Progress / Done / Canceled.	`services/teams.py` (`TeamService.create`); `repositories/teams.py` (`seed_default_workflow_states`)	`onboarding/pages/TeamStep.tsx` (`/onboarding/team`)	Partial	Team creation exists only in the onboarding step. There is no team rename or delete.
Workflow states	Five categories: backlog, unstarted, started, completed, canceled. Code branches only on the category. New issues start in the first `unstarted` state. `completed_at` is set on entering a completed or canceled state and cleared on leaving it.	`TeamRepository.find_default_workflow_state_id`; `IssueRepository.update` (`completed_at` CASE)	`teams/TeamPage.tsx`, `TeamStates.tsx` (`/team/:teamKey`)	Partial	States are read-only. There is no create, rename, reorder or delete at any layer.
Team issue list	Flat list of the team's issues	`issues(filter: {teamId})`	`teams/TeamIssuesPage.tsx` (`/team/:teamKey/issues`)	Full	
Team estimate scale	Admin-only setter: `none`, `points`, `hours` or `tshirt`. Existing estimates are not rewritten.	`TeamService.set_estimate_scale` (`teamEstimateScaleSet`)	none	Backend only	The analytics page reads `estimateScale`. The issue inspector does not label estimates by scale.

3.3 Issues

Feature	How it works	Backend	Frontend	UI	Notes / limits
Create issue	Title 1–500 characters, optional description, priority, assignee and other fields. Allocates the team's next number (gap-free counter row) and starts in the default `unstarted` state. Records a `created` activity entry. A non-null assignee is auto-subscribed and notified.	`services/issues.py` (`IssueService.create`); `TeamService.allocate_issue_number`	`issues/components/IssueComposer.tsx`; `issues/api/useCreateIssue.ts`	Full	The composer always files into `teams[0]` (`useCreateIssue.ts` line 59), so the team cannot be chosen at creation.
Identifier	`ENG-42`, computed as `team_key-number`. Not stored. Unique per `(team_id, number)`.	`domain/issues.py` (`IssueEntity.identifier`)	rows and inspector (`issues/components/IssueRow.tsx`)	Full	Detail URLs use the issue UUID (`/issues/:issueId`), not the identifier.
Edit fields	Patch update of title, description, status, priority (0–4), assignee (must be a member), estimate, due date. Null title, priority or state is refused (`NOT_NULLABLE`). Each changed field is logged as activity. Changes of assignee or status also emit notifications and domain events; a change to Urgent emits a domain event only.	`IssueService.update`; `services/activity.py` (`record_changes`)	`issues/components/IssueInspector.tsx`, `EditableText.tsx`; `issues/api/useIssueMutations.ts` (optimistic `IssueUpdate`)	Full	
Project / milestone / cycle on an issue	`issueSetProject` (a milestone requires a project: `PROJECT_REQUIRED`) and `issueSetCycle` (the cycle must belong to the issue's team)	`IssueService.set_project`, `set_cycle`	`IssueInspector.tsx` (project and cycle selects)	Full	These statements lack the `archived_at IS NULL` guard, so an archived issue can be moved (`app/repositories/issues.py` `set_cycle`, `set_project`: `WHERE workspace_id = $1 AND id = $2` only).
Archive	Sets `archived_at`. Archived issues disappear from every read. There is no delete because numbers are never reissued.	`IssueService.archive`; `IssueRepository.archive`	`IssueInspector.tsx`; `issues/api/cache.ts` (`removeArchivedIssue`)	Full	No unarchive and no archived-issues view exist, although the resolver docstring calls archiving "reversible" (`graphql/mutations/issues.py`).
Priority	0 = none, 1 = Urgent, 2 = High, 3 = Medium, 4 = Low. Ascending sort puts 0 last and descending puts it first (`NULLIF(priority,0)` with PostgreSQL's default NULL ordering).	`domain/issues.py` (`IssueOrderField`); `repositories/issues.py` (`_ORDER_KEYS`)	`issues/lib/priority.ts` (`PRIORITY_NAMES`)	Full	
Estimates	Must be an integer ≥0. The team's scale is read from the database: `tshirt` allows only 1–5 (XS–XL). Null is always allowed.	`domain/estimates.py` (`is_valid_estimate`); `IssueService._check_estimate_scale`	`IssueInspector.tsx` (estimate field)	Full	
Due dates and due windows	A `due_date` per issue. Filter windows `overdue`, `today`, `this_week` and `no_due_date` are resolved against the database `CURRENT_DATE`.	`domain/issues.py` (`DueWindow`, `IssueFilter`); `repositories/issues.py` (`_DUE_WINDOW_CLAUSES`)	Due date edited in `IssueInspector.tsx` and shown on `board/components/BoardCard.tsx`	Partial	No UI exposes the due-window filters (the `IssueFilterInput` fields `due`, `dueAfter` and `dueBefore` and the `DueWindow` enum are not used in `frontend/src`).

Feature	How it works	Backend	Frontend	UI	Notes / limits
Due-date reminders	Every 15 minutes, a `due_soon` notification is sent for live, non-terminal issues due today or tomorrow. The recipients are the assignee, the creator and subscribers. A ledger keyed on `(issue, due_date)` prevents repeats. Overdue issues are never reminded.	`services/schedule.py` (`ScheduleWorker._remind`, `REMINDER_LEAD_DAYS = 1`, `POLL_INTERVAL_SECONDS = 900`); `repositories/reminders.py` (`claim_due`)	Shown in the inbox	Full (background)	Uses the database's `CURRENT_DATE`. No session `TimeZone` is set in `app/db.py` (its only `server_settings` key is `statement_timeout`).
List, sort, paginate	`issues(filter, orderBy, first, after)`. Filters: team, assignee (null means unassigned), state, state category, label, priority, project, cycle, due window and range. Sort by priority, created, updated or due date, ascending or descending, with `id` as tie-break.	`IssueService.list`; `repositories/issues.py` (`_add_filters`)	`issues/pages/IssuesScreen.tsx` (`/issues`, flat or grouped list, "Load more"); `myIssues/MyIssuesPage.tsx` (`/my-issues`, `assigneeId = viewer`)	Partial	The All Issues list has no filter controls. Filtering in the UI is on the board and in saved views.
Sub-issues	Set or clear a parent. The parent may be on another team in the same workspace. Self-parenting (`SELF_PARENT`) and cycles at any depth (`CYCLE`, recursive CTE under a per-workspace advisory lock) are refused.	`services/relations.py` (`RelationService.set_parent`, `clear_parent`)	`collaboration/components/SubIssuesPanel.tsx`, `IssuePicker.tsx`	Full	
Relations	`blocks` / `blocked_by` / `related` / `duplicate`. Stored in canonical form, with activity recorded on both issues. A `blocked` notification goes to the blocked issue's watchers.	`RelationService.create_relation`, `delete_relation`	`collaboration/components/RelationsPanel.tsx`	Full	Blocking cycles (A blocks B, B blocks A) are allowed by design (migration 010).
Labels on issues	Attach or detach. At most 50 labels per issue (a soft cap that concurrent requests can exceed). At most one label per exclusive group, enforced by a database unique index.	`services/labels.py` (`LabelService.attach`, `detach`)	`collaboration/components/LabelsPanel.tsx`	Full	
Bulk update / archive	1–100 issue ids, all-or-nothing in one transaction. Rows are locked in id order. Patchable fields: state, assignee, priority, estimate, due date, project and milestone, cycle, plus add and remove labels.	`services/bulk.py` (`BulkService.update_many`, `archive_many`); `domain/bulk.py` (`BULK_MAX = 100`)	none (`issueBulkUpdate` and `issueBulkArchive` are absent from `frontend/src`)	Backend only	Bulk label attach bypasses the 50-label cap. No activity is recorded for the label half.
Templates	Named defaults: team (or workspace-wide), title, description, priority, estimate, assignee, project, cycle, labels. "Create from template" files an issue and then applies project, cycle and labels.	`services/templates.py` (`TemplateService.create/update/delete/apply`)	`templates/pages/TemplatesPage.tsx` (`/templates`), `components/TemplateForm.tsx`	Full	`apply` is not atomic: each step runs in its own transaction (`ponytail:` note), so a failure can leave a partly-templated issue.
Recurring issues	A schedule per template: daily, weekly (ISO weekdays) or monthly (day of month, clamped), every 1–52 intervals, with a `dueInDays` offset of 0–365. The 15-minute worker files one issue per due schedule.	`TemplateService.set_recurrence`, `clear_recurrence`; `domain/recurrence.py`; `ScheduleWorker._generate`	none (no `issueTemplateRecurrenceSet` and no `recurrence` in `frontend/src`)	Backend only	At-most-once delivery. After downtime only one occurrence is filed. At most 25 schedules are filed per pass.

Feature	How it works	Backend	Frontend	UI	Notes / limits
Subscribers (watchers)	Subscribe or unsubscribe self. Commenters and assignees are auto-subscribed.	`services/activity.py` (`ActivityService.subscribe`, `unsubscribe`, `auto_subscribe`)	none (no `issueSubscribe` and no `issueSubscribers` in `frontend/src`)	Backend only	`SUBSCRIBERS_MAX = 500`. Auto-subscription still drives inbox notifications.
Triage	An issue is in triage while `triage_entered_at` is set. Queue per team, oldest first. Actions: accept into a chosen state, decline (moves to the team's first canceled state), mark duplicate (adds a `duplicate` relation and declines), change team (new number, default state, stays in triage). An issue in triage cannot be archived.	`services/triage.py` (`TriageService`); `repositories/triage.py`	`triage/pages/TriagePage.tsx` (`/triage`), `components/TriageRow.tsx`	Partial	No UI sends an issue into triage: `TriageEnter` is defined in `triage/api` but no component calls it, and nothing else sets `triage_entered_at`. Triage moves send no notifications or Slack events.
Issue activity log	Per-field history in `issue_activity`, covering created, title, state, priority, assignee, project, cycle, archived, commented, label and relation changes. GitHub automation adds a `caused_by` value.	`services/activity.py` (`record`, `record_changes`, `ActivityService.list_for_issue`); `Issue.activity` field	`IssueInspector.tsx` shows only created, updated and completed timestamps (`ponytail:` comment at around line 271)	Backend only	The `Issue.activity` connection is not queried by the frontend.
Issue development (GitHub)	Linked PRs and commits for an issue, up to 50 of each. Also provides a branch-name suggestion such as `eng-142-fix-oauth`.	`services/github.py` (`development_for_issue`); `domain/github.py` (`branch_name_for`); `Issue.development` field	none (no `pullRequests` selection in `frontend/src`)	Backend only	

3.4 Labels and label groups

Feature	How it works	Backend	Frontend	UI	Notes / limits
Create label	Name 1–50 characters, unique per workspace. Colour `#rrggbb`, default `#6b7280`, stored lowercased.	`LabelService.create`	`collaboration/components/LabelsPanel.tsx` (`LabelCreate`)	Full	
Rename / recolour / delete label	Whole-record update; delete	`LabelService.update`, `delete` (`labelUpdate`, `labelDelete`)	none	Backend only	
Label groups	Create, update or delete a group (name 1–50, unique). A group can be marked `exclusive`; turning that on is refused (`GROUP_IN_USE`) if any issue already has two labels from the group. Deleting a group ungroups its labels. Up to 100 groups per workspace.	`LabelService.create_group`, `update_group`, `delete_group`, `set_label_group`	`labelGroups/pages/LabelGroupsPage.tsx` (`/label-groups`), `LabelGroupForm.tsx`, `LabelGroupPanel.tsx`	Full	The page loads one page of 50 labels (`labelGroups/api/queries.ts`, `ponytail:` note).

3.5 Comments

Feature	How it works	Backend	Frontend	UI	Notes / limits
Issue comments	Create (1–16384 characters) and delete. In one transaction: activity `commented`, auto-subscribe the author, and notify the assignee, the creator and subscribers (excluding the actor). Listed oldest first.	`services/comments.py` (`CommentService.create`, `delete`, `list_for_issue`)	`collaboration/components/CommentsPanel.tsx`	Full	Only the author can delete; admins get `NOT_FOUND`. No edit exists. Plain text only, with no mentions or reactions.

Feature	How it works	Backend	Frontend	UI	Notes / limits
Document comments	Same model on documents (1–16384 characters, author-only delete)	`services/documents.py` (`create_comment`, `delete_comment`)	`documents/components/DocumentPanel.tsx`	Full	No edit.

3.6 Views, filters and search

Feature	How it works	Backend	Frontend	UI	Notes / limits
Board (Kanban)	Team-scoped. Columns are the team's workflow states when grouped by status. Group by status, assignee, priority or project. Sort by priority, created, updated or due. Filter by status, assignee, label, priority, project or cycle. The view state lives in URL query parameters. Moving a card uses HTML5 drag or a per-card menu, with an optimistic update.	`issues` query and `issueUpdate`	`board/pages/BoardScreen.tsx` (`/board`); `board/lib/viewState.ts` (`GROUP_BY`, `SORT_BY`, `PARAM`); `board/api/useMoveIssue.ts`	Full	Drag is enabled only when columns are states (`BoardCard.tsx`, `draggable={columnsAreStates}`).
List density and grouping	Dense or comfortable rows; flat or grouped list. Both preferences persist in `localStorage`.	none	`app/layout/preferences.ts` (`useDensity`, `useGroupMode`)	Full	
Saved views	Name, `IssueFilter` stored as JSONB, order, layout (`list` or `board`), grouping and subgrouping, visibility (`personal` or `shared`). Personal views are invisible to others. Only the creator can edit or delete, even a shared view. Results are filtered on the server.	`services/saved_views.py` (`SavedViewService`); `repositories/saved_views.py` (`_VISIBLE_TO_VIEWER`)	`savedViews/pages/SavedViewsPage.tsx` (`/saved-views`), `components/SavedViewForm.tsx`	Full	Results render as `IssueRow` rows (`SavedViewsPage.tsx` line 391) whatever the stored layout. Filter editing is disabled for views whose filter contains an explicit null, which the input type cannot express (`savedViews/lib/savedViews.ts`).
Favorites	Pin a team, project or saved view (exactly one per favorite). Per-user ordering by position, up to 500.	`FavoriteService.add`, `remove`, `reorder`	`favorites/pages/FavoritesPage.tsx` (`/favorites`, up and down buttons)	Full	Reordering sends one request per moved row (`favorites/lib/favorites.ts`, `ponytail:` note).
Full-text search	PostgreSQL `websearch_to_tsquery('english')` over generated `tsvector` columns (title weighted A, description weighted B) for issues and projects, ranked by `ts_rank`. A query shaped like `KEY-123` puts that exact issue first. Query ≤200 characters. An empty query returns nothing without a database call.	`services/search.py` (`SearchService.search`); `domain/search.py` (`parse_issue_identifier`); `migrations/011_search.sql`	`search/SearchPage.tsx` (`/search?q=`, 250 ms debounce)	Full	The project arm has no archived filter (projects have no archive).
Hybrid (lexical + vector) search	The issue arm of `search` fuses the lexical rank and a pgvector cosine rank with reciprocal-rank fusion (`RRF_K = 60`, depth 100) in one SQL statement. Projects remain lexical.	`repositories/embeddings.py` (`search_hybrid`); `domain/semantic_search.py`	Same `SearchPage.tsx` and command palette	Full	Vectors are 384-dimensional from `HashingEmbedder` unless `sentence-transformers` is installed (§2.3). Search is an exact scan with no ANN index (migration 025).

Feature	How it works	Backend	Frontend	UI	Notes / limits
Duplicate / similar issues	Given a title and an optional description, returns vector-only matches with similarity ≥0.6, excluding the issue being edited.	`SearchService.suggest_duplicates` (`issueDuplicateSuggestions`)	`semanticSearch/pages/SemanticSearchPage.tsx` (`/semantic-search`)	Partial	The page is reachable only by URL: it is not linked from the sidebar or the palette (only `frontend/src/app/routes/routes.tsx` mounts `ROUTE_SEGMENTS.semanticSearch`; the `paths.semanticSearch` builder in `paths.ts` is never called). The composer does not call it.
Embedding index state and refresh	Counts of indexed, pending and failed embeddings. A manual refresh embeds up to 500 stale issues. A background worker (opt-in) re-embeds stale issues using `FOR UPDATE SKIP LOCKED` with retries.	`SearchService.indexing_state`, `refresh_embeddings`; `services/embedding_jobs.py` (`EmbeddingWorker`)	`semanticSearch/components/IndexPanel.tsx`	Full	Refresh has no role check. The worker is off by default (`EMBEDDING_WORKER_ENABLED`).
Command palette	Cmd/Ctrl+K toggles the palette. `/` opens it and `?` opens the shortcut list. Contains navigation commands, "New issue" when the current screen registered a composer, and live search (`search(first: 8)`).	`search` query	`command/CommandPalette.tsx`, `commands.tsx`, `useCommandSearch.tsx`, `ShortcutsView.tsx`	Full	
Keyboard shortcuts	Issue list: `c` opens the composer, Up/Down move between rows, Enter opens a row, Esc closes the composer or inspector. Shortcuts are ignored while typing in a field.	none	`issues/pages/IssuesScreen.tsx`; `command/lib/keyboard.ts`	Full	

3.7 Planning

Feature	How it works	Backend	Frontend	UI	Notes / limits
Cycles	Per-team, number 1–9999 and unique, optional name ≤200, `endsAt > startsAt` (timezone-aware). Overlapping cycles are refused by the GiST exclusion `cycles_no_overlap`. Create, update (full replace; team fixed) and delete. Deleting a cycle detaches its issues (`ON DELETE SET NULL`).	`services/cycles.py` (`CycleService`); `migrations/008_cycles.sql`	`cycles/pages/CycleListPage.tsx` (`/cycles`, current / upcoming / past), `CycleDetailPage.tsx` (`/cycles/:cycleId`, add unscheduled issues), `components/CycleForm.tsx`	Full	Cycles are created manually with an explicit number; nothing auto-schedules them. No top-level sidebar entry; reached through each team's "Cycles" row or the palette.
Projects	Name 1–200, description ≤10000, state (`planned`, `started`, `paused`, `completed`, `canceled`), target date, lead (must be a member), many-to-many teams. Deleting a project detaches issues, milestones, teams, updates, dependencies and initiative links in one transaction.	`services/projects.py` (`ProjectService`)	`projects/pages/ProjectListPage.tsx` (`/projects`), `ProjectDetailPage.tsx` (`/projects/:projectId`), `ProjectForm.tsx`, `ProjectTeams.tsx`, `ProjectIssues.tsx`, `ProgressBar.tsx`	Full	Deleting a project that has documents hits `documents_project_fk ON DELETE RESTRICT`, which is unmapped and likely returns a masked internal error (see §15.1).
Project milestones	Create (appended at `MAX(position)+1`), update, delete. Ordered by position.	`ProjectService.create_milestone`, `update_milestone`, `delete_milestone`	`projects/components/ProjectMilestones.tsx`	Full	The position calculation has no lock, so concurrent creates can collide.

Feature	How it works	Backend	Frontend	UI	Notes / limits
Project updates and health	Post an update (health plus body ≤10000). This sets `projects.health` and emits `project_update_published` and, when health changed, `project_health_changed` domain events (shown in Slack).	`ProjectService.post_update`; `services/events.py` (`record_project_update`)	`projects/components/HealthRing.tsx` displays health; there is no posting UI (`projectUpdatePost` is absent in `frontend/src`)	Partial	`projectUpdate` has no health field, so health cannot be set from the UI.
Project dependencies	A directed "blocks" edge between projects. Self-dependency and cycles are refused (recursive CTE under an advisory lock).	`ProjectService.add_dependency`, `remove_dependency`	none	Backend only	
Initiatives	Name, description, status (`planned`, `active`, `completed`, `canceled`), health, target date, owner. Tree via parent, at most 4 edges deep (`TOO_DEEP`), no cycles (`CYCLE`). Many-to-many projects. Health updates. Deleting an initiative makes its children top-level.	`services/initiatives.py` (`InitiativeService`)	`initiatives/pages/InitiativesPage.tsx` (`/initiatives`), `components/InitiativeDetailPanel.tsx`, `InitiativeForm.tsx`	Full	Initiative updates emit no domain event (unlike projects). The documents RESTRICT issue noted for projects also applies here.
Roadmap	Initiatives and projects placed by the month of `targetDate`, as markers rather than bars (there is no start date in the schema)	`initiatives`, `projects` queries	`roadmap/pages/RoadmapPage.tsx` (`/roadmap`), `lib/roadmap.ts`	Full	Read-only view.
Documents	Title 1–200. ProseMirror JSON from an allowlist of nodes and marks, with `http`, `https` and `mailto` links only, ≤10000 nodes and depth ≤20. A document attaches to a project, an initiative or neither (not both). An edit snapshots a revision when requested, when the editor changes, or after a 10-minute gap. Restore snapshots the current state first.	`services/documents.py` (`DocumentService.create/edit/restore/delete`); `domain/documents.py` (`parse_content`, `REVISION_GAP`)	`documents/pages/DocumentsPage.tsx` (`/documents`), `components/DocumentBody.tsx`, `DocumentForm.tsx`, `DocumentPanel.tsx`	Partial	The UI renders rich content, but editing is a plain-text textarea, offered only when the document is plain paragraphs (`documents/lib/content.ts`, `isPlainTextDocument`, `ponytail:` note). A document's parent cannot change after creation.
Releases	Created from a tracked GitHub repository plus a 40-hex head commit and an optional previous commit. If the previous commit is omitted, it defaults to the last deployed release in that environment. The issues and merged PRs in the commit window are frozen and deterministic notes are rendered (up to 200 items). Status moves pending → deployed or failed, and deployed → rolled_back.	`services/releases.py` (`ReleaseService.create`, `set_status`, `delete`); `repositories/releases.py` (`resolve_range`)	`releases/pages/ReleasesPage.tsx` (`/releases`), `ReleaseDetailPanel.tsx`, `ReleaseForm.tsx`	Full	Requires a connected GitHub installation with webhook-ingested commits (`NOT_FOUND` / `UNDATED` otherwise). No general edit.
Environments	Name 1–100, unique. Kind is `development`, `staging`, `production` or `custom`.	`ReleaseService.create_environment`, `list_environments`	`environments/pages/EnvironmentsPage.tsx` (`/environments`)	Full	No update or delete. "What is deployed" is derived from the first 25 releases only (`ponytail:` note).

Feature	How it works	Backend	Frontend	UI	Notes / limits
Analytics	Window of 1–180 days (default 30, UTC days). Created and completed/canceled per day, completion rate, lead time and cycle time (p50/p90), issue age, state and priority mix, workload (top 20), overdue count, team completion, project progress (top 25), cycle progress (top 12). Estimate sums are suppressed for t-shirt teams.	`services/analytics.py` (`AnalyticsService.overview`); `repositories/analytics.py`	`analytics/pages/AnalyticsPage.tsx` (`/analytics`, 7/30/90/180-day options in `api/queries.ts` `RANGE_OPTIONS`), `components/ThroughputChart.tsx` (hand-drawn SVG)	Full	No burndown, burnup or velocity. Reads at READ COMMITTED, so the figures are not one consistent snapshot (`ponytail:` note).

3.8 Inbox and notifications

Feature	How it works	Backend	Frontend	UI	Notes / limits
Inbox	Per-user notifications of kind `assigned`, `commented`, `blocked`, `status_changed` or `due_soon`. Each event sends one row per recipient (the assignee, the creator for comments and reminders, and subscribers; active members only; never the actor). Unread-only filter. Mark read (idempotent) and mark all read. Unread count.	`services/activity.py` (`notify`, `ActivityService.list_notifications/mark_read/mark_all_read/unread_count`); `repositories/notifications.py` (`notify_about_issue`)	`inbox/InboxPage.tsx` (`/inbox`); unread badge in `navigation/SidebarNav.tsx`	Full	No archive, snooze, delete or mark-unread. No grouping across events. No real-time updates: the badge changes only on refetch or a cache write.

3.9 Integrations

Feature	How it works	Backend	Frontend	UI	Notes / limits
GitHub: connect	Admin or owner. `GET /github/install` sets an unsigned, session-bound state cookie (10 minutes) and redirects to GitHub. The callback checks state and session in constant time, exchanges the OAuth code to list the user's installations, records a pending claim (15-minute TTL), and confirms it when the grant includes the installation.	`rest/github.py` (`install`, `callback`); `services/github.py` (`GithubService.connect`, `confirm_claim`, `installations_for_grant`)	`settings/SettingsPage.tsx` and `onboarding/pages/IntegrationsStep.tsx` (link to `/github/install?workspace=…&return_to=…`)	Full	Status is `unconfigured`, `disconnected`, `pending` or `connected`. Routes are also mounted under `/integrations/github/*`. No GitHub App JWT or installation tokens, so repository lists are capped at one page of 100.
GitHub: disconnect / tracked repositories	Disconnect deletes the installation data (it does not uninstall on GitHub). An admin can untrack repositories, which narrows the set; a repository referenced by a release cannot be untracked.	`GithubService.disconnect`, `set_tracked_repositories`	`SettingsPage.tsx` (`WorkspaceGithubDisconnect`, `WorkspaceGithubRepositoriesSet`)	Full	
GitHub: webhooks and linking	`POST /github/webhook` with HMAC-SHA256 verification. Handles `installation`, `installation_repositories`, `pull_request` and `push`, deduplicated by delivery id. Any bare `KEY-123` mention in a PR title, body or branch, or a commit message, links the issue (first 10000 characters, at most 20 identifiers). No "fixes"/"closes" keywords are needed. A merged PR emits a `pull_request_merged` domain event.	`rest/github.py` (`webhook`); `GithubService.apply_webhook`; `domain/github.py` (`issue_identifiers`, `MENTION_LIMIT = 20`)	none (linking is automatic; the per-issue PR panel has no UI, see 3.3)	Partial	`github_deliveries` is never pruned.

Feature	How it works	Backend	Frontend	UI	Notes / limits
GitHub: state automation	Per-team opt-in. Opening, reopening or marking a PR ready for review moves linked issues to a started state, and merging moves them to completed. Moves are forward-only. Defaults are the team's first started and completed states. Each move records activity with `caused_by` and sends a `status_changed` notification.	`GithubService.set_issue_automation`, `_automate_state`; `domain/github.py` (`automated_move_for`, `START_ACTIONS`); `migrations/030_github_automations.sql`	`SettingsPage.tsx` (`WorkspaceGithubAutomationSet`, a checkbox plus two state selects)	Full	
Slack: connect / disconnect	Admin or owner. `GET /slack/oauth/start` requests scopes `channels:read` and `chat:write`, with a state cookie. The callback exchanges the code. One Slack team maps to one workspace (409 on conflict). The bot token is stored in plaintext (`DatabaseTokenStore`). Disconnect does not revoke the token at Slack.	`rest/slack.py`; `services/slack.py` (`SlackService.connect`, `disconnect`, `REQUESTED_SCOPES`)	`SettingsPage.tsx`, `IntegrationsStep.tsx` (link to `/slack/oauth/start`; `WorkspaceSlackDisconnect`)	Full	Status is `unconfigured`, `disconnected` or `connected`. An unconfigured Slack returns 503, where GitHub returns 404.
Slack: channel and notification preferences	Sync public channels (`conversations.list`, up to 10 pages). Choose one default channel. Turn each of the six event kinds on or off (off by default). Send a test message.	`SlackService.sync_channels`, `set_default_channel`, `set_notification_preference`, `send_test_notification`	none (`slackChannelsSync`, `slackDefaultChannelSet`, `slackNotificationPreferenceSet` and `slackTestNotification` are absent from `frontend/src`)	Backend only	Because preferences default to off and the UI cannot turn them on, a Slack workspace connected through the UI receives no event messages.
Slack: event delivery	Transactional outbox `domain_events`: `issue_assigned`, `issue_completed`, `issue_priority_urgent`, `project_health_changed`, `project_update_published`, `pull_request_merged`. A background loop claims one row at a time (`SKIP LOCKED`) and posts a one-line message to the default channel. Up to 5 attempts in total (`MAX_ATTEMPTS = 5`), with linear backoff of 1, 2, 3 and 4 minutes between them. Terminal failures are kept with a reason.	`services/events.py`; `services/notifications.py` (`SlackNotifier`, `run_delivery_loop`); `repositories/events.py` (`claim_next`)	none	Backend only (background)	`IssueService.create` emits no domain event, so creating an issue with an assignee posts nothing. Deep links only when `PUBLIC_BASE_URL` is set.
Slack: inbound events	`POST /slack/events` verifies the v0 signature (±300 s), answers `url_verification`, ignores the bot's own messages and deduplicates by event id.	`rest/slack.py` (`slack_events`, `_handle_event_callback`)	none	Backend only	No ingestion: the handler body is "Deliberately empty".

3.10 Onboarding, navigation and shell

Feature	How it works	Backend	Frontend	UI	Notes / limits
Landing page	Public `/` with sign-in and register calls to action	none	`landing/LandingPage.tsx`	Full	

Feature	How it works	Backend	Frontend	UI	Notes / limits
Onboarding wizard	Workspace → Team → Invite → Integrations. `resolveStep` redirects to the first missing step or to `/:slug/issues`.	`workspaceCreate`, `teamCreate`, `invitationCreate`, `githubIntegration` / `slackIntegration`	`onboarding/pages/OnboardingLayout.tsx`, `WorkspaceStep.tsx`, `TeamStep.tsx`, `InviteStep.tsx`, `IntegrationsStep.tsx`; `onboarding/lib/progress.ts`	Full	A signed-in user with no memberships is redirected to `/onboarding` (`app/layout/AppLayout.tsx`).
Sidebar navigation	Sections: My Issues, Inbox, Triage, All Issues, Board, Projects, Initiatives, Roadmap, Documents; Saved Views, Favorites; Members, Analytics, Releases, Environments, Label Groups, Templates, Settings. Each team expands to Team, Issues and Cycles. Collapse state persists.	`ShellSidebar` query (`app/workspace/operations.graphql`)	`navigation/navigationItems.tsx`, `SidebarNav.tsx`; `app/layout/AppLayout.tsx`	Full	`/semantic-search` is not linked. Cycles has no top-level entry.
Cross-tenant guard (UI)	The shell never queries a slug the viewer is not a member of and renders "Workspace not found". The server separately returns `NOT_FOUND` for non-members.	`MembershipService.authorized_scope_for_slug`; `graphql/scope.py`	`app/layout/AppLayout.tsx` (`WorkspaceNotFound`)	Full	Covered by Playwright `e2e/journeys.spec.ts` test 3.

Gaps / to verify:

- **UI coverage method.** The Backend only and Partial labels rest on searching `frontend/src` for operation and field names. A dynamically built operation would not be detected; none was seen, since all operations are codegen documents.
- **Saved view layout.** The `board` layout appears to be stored and labelled but not rendered as a board in `SavedViewsPage.tsx`. Only the presence of `IssueRow` at around line 391 was checked; the full render branch was not read.
- **Second workspace via UI.** `resolveStep` returns a redirect for the workspace step whenever `hasWorkspace` is true, so a second workspace appears impossible to create in the UI. Whether another entry point (for example the workspace switcher) offers it was not checked beyond the operation list: only `OnboardingWorkspaceCreate` calls `workspaceCreate`.
- **Team creation after onboarding.** `/onboarding/team` is still routable for a user who already has a team (`resolveStep` returns null for that step). It is not linked from the product shell; this was not exercised in a browser.
- **Slack preferences.** "Connected via UI ⇒ no messages" is inferred from preferences defaulting to off (`SlackService._settings`, `SlackNotifier._route`) together with the absence of a preference UI. It was not tested end to end.
- **Project and initiative delete with documents.** The 500 / masked-error outcome is inferred from FK definitions and the absence of a mapping. Not run.
- **Reminder timezone.** Whether production PostgreSQL sessions run in UTC (which `CURRENT_DATE` for reminders and due windows depends on) was not verified.

4. Tech stack

Versions below are the exact resolved pins: Python from the hash-locked `requirements.txt` / `requirements-dev.txt` (compiled with `uv pip compile ... --universal --python-version 3.12 --generate-hashes`, header of `requirements.in`), JavaScript from `frontend/package-lock.json` (lockfileVersion 3). Snapshot: HEAD `577b868`.

4.1 Languages

Name	Version	Why it is there
Python	3.12 (image pins 3.12.14)	Backend, migration runner, scripts. `ARG PYTHON_VERSION=3.12.14` (`Dockerfile`); `python-version: "3.12"` (`.github/workflows/ci.yml`); ruff `target-version = "py312"`, mypy `python_version = "3.12"` (`pyproject.toml`); launcher requires 3.12+ (`run-vector-local.ps1` `$MinimumPythonMinor = 12`).
TypeScript	6.0.3	Frontend (`frontend/src`, `.ts`/`.tsx`); `tsc -b` is the `typecheck` script and the first half of `build` (`frontend/package.json`). Compile target `es2023` (`frontend/tsconfig.app.json`).
SQL (PostgreSQL 18 dialect)	PG 18	32 numbered migrations `migrations/001_issues.sql` … `032_auth_hardening.sql`, plus the design-only `schema_drafts/full_schema.sql`. Plain SQL only: no `CREATE FUNCTION`/`CREATE TRIGGER`/PL/pgSQL in any migration (grep). Requires PG 18 because `001_issues.sql` defaults keys to `uuidv7()`.
GraphQL SDL	—	`frontend/schema.graphql` (exported from the Strawberry schema by `frontend/scripts/export_schema.py`) and frontend operation documents feeding codegen.
CSS	—	Styling; 53 of 57 tracked `.css` files are CSS Modules (`*.module.css`).
PowerShell	5.1+	Windows one-command launcher (`run-vector-local.ps1`, `#Requires -Version 5.1`).
YAML	—	CI (`.github/workflows/ci.yml`), `docker-compose.yml`, `k8s/*.yaml`, `render.yaml`, `.pre-commit-config.yaml`.
TOML	—	`pyproject.toml` (pytest/ruff/mypy config only, no `[project]` table), `.gitleaks.toml`.
JavaScript (ESM)	Node ≥ 20.19	Build/codegen helpers `frontend/scripts/graphql-schema.mjs`, `frontend/scripts/graphql-codegen.mjs`; `frontend/eslint.config.js`. `engines.node: ">=20.19"` (`frontend/package.json`).
nginx config	—	`frontend/nginx.conf` for the separate web image.

4.2 Backend runtime dependencies

Direct dependencies as declared in `requirements.in`; version is the pin in `requirements.txt`. Only `strawberry-graphql` is exact-pinned in the `.in`; `starlette` has a floor.

Name	Version	Why it is there
fastapi	0.141.1	Outer ASGI framework and REST routes (`app/main.py` `create_app`, `app/rest/*.py`).
starlette	1.6.0 (`>=1.6.0` floor)	`app/http_limits.py` imports `starlette.middleware.body_limit`, new in 1.6.0 (comment in `requirements.in`); `StaticFiles` in `app/spa.py`.
uvicorn[standard]	0.52.4	ASGI server; image CMD `exec uvicorn "app.main:create_app" "--factory"` (`Dockerfile`). Extra pulls `httptools` 0.8.0, `uvloop` 0.22.1 (non-Windows), `watchfiles` 1.2.0, `websockets` 17.0.1, `python-dotenv` 1.2.3, `pyyaml` 6.0.3 (`requirements.txt`).
asyncpg	0.31.0	PostgreSQL driver and pool (`app/db.py`), migration runner (`scripts/apply_migration.py`), `scripts/inspect_deployment_db.py`.
pydantic	2.13.4	`SecretStr` in `app/config.py` (named directly per `requirements.in` comment). `pydantic-core` 2.46.4 transitive.
pydantic-settings	2.15.0	`Settings(BaseSettings)` with `env_file=".env"`, `extra="forbid"` (`app/config.py`).

Name	Version	Why it is there
strawberry-graphql[fastapi]	0.326.0	GraphQL schema, types, router (`app/graphql/`). `graphql-core` 3.2.12 transitive.
structlog	26.1.0	Structured logging via `structlog.get_logger` (`app/main.py` `startup.ready` event, `app/services/*.py`, `app/rest/github.py`); `app/logging.py` is only a `# TODO: structlog` stub.
argon2-cffi	25.1.0	Argon2id password hashing (`app/services/passwords.py`); hash warm-up at startup (`app/main.py` `warm_password_hashing`). `argon2-cffi-bindings` 26.1.0, `cffi` 2.1.1 transitive.
cryptography	50.0.1	**Declared but unused.** `requirements.in` says it is for RS256 signing of the GitHub App JWT, but nothing in `app/`, `scripts/` or `tests/` imports it (grep for `cryptography` returns no hits). Ships in the runtime image as dead weight.
httpx	0.28.1	Async HTTP client for GitHub and Slack API calls (`app/services/github.py`, `app/services/slack.py`); also the test client transport.

Transitive packages imported directly elsewhere: `certifi` 2026.7.22 is imported by `scripts/inspect_deployment_db.py` but is not declared in `requirements.in` (arrives via httpx).

4.3 Backend dev / tooling dependencies

`requirements-dev.in` starts with `-r requirements.in`, so `requirements-dev.txt` is a superset. The runtime image installs `requirements.txt` only (`Dockerfile` stage `builder`).

Name	Version	Why it is there
pytest	9.1.1	Test runner; `addopts = "-m 'not db'"`, marker `db` (`pyproject.toml`).
pytest-asyncio	1.4.0	`asyncio_mode = "auto"` (`pyproject.toml`).
httpx	0.28.1	ASGI transport for the FastAPI test client (`requirements-dev.in`).
ruff	0.14.14	Lint (`select = E4,E7,E9,F,I,B,ASYNC`) and format (`pyproject.toml`; CI `lint` job).
mypy	1.19.1	`mypy app/` with `plugins = ["pydantic.mypy"]` (`pyproject.toml`; CI `lint` job).
pre-commit	4.4.0	Runs `.pre-commit-config.yaml` (CI `lint` job).
uv	0.12.9	The lock compiler, pinned so `locks` CI job recompiles reproducibly (`requirements-dev.in`).

No coverage tool is declared (`requirements-dev.in`, `pyproject.toml`).

`uv.lock` is empty. The tracked `uv.lock` contains only `version = 1`, `revision = 3`, `requires-python = ">=3.12"` and no packages; no tooling (CI, `Dockerfile`, launcher) reads it. The real locks are `requirements*.txt`.

4.4 Frontend dependencies

Resolved versions from `frontend/package-lock.json` (`node_modules/<name>.version`).

Name	Version	Why it is there
react	19.2.8	UI library (`frontend/src`).
react-dom	19.2.8	DOM renderer.
react-router-dom	7.18.3	Client routing (`frontend/src/app/App.tsx`, `app/layout/*`).
@apollo/client	4.2.12	GraphQL client and cache (`frontend/src/lib/graphql/client.ts`, `app/providers/AppProviders.tsx`).
graphql	16.14.2	Peer dependency of `@apollo/client` 4 and of the codegen toolchain; imported directly in tests (`src/features/issues/api/documents.test.ts`).

Name	Version	Why it is there
rxjs	7.8.2	Required peer of `@apollo/client` 4 (`peerDependencies.rxjs ^7.3.0` in the lock); imported directly only in tests (`src/test/controlledLink.ts`, `src/features/auth/api/sessionExpiry.test.ts`).

4.5 Frontend dev dependencies

Name	Version	Why it is there
vite	8.2.2	Dev server, proxy and production build (`frontend/vite.config.ts`; scripts `dev`, `build`, `preview`).
@vitejs/plugin-react	6.1.1	React plugin for Vite and Vitest (`vite.config.ts`, `vitest.config.ts`).
typescript	6.0.3	`tsc -b` (`typecheck`, `build`).
vitest	5.0.0	Unit/component tests, `environment: 'jsdom'` (`frontend/vitest.config.ts`; script `test`).
jsdom	29.1.1	Vitest DOM environment.
@testing-library/react	16.3.3	Component test rendering (`*.test.tsx`).
@testing-library/dom	10.4.1	Peer of `@testing-library/react`; not imported directly.
@testing-library/user-event	14.6.7	User interaction simulation in tests.
@testing-library/jest-dom	7.0.1	DOM matchers (`src/test/setup.ts`).
@playwright/test	1.63.0	Browser E2E suite (`frontend/playwright.config.ts`, `frontend/e2e/`); exact pin.
@graphql-codegen/cli	7.4.0	Typed GraphQL codegen (`frontend/codegen.ts`, `scripts/graphql-codegen.mjs`); exact pin.
@graphql-codegen/typescript	6.1.0	Generates `src/generated/schema.ts`; exact pin.
@graphql-codegen/typescript-operations	6.1.6	Generates `src/generated/operations.ts`; exact pin.
@graphql-codegen/typed-document-node	7.1.0	Typed document nodes for Apollo; exact pin.
eslint	10.10.0	Lint (`frontend/eslint.config.js`; script `lint`).
typescript-eslint	8.69.0	TS rules for ESLint (`eslint.config.js`).
@types/react	19.2.18	React typings.
@types/react-dom	19.2.7	React DOM typings.

4.6 Data

Name	Version	Why it is there
PostgreSQL	18 (staging Neon reports 18.6)	System of record. 18 is mandatory: `uuidv7()` default in `migrations/001_issues.sql`; staging version per `DEPLOYMENT_ENV_VARS.md` "Neon".
pgvector/pgvector image	`pg18` tag	Local, compose, k8s, CI and test-fixture database image (`docker-compose.yml`, `k8s/20-postgres.yaml`, `run-vector-local.ps1`, `.github/workflows/ci.yml`, `tests/conftest.py`). Official `postgres:18` plus pgvector.
btree_gist	not pinned	`CREATE EXTENSION IF NOT EXISTS btree_gist` (`migrations/008_cycles.sql`); enables the `cycles_no_overlap EXCLUDE USING gist` with `=` on UUID columns.
btree_gin	not pinned	`CREATE EXTENSION IF NOT EXISTS btree_gin` (`migrations/011_search.sql`); enables composite `GIN (workspace_id, search_vector)` indexes.
vector (pgvector)	not pinned	`CREATE EXTENSION IF NOT EXISTS vector` (`migrations/025_semantic_search.sql`); `embedding vector(384)` column. No HNSW/IVFFlat index by design (comment block in 025).

No extension `VERSION` clause is used, so extension versions are whatever the image/Neon provides.

4.7 Infrastructure and tooling

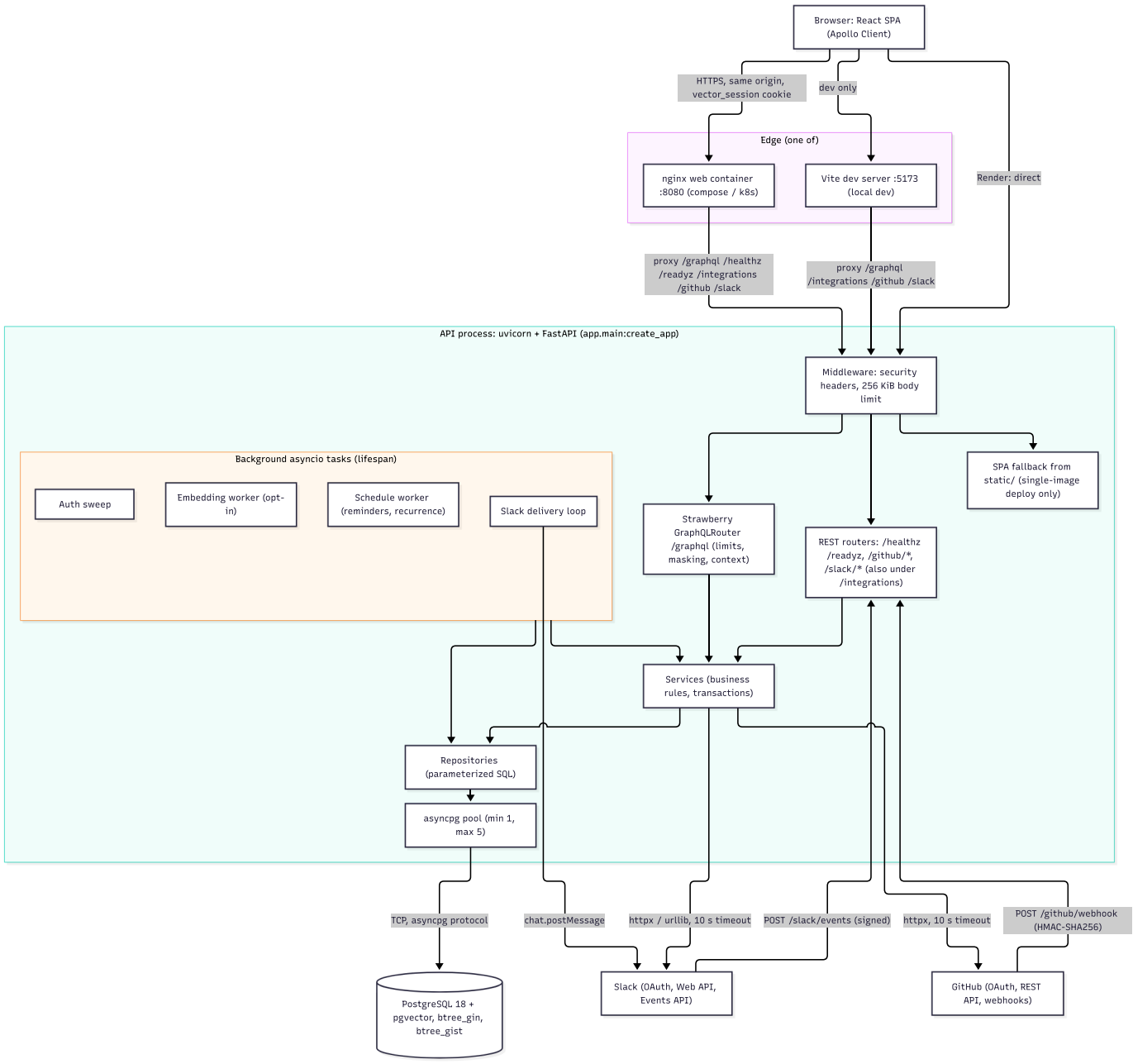
Name	Version	Why it is there
Docker base `python`	`3.12.14-slim-bookworm`	`builder` and `runtime` stages of `Dockerfile`. Tag-pinned, no digest.
Docker base `node`	`22-alpine`	Frontend build stages in `Dockerfile` (`frontend`) and `frontend/Dockerfile` (`builder`).
`nginxinc/nginx-unprivileged`	`1.29-alpine`	Runtime of the separate web image (`frontend/Dockerfile`), config `frontend/nginx.conf`.
Docker Compose	—	Local deployment-shape stack, 4 services (`docker-compose.yml`).
Kubernetes	plain manifests	`k8s/00-namespace.yaml` … `k8s/40-web.yaml`; no Helm/kustomize (`README.md`).
Render	`plan: free`, `region: oregon`	Staging/demo web service `vector-staging`, Docker runtime, auto-deploy from `main` (`render.yaml`).
Neon	PostgreSQL 18.6	Managed Postgres for staging, DSN with `sslmode=verify-full` (`DEPLOYMENT_ENV_VARS.md`).
GitHub Actions	`ubuntu-24.04` runners	CI, 10 jobs (`.github/workflows/ci.yml`). Actions: `actions/checkout@v7`, `actions/setup-python@v7`, `actions/setup-node@v7`, `actions/cache@v6`, `actions/upload-artifact@v6` (major tags, not SHAs).
gitleaks	8.30.1	CI `secret-scan` (tarball sha256-verified), `.gitleaks.toml` `minVersion = "8.30.1"`, default rules plus custom `vector-postgres-dsn-password`.
pre-commit hooks	ruff-pre-commit `v0.14.14`; pre-commit-hooks `v6.0.0`	`ruff-check --fix`, `ruff-format`, `trailing-whitespace`, `end-of-file-fixer` (both exclude `^migrations/`), `check-yaml --allow-multiple-documents`, `check-toml`, `check-merge-conflict` (`.pre-commit-config.yaml`).
uv	0.12.9	Lock compilation only (`requirements-dev.in`, CI `locks`).
Playwright (Chromium)	1.63.0	E2E in CI (`npx playwright install --with-deps chromium`, `e2e` job); 1 worker, 0 retries (`frontend/playwright.config.ts`).

> **Gaps / to verify:**
>
> - `cryptography` 50.0.1 is a declared runtime dependency with no import anywhere; confirm whether GitHub App RS256 signing is planned or the dependency should be dropped.
> - `uv.lock` is an empty, unused artifact.
> - pgvector, btree_gist and btree_gin versions are not pinned anywhere in the repo; actual versions depend on the `pgvector/pgvector:pg18` tag at pull time and on Neon.
> - Base images and Actions are tag-pinned, not digest/SHA-pinned.

5. Architecture

Vector is a single FastAPI process that serves the GraphQL API, a few REST endpoints, and (in the single-image deployment) the built React SPA. It also runs four background `asyncio` tasks. PostgreSQL 18 is the only stateful dependency: there is no Redis, message broker, cron container or external search engine (`app/main.py :: lifespan`, `_start_embedding_worker` docstring; `migrations/011_search.sql` header).

5.1 Component diagram



Notes on the diagram:

- "BG → REPO" is real: the schedule worker and Slack notifier hold repositories directly (`NotificationRepository`, `EventRepository`, `SlackRepository`) because they have no request user to build an `AuthorizedWorkspaceScope` from; they still acquire connections themselves, as services do (`app/main.py :: _build_schedule_worker`, `_build_notifier`).
- Extensions: `btree_gist` (`migrations/008_cycles.sql`), `btree_gin` (`011_search.sql`), `vector` (`025_semantic_search.sql`).

5.2 Components and how they communicate

Component	Technology	Responsibility	Talks to (protocol)	Source
SPA	React, Vite build, Apollo Client	UI; all data via GraphQL	API at relative `/graphql` over HTTP POST JSON, `credentials: 'same-origin'`	`frontend/src/lib/graphql/client.ts :: createApolloClient`, `frontend/src/lib/config/env.ts :: graphqlUrl`
nginx `web` (compose/k8s)	`nginxinc/nginx-unprivileged:1.29-alpine`	Serves `dist/`, SPA fallback, reverse-proxies API paths	`proxy_pass http://api:8000` for `^/(graphql|healthz|readyz|integrations|github|slack)(/|$)`; sets `Host`, `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`	`frontend/nginx.conf`, `frontend/Dockerfile`
Vite dev server	Vite	Dev-only proxy of `/graphql`, `/integrations`, `/github`, `/slack` to `http://127.0.0.1:8000` with `changeOrigin: false`	HTTP	`frontend/vite.config.ts :: proxy`
FastAPI app	FastAPI 0.141.1 on uvicorn, factory `app.main:create_app`	Composition root: middleware, routers, lifespan	ASGI	`app/main.py :: create_app`
Middleware	Starlette `RequestBodyLimitMiddleware`; `@app.middleware("http")`	256 KiB body cap; CSP/nosniff/Referrer/Permissions/HSTS headers	in-process	`app/http_limits.py :: add_request_body_limit`, `app/http_headers.py :: add_security_headers`
GraphQL router	Strawberry 0.326.0 `GraphQLRouter` at `/graphql`	Parse, validate (depth/alias/selection/complexity limits), execute, mask errors; POST only; GraphiQL outside production	in-process	`app/graphql/router.py :: build_graphql_router`, `app/graphql/schema.py :: build_schema`
REST routers	FastAPI `APIRouter`	`/healthz` (no DB), `/readyz` (DB probe); GitHub install/callback/webhook; Slack OAuth/events. GitHub and Slack routers are each mounted twice (bare and under `/integrations`)	HTTP	`app/rest/health.py`, `app/rest/github.py`, `app/rest/slack.py`, `app/main.py :: create_app`
SPA host (single image)	`StaticFiles` subclass + catch-all route	Serves `static/assets/*` (immutable cache) and `index.html` (`no-store`); no-op when `static/index.html` is absent	in-process	`app/spa.py :: add_spa`
Services	Plain Python classes	Validation, business rules, transaction boundaries	Repositories (in-process), external HTTP	`app/services/*`

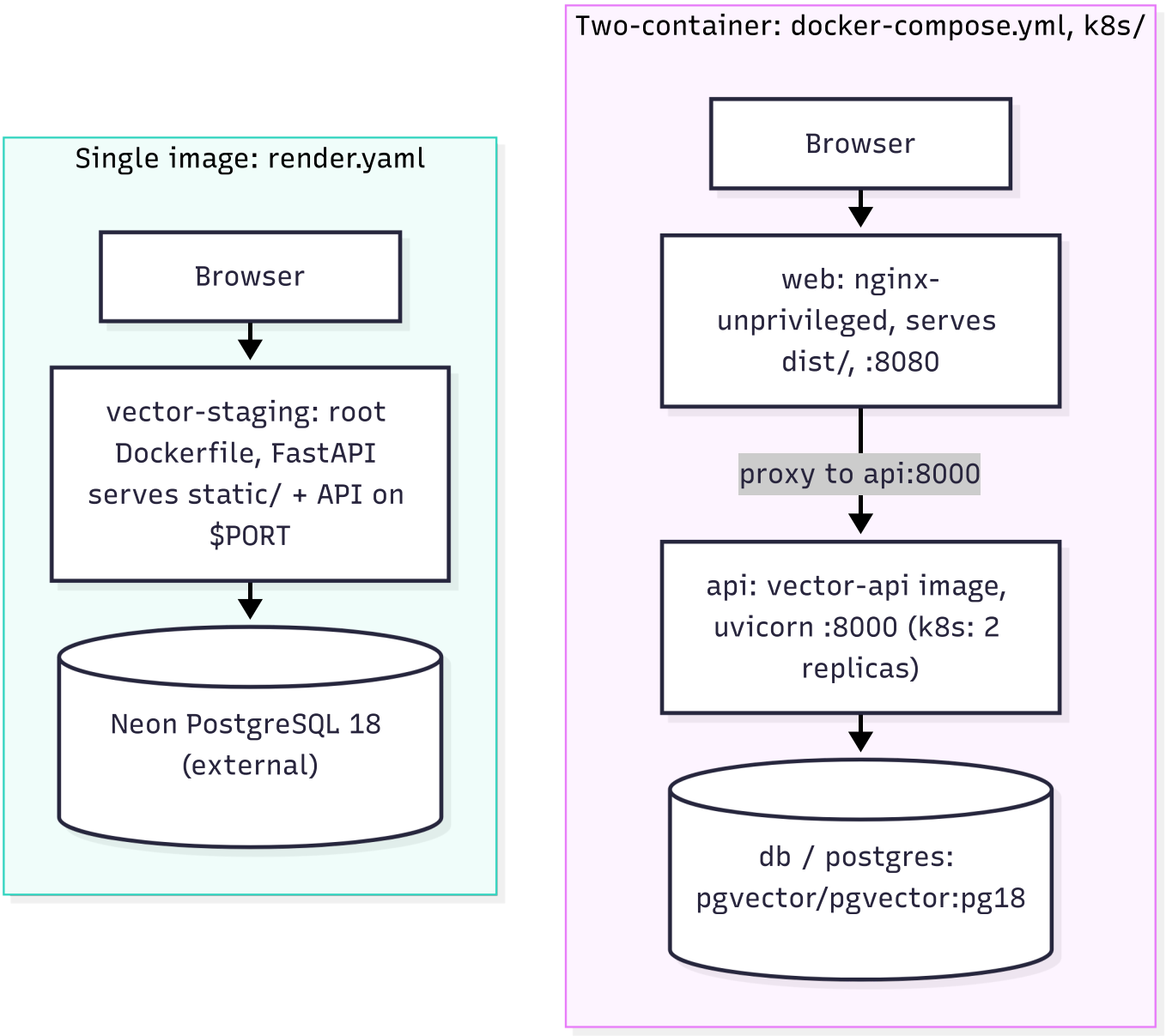
Component	Technology	Responsibility	Talks to (protocol)	Source
Repositories	asyncpg	Parameterized SQL, `Record` → frozen dataclass	PostgreSQL over the connection passed in	`app/repositories/*`
Pool	`asyncpg.create_pool(min_size=1, max_size=5, command_timeout=10.0, server_settings={"statement_timeout": "9500"})`	One pool per process, shared by requests and background tasks	TCP	`app/db.py :: connect`
Background tasks	`asyncio.create_task` in lifespan	Embedding indexing, Slack delivery, reminders/recurrence, session/rate-limit sweep	Pool, Slack HTTP	`app/main.py :: lifespan` (detail in §10.7)
PostgreSQL 18	`pgvector/pgvector:pg18` locally; Neon in Render staging	Source of truth, FTS, vector distance, queues (`FOR UPDATE SKIP LOCKED`)	—	`docker-compose.yml`, `render.yaml` comments, `DEPLOYMENT_ENV_VARS.md`
GitHub	GitHub App + OAuth	Install/ownership check, webhooks (PRs, pushes, installations)	Inbound HTTPS webhooks; outbound httpx to `github.com` / `api.github.com`	`app/rest/github.py`, `app/services/github.py`
Slack	Slack app	OAuth install, channel list, outbound messages; inbound events (verified, deduped, not ingested)	Outbound httpx/urllib to `slack.com/api`; inbound `POST /slack/events`	`app/rest/slack.py`, `app/services/slack.py`

Cross-cutting communication rules:

- **Same origin, no CORS.** The backend installs no CORS middleware. The SPA always calls a relative `/graphql`, and a proxy (nginx, Vite) or the single-image FastAPI makes that same-origin (`frontend/vite.config.ts` comment on `proxy`; `app/spa.py` docstring).
- **Authentication is a cookie.** `vector_session`, `Path=/`, `SameSite=Lax`, HttpOnly, `Secure` only when `ENVIRONMENT=production` (`app/http_cookies.py :: SESSION_COOKIE_NAME, is_secure_environment`). The GraphQL context reads it through `read_session_token` (`app/graphql/context.py :: VectorContext.session_token`).
- **Client IP behind proxies** comes from the Nth-from-right `X-Forwarded-For` entry when `TRUSTED_PROXY_HOPS > 0`; `X-Real-IP` is never trusted (`app/http_client_ip.py :: read_client_ip`). Compose, k8s and Render all set `TRUSTED_PROXY_HOPS=1`.
- **Tenant is never ambient.** Every workspace-scoped GraphQL field takes a `workspaceSlug` and resolves membership itself (`app/graphql/scope.py :: authorized_scope`).

5.3 Deployment topologies

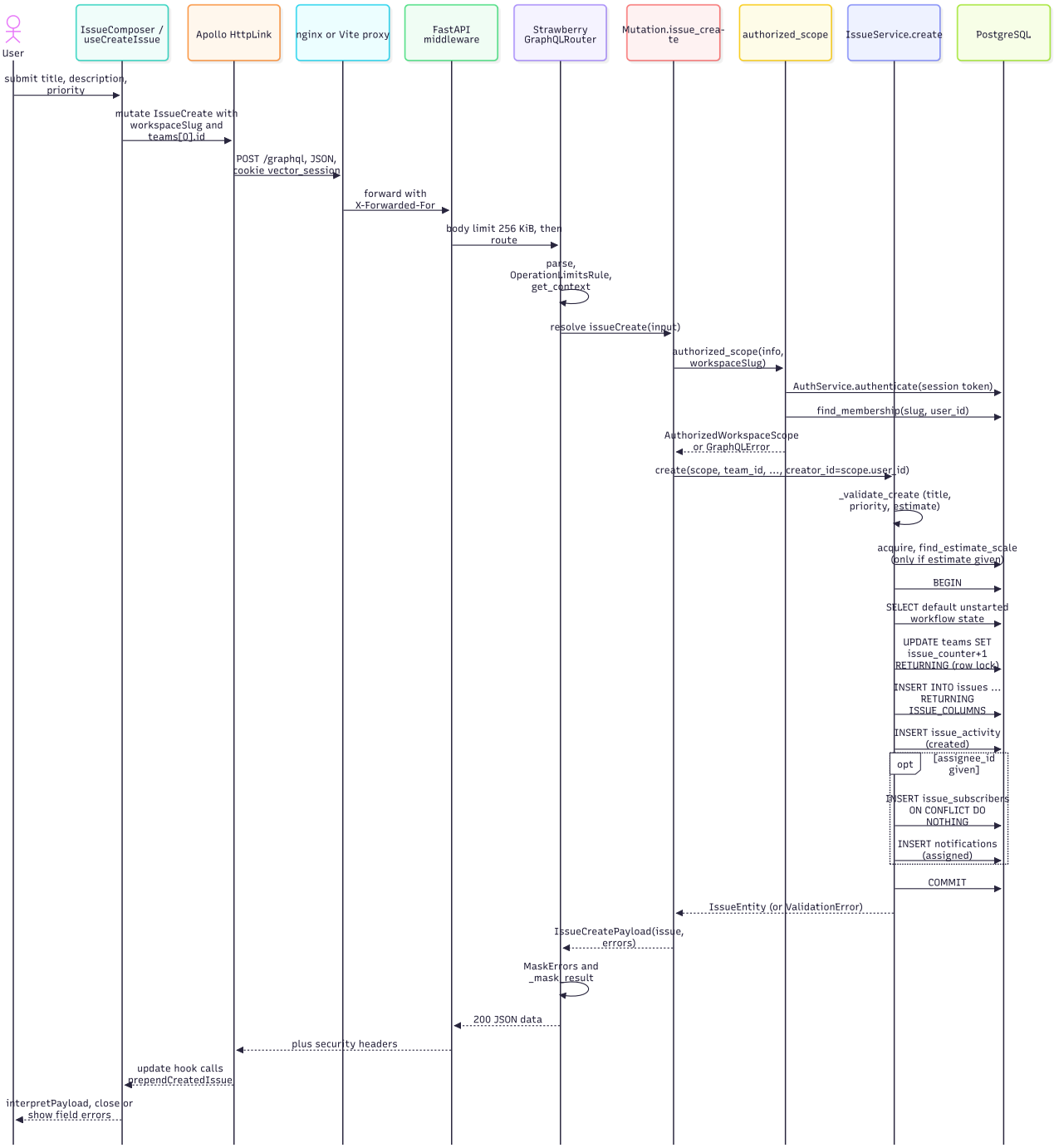
Two shapes ship from the same backend image (root `Dockerfile`). Full detail, probes and env vars are in §11.



Aspect	Compose / Kubernetes	Render (staging/demo)
SPA served by	nginx `web` container (`frontend/Dockerfile`)	FastAPI from `static/` (`Dockerfile` copies `/build/dist` to `./static`; `app/spa.py :: add_spa`)
API replicas	1 (compose; `--scale` possible), 2 (`k8s/30-api.yaml`)	1 (`render.yaml`, `plan: free`)
Migrations	Image CMD runs `python -m scripts.apply_migration --all` before `exec uvicorn`; compose also has a one-shot `migrate` service	Same image CMD
Database	`pgvector/pgvector:pg18` container / Deployment	Hand-created Neon PG 18 project
Health checks	`/readyz` (compose api, k8s readiness), `/healthz` (k8s startup/liveness)	`healthCheckPath: /healthz`

Multiple API replicas are safe because every background queue claims work with `FOR UPDATE SKIP LOCKED`, a primary-key ledger, or idempotent DELETEs (`app/main.py :: _start_embedding_worker`, `_build_schedule_worker` docstrings; `lifespan` comment on `sweep`).

5.4 Request lifecycle: "create an issue"



1. UI trigger. The shell's create button (`frontend/src/app/layout/CreateIssueButton.tsx`) invokes the handler that `IssuesScreen` registers via `useRegisterCreateIssueAction(openComposer)`, which renders `IssueComposer` (`frontend/src/features/issues/pages/IssuesScreen.tsx`, `frontend/src/app/layout/createIssueAction.ts`). `IssueComposer.submit` calls `createIssue({ title, description, priority })`; a blank description is sent as `null` (`frontend/src/features/issues/components/IssueComposer.tsx`). The composer has no team, assignee, estimate or due-date controls even though the backend input accepts the last three.

2. Mutation document and hook. `useCreateIssue` (`frontend/src/features/issues/api/useCreateIssue.ts`) runs `useMutation(IssueCreateDocument)` with variables `{ input: { ...draft, workspaceSlug, teamId } }`. `teamId` is always `teams[0]?.id` from `useWorkspaceContext`; with no team the hook returns `NO_TEAM` without a request. The document is defined in `frontend/src/features/issues/api/operations.graphql`:

```
mutation IssueCreate($input: IssueCreateInput!) {
  issueCreate(input: $input) {
    issue { ...IssueDetailFields }
    errors { ...ValidationFields }
  }
}
```

`IssueCreateDocument` is generated into `frontend/src/generated/operations.ts` by GraphQL codegen and re-exported from `frontend/src/features/issues/api/documents.ts`.

3. GraphQL client. Apollo's link chain is `sessionExpiryLink` → `HttpLink({ uri: graphqlUrl, credentials: 'same-origin' })` (`frontend/src/lib/graphql/client.ts`). `graphqlUrl` defaults to `/graphql` unless `VITE_GRAPHQL_URL` is set (`frontend/src/lib/config/env.ts`). The browser attaches the `vector_session` cookie. `sessionExpiryLink` watches for `extensions.code === 'UNAUTHENTICATED'` (`frontend/src/features/auth/api/sessionExpiry.ts`).

4. Proxy. Compose/k8s: nginx matches `^/(graphql|…)` and proxies to `api:8000` with forwarding headers (`frontend/nginx.conf`). Local dev: Vite proxies `/graphql` to `http://127.0.0.1:8000` (`frontend/vite.config.ts`). Render: no proxy; the browser hits uvicorn behind Render's edge, and uvicorn trusts forwarded headers via `FORWARDED_ALLOW_IPS="*"` (`render.yaml`).

5. FastAPI middleware. `create_app` registers `add_request_body_limit` then `add_security_headers` (`app/main.py :: create_app`). Starlette's `add_middleware` inserts each at position 0, so the effective outer-to-inner chain is: `ServerErrorMiddleware` → security-headers middleware → `RequestBodyLimitMiddleware` (256 KiB; rejects an oversized `Content-Length` before reading and aborts a streamed overrun) → `ExceptionMiddleware` → router (`app/http_limits.py`, `app/http_headers.py`). Headers are applied with `setdefault`, so a route's own value wins. There is no CORS, GZip, TrustedHost or HTTPS-redirect middleware.

6. GraphQL router, limits, context. The request reaches Strawberry's `GraphQLRouter` mounted at `/graphql` (`context_getter=get_context`, `allow_queries_via_get=False`) (`app/graphql/router.py :: build_graphql_router`). During validation `OperationLimitsRule` enforces `MAX_DEPTH=10`, `MAX_ALIASES=15`, `MAX_SELECTIONS=2000`, `MAX_COMPLEXITY=1000` (`app/graphql/limits.py`). `get_context` builds a fresh `VectorContext` (a Strawberry `BaseContext`) with ~22 services, each given the process pool and new stateless repositories, e.g. `IssueService(pool=pool, repository=IssueRepository(), teams=team_service)` (`app/graphql/context.py :: get_context`). The viewer is not resolved yet; `VectorContext.viewer()` is a memoised future.

7. Resolver and scope. `Mutation.issue_create(info, input: IssueCreateInput)` (`app/graphql/mutations/issues.py`) first calls `authorized_scope(info, input.workspace_slug)` outside any `try` (`app/graphql/scope.py`):

1. `viewer_user_id` awaits `context.viewer()` → `AuthService.authenticate(session_token)`; no user raises `GraphQLError("Authentication required", extensions={"code": "UNAUTHENTICATED"})` before any workspace lookup (`app/graphql/viewer.py :: viewer_user_id`).
2. `MembershipService.authorized_scope_for_slug(slug, user_id)` runs one membership statement (`workspace_members JOIN workspaces … removed_at IS NULL`). "No such slug" and "not a member" both raise `WorkspaceAccessDeniedError`, translated to `GraphQLError("Workspace not found", extensions={"code": "NOT_FOUND"}) from None`.
3. The result is an `AuthorizedWorkspaceScope(workspace_id, user_id, role)` (`app/domain/tenancy.py`). Any member role may create issues.

8. Input → service call. `IssueCreateInput` (`app/graphql/inputs/issue.py`) has `workspace_slug`, `team_id`, `title`, `description=None`, `priority=0`, `assignee_id=None`, `estimate=None`, `due_date=None`, and deliberately no `workflowStateId`, `creatorId` or `completedAt`. The resolver passes the fields as keyword arguments plus `creator_id=scope.user_id` to `info.context.issue_service.create(...)`, inside `try/except ValidationError`. There is no separate input→domain object for create; scalars are passed directly (the patch path for updates is §10.5).

9. Service (`app/services/issues.py :: IssueService.create`).

1. `_validate_create` — pure, collects every violation then raises once: title length 1–500 as supplied (not trimmed; `REQUIRED`/`TOO_LONG`), priority 0–4 (`OUT_OF_RANGE`), estimate ≥ 0 (`OUT_OF_RANGE`) (`_title_issues`, `_priority_issues`,

`_estimate_issues`).
2. `async with self._pool.acquire()`.
3. `_check_estimate_scale` — only when an estimate is given, reads the team's scale via `TeamService.estimate_scale` and raises `OUT_OF_RANGE` if the value is not on it. This runs on the connection but **outside** the transaction, keeping the later row lock short.
4. `async with connection.transaction():`
   - `TeamService.default_workflow_state_id` → `TeamRepository.find_default_workflow_state_id`: `SELECT id FROM workflow_states WHERE workspace_id = $1 AND team_id = $2 AND type = 'unstarted' ORDER BY position, id LIMIT 1`; none → `TeamNotFoundError`.
   - `TeamService.allocate_issue_number` → `TeamRepository.allocate_issue_number`: `UPDATE teams SET issue_counter = issue_counter + 1 WHERE workspace_id = $1 AND id = $2 RETURNING issue_counter`. This row-locks the team until commit, serialising creates per team; because it is an ordinary column, a rollback returns the number, so numbering is gapless (unlike a sequence).
   - `IssueRepository.create` (step 10).
   - `activity.record(..., kind=ActivityKind.CREATED, to_value=entity.title, actor_id=creator_id)` → one `issue_activity` row (`app/services/activity.py :: record`).
   - Only if `assignee_id` is set: `activity.auto_subscribe` (`issue_subscribers` insert, `ON CONFLICT DO NOTHING`) and `activity.notify(kind=NotificationKind.ASSIGNED, actor_id=creator_id)` → `NotificationRepository.notify_about_issue`, one `INSERT … SELECT DISTINCT` over assignee ∪ subscribers, active members only, minus the actor (`app/repositories/notifications.py`). Self-assignment therefore produces no notification.
5. Commit on leaving the block. Errors are mapped **after** rollback: `TeamNotFoundError` → `ValidationError` teamId `NOT_FOUND`; `asyncpg.ForeignKeyViolationError` → `_validation_error_for(exc.constraint_name)` (`issues_team_fk` → teamId `NOT_FOUND`, `issues_assignee_fk` → assigneeId `NOT_A_MEMBER`, `issues_workflow_state_fk` → workflowStateId `NOT_FOUND`); an unlisted constraint is re-raised and later masked.

What create does **not** do (verified in `IssueService.create`):

- No domain event: `record_issue_event` is not called, so creating an issue with an assignee emits no `issue_assigned` Slack event (an update that assigns does, via `activity.record_changes`).
- No embedding job: there is no enqueue on any write path; the only enqueue is the worker's polling `EmbeddingJobRepository.enqueue_stale` (§10.9). Lexical search sees the issue immediately because `issues.search_vector` is a generated column (`migrations/011_search.sql`).
- No labels, cycle, project or parent are set; labels are attached by separate mutations, and `TemplateService.apply` composes `create` + `set_project` + `set_cycle` + label attach in separate transactions.
- No due-date reminder is scheduled; the schedule worker discovers due issues by polling (§10.7).

10. Repository SQL (`app/repositories/issues.py :: IssueRepository.create`).

```
INSERT INTO issues (workspace_id, team_id, number, workflow_state_id, title,
                    description, priority, assignee_id, creator_id, estimate, due_date)
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11)
RETURNING {ISSUE_COLUMNS}
```

`workspace_id` comes from the scope. Team, assignee and state are not pre-checked: composite FKs `issues_team_fk (workspace_id, team_id)`, `issues_assignee_fk (workspace_id, assignee_id) → workspace_members`, `issues_workflow_state_fk` reject cross-tenant values inside the same statement (method docstring). `completed_at` and `cycle_id` are not written. `ISSUE_COLUMNS` includes `team_key` via a correlated subquery on `teams`.

11. Mapping. `IssueRepository._to_entity(row: asyncpg.Record)` builds the frozen, slotted `IssueEntity` (`app/domain/issues.py`); `identifier` is the computed property `f"{team_key}-{number}"`. The resolver returns `IssueCreatePayload(issue=IssueType.from_entity(entity, scope), errors=[])` (`app/graphql/types/issue.py :: IssueType.from_entity`). Nested fields requested by `IssueDetailFields` resolve lazily, e.g. `labels` through the per-request `issue_labels` DataLoader and `cycle` through `cycle_loader`.

12. Errors and response.

- Expected input failures: `ValidationError` → `IssueCreatePayload(issue=None, errors=_errors(exc))`, each `ValidationErrorType{field, code, message}`, inside `data` with HTTP 200 (`app/graphql/types/errors.py :: ValidationErrorType.from_domain`).
- Auth/tenancy failures: top-level GraphQL errors with `extensions.code` `UNAUTHENTICATED` or `NOT_FOUND`.
- Anything else (e.g. an unmapped FK violation): replaced with "Internal server error" by `MaskErrors` and again by `_MaskedSchema.execute → _mask_result`; only `BAD_USER_INPUT`, `UNAUTHENTICATED`, `NOT_FOUND` and errors without an `original_error` pass through (`app/graphql/schema.py :: PUBLIC_ERROR_CODES, is_public_error`).
- On the way out the security-headers middleware adds `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` and the other headers (`app/http_headers.py`).

13. Client cache. The `update` callback calls `prependCreatedIssue(cache, created, workspaceSlug)` to insert the row into cached issue lists (`frontend/src/features/issues/api/cache.ts`); `interpretPayload` turns payload `errors` into per-field messages and `failureOutcome` turns transport/top-level errors into a banner (`frontend/src/features/issues/api/outcome.ts`).

> **Gaps / to verify:**
>
> - Middleware order is inferred from Starlette's `add_middleware` (`insert(0, …)`) semantics; FastAPI may add its own internal middleware (e.g. an exit-stack layer) that is not shown.
> - `IssueService.create` docstring says `creator_id` "is None for an unauthenticated caller", but the resolver always requires a viewer via `authorized_scope`; the docstring is stale.
> - `IssueComposer` docstring says `IssueCreateInput` is `{ title, description, priority }`; the backend input also accepts `assigneeId`, `estimate`, `dueDate` (`app/graphql/inputs/issue.py`). The UI does not expose them and always files into the workspace's first team.
> - Render edge/proxy behaviour (TLS termination, header rewriting) is outside the repo; only `render.yaml` settings were read.

6. Data model

6.1 Overview

The schema is defined only by `migrations/001_issues.sql` … `migrations/032_auth_hardening.sql`, applied in numeric order by `scripts/apply_migration.py`. After all 32 migrations it contains **55 tables**, plus the runner-owned ledger `schema_migrations` (created by `scripts/apply_migration.py` `CREATE_LEDGER_SQL`, not by a migration).

Fact	Value	Source
Tables created by migrations	55 (`CREATE TABLE` count across `migrations/*.sql`)	`migrations/*.sql`
Foreign keys	102; 90 are `ON DELETE RESTRICT ON UPDATE RESTRICT`	`migrations/*.sql`
Explicit indexes	91 (89 `CREATE INDEX` + 3 `CREATE UNIQUE INDEX`, minus `issues_created_at_id_idx` dropped in 002)	`migrations/*.sql`, `migrations/002_tenancy.sql`
Triggers / PL/pgSQL functions / `CREATE TYPE`	none	`migrations/*.sql`
Target server	PostgreSQL 18 (`uuidv7()` is built in); tests use the `pgvector/pgvector:pg18` image	`migrations/001_issues.sql`, `migrations/025_semantic_search.sql` (comment)

Extensions

Extension	Created in	Used for
`btree_gist`	`migrations/008_cycles.sql`	`cycles_no_overlap` EXCLUDE constraint mixing `=` on UUIDs with `&&` on a `tstzrange`
`btree_gin`	`migrations/011_search.sql`	GIN indexes keyed on `(workspace_id, search_vector)`
`vector` (pgvector)	`migrations/025_semantic_search.sql`	`vector(384)` column and `<=>` cosine distance

`pg_trgm` is discussed and declined (`migrations/011_search.sql` comment); `citext` is declined in favour of a lowercase `CHECK` (`migrations/003_auth.sql` comment on `users.email`). `IF NOT EXISTS` appears only on `CREATE EXTENSION`.

Conventions

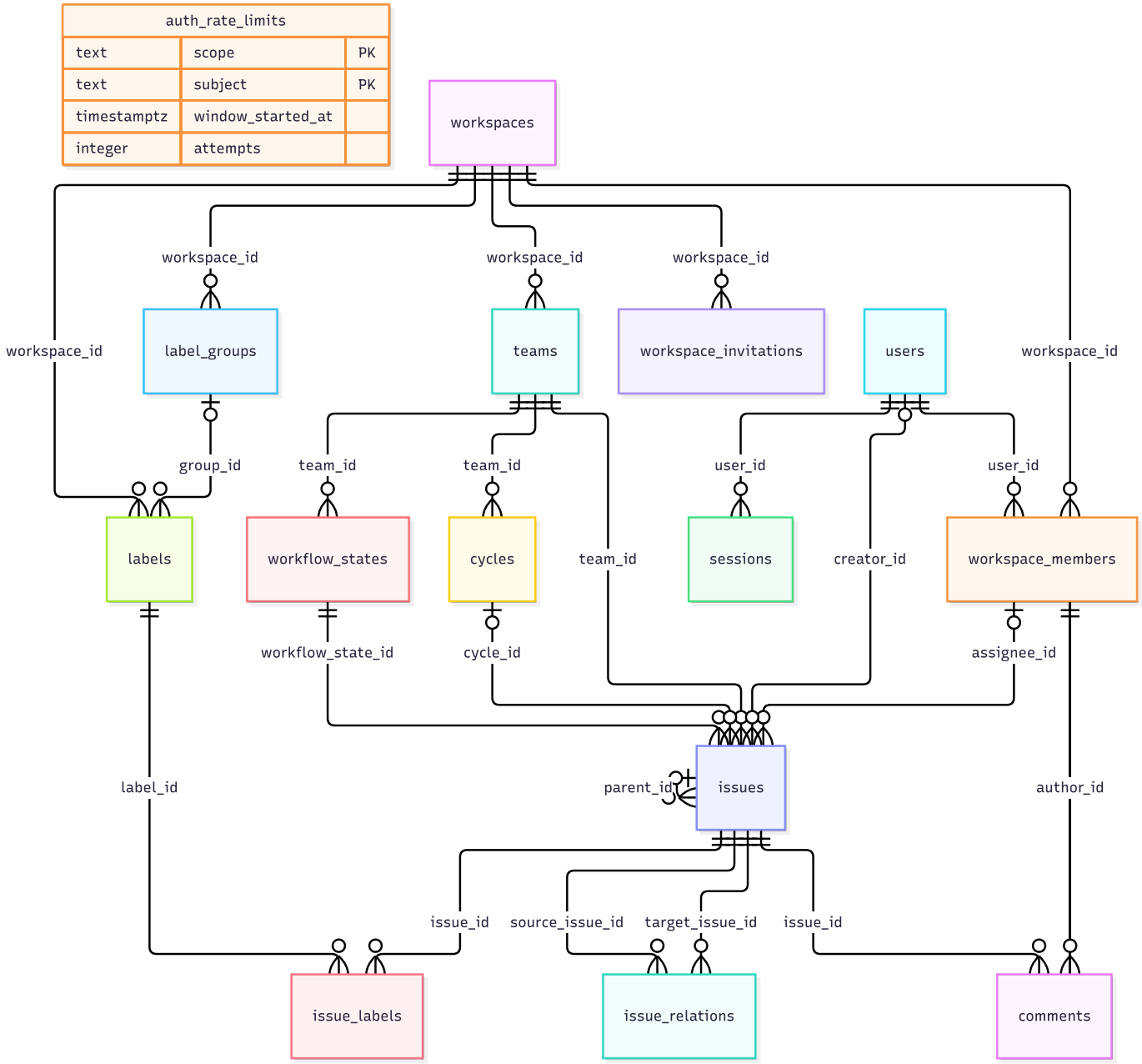
Convention	Detail	Source
Surrogate keys	27 tables have `id UUID … DEFAULT uuidv7()`. 25 use it as a single-column `PRIMARY KEY`; `environments` and `releases` use `PRIMARY KEY (workspace_id, id)`. The other 28 tables use natural/composite keys (join tables, per-workspace singletons, ledgers).	`migrations/*.sql`; `migrations/024_releases.sql`
Tenancy	Every tenant-owned table has `workspace_id UUID NOT NULL`. Exceptions with no tenant column: `workspaces`, `users`, `sessions`, `auth_rate_limits`, `github_deliveries`, `slack_event_deliveries`, `schema_migrations`.	`migrations/*.sql`
Composite tenant FKs	Parents expose `UNIQUE (workspace_id, id)` (e.g. `teams_workspace_id_key`, `issues_workspace_id_key`, `projects_workspace_id_key`) and children reference `(workspace_id, <fk>)`, so a cross-workspace reference cannot be stored. Team-scoped parents expose 3-column keys (`workflow_states_team_id_key`, `cycles_workspace_team_id_key`) so an issue's state and cycle must belong to the issue's team.	`migrations/002_tenancy.sql`, `005_team_workflows.sql`, `007_labels_comments.sql`, `008_cycles.sql`, `009_projects.sql`

Convention	Detail	Source
People references	17 FKs target `workspace_members (workspace_id, user_id)` (an assignee/author must be a member). Three reference `users(id)` directly with `ON DELETE SET NULL`: `issues_creator_fk`, `issue_activity_actor_fk`, `notifications_actor_fk`.	`migrations/006_issue_fields.sql`, `012_activity_notifications.sql`
`issues.workspace_id`	No direct FK to `workspaces`; guaranteed transitively through `issues_team_fk`. Composite FKs use default `MATCH SIMPLE`, which is why `issues.workspace_id/team_id` must stay `NOT NULL`.	`migrations/002_tenancy.sql` (comment)
Timestamps	`TIMESTAMPTZ`; `created_at` (and `updated_at` where present) `NOT NULL DEFAULT now()`. `updated_at` is maintained by application SQL, not triggers. Several tables have no `updated_at` (see 6.3).	`migrations/*.sql`
Enumerations	`TEXT` + `CHECK (col IN (…))`; no PostgreSQL enum types. Domain enums mirror the CHECK lists (e.g. `ActivityKind`, `NotificationKind`, `DomainEventKind`).	`migrations/*.sql`; `app/domain/activity.py`, `notifications.py`, `events.py`
Archival / soft delete	`issues.archived_at` (most issue-list indexes are partial on `archived_at IS NULL`); `workspace_members.removed_at` tombstone (rejoin revives the row via `ON CONFLICT ON CONSTRAINT workspace_members_pkey DO UPDATE`). No `deleted_at` columns; projects, initiatives and documents have no archive column.	`migrations/006_issue_fields.sql`, `026_member_departure.sql`; `app/repositories/memberships.py`
Delete policy	RESTRICT by default. `ON DELETE CASCADE`: `sessions_user_fk`, `issue_labels_issue_fk`, `issue_labels_label_fk`, `issue_labels_exclusivity_fk`, `comments_issue_fk`, `embedding_jobs_issue_fk`. `ON DELETE SET NULL`: `issues_assignee_fk` (`SET NULL (assignee_id)`), `issues_cycle_fk` (`SET NULL (cycle_id)`), `issues_creator_fk`, `issue_activity_actor_fk`, `notifications_actor_fk`. `ON UPDATE CASCADE`: `labels_group_fk`, `issue_labels_exclusivity_fk`.	`migrations/003`, `006`, `007`, `008`, `012`, `021`, `028`
Pagination	Keyset indexes of the form `(workspace_id, <sort> DESC, id DESC)`; `id` is the tiebreaker.	`migrations/002_tenancy.sql`, `015_issue_list_indexes.sql`
Derived columns	`GENERATED ALWAYS … STORED`: `issues.search_vector`, `projects.search_vector`, `labels.exclusivity_key`, `issues.embedding_source_digest`.	`migrations/011_search.sql`, `021_triage_label_groups.sql`, `025_semantic_search.sql`

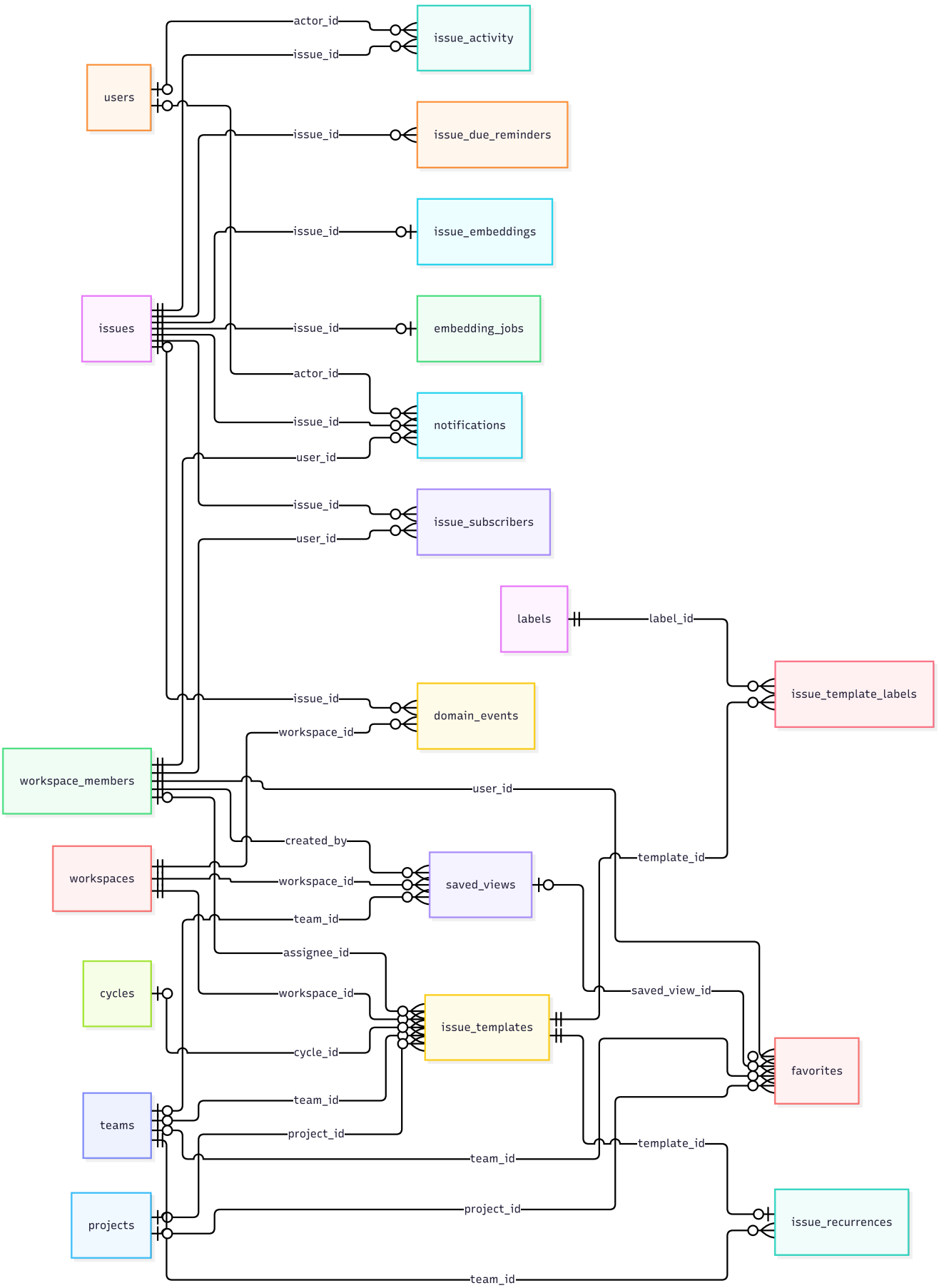
6.2 ER diagrams

Four diagrams by domain area; every table appears in at least one. Relationship labels are the FK column on the child side. Tables shared between diagrams (e.g. `issues`, `workspace_members`) are repeated as anchors. Columns are listed in 6.3.

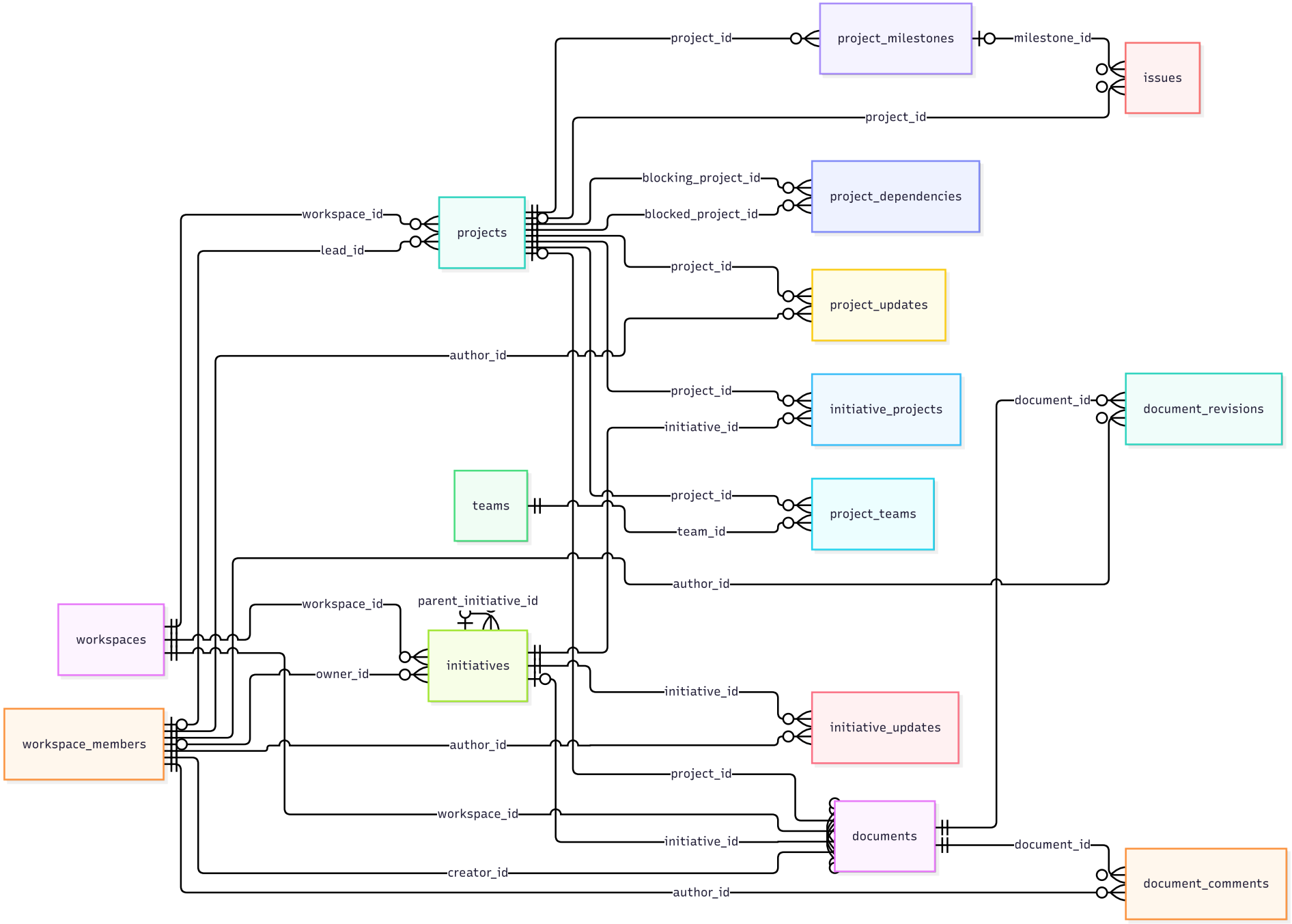
6.2.1 Tenancy, identity, issue core



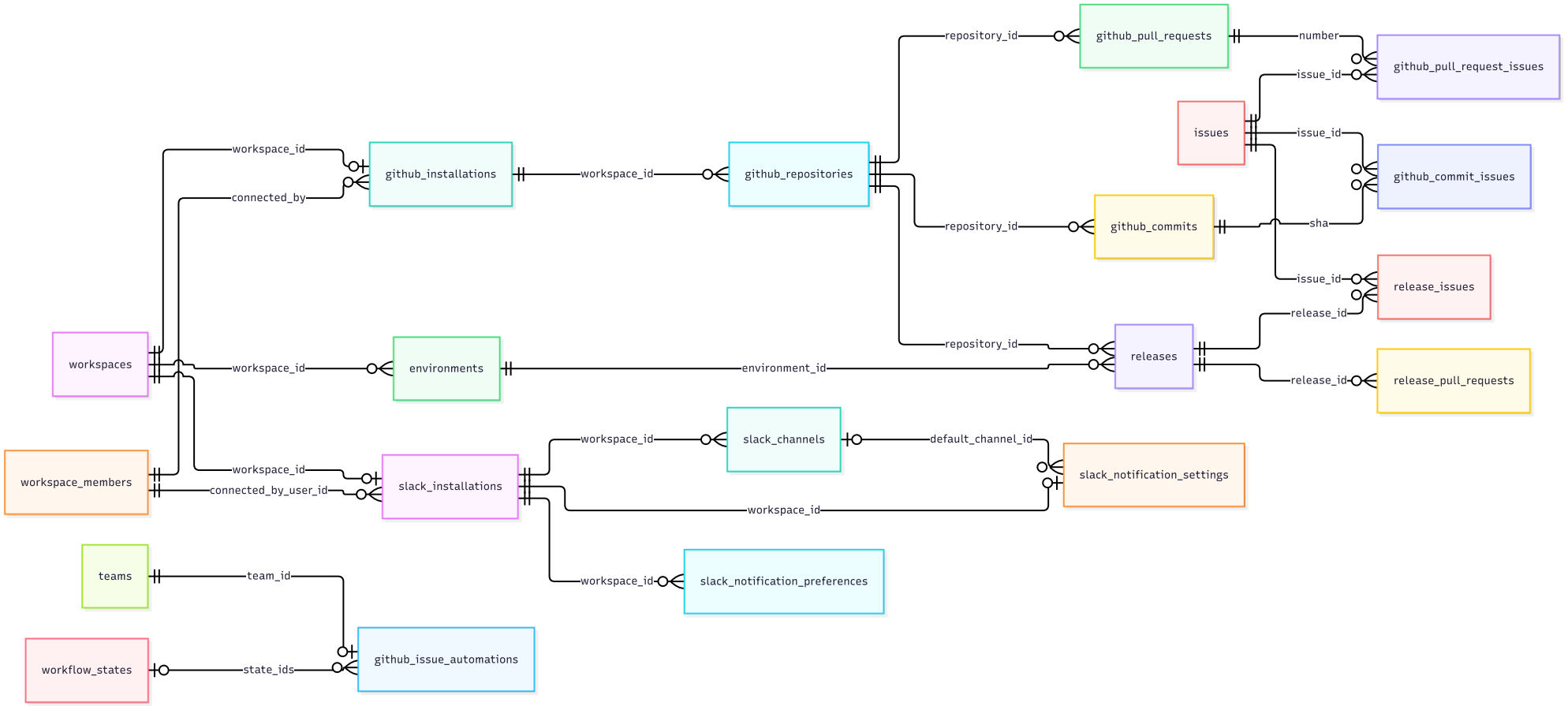
6.2.2 Issue satellites: activity, notifications, templates, embeddings, views, events



6.2.3 Planning: projects, initiatives, documents


project_milestones
milestone_id
issues
project_id
project_id
blocking_project_id
blocked_project_id
project_dependencies
workspace_id
lead_id
projects
project_id
project_updates
author_id
project_id
initiative_id
initiative_projects
document_id
document_revisions
teams
project_id
team_id
project_teams
author_id
workspaces
workspace_id
parent_initiative_id
owner_id
initiatives
initiative_id
initiative_updates
workspace_members
author_id
project_id
workspace_id
initiative_id
documents
document_id
document_comments
creator_id
author_id

6.2.4 Integrations, releases, ops



github_deliveries		
text	delivery_id	PK
text	event	
timestamptz	received_at	

slack_event_deliveries		
text	event_id	PK
timestamptz	received_at	

schema_migrations		
text	version	PK
timestamptz	applied_at	
text	checksum	

`github_issue_automations` has two FKs into `workflow_states` (`started_state_id`, `completed_state_id`), drawn as one edge. `release_pull_requests` deliberately has no FK to `github_pull_requests` (it stores a title/url snapshot) (`migrations/024_releases.sql`).

6.3 Entity reference

Notation: NN = `NOT NULL`; "—" = no default. Unless stated, every FK is `ON DELETE RESTRICT ON UPDATE RESTRICT`. "(0xx)" after a column = migration that added it. `uuid` columns named `id` default to `uuidv7()`.

6.3.1 Tenancy and identity

`workspaces` (`migrations/002_tenancy.sql`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
slug	text	NN	—	`workspaces_slug_key` UNIQUE; `workspaces_slug_format` CHECK `^[a-z0-9]([a-z0-9-]*[a-z0-9])?$`
name	text	NN	—	
created_at	timestamptz	NN	now()	

- FKs: none. No `updated_at`. Seed row: id `00000000-0000-7000-8000-000000000001`, slug `vector`, name `Vector`.

`teams` (`002`; altered `005`, `029`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK
name	text	NN	—	
created_at	timestamptz	NN	now()	
key (005)	text	NN	— (backfilled `CORE` / `TEAM<n>`)	`teams_key_format` CHECK `^[A-Z][A-Z0-9]{0,9}$`
issue_counter (005)	bigint	NN	0	`teams_issue_counter_non_negative` (>= 0)
estimate_scale (029)	text	NN	'none'	`teams_estimate_scale_known` IN ('none','points','hours','tshirt')

- Unique: `teams_workspace_id_key (workspace_id, id)` (FK target); `teams_workspace_key_unique (workspace_id, key)`.
- FKs: `teams_workspace_fk (workspace_id) → workspaces(id)`.
- Seed row (002): id `00000000-0000-7000-8000-000000000002`, name `Core`. No `updated_at`.

`users` (`migrations/003_auth.sql`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
email	text	NN	—	`users_email_key` UNIQUE; `users_email_lowercase` (`email = lower(email)`); `users_email_shape` `^[^@[:space:]]+@[^@[:space:]]+$`; `users_email_length` 3..320
password_hash	text	NN	—	`users_password_hash_argon2id` (`LIKE '$argon2id$%'`)

Column	Type	Null	Default	Constraints
name	text	null	—	
created_at	timestamptz	NN	now()	
updated_at	timestamptz	NN	now()	

- FKs: none. The unapplied draft 033 would change this table (6.5).

`workspace_members` (`migrations/004_membership.sql`; altered `026`)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part; FK
user_id	uuid	NN	—	PK part; FK
role	text	NN	—	`workspace_members_role_check` IN ('member','admin','owner')
created_at	timestamptz	NN	now()	
removed_at (026)	timestamptz	null	—	`workspace_members_removed_after_created` (NULL or >= created_at)

- PK `workspace_members_pkey (workspace_id, user_id)`; target of 17 composite FKs.
- FKs: `workspace_members_workspace_fk → workspaces(id)`; `workspace_members_user_fk → users(id)`.
- Indexes: `workspace_members_user_idx (user_id)`.

`workspace_invitations` (`migrations/004_membership.sql`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK
email	text	NN	—	(no shape/lowercase CHECK)
role	text	NN	—	IN ('member','admin','owner')
token_hash	text	NN	—	UNIQUE; `…_token_hash_format` `^[a-f0-9]{64}$` (hex SHA-256)
expires_at	timestamptz	NN	—	`…_expiry_after_creation` (expires_at > created_at)
accepted_at	timestamptz	null	—	
created_at	timestamptz	NN	now()	

- FKs: `workspace_invitations_workspace_fk → workspaces(id)`. Indexes: `workspace_invitations_workspace_idx (workspace_id)`.

6.3.2 Auth

`sessions` (`migrations/003_auth.sql`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
user_id	uuid	NN	—	FK
token_hash	bytea	NN	—	`sessions_token_hash_key` UNIQUE; `sessions_token_hash_length` (`octet_length = 32`, SHA-256)
created_at	timestamptz	NN	now()	

Column	Type	Null	Default	Constraints
last_used_at	timestamptz	null	—	
expires_at	timestamptz	NN	—	

- FKs: `sessions_user_fk (user_id) → users(id)` **ON DELETE CASCADE**.
- Indexes: `sessions_user_id_idx (user_id)`, `sessions_expires_at_idx (expires_at)`. No `revoked_at`; logout deletes the row (`003` comment).

`auth_rate_limits` (migrations/032_auth_hardening.sql)

Column	Type	Null	Default	Constraints
scope	text	NN	—	PK part; length 1..64
subject	text	NN	—	PK part; length 1..128
window_started_at	timestamptz	NN	—	
attempts	integer	NN	—	`auth_rate_limits_attempts_positive` (> 0)

- PK `(scope, subject)` (unnamed → `auth_rate_limits_pkey`). No FKs, no tenant column. Indexes: `auth_rate_limits_window_idx (window_started_at)`.
- Fixed-window counter; scopes named in the file comment: `login:ip`, `login:email`, `register:ip`. Accessed only by `app/repositories/rate_limits.py`.

6.3.3 Issues and workflow

`workflow_states` (migrations/005_team_workflows.sql)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK part
team_id	uuid	NN	—	FK part
name	text	NN	—	`workflow_states_name_present` (`btrim(name) <> ''`)
type	text	NN	—	`workflow_states_type_check` IN ('backlog','unstarted','started','completed','canceled')
position	integer	NN	—	>= 0
color	text	null	—	`workflow_states_color_format` `^#[0-9a-f]{6}$`
created_at	timestamptz	NN	now()	

- Unique: `workflow_states_team_name_key (team_id, name)`; `workflow_states_team_id_key (workspace_id, team_id, id)` (FK target).
- FKs: `workflow_states_team_fk (workspace_id, team_id) → teams(workspace_id, id)`.
- Seed (005): every existing team gets Backlog / Todo / In Progress / Done / Canceled at positions 0..4. No `updated_at`.

`issues` (001; altered `002`, `005`, `006`, `007`, `008`, `009`, `010`, `011`, `021`, `025`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
title	text	NN	—	
description	text	null	—	

Column	Type	Null	Default	Constraints
priority	smallint	NN	0	`issues_priority_range` 0..4
completed_at	timestamptz	null	—	
created_at	timestamptz	NN	now()	
updated_at	timestamptz	NN	now()	
workspace_id (002)	uuid	NN	—	FK part
team_id (002)	uuid	NN	—	FK part
number (005)	bigint	NN	—	`issues_number_positive` (>= 1); UNIQUE (team_id, number)
workflow_state_id (005)	uuid	NN	—	FK
assignee_id (006)	uuid	null	—	FK
creator_id (006)	uuid	null	—	FK
estimate (006)	integer	null	—	`issues_estimate_non_negative`
due_date (006)	date	null	—	
archived_at (006)	timestamptz	null	—	
cycle_id (008)	uuid	null	—	FK
project_id (009)	uuid	null	—	FK
milestone_id (009)	uuid	null	—	FK; `issues_milestone_requires_project`
parent_id (010)	uuid	null	—	FK; `issues_parent_not_self` (`IS DISTINCT FROM id`)
search_vector (011)	tsvector	generated	`setweight(to_tsvector('english', coalesce(title,'')),'A') || setweight(to_tsvector('english', coalesce(description,'')),'B')` STORED	
triage_entered_at (021)	timestamptz	null	—	`issues_triage_is_not_archived` (archived_at IS NULL OR triage_entered_at IS NULL)
embedding_source_digest (025)	text	generated	`md5(coalesce(title,'')) || md5(coalesce(description,''))` STORED	

- Unique: `issues_team_number_key (team_id, number)` (005); `issues_workspace_id_key (workspace_id, id)` (007, FK target for all issue children).
- FKs:

FK	Columns → target	ON DELETE
`issues_team_fk`	(workspace_id, team_id) → teams(workspace_id, id)	RESTRICT
`issues_workflow_state_fk`	(workspace_id, team_id, workflow_state_id) → workflow_states(workspace_id, team_id, id)	RESTRICT
`issues_assignee_fk`	(workspace_id, assignee_id) → workspace_members(workspace_id, user_id)	SET NULL (assignee_id)

FK	Columns → target	ON DELETE
`issues_creator_fk`	(creator_id) → users(id)	SET NULL
`issues_cycle_fk`	(workspace_id, team_id, cycle_id) → cycles(workspace_id, team_id, id)	SET NULL (cycle_id)
`issues_project_fk`	(workspace_id, project_id) → projects(workspace_id, id)	RESTRICT
`issues_milestone_fk`	(workspace_id, project_id, milestone_id) → project_milestones(workspace_id, project_id, id)	RESTRICT
`issues_parent_fk`	(workspace_id, parent_id) → issues(workspace_id, id)	RESTRICT

- Indexes (001's `issues_created_at_id_idx` was dropped in 002):

Index	Definition	Mig
`issues_workspace_created_at_id_idx`	(workspace_id, created_at DESC, id DESC)	002
`issues_workspace_team_idx`	(workspace_id, team_id)	002
`issues_workflow_state_idx`	(workspace_id, team_id, workflow_state_id)	005
`issues_workspace_assignee_idx`	(workspace_id, assignee_id)	006
`issues_creator_idx`	(creator_id)	006
`issues_workspace_live_created_at_id_idx`	(workspace_id, created_at DESC, id DESC) WHERE archived_at IS NULL	006
`issues_workspace_team_cycle_idx`	(workspace_id, team_id, cycle_id) WHERE cycle_id IS NOT NULL	008
`issues_workspace_project_milestone_idx`	(workspace_id, project_id, milestone_id)	009
`issues_workspace_parent_created_at_id_idx`	(workspace_id, parent_id, created_at DESC, id DESC) WHERE parent_id IS NOT NULL	010
`issues_workspace_live_search_idx`	GIN (workspace_id, search_vector) WHERE archived_at IS NULL	011
`issues_workspace_live_assignee_created_at_id_idx`	(workspace_id, assignee_id, created_at DESC, id DESC) WHERE archived_at IS NULL	015
`issues_workspace_live_updated_at_id_idx`	(workspace_id, updated_at DESC, id DESC) WHERE archived_at IS NULL	015
`issues_workspace_live_priority_id_idx`	(workspace_id, NULLIF(priority, 0) DESC, id DESC) WHERE archived_at IS NULL	015
`issues_workspace_live_due_date_id_idx`	(workspace_id, due_date, id) WHERE archived_at IS NULL	015
`issues_workspace_team_triage_idx`	(workspace_id, team_id, triage_entered_at, id) WHERE triage_entered_at IS NOT NULL	021
`issues_workspace_completed_at_idx`	(workspace_id, completed_at) WHERE completed_at IS NOT NULL	031

- Issue numbering: `teams.issue_counter` is incremented atomically (`UPDATE … SET issue_counter = issue_counter + 1 … RETURNING`, `app/repositories/teams.py`); the `KEY-N` identifier is computed (`app/domain/issues.py` `IssueEntity.identifier`), not stored. Triage has no table of its own; it is `issues.triage_entered_at` (021).

6.3.4 Planning: cycles, projects, initiatives, documents, releases

`cycles (migrations/008_cycles.sql)`

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK part
team_id	uuid	NN	—	FK part
number	integer	NN	—	`cycles_number_positive` (> 0)
name	text	null	—	
starts_at	timestamptz	NN	—	

Column	Type	Null	Default	Constraints
ends_at	timestamptz	NN	—	`cycles_dates_ordered` (ends_at > starts_at)
created_at, updated_at	timestamptz	NN	now()	

- Unique: `cycles_workspace_team_id_key (workspace_id, team_id, id)` (FK target); `cycles_team_number_key (workspace_id, team_id, number)`.
- Exclusion: `cycles_no_overlap EXCLUDE USING gist (workspace_id WITH =, team_id WITH =, tstzrange(starts_at, ends_at, '[)') WITH &&)`.
- FKs: `cycles_team_fk (workspace_id, team_id) → teams`. No extra indexes.

`projects` (migrations/009_projects.sql; altered `011`, `022`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK
name	text	NN	—	
description	text	null	—	
state	text	NN	—	`projects_state_check` IN ('planned','started','paused','completed','canceled')
target_date	date	null	—	
lead_id	uuid	null	—	FK
created_at, updated_at	timestamptz	NN	now()	
search_vector (011)	tsvector	generated	name weight A || description weight B, `'english'`, STORED	
health (022)	text	null	—	`projects_health_check` NULL or IN ('on_track','at_risk','off_track')

- Unique: `projects_workspace_id_key (workspace_id, id)`.
- FKs: `projects_workspace_fk → workspaces`; `projects_lead_fk (workspace_id, lead_id) → workspace_members`.
- Indexes: `projects_workspace_created_at_id_idx (workspace_id, created_at DESC, id DESC)`; `projects_workspace_lead_idx (workspace_id, lead_id)`; `projects_workspace_search_idx` GIN (workspace_id, search_vector) (no archive predicate).

`project_teams` (009)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
project_id	uuid	NN	—	PK part; FK
team_id	uuid	NN	—	PK part; FK
created_at	timestamptz	NN	now()	

- PK `project_teams_pkey (workspace_id, project_id, team_id)`. FKs: `project_teams_project_fk → projects(workspace_id, id)`; `project_teams_team_fk → teams(workspace_id, id)`. Indexes: `project_teams_workspace_team_idx (workspace_id, team_id)`.

`project_milestones` (009)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	FK part
project_id	uuid	NN	—	FK part
name	text	NN	—	
target_date	date	null	—	
position	integer	NN	—	>= 0
created_at, updated_at	timestamptz	NN	now()	

- Unique: `project_milestones_project_id_key (workspace_id, project_id, id)` (FK target). FKs: `project_milestones_project_fk → projects`. Indexes: `project_milestones_workspace_project_position_idx (workspace_id, project_id, position, id)`.

`project_updates` (migrations/022_initiatives.sql)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
project_id	uuid	NN	—	FK
health	text	NN	—	IN ('on_track','at_risk','off_track')
body	text	NN	—	length 1..10000
author_id	uuid	NN	—	FK
created_at	timestamptz	NN	now()	

- FKs: `project_updates_project_fk → projects`; `project_updates_author_fk → workspace_members`. Indexes: `project_updates_workspace_project_created_at_id_idx (workspace_id, project_id, created_at DESC, id DESC)`; `project_updates_workspace_author_idx (workspace_id, author_id)`.

`project_dependencies` (022)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
blocking_project_id	uuid	NN	—	PK part; FK
blocked_project_id	uuid	NN	—	PK part; FK; `project_dependencies_not_self`
created_at	timestamptz	NN	now()	

- FKs: `project_dependencies_blocking_fk`, `project_dependencies_blocked_fk → projects(workspace_id, id)`. Indexes: `project_dependencies_workspace_blocked_idx (workspace_id, blocked_project_id)`.

`initiatives` (022)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK; `initiatives_parent_not_self`
workspace_id	uuid	NN	—	FK
name	text	NN	—	length 1..200
description	text	null	—	
status	text	NN	—	IN ('planned','active','completed','canceled')
health	text	null	—	NULL or IN ('on_track','at_risk','off_track')
target_date	date	null	—	

Column	Type	Null	Default	Constraints
owner_id	uuid	null	—	FK
parent_initiative_id	uuid	null	—	FK (self)
created_at, updated_at	timestamptz	NN	now()	

- Unique: `initiatives_workspace_id_key (workspace_id, id)`. FKs: `initiatives_workspace_fk → workspaces`; `initiatives_owner_fk → workspace_members`; `initiatives_parent_fk (workspace_id, parent_initiative_id) → initiatives(workspace_id, id)`.
- Indexes: `initiatives_workspace_created_at_id_idx`; `initiatives_workspace_owner_idx`; `initiatives_workspace_parent_created_at_id_idx (workspace_id, parent_initiative_id, created_at DESC, id DESC)`.

`initiative_projects` (022)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
initiative_id	uuid	NN	—	PK part; FK
project_id	uuid	NN	—	PK part; FK
created_at	timestamptz	NN	now()	

- FKs: `initiative_projects_initiative_fk → initiatives`; `initiative_projects_project_fk → projects`. Indexes: `initiative_projects_workspace_project_idx (workspace_id, project_id)`.

`initiative_updates` (022)

Same shape and CHECKs as `project_updates`, with `initiative_id uuid NN → initiatives(workspace_id, id)` (`initiative_updates_initiative_fk`) and `author_id → workspace_members` (`initiative_updates_author_fk`). Indexes: `initiative_updates_workspace_initiative_created_at_id_idx (workspace_id, initiative_id, created_at DESC, id DESC)`; `initiative_updates_workspace_author_idx`.

`documents` (migrations/023_documents.sql)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK
project_id	uuid	null	—	FK
initiative_id	uuid	null	—	FK
title	text	NN	—	`char_length` 1..200
content	jsonb	NN	—	`documents_content_is_object (jsonb_typeof = 'object')`; `documents_content_length (length(content::text) <= 524288)`
creator_id	uuid	NN	—	FK
last_edited_by	uuid	NN	—	FK
created_at, updated_at	timestamptz	NN	now()	

- Checks: `documents_one_parent (num_nonnulls(project_id, initiative_id) <= 1)`. Unique: `documents_workspace_id_key (workspace_id, id)`.
- FKs: `documents_workspace_fk`; `documents_project_fk`; `documents_initiative_fk`; `documents_creator_fk`, `documents_last_editor_fk → workspace_members`.

- Indexes: `documents_workspace_created_at_id_idx`; `documents_workspace_project_created_at_id_idx`; `documents_workspace_initiative_created_at_id_idx` (all `…, created_at DESC, id DESC`); `documents_workspace_creator_idx`; `documents_workspace_last_editor_idx`.

`document_revisions` (023)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
document_id	uuid	NN	—	FK
title	text	NN	—	`char_length` 1..200
content	jsonb	NN	—	object; <= 524288 chars
author_id	uuid	NN	—	FK
created_at	timestamptz	NN	now()	

- FKs: `document_revisions_document_fk → documents(workspace_id, id)`; `document_revisions_author_fk → workspace_members`. Indexes: `document_revisions_workspace_document_created_at_id_idx`; `document_revisions_workspace_author_idx`.

`document_comments` (023)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
document_id	uuid	NN	—	FK
author_id	uuid	NN	—	FK
body	text	NN	—	length 1..16384
edited_at	timestamptz	null	—	
created_at, updated_at	timestamptz	NN	now()	

- FKs: `document_comments_document_fk → documents` (RESTRICT, unlike `comments_issue_fk` which cascades); `document_comments_author_fk → workspace_members`. Indexes: `document_comments_workspace_document_created_idx (workspace_id, document_id, created_at, id)`; `document_comments_workspace_author_idx`.

`environments` (migrations/024_releases.sql)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part; FK
id	uuid	NN	uuidv7()	PK part
name	text	NN	—	length 1..100; UNIQUE (workspace_id, name)
kind	text	NN	—	IN ('development','staging','production','custom')
created_at	timestamptz	NN	now()	

- PK `environments_pkey (workspace_id, id)`. FKs: `environments_workspace_fk → workspaces`.

`releases` (024)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK part
name	text	NN	—	length 1..200
environment_id	uuid	NN	—	FK
repository_id	bigint	NN	—	FK
commit_sha	text	NN	—	`^[0-9a-f]{40}$`
previous_commit_sha	text	null	—	40-hex; `releases_previous_commit_differs`
status	text	NN	—	IN ('pending','deployed','failed','rolled_back')
notes	text	NN	—	length 1..100000
deployed_at	timestamptz	null	—	`releases_deployed_at_matches_status` (status IN ('deployed','rolled_back')) = (deployed_at IS NOT NULL)
created_at, updated_at	timestamptz	NN	now()	

- FKs: `releases_environment_fk → environments(workspace_id, id)`; `releases_repository_fk → github_repositories(workspace_id, repository_id)`.
- Indexes: `releases_workspace_created_at_id_idx`; `releases_workspace_environment_idx`; `releases_workspace_repository_environment_deployed_idx (workspace_id, repository_id, environment_id, deployed_at DESC, id DESC)`.

`release_issues` (024)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
release_id	uuid	NN	—	PK part; FK
issue_id	uuid	NN	—	PK part; FK

- FKs: `release_issues_release_fk → releases`; `release_issues_issue_fk → issues`. Indexes: `release_issues_workspace_issue_idx (workspace_id, issue_id)`. No timestamps.

`release_pull_requests` (024)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
release_id	uuid	NN	—	PK part; FK
repository_id	bigint	NN	—	PK part
number	integer	NN	—	PK part; > 0
title	text	NN	—	length 1..1024
url	text	null	—	NULL or length 1..2048

- FKs: `release_pull_requests_release_fk → releases` only (no FK to `github_pull_requests`). No timestamps.

6.3.5 Collaboration: labels, comments, relations, subscribers, templates, saved views

`labels` (`migrations/007_labels_comments.sql`; altered `021`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK
name	text	NN	—	length 1..50
color	text	NN	—	`^#[0-9a-f]{6}$`
created_at, updated_at	timestamptz	NN	now()	
group_id (021)	uuid	null	—	FK part
group_exclusive (021)	boolean	null	—	`labels_group_exclusive_paired` ((group_id IS NULL) = (group_exclusive IS NULL))
exclusivity_key (021)	uuid	generated	`CASE WHEN group_exclusive THEN group_id ELSE id END STORED`	

- Unique: `labels_workspace_id_key (workspace_id, id)`; `labels_workspace_id_exclusivity_key (workspace_id, id, exclusivity_key)` (FK target).
- FKs: `labels_workspace_fk → workspaces`; `labels_group_fk (workspace_id, group_id, group_exclusive) → label_groups(workspace_id, id, exclusive)` ON DELETE RESTRICT **ON UPDATE CASCADE**.
- Indexes: `labels_workspace_name_key` UNIQUE (workspace_id, lower(name)); `labels_workspace_name_idx (workspace_id, name, id)`; `labels_workspace_group_idx (workspace_id, group_id, group_exclusive)`.

`label_groups` (`migrations/021_triage_label_groups.sql`)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK
name	text	NN	—	length 1..50
exclusive	boolean	NN	—	
created_at, updated_at	timestamptz	NN	now()	

- Unique: `label_groups_workspace_id_exclusive_key (workspace_id, id, exclusive)` (FK target). FKs: `label_groups_workspace_fk → workspaces`.
- Indexes: `label_groups_workspace_name_key` UNIQUE (workspace_id, lower(name)); `label_groups_workspace_name_idx (workspace_id, name, id)`.

`issue_labels` (`007`; altered `021`)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
issue_id	uuid	NN	—	PK part; FK
label_id	uuid	NN	—	PK part; FK
created_at	timestamptz	NN	now()	
exclusivity_key (021)	uuid	NN (backfilled)	—	FK part

- PK `issue_labels_pkey (workspace_id, issue_id, label_id)`.
- FKs: `issue_labels_issue_fk → issues(workspace_id, id)` **ON DELETE CASCADE**; `issue_labels_label_fk → labels(workspace_id, id)` **ON DELETE CASCADE**; `issue_labels_exclusivity_fk (workspace_id, label_id, exclusivity_key) → labels(workspace_id, id, exclusivity_key)` **ON DELETE CASCADE ON UPDATE CASCADE**.
- Indexes: `issue_labels_workspace_label_idx (workspace_id, label_id)`; `issue_labels_exclusive_group_key` UNIQUE (workspace_id, issue_id, exclusivity_key) WHERE exclusivity_key <> label_id — at most one label per exclusive group per issue, enforced in the database.

`comments (007)`

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
issue_id	uuid	NN	—	FK
author_id	uuid	NN	—	FK
body	text	NN	—	length 1..16384
edited_at	timestamptz	null	—	
created_at, updated_at	timestamptz	NN	now()	

- FKs: `comments_issue_fk → issues(workspace_id, id)` **ON DELETE CASCADE**; `comments_author_fk → workspace_members`. Indexes: `comments_workspace_issue_created_idx (workspace_id, issue_id, created_at, id)`; `comments_workspace_author_idx (workspace_id, author_id)`.

`issue_relations (migrations/010_issue_relations.sql)`

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
source_issue_id	uuid	NN	—	FK; `issue_relations_not_self`
target_issue_id	uuid	NN	—	FK
type	text	NN	—	`issue_relations_type_known` IN ('blocks','related','duplicate')
created_at	timestamptz	NN	now()	

- Checks: `issue_relations_symmetric_ordered (type = 'blocks' OR source_issue_id < target_issue_id)`. Unique: `issue_relations_unique (workspace_id, source_issue_id, target_issue_id, type)`.
- FKs: `issue_relations_source_fk`, `issue_relations_target_fk → issues(workspace_id, id)`. Indexes: `issue_relations_workspace_source_created_at_id_idx`, `issue_relations_workspace_target_created_at_id_idx (…, created_at DESC, id DESC)`.
- Sub-issues are not here; they use `issues.parent_id` (010). The domain `RelationType.BLOCKED_BY` is never stored; it is written as `blocks` with swapped ends (`app/domain/relations.py`).

`issue_subscribers (migrations/020_subscribers_templates.sql)`

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
issue_id	uuid	NN	—	PK part; FK
user_id	uuid	NN	—	PK part; FK
created_at	timestamptz	NN	now()	

- FKs: `issue_subscribers_issue_fk → issues`; `issue_subscribers_user_fk → workspace_members`. Indexes: `issue_subscribers_workspace_user_idx (workspace_id, user_id)`.

`issue_templates (020)`

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK

Column	Type	Null	Default	Constraints
team_id	uuid	null	—	FK
name	text	NN	—	length 1..100
title	text	null	—	NULL or 1..500
description	text	null	—	NULL or 1..16384
priority	integer	null	—	NULL or 0..4
estimate	integer	null	—	NULL or >= 0
assignee_id	uuid	null	—	FK
project_id	uuid	null	—	FK
cycle_id	uuid	null	—	FK; `issue_templates_cycle_requires_team`
created_at, updated_at	timestamptz	NN	now()	

- Unique: `issue_templates_workspace_id_key (workspace_id, id)`.
- FKs: `issue_templates_workspace_fk → workspaces`; `issue_templates_team_fk → teams`; `issue_templates_assignee_fk → workspace_members`; `issue_templates_project_fk → projects`; `issue_templates_cycle_fk (workspace_id, team_id, cycle_id) → cycles(workspace_id, team_id, id)` (RESTRICT, unlike `issues_cycle_fk`).
- Indexes: `issue_templates_workspace_name_idx (workspace_id, name, id)`; `issue_templates_workspace_team_cycle_idx`; `issue_templates_workspace_assignee_idx`; `issue_templates_workspace_project_idx`.

`issue_template_labels` `(020)`

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
template_id	uuid	NN	—	PK part; FK
label_id	uuid	NN	—	PK part; FK
created_at	timestamptz	NN	now()	

- FKs: `issue_template_labels_template_fk → issue_templates`; `issue_template_labels_label_fk → labels` (both RESTRICT). Indexes: `issue_template_labels_workspace_label_idx`.

`issue_recurrences` `(migrations/029_estimates_dates.sql)`

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
template_id	uuid	NN	—	PK part; FK
team_id	uuid	NN	—	FK
frequency	text	NN	—	IN ('daily','weekly','monthly')
interval_count	smallint	NN	—	1..52
weekdays	smallint[]	null	—	required iff weekly; 1..7 elements, each in 1..7
day_of_month	smallint	null	—	required iff monthly; 1..31
starts_on	date	NN	—	
due_in_days	smallint	null	—	NULL or 0..365
next_run_on	date	NN	—	`issue_recurrences_runs_after_start` (>= starts_on)
created_at, updated_at	timestamptz	NN	now()	

- PK `issue_recurrences_pkey (workspace_id, template_id)` (one rule per template). FKs: `issue_recurrences_template_fk → issue_templates`; `issue_recurrences_team_fk → teams`.

- Indexes: `issue_recurrences_due_idx (next_run_on, workspace_id, template_id)`; `issue_recurrences_workspace_team_idx (workspace_id, team_id)`.

`issue_due_reminders` (029)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
issue_id	uuid	NN	—	PK part; FK
due_date	date	NN	—	PK part
reminded_at	timestamptz	NN	now()	

- PK `(workspace_id, issue_id, due_date)`: one reminder per issue per due date. FKs: `issue_due_reminders_issue_fk → issues`.

`saved_views` (migrations/019_saved_views.sql)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	FK
team_id	uuid	null	—	FK
name	text	NN	—	`char_length` 1..200
filter	jsonb	NN	—	`jsonb_typeof = 'object'`
order_field	text	NN	—	IN ('priority','created_at','updated_at','due_date')
order_direction	text	NN	—	IN ('asc','desc')
layout	text	NN	—	IN ('list','board')
grouping	text	null	—	NULL or IN ('workflow_state','assignee','priority','project','cycle','team')
subgrouping	text	null	—	same list; `saved_views_subgrouping_requires_grouping` (needs grouping, must differ)
visibility	text	NN	—	IN ('personal','shared')
created_by	uuid	NN	—	FK
created_at, updated_at	timestamptz	NN	now()	

- Unique: `saved_views_workspace_id_key (workspace_id, id)`. FKs: `saved_views_workspace_fk`; `saved_views_team_fk → teams`; `saved_views_creator_fk → workspace_members`.
- Indexes: `saved_views_workspace_name_id_idx (workspace_id, name, id)`; `saved_views_workspace_team_idx`; `saved_views_workspace_creator_idx`.

`favorites` (019)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
user_id	uuid	NN	—	FK
team_id	uuid	null	—	FK
project_id	uuid	null	—	FK
saved_view_id	uuid	null	—	FK

Column	Type	Null	Default	Constraints
position	integer	NN	—	>= 0
created_at	timestamptz	NN	now()	

- Checks: `favorites_one_target` (`num_nonnulls(team_id, project_id, saved_view_id) = 1`). Unique: `favorites_team_key`, `favorites_project_key`, `favorites_saved_view_key` (each `(workspace_id, user_id, <target>)`).
- FKs: `favorites_member_fk → workspace_members`; `favorites_team_fk`; `favorites_project_fk`; `favorites_saved_view_fk → saved_views(workspace_id, id)`.
- Indexes: `favorites_workspace_user_position_idx (workspace_id, user_id, position, id)`; `favorites_workspace_team_idx`; `favorites_workspace_project_idx`; `favorites_workspace_saved_view_idx`.

6.3.6 Activity, notifications, events, jobs

`issue_activity` (migrations/012_activity_notifications.sql; altered 030)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
issue_id	uuid	NN	—	FK
actor_id	uuid	null	—	FK
kind	text	NN	—	`issue_activity_kind_known` IN ('created','title_changed','state_changed','priority_changed','assignee_changed','archived','commented','label_attached','label_detached','relation_added','project_changed','cycle_changed')
from_value	text	null	—	
to_value	text	null	—	
created_at	timestamptz	NN	now()	
caused_by (030)	text	null	—	NULL or length 1..200

- FKs: `issue_activity_issue_fk → issues`; `issue_activity_actor_fk (actor_id) → users(id)` **ON DELETE SET NULL**. Indexes: `issue_activity_workspace_issue_created_idx (workspace_id, issue_id, created_at DESC, id DESC)`; `issue_activity_actor_idx (actor_id)`.

`notifications` (012; CHECK replaced in 020, 029)

Column	Type	Null	Default	Constraints
id	uuid	NN	uuidv7()	PK
workspace_id	uuid	NN	—	
user_id	uuid	NN	—	FK (recipient)
actor_id	uuid	null	—	FK; `notifications_actor_is_not_recipient`
issue_id	uuid	NN	—	FK
kind	text	NN	—	`notifications_kind_known` IN ('assigned','commented','blocked','status_changed','due_soon')
read_at	timestamptz	null	—	

Column	Type	Null	Default	Constraints
created_at	timestamptz	NN	now()	

- `kind` history: 3 values (012) → + `status_changed` (020) → + `due_soon` (029).
- FKs: `notifications_issue_fk → issues`; `notifications_user_fk → workspace_members`; `notifications_actor_fk → users(id)` **ON DELETE SET NULL**.
- Indexes: `notifications_workspace_user_unread_idx (workspace_id, user_id, created_at DESC, id DESC) WHERE read_at IS NULL`; `notifications_workspace_user_created_idx` (same columns, no predicate); `notifications_workspace_issue_idx`; `notifications_actor_idx`.

`domain_events` (`migrations/027_domain_events.sql`) — transactional outbox

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part; FK
kind	text	NN	—	PK part; IN ('issue_assigned','issue_completed','issue_priority_urgent','project_health_changed','project_update_published','pull_request_merged')
dedupe_key	text	NN	—	PK part; length 1..200
issue_id	uuid	null	—	FK
subject	text	NN	—	length 1..200
summary	text	NN	—	length 1..300
path	text	NN	—	`^/[!-~]*$`; length 1..512
occurred_at	timestamptz	NN	now()	
slack_state	text	NN	'pending'	IN ('pending','delivered','failed','skipped')
slack_failure	text	null	—	NULL or IN ('not_connected','missing_scope','no_default_channel','channel_unavailable','slack_refused','slack_unreachable','preference_disabled','no_installation')
slack_attempts	integer	NN	0	>= 0
slack_next_attempt_at	timestamptz	NN	now()	
slack_delivered_at	timestamptz	null	—	

- Checks: `domain_events_delivered_has_an_instant` ((state = 'delivered') = (delivered_at IS NOT NULL)); `domain_events_failed_has_a_reason`; `domain_events_progress_has_no_reason` (pending/delivered ⇒ failure IS NULL).
- PK `domain_events_pkey (workspace_id, kind, dedupe_key)` dedupes facts. FKs: `domain_events_workspace_fk → workspaces`; `domain_events_issue_fk → issues`.
- Indexes: `domain_events_pending_idx (slack_next_attempt_at) WHERE slack_state = 'pending'`; `domain_events_workspace_issue_idx (workspace_id, issue_id)`.

`embedding_jobs` (`migrations/028_embedding_jobs.sql`)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
issue_id	uuid	NN	—	PK part; FK
source_digest	text	NN	—	`^[0-9a-f]{64}$`
attempts	smallint	NN	0	0..100

Column	Type	Null	Default	Constraints
next_attempt_at	timestamptz	NN	now()	
last_error	text	null	—	NULL or length 1..500
created_at, updated_at	timestamptz	NN	now()	

- FKs: `embedding_jobs_issue_fk → issues(workspace_id, id)` **ON DELETE CASCADE**. Indexes: `embedding_jobs_due_idx (next_attempt_at, workspace_id, issue_id)`.

6.3.7 Search and embeddings

Full-text search uses the generated `search_vector` columns on `issues` and `projects` with GIN indexes (`migrations/011_search.sql`; see 6.3.3, 6.3.4). Semantic search uses one table:

`issue_embeddings` (`migrations/025_semantic_search.sql`)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
issue_id	uuid	NN	—	PK part; FK
model	text	NN	—	length 1..200
source_digest	text	NN	—	`^[0-9a-f]{64}$` (compared with `issues.embedding_source_digest`)
embedding	vector(384)	NN	—	
created_at, updated_at	timestamptz	NN	now()	

- FKs: `issue_embeddings_issue_fk → issues` (RESTRICT). Indexes: `issue_embeddings_workspace_model_digest_idx (workspace_id, model, source_digest)`. No HNSW/IVFFlat index: queries are exact per-workspace scans with `<=>` (025 comment "THE INDEX DECISION"). Dimension matches `EMBEDDING_DIMENSIONS = 384` (`app/domain/semantic_search.py`).

6.3.8 Integrations: GitHub

`github_installations` (`migrations/013_github_integration.sql`; altered `016`)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK (one installation per workspace); FK
installation_id	bigint	NN	—	UNIQUE; > 0
account_login	text	null	—	NULL or `^[A-Za-z0-9][A-Za-z0-9-]{0,38}$`
connected_by	uuid	NN	—	FK
connected_at	timestamptz	NN	now()	
updated_at	timestamptz	NN	now()	
confirmed_at (016)	timestamptz	null	—	`github_installations_unconfirmed_holds_no_account` (confirmed_at IS NOT NULL OR account_login IS NULL)

- FKs: `github_installations_workspace_fk → workspaces`; `github_installations_connected_by_fk → workspace_members`. Indexes: `github_installations_connected_by_idx (workspace_id, connected_by)`.

`github_repositories` (`013`; altered `030`)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part; FK
repository_id	bigint	NN	—	PK part; > 0
full_name	text	NN	—	`^[A-Za-z0-9][A-Za-z0-9-]{0,38}/[A-Za-z0-9._-]{1,100}$`
created_at	timestamptz	NN	now()	
tracked (030)	boolean	NN	TRUE	

- FKs: `github_repositories_installation_fk (workspace_id) → github_installations(workspace_id)`. Indexes: `github_repositories_repository_id_idx (repository_id)` (cross-workspace webhook lookup).

`github_deliveries` (`migrations/017_github_development.sql`) — webhook idempotency

Column	Type	Null	Default	Constraints
delivery_id	text	NN	—	PK; length 1..200
event	text	null	—	
received_at	timestamptz	NN	now()	

- No tenant column, no FKs. Indexes: `github_deliveries_received_at_idx (received_at)`.

`github_pull_requests` (017)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
repository_id	bigint	NN	—	PK part
number	integer	NN	—	PK part; > 0
title	text	NN	—	length 1..1024
state	text	NN	—	IN ('open','closed')
draft	boolean	NN	FALSE	
merged_at	timestamptz	null	—	`github_pull_requests_merged_is_closed`
head_ref	text	null	—	NULL or `^[!-~]{1,255}$`
url	text	null	—	NULL or length 1..2048
github_updated_at	timestamptz	null	—	
created_at, updated_at	timestamptz	NN	now()	

- PK `github_pull_requests_pkey (workspace_id, repository_id, number)`; additionally `github_pull_requests_workspace_repository_number_key UNIQUE` on the same columns.
- FKs: `github_pull_requests_repository_fk → github_repositories(workspace_id, repository_id)`. Indexes: `github_pull_requests_workspace_repository_updated_idx (workspace_id, repository_id, github_updated_at DESC, number DESC)`.

`github_pull_request_issues` (017)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
repository_id	bigint	NN	—	PK part; FK part
number	integer	NN	—	PK part; FK part
issue_id	uuid	NN	—	PK part; FK
source	text	NN	—	PK part; IN ('title','body','branch')

Column	Type	Null	Default	Constraints
created_at	timestamptz	NN	now()	

- FKs: `github_pull_request_issues_pull_request_fk → github_pull_requests`; `github_pull_request_issues_issue_fk → issues`. Indexes: `github_pull_request_issues_workspace_issue_idx (workspace_id, issue_id)`.

`github_commits` (017)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
repository_id	bigint	NN	—	PK part
sha	text	NN	—	PK part; `^[0-9a-f]{40}$`
message	text	NN	—	length 1..8192
url	text	null	—	NULL or length 1..2048
committed_at	timestamptz	null	—	
created_at	timestamptz	NN	now()	

- PK `github_commits_pkey (workspace_id, repository_id, sha)`; additionally `github_commits_workspace_repository_sha_key UNIQUE` on the same columns. FKs: `github_commits_repository_fk → github_repositories`.

`github_commit_issues` (017)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
repository_id	bigint	NN	—	PK part; FK part
sha	text	NN	—	PK part; FK part
issue_id	uuid	NN	—	PK part; FK
created_at	timestamptz	NN	now()	

- FKs: `github_commit_issues_commit_fk → github_commits`; `github_commit_issues_issue_fk → issues`. Indexes: `github_commit_issues_workspace_issue_idx`.

`github_issue_automations` (migrations/030_github_automations.sql)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part
team_id	uuid	NN	—	PK part; FK
started_state_id	uuid	null	—	FK
completed_state_id	uuid	null	—	FK
created_at, updated_at	timestamptz	NN	now()	

- Checks: `github_issue_automations_moves_something` (at least one state set). FKs: `github_issue_automations_team_fk → teams`; `…_started_state_fk`, `…_completed_state_fk (workspace_id, team_id, *_state_id) → workflow_states(workspace_id, team_id, id)`.

6.3.9 Integrations: Slack

`slack_installations` (migrations/014_slack_integration.sql)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK; FK
slack_team_id	text	NN	—	`slack_installations_team_key` UNIQUE
slack_team_name	text	NN	—	
bot_user_id	text	NN	—	
scopes	text[]	NN	—	
bot_token_backend	text	NN	—	(no CHECK)
bot_token_reference	text	NN	—	
connected_by_user_id	uuid	NN	—	FK
connected_at	timestamptz	NN	now()	

- FKs: `slack_installations_workspace_fk → workspaces`; `slack_installations_connected_by_fk → workspace_members`. Per the 014 comment, the only token backend is `'database'`, where the reference is the bot token itself.

`slack_event_deliveries` (014) — event idempotency

Column	Type	Null	Default	Constraints
event_id	text	NN	—	PK
received_at	timestamptz	NN	now()	

- No tenant column, no FKs. Indexes: `slack_event_deliveries_received_at_idx (received_at)`.

`slack_channels` (migrations/018_slack_channels.sql)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part; FK
channel_id	text	NN	—	PK part; `^[A-Z0-9]{2,64}$`
name	text	NN	—	length 1..80
is_private	boolean	NN	FALSE	
is_archived	boolean	NN	FALSE	
is_member	boolean	NN	FALSE	
is_accessible	boolean	NN	TRUE	
synced_at	timestamptz	NN	now()	
created_at	timestamptz	NN	now()	

- FKs: `slack_channels_installation_fk (workspace_id) → slack_installations(workspace_id)`.

`slack_notification_settings` (018)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK; FK
default_channel_id	text	null	—	FK part
default_channel_name	text	null	—	NULL or length 1..80; `…_channel_pair` (both NULL or both set)
updated_at	timestamptz	NN	now()	

- FKs: `slack_notification_settings_installation_fk → slack_installations`; `slack_notification_settings_default_channel_fk (workspace_id, default_channel_id) → slack_channels`.

`slack_notification_preferences` (018)

Column	Type	Null	Default	Constraints
workspace_id	uuid	NN	—	PK part; FK
event	text	NN	—	PK part; same six values as `domain_events.kind`
enabled	boolean	NN	—	
updated_at	timestamptz	NN	now()	

- FKs: `slack_notification_preferences_installation_fk → slack_installations`.

6.3.10 Analytics

No analytics tables. `migrations/031_analytics.sql` adds only the partial index `issues_workspace_completed_at_idx`; aggregates are computed on read by `app/repositories/analytics.py` (`AnalyticsRepository`) over `issues`, `teams`, `workflow_states`, `cycles`, `projects`, `issue_activity`, `users`.

6.3.11 Ops

`schema_migrations` (`scripts/apply_migration.py` `CREATE_LEDGER_SQL`, not a migration)

Column	Type	Null	Default	Constraints
version	text	NN	—	PK
applied_at	timestamptz	NN	now()	
checksum	text	NN	—	SHA-256 hex of the file (`compute_checksum`)

- Created with `CREATE TABLE IF NOT EXISTS`. The runner takes advisory lock `ADVISORY_LOCK_KEY = 0x56454354` and applies each file in its own transaction (`scripts/apply_migration.py`). The container start command runs `python -m scripts.apply_migration --all` (`Dockerfile`).
- `auth_rate_limits` (6.3.2) is the other operational table.

6.4 Table → domain entity → repository

Domain classes live in `app/domain/<module>.py`; repositories in `app/repositories/<module>.py`. "Also read by" lists other repository modules whose SQL touches the table (`FROM`/`INTO`/`UPDATE`/`JOIN`).

Table(s)	Domain class(es)	Primary repository	Also read by
workspaces	none dedicated; `tenancy.WorkspaceScope`, `memberships.WorkspaceMembershipEntity` carry slug/name	`workspaces.WorkspaceRepository`	events, memberships
teams	`teams.TeamEntity`, `estimates.EstimateScale`	`teams.TeamRepository`	analytics, events, github, issues, relations, releases
workflow_states	`teams.WorkflowStateEntity`	`teams.TeamRepository`	analytics, bulk, events, github, issues, reminders, triage
users	`auth.UserEntity`	`users.UserRepository`	analytics, memberships
sessions	`auth.SessionEntity`	`sessions.SessionRepository`	—

Table(s)	Domain class(es)	Primary repository	Also read by
workspace_members	`memberships.WorkspaceMembershipEntity`, `memberships.WorkspaceMemberEntity`	`memberships.MembershipRepository`	notifications
workspace_invitations	`memberships.WorkspaceInvitationEntity`	`invitations.InvitationRepository`	—
auth_rate_limits	none	`rate_limits.RateLimitRepository`	—
issues	`issues.IssueEntity`; `triage.TriageIssueEntity`	`issues.IssueRepository`	analytics, bulk (`BulkRepository`), embedding_jobs, embeddings, events, github, memberships, notifications, relations, releases, reminders, triage (`TriageRepository`)
cycles	`cycles.CycleEntity`	`cycles.CycleRepository`	analytics
labels	`labels.LabelEntity`	`labels.LabelRepository`	issue_labels, label_groups
label_groups	`labels.LabelGroupEntity`	`label_groups.LabelGroupRepository`	labels
issue_labels	none (labels returned as `LabelEntity`)	`issue_labels.IssueLabelRepository`	issues
comments	`comments.CommentEntity`	`comments.CommentRepository`	—
issue_relations	`relations.IssueRelationEntity`	`relations.RelationRepository`	—
issue_activity	`activity.ActivityEntity`	`activity.ActivityRepository`	analytics
notifications	`notifications.NotificationEntity`	`notifications.NotificationRepository`	memberships
issue_subscribers	`subscribers.SubscriberEntity`	`subscribers.SubscriberRepository`	memberships, notifications
issue_templates, issue_template_labels	`templates.IssueTemplateEntity`	`templates.TemplateRepository`	memberships (templates only)
issue_recurrences	`recurrence.RecurrenceEntity`	`recurrences.RecurrenceRepository`	—
issue_due_reminders	`reminders.DueIssue`	`reminders.DueReminderRepository`	—
issue_embeddings	`semantic_search.EmbeddingSource`	`embeddings.EmbeddingRepository`	embedding_jobs

Table(s)	Domain class(es)	Primary repository	Also read by
embedding_jobs	`embedding_jobs.ClaimedEmbeddingJob`	`embedding_jobs.EmbeddingJobRepository`	—
projects, project_teams, project_milestones, project_updates, project_dependencies	`projects.ProjectEntity`, `ProjectMilestoneEntity`, `ProjectUpdateEntity`, `ProjectDependencies`	`projects.ProjectRepository`	analytics, events, memberships (projects only)
initiatives, initiative_projects, initiative_updates	`initiatives.InitiativeEntity`, `InitiativeUpdateEntity`	`initiatives.InitiativeRepository`	memberships (initiatives only)
documents, document_revisions, document_comments	`documents.DocumentEntity`, `DocumentRevisionEntity`, `DocumentCommentEntity`	`documents.DocumentRepository`	—
saved_views	`saved_views.SavedViewEntity`	`saved_views.SavedViewRepository`	memberships
favorites	`saved_views.FavoriteEntity`	`saved_views.FavoriteRepository`	memberships
github_installations, github_repositories	`github.GithubInstallationEntity`, `GithubRepositoryEntity`	`github.GithubRepository`	memberships (installations)
github_deliveries	none	`github.GithubRepository`	—
github_pull_requests, github_pull_request_issues, github_commits, github_commit_issues	`github.GithubPullRequestEntity`, `GithubCommitEntity`	`github.GithubRepository`	releases; events (pull_request_issues)
github_issue_automations	`github.GithubAutomationEntity`	`github.GithubRepository`	—
slack_installations, slack_channels, slack_notification_settings, slack_notification_preferences, slack_event_deliveries	`slack.SlackInstallationEntity`, `SlackChannelEntity`, `SlackNotificationSettingsEntity`, `SlackNotificationPreference`	`slack.SlackRepository`	memberships (installations)
domain_events	`events.DomainEventEntity`	`events.EventRepository`	—
environments, releases, release_issues, release_pull_requests	`releases.EnvironmentEntity`, `ReleaseEntity`, `ReleaseIssueRef`, `ReleasePullRequestRef`	`releases.ReleaseRepository`	github (releases)
(cross-table aggregates)	`analytics.*`	`analytics.AnalyticsRepository`	—
schema_migrations	none	`scripts/apply_migration.py` (no repository)	—

6.5 Migration history

`migrations/001_issues.sql` is immutable (`CLAUDE.md`). It reached production before the ledger existed, so the runner **adopts** it: if `schema_migrations` is empty and the live `issues` table matches the `INITIAL_COLUMNS` fingerprint, a ledger row is written instead of re-running it (`scripts/apply_migration.py` `_adopt_initial_migration`). `tests/test_phase1a2_gates.py` asserts the exact list of migration files (`EXPECTED_MIGRATIONS`).

#	File	Summary
001	`001_issues.sql`	`issues` (id, title, description, priority 0..4, completed_at, timestamps) + `issues_created_at_id_idx`. Immutable; adopted by the runner.
002	`002_tenancy.sql`	`workspaces`, `teams`; seeds workspace `vector` / team `Core`; backfills NOT NULL `issues.workspace_id`, `team_id`; `issues_team_fk`; replaces 001's index with workspace-leading keyset index.
003	`003_auth.sql`	`users` (lowercase email, argon2id CHECK), `sessions` (32-byte SHA-256 BYTEA token hash, CASCADE on user delete).
004	`004_membership.sql`	`workspace_members` (PK workspace_id, user_id; role), `workspace_invitations` (hex token hash, expiry).
005	`005_team_workflows.sql`	`teams.key`, `teams.issue_counter`; `workflow_states` + 5 seeded states per team; backfilled `issues.number`, `issues.workflow_state_id` + composite FK.
006	`006_issue_fields.sql`	`issues.assignee_id`, `creator_id`, `estimate`, `due_date`, `archived_at`; SET NULL FKs; live (non-archived) keyset index.
007	`007_labels_comments.sql`	`issues_workspace_id_key`; `labels`, `issue_labels`, `comments` (cascade from issue).
008	`008_cycles.sql`	`btree_gist`; `cycles` with no-overlap EXCLUDE; `issues.cycle_id` (SET NULL).
009	`009_projects.sql`	`projects`, `project_teams`, `project_milestones`; `issues.project_id`, `milestone_id`.
010	`010_issue_relations.sql`	`issues.parent_id` (sub-issues); `issue_relations`.
011	`011_search.sql`	`btree_gin`; generated `search_vector` on `issues` and `projects`; GIN indexes.
012	`012_activity_notifications.sql`	`issue_activity`, `notifications`.
013	`013_github_integration.sql`	`github_installations`, `github_repositories`.
014	`014_slack_integration.sql`	`slack_installations`, `slack_event_deliveries`.
015	`015_issue_list_indexes.sql`	Four partial keyset indexes for issue-list sorts (assignee, updated_at, `NULLIF(priority, 0)`, due_date).
016	`016_github_installation_trust.sql`	`github_installations.confirmed_at` + `unconfirmed_holds_no_account` CHECK.
017	`017_github_development.sql`	`github_deliveries`, `github_pull_requests`, `github_pull_request_issues`, `github_commits`, `github_commit_issues`.
018	`018_slack_channels.sql`	`slack_channels`, `slack_notification_settings`, `slack_notification_preferences`.
019	`019_saved_views.sql`	`saved_views`, `favorites`.
020	`020_subscribers_templates.sql`	`issue_subscribers`; notification kind `status_changed`; `issue_templates`, `issue_template_labels`.
021	`021_triage_label_groups.sql`	`issues.triage_entered_at`; `label_groups`; label group columns + generated `exclusivity_key`; `issue_labels.exclusivity_key` + exclusive-group unique index.
022	`022_initiatives.sql`	`initiatives`, `initiative_projects`, `projects.health`, `project_updates`, `initiative_updates`, `project_dependencies`.
023	`023_documents.sql`	`documents` (JSONB content, optional project/initiative parent), `document_revisions`, `document_comments`.
024	`024_releases.sql`	`environments`, `releases`, `release_issues`, `release_pull_requests`.

#	File	Summary
025	`025_semantic_search.sql`	`vector` extension; generated `issues.embedding_source_digest`; `issue_embeddings vector(384)`; no ANN index.
026	`026_member_departure.sql`	`workspace_members.removed_at` tombstone + CHECK.
027	`027_domain_events.sql`	`domain_events` outbox with Slack delivery state CHECKs.
028	`028_embedding_jobs.sql`	`embedding_jobs` (attempts, backoff, last error).
029	`029_estimates_dates.sql`	`teams.estimate_scale`; notification kind `due_soon`; `issue_due_reminders`, `issue_recurrences`.
030	`030_github_automations.sql`	`github_repositories.tracked`; `issue_activity.caused_by`; `github_issue_automations`.
031	`031_analytics.sql`	Partial index `issues_workspace_completed_at_idx`.
032	`032_auth_hardening.sql`	`auth_rate_limits` + `auth_rate_limits_window_idx`.

Not applied: `schema_drafts/`

File	Status	Content
`schema_drafts/033_google_identity.draft.sql`	Untracked in git (`git status`: `??`); deliberately outside `migrations/` because boot runs `apply_migration --all` (file header).	`users.password_hash` → nullable, argon2id CHECK relaxed to `password_hash IS NULL OR …`; new `users.google_subject TEXT` with `users_google_subject_key UNIQUE` and length 1..255 CHECK; `users_has_a_credential` (password_hash or google_subject NOT NULL). Header policy: a Google sign-in never auto-links to an existing email (enforced by `users_email_key`). Adoption steps: move to `migrations/033_google_identity.sql`, add to `EXPECTED_MIGRATIONS`, apply via runner.
`schema_drafts/full_schema.sql`	Tracked design document, never executed (`CLAUDE.md`). Header calls itself `002_full_schema.sql`; wrapped in `BEGIN; … COMMIT;`.	27 tables. Key differences from the applied schema: uses `CITEXT` for `users.email`; uses `ON DELETE CASCADE` broadly (43 occurrences) vs RESTRICT in migrations; `BIGSERIAL` PKs on append-only log tables; draft-only tables `auth_identities`, `team_members`, `reactions`, `api_keys`, `webhooks`, `webhook_deliveries`, `integrations`, `integration_links`, `outbox_events`, `inbox_events`, `sync_events`, `issue_history`. The applied schema instead has provider-specific `github_*` / `slack_*` tables, `domain_events` (outbox) and `issue_activity` (history), plus many tables the draft lacks (label groups, templates, recurrences, initiatives, documents, saved views, releases, embeddings, rate limits, invitations).

6.6 Gaps / to verify

> **Gaps / to verify:**
>
> - `github_pull_requests` and `github_commits` declare a separate `UNIQUE` constraint over exactly their PK columns (`migrations/017_github_development.sql`), which PostgreSQL backs with a second unique index; the file comment implies only one. Confirm with `\d` on a live database.
> - Which migrations are applied in staging/production cannot be determined from the repository (the ledger lives in the database); `python -m scripts.apply_migration` status output would show it.
> - Server versions on Neon (PostgreSQL 18 for `uuidv7()`, pgvector version) are not verifiable from the repo.
> - `issue_activity.kind` has no values for due-date, estimate, milestone, parent, triage or unarchive changes; whether those changes are logged elsewhere was not checked.
> - `workspace_invitations.email` has no lowercase/shape CHECK (unlike `users.email`); normalisation is presumably in application code — not verified.

- Sweeps of `github_deliveries`, `slack_event_deliveries` and `auth_rate_limits` (indexes on `received_at` / `window_started_at` exist) were not traced to code.
- `releases.repository_id` is `NOT NULL` with an FK to `github_repositories`, so a release requires a connected GitHub repository; whether that is intended product behaviour is not documented in the SQL.
- `IssueEntity` (`app/domain/issues.py`) does not carry `parent_id` or `triage_entered_at`; how those reach the API was not traced here.
- If draft 033 is adopted, `UserCredentials.password_hash: str` (`app/domain/auth.py`, non-optional) and the code that reads it must change; the extent of that change was not traced.

7. API

Vector exposes one GraphQL endpoint for all first-party product operations, plus a small set of REST routes for health probes, OAuth callbacks and provider webhooks. The authoritative contract is the exported SDL `frontend/schema.graphql`. It is regenerated by `frontend/scripts/export_schema.py` and checked against the code in CI (the "GraphQL schema contract" job in `.github/workflows/ci.yml`, and `npm run graphql:schema:check` in `frontend/package.json`).

7.1 Transport and conventions

Endpoint

Aspect	Behaviour	Source
Path	`/graphql`. Strawberry `GraphQLRouter` mounted with `prefix="/graphql"`, registered before every REST router and before the SPA catch-all	`app/main.py` → `create_app`; `app/graphql/router.py` → `build_graphql_router`
Operations	`POST` with an `application/json` body runs queries and mutations. Other content types get a 400 (multipart uploads are not enabled)	`app/graphql/router.py` (`GraphQLRouter` defaults)
GET	`allow_queries_via_get=False`. `GET /graphql` serves only the GraphiQL IDE, and only when that IDE is enabled	`app/graphql/router.py` → `build_graphql_router`
GraphiQL	`graphql_ide="graphiql"` unless `ENVIRONMENT == "production"`, where it is `None`	`app/graphql/router.py`
Introspection	Disabled in production by appending `strawberry.extensions.DisableIntrospection`. Enabled in `development`/`test`	`app/graphql/schema.py` → `build_schema`
Environments	`Environment = Literal["development", "test", "production"]`	`app/config.py`
Batching	Disabled. The schema is built without a `batching_config`, so a JSON-array body gets a 400 "Batching is not enabled"	`app/graphql/schema.py` → `_MaskedSchema`
Subscriptions	None declared	`app/graphql/schema.py` → `_MaskedSchema` docstring
Root composition	`QUERY_TYPES` (19 classes) and `MUTATION_TYPES` (21 classes) merged with `strawberry.tools.merge_types`. Tuple order is SDL field order	`app/graphql/schema.py`
Identity	HttpOnly cookie `vector_session` (path `/`, `SameSite=Lax`, `Secure` in production). Resolved once per request through `VectorContext.viewer()` → `AuthService.authenticate`	`app/http_cookies.py`; `app/graphql/context.py` → `get_context`
CORS	No `CORSMiddleware`. The SPA is same-origin	`app/main.py`

Request limits. The limits are enforced as validation rules in every environment (`app/graphql/limits.py` → `OperationLimitsRule`, installed via `operation_limit_extensions()`). The body cap is app-wide middleware.

Limit	Value	Notes
`MAX_DEPTH`	10	Nesting levels of composite fields
`MAX_ALIASES`	15	Aliases per operation
`MAX_SELECTIONS`	2000	Total field selections after fragment expansion

Limit	Value	Notes
`MAX_COMPLEXITY`	1000	Weighted cost. A leaf costs 1. A composite field costs page size × the inner cost. Page size comes from a literal `first`/`last`, else the argument default, else `ASSUMED_PAGE_SIZE = 100` (e.g. a variable); a field with no `first`/`last` argument counts as 1. `__schema`/`__type` are not charged
Request body	256 KiB (`MAX_REQUEST_BODY_BYTES = 256 * 1024`)	`app/http_limits.py` → `add_request_body_limit`. Applies to all routes, REST included
Service page caps	`first` must be between 1 and 100 (`FIRST_MIN`/`FIRST_MAX`) for list services	e.g. `app/services/issues.py`
Other caps	`embeddingsRefresh.limit` ≤ 500 (`REFRESH_MAX`). Bulk ops take ≤ 100 issue IDs (`BULK_MAX`). `workspaceAnalytics.days` must be 1..180 (`ANALYTICS_MAX_DAYS`, default 30)	`app/services/search.py`; `app/domain/bulk.py`; `app/domain/analytics.py`

A request that breaks a limit gets a validation error such as "Query is nested N levels deep; the maximum is 10." All budgets it breaks are reported together (`app/graphql/limits.py` → `_report`).

Error masking (`app/graphql/schema.py`). Masking is applied in all environments.

- `PUBLIC_ERROR_CODES = {"BAD_USER_INPUT", "UNAUTHENTICATED", "NOT_FOUND"}`.
- An error reaches the client verbatim only in two cases: it carries one of those codes in `extensions.code`, or it has no `original_error` (graphql-core parse/validation errors). Every other error is rewritten to `"Internal server error"` (`MASKED_ERROR_MESSAGE`) with no extensions (`is_public_error`).
- Masking runs twice: once through the `MaskErrors` extension, and again in the `_MaskedSchema.execute` override. The second pass covers exceptions that escape during validation.

Two error channels. The SDL has no unions and no interfaces.

Channel	When	Shape	Source
`payload.errors`	Expected domain validation failures on mutations	`errors: [ValidationErrorType!]!`, where `ValidationErrorType { field: String!, code: String!, message: String! }`. The result field is `null` on failure	`app/graphql/types/errors.py` → `ValidationErrorType.from_domain`
Top-level `UNAUTHENTICATED`	No valid session on a protected field. Raised before any workspace lookup	`"Authentication required"`, `extensions.code = "UNAUTHENTICATED"`	`app/graphql/viewer.py` → `viewer_user_id`
Top-level `NOT_FOUND`	The workspace slug does not exist, the caller is not a member, or the caller lacks the required role. These cases cannot be told apart	`"Workspace not found"`	`app/graphql/scope.py` → `authorized_scope` (`WORKSPACE_NOT_FOUND_MESSAGE`)
Top-level `BAD_USER_INPUT`	Validation failures on queries (which have no payload), and a few mutations noted below	`extensions = {code, issues: [{field, code, message}]}`	`app/graphql/errors.py` → `bad_user_input`

Typical `ValidationErrorType.code` values include `REQUIRED`, `TOO_LONG`, `INVALID`, `NOT_FOUND`, `DUPLICATE`, `OUT_OF_RANGE`, `INVALID_CURSOR`, `CYCLE`, `NOT_A_MEMBER`, `EMAIL_TAKEN`, `SLUG_TAKEN`, `LAST_OWNER`, `INVALID_CREDENTIALS` and `INVALID_TRANSITION`. They are mostly raised from `app/services/*.py`; `INVALID_CREDENTIALS` comes from `app/graphql/mutations/auth.py`, and some `INVALID`/`NOT_FOUND` issues are built in `app/graphql/mutations/*.py` (e.g. `github.py`, `releases.py`, `issues.py`).

Pagination. Pagination is forward-only and keyset-based.

- Connections have the shape `{ nodes: [T!]!, pageInfo: PageInfo! }`, with `PageInfo { hasNextPage: Boolean!, endCursor: String }` (`app/graphql/types/pagination.py`).
- Arguments are `first` / `after`. There is no `last`/`before`, no `edges`, and no `startCursor`.

- `IssueConnection` adds `totalCount: Int!`.
- The SDL defines 15 `*Connection` types.
- Cursors are opaque URL-safe base64 of compact, sorted-key JSON (`app/domain/pagination.py` → `_encode_payload`). Examples:
  - `encode_issue_cursor` encodes `{created_at, id}`.
  - `encode_issue_list_cursor` encodes the order, the sort key and the id.
  - `encode_label_cursor` encodes the name and the id.
- A malformed cursor produces the `INVALID_CURSOR` validation code.

DataLoaders. Loaders are built per request on `VectorContext` (`app/graphql/context.py`; `app/graphql/loaders/*.py`). Each loader groups its keys by workspace and makes one service call per workspace.

Loader	Batches via	Serves
`project_loader`	`ProjectService.get_many_by_ids`	`Issue.project`
`project_milestones_loader` / `project_updates_loader` / `project_dependencies_loader`	`ProjectService.list_milestones_for_projects` / `list_updates_for_projects` / `list_dependencies_for_projects`	`Project.milestones` / `.updates` / `.dependencies`
`initiative_updates_loader`	`InitiativeService.list_updates_for_initiatives`	`Initiative.updates`
`document_content_loader` / `revision_content_loader`	`DocumentService.contents_for_documents` / `contents_for_revisions`	`Document.content` / `DocumentRevision.content`
`cycle_loader` (`CycleLoader`)	`CycleService.get_many`	`Issue.cycle`
`issue_labels`	`LabelService.labels_for_issues`	`Issue.labels`
`issue_summaries`	`IssueService.get_many_by_ids`	`Notification.issue`

These nested fields are **not** batched and call a service once per parent: `Issue.parent`, `Issue.children`, `Issue.relations`, `Issue.activity`, `Issue.development`, `Issue.comments`, `IssueConnection.totalCount`, `Document.revisions`, `Document.comments` and `SavedView.issues` (`app/graphql/types/issue.py`, `document.py`, `saved_view.py`).

7.2 Authorization levels

Resolvers establish identity, and services enforce roles. Every workspace-scoped resolver calls `authorized_scope(info, workspaceSlug)` (`app/graphql/scope.py`), or a role-checking equivalent: `github_scope` in `app/graphql/queries/github.py`, `slack_admin_scope` in `app/graphql/queries/slack.py`, and `_scope` in `app/graphql/mutations/memberships.py`. Each of these resolves the viewer first and then checks membership through `MembershipService.authorized_scope_for_slug`.

Level	Meaning	Failure seen by client
anon	No identity check	—
auth	Signed in; no workspace involved (`viewer_user_id`)	`UNAUTHENTICATED`
member	Signed in and a member of `workspaceSlug` (any role in `WorkspaceRole { MEMBER, ADMIN, OWNER }`)	`UNAUTHENTICATED`, or `NOT_FOUND` for a non-member or unknown slug
admin	**member** plus role `ADMIN` or `OWNER`. Enforced in the service by `app/domain/tenancy.py` → `require_workspace_admin`, `app/services/github.py` → `require_workspace_admin`, or `SlackService.require_admin`	As **member**. A member without the role also gets `NOT_FOUND`
owner	Owner-only sub-rules inside admin operations: only an owner may grant or modify the `OWNER` role, or invite as owner (`app/services/memberships.py`)	`NOT_FOUND`
author	**member** plus authorship of the target row. The service matches `author_id` in its `WHERE` clause (`CommentService.delete`, `DocumentService.delete_comment`)	`payload.errors` with code `NOT_FOUND`

In the tables below, "VE" means that a domain `ValidationError` is mapped into `payload.errors`. Every member- and admin-level field can also return top-level `UNAUTHENTICATED` and `NOT_FOUND`.

7.3 Queries (42 fields)

Fields are enumerated from `type Query` in `frontend/schema.graphql`. Unless noted, every field takes `workspaceSlug: String!` as its first argument. Resolver paths are relative to `app/graphql/queries/`.

Identity and workspace

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`me`	—	`User`	anon	`null` when signed out	`auth.py` `me` → `context.viewer()` (`AuthService.authenticate`)
`myWorkspaces`	—	`[WorkspaceMembership!]!`	auth		`memberships.py` `my_workspaces` → `MembershipService.list_for_user`
`myWorkspace`	`slug: String!`	`WorkspaceMembership!`	auth + member	`NOT_FOUND` "Workspace not found"	`memberships.py` `my_workspace` → `MembershipService.membership_for_slug`
`workspaceMembers`	`workspaceSlug`	`[WorkspaceMember!]!`	member	Includes departed members (`removedAt`)	`memberships.py` `workspace_members` → `MembershipService.list_members`
`invitations`	`workspaceSlug`	`[WorkspaceInvitation!]!`	admin	Pending invitations only	`memberships.py` `invitations` → `MembershipService.list_invitations`
`teams`	`workspaceSlug`	`[Team!]!`	member	Teams with their workflow states	`teams.py` `teams` → `TeamService.list_workflows`

Issues

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`issue`	`workspaceSlug, id: UUID!`	`Issue`	member	`null` if absent	`issues.py` `issue` → `IssueService.get_by_id`
`issues`	`workspaceSlug, filter: IssueFilterInput, orderBy: IssueOrderInput, first: Int! = 50, after: String`	`IssueConnection!`	member	`BAD_USER_INPUT` "Invalid issue list arguments"	`issues.py` `issues` → `IssueService.list`

`IssueFilterInput` (all fields optional):

Field	Type
`teamId`, `assigneeId`, `workflowStateId`, `labelId`, `projectId`, `cycleId`	`UUID`
`stateCategory`	`WorkflowStateCategory` (`BACKLOG`, `UNSTARTED`, `STARTED`, `COMPLETED`, `CANCELED`)
`priority`	`Int`
`due`	`DueWindow` (`OVERDUE`, `TODAY`, `THIS_WEEK`, `NO_DUE_DATE`)
`dueAfter`, `dueBefore`	`Date`

`IssueOrderInput` takes `field: IssueOrderField! = CREATED_AT` (one of `PRIORITY`, `CREATED_AT`, `UPDATED_AT`, `DUE_DATE`) and `direction: OrderDirection! = DESC`.

Labels, cycles, projects, initiatives, documents

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`label`	`workspaceSlug, id: UUID!`	`Label`	member	`null` if absent	`labels.py` `label` → `LabelService.get_by_id`
`labelGroups`	`workspaceSlug`	`[LabelGroup!]!`	member		`labels.py` `label_groups` → `LabelService.list_groups`
`labelGroup`	`workspaceSlug, id: UUID!`	`LabelGroup`	member	`null` if absent	`labels.py` `label_group` → `LabelService.get_group`
`labels`	`workspaceSlug, first: Int! = 50, after: String`	`LabelConnection!`	member	`BAD_USER_INPUT`	`labels.py` `labels` → `LabelService.list`
`cycle`	`workspaceSlug, id: UUID!`	`Cycle`	member	`null` if absent	`cycles.py` `cycle` → `CycleService.get_by_id`
`cycles`	`workspaceSlug, teamId: UUID!`	`[Cycle!]!`	member		`cycles.py` `cycles` → `CycleService.list_for_team`
`project`	`workspaceSlug, id: UUID!`	`Project`	member	`null` if absent	`projects.py` `project` → `ProjectService.get_by_id`
`projects`	`workspaceSlug, first: Int! = 50, after: String`	`ProjectConnection!`	member	`BAD_USER_INPUT`	`projects.py` `projects` → `ProjectService.list`
`initiative`	`workspaceSlug, id: UUID!`	`Initiative`	member	`null` if absent	`initiatives.py` `initiative` → `InitiativeService.get_by_id`
`initiatives`	`workspaceSlug, first: Int! = 50, after: String`	`InitiativeConnection!`	member	`BAD_USER_INPUT`	`initiatives.py` `initiatives` → `InitiativeService.list`
`document`	`workspaceSlug, id: UUID!`	`Document`	member	`null` if absent	`documents.py` `document` → `DocumentService.get_by_id`
`documents`	`workspaceSlug, projectId: UUID, initiativeId: UUID, first: Int! = 50, after: String`	`DocumentConnection!`	member	`BAD_USER_INPUT`. An omitted argument and an explicit `null` are distinguished via `strawberry.UNSET`	`documents.py` `documents` → `DocumentService.list`

Search

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`search`	`workspaceSlug, query: String!, first: Int! = 20`	`SearchResults!` (`issues`, `projects`)	member	`BAD_USER_INPUT` "Invalid search arguments". Does the scope check inline rather than calling `authorized_scope`	`search.py` `search` → `SearchService.search`

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`issueDuplicateSuggestions`	`workspaceSlug, title: String!, description: String, excludeIssueId: UUID, first: Int! = 5`	`[DuplicateSuggestion!]!`	member	`BAD_USER_INPUT`	`search.py issue_duplicate_suggestions` → `SearchService.suggest_duplicates`
`embeddingIndexingState`	`workspaceSlug`	`EmbeddingIndexingState!`	member		`search.py embedding_indexing_state` → `SearchService.indexing_state`

Integrations

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`githubIntegration`	`workspaceSlug`	`GithubIntegration!`	admin	Authorized via `github_scope`	`github.py github_integration` → `GithubService.integration_for`
`slackIntegration`	`workspaceSlug`	`SlackIntegration!`	admin	Authorized via `slack_admin_scope`	`slack.py slack_integration` → `SlackService.status`
`slackChannels`	`workspaceSlug`	`[SlackChannel!]!`	admin		`slack.py slack_channels` → `SlackService.channels`
`slackNotificationSettings`	`workspaceSlug`	`SlackNotificationSettings!`	admin		`slack.py slack_notification_settings` → `SlackService.notification_settings`

Inbox, views, subscriptions, templates

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`notifications`	`workspaceSlug, unreadOnly: Boolean! = false, first: Int! = 50, after: String`	`NotificationConnection!`	member	Viewer's own inbox. `BAD_USER_INPUT`	`notifications.py notifications` → `ActivityService.list_notifications`
`notificationUnreadCount`	`workspaceSlug`	`Int!`	member		`notifications.py notification_unread_count` → `ActivityService.unread_count`
`savedView`	`workspaceSlug, id: UUID!`	`SavedView`	member	`null` if absent	`saved_views.py saved_view` → `SavedViewService.get_by_id`
`savedViews`	`workspaceSlug, teamId: UUID, first: Int! = 50, after: String`	`SavedViewConnection!`	member	`BAD_USER_INPUT`	`saved_views.py saved_views` → `SavedViewService.list`
`favorites`	`workspaceSlug`	`[Favorite!]!`	member	Per user and per workspace	`saved_views.py favorites` → `FavoriteService.list`
`issueSubscribers`	`workspaceSlug, issueId: UUID!`	`[IssueSubscriber!]!`	member		`subscribers.py issue_subscribers` → `ActivityService.list_subscribers`
`issueViewerIsSubscribed`	`workspaceSlug, issueId: UUID!`	`Boolean!`	member		`subscribers.py issue_viewer_is_subscribed` → `ActivityService.is_subscribed`
`issueTemplates`	`workspaceSlug, teamId: UUID`	`[IssueTemplate!]!`	member		`templates.py issue_templates` → `TemplateService.list`
`issueTemplate`	`workspaceSlug, id: UUID!`	`IssueTemplate`	member	`null` if absent	`templates.py issue_template` → `TemplateService.get`

Triage, releases, analytics

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`triageIssues`	`workspaceSlug, teamId: UUID!, first: Int! = 25, after: String`	`TriageIssueConnection!`	member	`BAD_USER_INPUT`	`triage.py triage_issues` → `TriageService.queue`
`triageCount`	`workspaceSlug, teamId: UUID!`	`Int!`	member		`triage.py triage_count` → `TriageService.waiting_count`
`environments`	`workspaceSlug`	`[Environment!]!`	member		`releases.py environments` → `ReleaseService.list_environments`
`release`	`workspaceSlug, id: UUID!`	`Release`	member	`null` if absent	`releases.py release` → `ReleaseService.get_by_id`
`releases`	`workspaceSlug, first: Int! = 50, after: String`	`ReleaseConnection!`	member	`BAD_USER_INPUT`	`releases.py releases` → `ReleaseService.list`
`workspaceAnalytics`	`workspaceSlug, days: Int! = 30`	`WorkspaceAnalytics!`	member	`BAD_USER_INPUT` "Invalid analytics window" when `days` is outside 1..180	`analytics.py workspace_analytics` → `AnalyticsService.overview`

7.4 Mutations (95 fields)

Fields are enumerated from `type Mutation` in `frontend/schema.graphql`. Unless noted, each field takes one `input: <Name>Input!` argument carrying `workspaceSlug: String!`, and returns `<Name>Payload!` with the shape `{ <result>, errors: [ValidationErrorType!]! }`. Resolver paths are relative to `app/graphql/mutations/`.

Auth and workspace administration

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`register`	`input: RegisterInput!` (`email`, `password`, `name`)	`RegisterPayload!`	anon	VE (`EMAIL_TAKEN`, `TOO_MANY_ATTEMPTS`, …). Sets the session cookie	`auth.py register` → `AuthService.register`
`login`	`input: LoginInput!` (`email`, `password`)	`LoginPayload!`	anon	`AuthenticationError` becomes a single `INVALID_CREDENTIALS` issue. Sets the cookie	`auth.py login` → `AuthService.log_in`
`logout`	—	`LogoutPayload!` (`signedOut`)	anon	Clears the cookie. No-op without a session	`auth.py logout` → `AuthService.log_out`
`workspaceCreate`	`input: WorkspaceCreateInput!`	`WorkspacePayload!`	auth	VE (`SLUG_TAKEN`, …). The caller becomes owner	`memberships.py workspace_create` → `MembershipService.create_workspace`
`teamCreate`	`input: TeamCreateInput!`	`TeamPayload!`	admin	VE (`KEY_TAKEN`, …)	`memberships.py team_create` → `TeamService.create`
`teamEstimateScaleSet`	`input: TeamEstimateScaleSetInput!`	`TeamPayload!`	admin	VE. `TeamNotFoundError` becomes `NOT_FOUND` on field `teamId`	`memberships.py team_estimate_scale_set` → `TeamService.set_estimate_scale`

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`memberRoleUpdate`	`input: MemberRoleUpdateInput!`	`WorkspaceMemberPayload!`	admin (+owner rules)	VE (`LAST_OWNER`, …)	`memberships.py` `member_role_update` → `MembershipService.update_member_role`
`memberRemove`	`input: MemberRemoveInput!`	`MemberRemovePayload!`	admin (+owner rules)	VE (`LAST_OWNER`, `STILL_LEADS_PROJECT`, `CONNECTED_GITHUB`, …)	`memberships.py` `member_remove` → `MembershipService.remove_member`
`invitationCreate`	`input: InvitationCreateInput!`	`InvitationCreatePayload!`	admin (inviting as owner requires owner)	VE. The raw token is returned once	`memberships.py` `invitation_create` → `MembershipService.create_invitation`
`invitationRevoke`	`input: InvitationRevokeInput!`	`InvitationRevokePayload!`	admin	VE	`memberships.py` `invitation_revoke` → `MembershipService.revoke_invitation`
`invitationAccept`	`input: InvitationAcceptInput!` (token, no slug)	`InvitationAcceptPayload!`	auth	VE	`memberships.py` `invitation_accept` → `MembershipService.accept_invitation`

Issues

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`issueCreate`	`input: IssueCreateInput!`	`IssueCreatePayload!`	member	VE. The creator is the viewer; there is no `creatorId` input	`issues.py` `issue_create` → `IssueService.create`
`issueUpdate`	`id: UUID!, input: IssueUpdateInput!`	`IssueUpdatePayload!`	member	VE, or `NOT_FOUND` on field `id`. Partial patch via `input.to_patch()`	`issues.py` `issue_update` → `IssueService.update`
`issueArchive`	`workspaceSlug: String!, id: UUID!`	`IssueArchivePayload!`	member	`NOT_FOUND` on field `id` (`ISSUE_NOT_FOUND`). There is no `ValidationError` catch. Archives the issue rather than deleting it	`issues.py` `issue_archive` → `IssueService.archive`
`issueSetCycle`	`input: IssueSetCycleInput!`	`IssueSetCyclePayload!`	member	VE	`cycles.py` `issue_set_cycle` → `IssueService.set_cycle`
`issueSetProject`	`input: IssueSetProjectInput!`	`IssueSetProjectPayload!`	member	VE	`projects.py` `issue_set_project` → `IssueService.set_project`
`issueBulkUpdate`	`input: IssueBulkUpdateInput!`	`IssueBulkPayload!` (`issues: [IssueSummary!]!`, `count`, `errors`)	member	VE. Takes ≤ 100 IDs	`bulk.py` `issue_bulk_update` → `BulkService.update_many`
`issueBulkArchive`	`input: IssueBulkArchiveInput!`	`IssueBulkPayload!`	member	VE	`bulk.py` `issue_bulk_archive` → `BulkService.archive_many`

`IssueCreateInput` and `IssueUpdateInput` fields:

Input	Fields
`IssueCreateInput`	`workspaceSlug: String!`, `teamId: UUID!`, `title: String!`, `description: String`, `priority: Int! = 0`, `assigneeId: UUID`, `estimate: Int`, `dueDate: Date`
`IssueUpdateInput`	`workspaceSlug: String!`, and optionally `title`, `description`, `priority: Int`, `workflowStateId: UUID`, `assigneeId: UUID`, `estimate: Int`, `dueDate: Date`. Omitted fields are left unchanged

Labels

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`labelCreate`	`input: LabelCreateInput!`	`LabelPayload!`	member	VE	`labels.py label_create` → `LabelService.create`
`labelUpdate`	`input: LabelUpdateInput!`	`LabelPayload!`	member	VE	`labels.py label_update` → `LabelService.update`
`labelDelete`	`input: LabelDeleteInput!`	`LabelDeletePayload!`	member	VE	`labels.py label_delete` → `LabelService.delete`
`labelGroupCreate`	`input: LabelGroupCreateInput!`	`LabelGroupPayload!`	member	VE	`labels.py label_group_create` → `LabelService.create_group`
`labelGroupUpdate`	`input: LabelGroupUpdateInput!`	`LabelGroupPayload!`	member	VE	`labels.py label_group_update` → `LabelService.update_group`
`labelGroupDelete`	`input: LabelGroupDeleteInput!`	`LabelGroupDeletePayload!`	member	VE (`GROUP_IN_USE`, …)	`labels.py label_group_delete` → `LabelService.delete_group`
`labelSetGroup`	`input: LabelSetGroupInput!`	`LabelPayload!`	member	VE	`labels.py label_set_group` → `LabelService.set_label_group`
`issueLabelAttach`	`input: IssueLabelInput!`	`IssueLabelPayload!`	member	VE (`EXCLUSIVE_GROUP`, `ALREADY_ATTACHED`, …)	`labels.py issue_label_attach` → `LabelService.attach`, then `IssueService.get_by_id`
`issueLabelDetach`	`input: IssueLabelInput!`	`IssueLabelPayload!`	member	VE	`labels.py issue_label_detach` → `LabelService.detach`, then `IssueService.get_by_id`

Comments, cycles, relations

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`commentCreate`	`input: CommentCreateInput!`	`CommentCreatePayload!`	member	VE. The author is the viewer	`comments.py comment_create` → `CommentService.create`
`commentDelete`	`input: CommentDeleteInput!`	`CommentDeletePayload!`	author	VE `NOT_FOUND` for another user's comment	`comments.py comment_delete` → `CommentService.delete`
`cycleCreate`	`input: CycleCreateInput!`	`CyclePayload!`	member	VE (`OVERLAPPING`, …)	`cycles.py cycle_create` → `CycleService.create`
`cycleUpdate`	`input: CycleUpdateInput!`	`CyclePayload!`	member	VE	`cycles.py cycle_update` → `CycleService.update`
`cycleDelete`	`workspaceSlug: String!, id: UUID!`	`CycleDeletePayload!`	member	VE	`cycles.py cycle_delete` → `CycleService.delete`
`issueSetParent`	`input: IssueSetParentInput!`	`IssueParentPayload!`	member	VE (`CYCLE`, `SELF_PARENT`)	`relations.py issue_set_parent` → `RelationService.set_parent`

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`issueClearParent`	`input: IssueClearParentInput!`	`IssueParentPayload!`	member	VE	`relations.py issue_clear_parent` → `RelationService.clear_parent`
`issueRelationCreate`	`input: IssueRelationCreateInput!`	`IssueRelationCreatePayload!`	member	VE (`DUPLICATE`, `SELF_RELATION`)	`relations.py issue_relation_create` → `RelationService.create_relation`
`issueRelationDelete`	`input: IssueRelationDeleteInput!`	`IssueRelationDeletePayload!`	member	VE	`relations.py issue_relation_delete` → `RelationService.delete_relation`

Projects

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`projectCreate`	`input: ProjectCreateInput!`	`ProjectPayload!`	member	VE	`projects.py project_create` → `ProjectService.create`
`projectUpdate`	`input: ProjectUpdateInput!`	`ProjectPayload!`	member	VE. UNSET-aware patch	`projects.py project_update` → `ProjectService.update`
`projectDelete`	`input: ProjectDeleteInput!`	`ProjectDeletePayload!`	member	VE	`projects.py project_delete` → `ProjectService.delete`
`projectTeamAdd`	`input: ProjectTeamInput!`	`ProjectPayload!`	member	VE	`projects.py project_team_add` → `ProjectService.add_team`
`projectTeamRemove`	`input: ProjectTeamInput!`	`ProjectPayload!`	member	VE	`projects.py project_team_remove` → `ProjectService.remove_team`
`projectMilestoneCreate`	`input: ProjectMilestoneCreateInput!`	`ProjectMilestonePayload!`	member	VE	`projects.py project_milestone_create` → `ProjectService.create_milestone`
`projectMilestoneUpdate`	`input: ProjectMilestoneUpdateInput!`	`ProjectMilestonePayload!`	member	VE	`projects.py project_milestone_update` → `ProjectService.update_milestone`
`projectMilestoneDelete`	`input: ProjectMilestoneDeleteInput!`	`ProjectMilestoneDeletePayload!`	member	VE	`projects.py project_milestone_delete` → `ProjectService.delete_milestone`
`projectUpdatePost`	`input: ProjectUpdatePostInput!`	`ProjectUpdatePayload!`	member	VE. The author is the viewer	`projects.py project_update_post` → `ProjectService.post_update`
`projectDependencyAdd`	`input: ProjectDependencyInput!`	`ProjectDependencyPayload!`	member	VE (`CYCLE`, `SELF_DEPENDENCY`)	`projects.py project_dependency_add` → `ProjectService.add_dependency`
`projectDependencyRemove`	`input: ProjectDependencyInput!`	`ProjectDependencyPayload!`	member	VE	`projects.py project_dependency_remove` → `ProjectService.remove_dependency`

Initiatives

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`initiativeCreate`	`input: InitiativeCreateInput!`	`InitiativePayload!`	member	VE	`initiatives.py initiative_create` → `InitiativeService.create`
`initiativeUpdate`	`input: InitiativeUpdateInput!`	`InitiativePayload!`	member	VE	`initiatives.py initiative_update` → `InitiativeService.update`

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`initiativeDelete`	`input: InitiativeDeleteInput!`	`InitiativeDeletePayload!`	member	VE	`initiatives.py` `initiative_delete` → `InitiativeService.delete`
`initiativeProjectAdd`	`input: InitiativeProjectInput!`	`InitiativePayload!`	member	VE (`ALREADY_ASSOCIATED`)	`initiatives.py` `initiative_project_add` → `InitiativeService.add_project`
`initiativeProjectRemove`	`input: InitiativeProjectInput!`	`InitiativePayload!`	member	VE	`initiatives.py` `initiative_project_remove` → `InitiativeService.remove_project`
`initiativeSetParent`	`input: InitiativeSetParentInput!`	`InitiativePayload!`	member	VE (`CYCLE`, `TOO_DEEP`, `SELF_PARENT`)	`initiatives.py` `initiative_set_parent` → `InitiativeService.set_parent`
`initiativeClearParent`	`input: InitiativeClearParentInput!`	`InitiativePayload!`	member	VE	`initiatives.py` `initiative_clear_parent` → `InitiativeService.clear_parent`
`initiativeUpdatePost`	`input: InitiativeUpdatePostInput!`	`InitiativeUpdatePayload!`	member	VE	`initiatives.py` `initiative_update_post` → `InitiativeService.post_update`

Documents

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`documentCreate`	`input: DocumentCreateInput!`	`DocumentPayload!`	member	VE	`documents.py` `document_create` → `DocumentService.create`
`documentEdit`	`input: DocumentEditInput!`	`DocumentPayload!`	member	VE, including `CONFLICT`	`documents.py` `document_edit` → `DocumentService.edit`
`documentRestore`	`input: DocumentRestoreInput!`	`DocumentPayload!`	member	VE	`documents.py` `document_restore` → `DocumentService.restore`
`documentDelete`	`input: DocumentDeleteInput!`	`DocumentDeletePayload!`	member	VE	`documents.py` `document_delete` → `DocumentService.delete`
`documentCommentCreate`	`input: DocumentCommentCreateInput!`	`DocumentCommentPayload!`	member	VE	`documents.py` `document_comment_create` → `DocumentService.create_comment`
`documentCommentDelete`	`input: DocumentCommentDeleteInput!`	`DocumentCommentDeletePayload!`	author	VE	`documents.py` `document_comment_delete` → `DocumentService.delete_comment`

GitHub and Slack integrations

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`githubDisconnect`	`input: GithubDisconnectInput!`	`GithubIntegration!` (**no payload**)	admin	`GithubRepositoriesInUseError` becomes a top-level `BAD_USER_INPUT`	`github.py` `github_disconnect` → `GithubService.disconnect`
`githubRepositoriesSet`	`input: GithubRepositoriesSetInput!` (`repositoryIds: [ID!]!`)	`GithubIntegrationPayload!`	admin	VE. A non-numeric ID gives `INVALID` and a repository in use gives `IN_USE`, both on field `repositoryIds`	`github.py` `github_repositories_set` → `GithubService.set_tracked_repositories`

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`githubAutomationSet`	`input: GithubAutomationSetInput!`	`GithubIntegrationPayload!`	admin	VE	`github.py github_automation_set` → `GithubService.set_issue_automation`
`slackDisconnect`	`input: SlackDisconnectInput!`	`SlackDisconnectPayload!` (`integration` only, no `errors`)	admin		`slack.py slack_disconnect` → `SlackService.disconnect`
`slackChannelsSync`	`input: SlackChannelsSyncInput!`	`SlackChannelsSyncPayload!` (`channels`, `failure: SlackFailure`)	admin	Failure reported through the `failure` enum, not `errors`	`slack.py slack_channels_sync` → `SlackService.sync_channels`
`slackDefaultChannelSet`	`input: SlackDefaultChannelSetInput!`	`SlackNotificationSettingsPayload!`	admin	VE	`slack.py slack_default_channel_set` → `SlackService.set_default_channel`
`slackNotificationPreferenceSet`	`input: SlackNotificationPreferenceSetInput!`	`SlackNotificationSettingsPayload!`	admin	VE	`slack.py slack_notification_preference_set` → `SlackService.set_notification_preference`
`slackTestNotification`	`input: SlackTestNotificationInput!`	`SlackTestNotificationPayload!` (`delivered`, `failure`)	admin	Failure reported through the `failure` enum	`slack.py slack_test_notification` → `SlackService.send_test_notification`

Notifications, saved views, favorites, subscriptions

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`notificationMarkRead`	`input: NotificationMarkReadInput!`	`NotificationMarkReadPayload!`	member	VE	`notifications.py notification_mark_read` → `ActivityService.mark_read`
`notificationMarkAllRead`	`input: NotificationMarkAllReadInput!`	`NotificationMarkAllReadPayload!`	member	No VE catch	`notifications.py notification_mark_all_read` → `ActivityService.mark_all_read`
`savedViewCreate`	`input: SavedViewCreateInput!`	`SavedViewPayload!`	member	VE	`saved_views.py saved_view_create` → `SavedViewService.create`
`savedViewUpdate`	`input: SavedViewUpdateInput!`	`SavedViewPayload!`	member	VE	`saved_views.py saved_view_update` → `SavedViewService.update`
`savedViewDelete`	`input: SavedViewDeleteInput!`	`SavedViewDeletePayload!`	member	VE	`saved_views.py saved_view_delete` → `SavedViewService.delete`
`favoriteAdd`	`input: FavoriteAddInput!`	`FavoritePayload!`	member	VE (`ALREADY_FAVORITED`)	`saved_views.py FavoriteMutation.favorite_add` → `FavoriteService.add`
`favoriteRemove`	`input: FavoriteRemoveInput!`	`FavoriteDeletePayload!`	member	VE	`saved_views.py favorite_remove` → `FavoriteService.remove`
`favoriteReorder`	`input: FavoriteReorderInput!`	`FavoritePayload!`	member	VE	`saved_views.py favorite_reorder` → `FavoriteService.reorder`

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`issueSubscribe`	`input: IssueSubscriptionInput!`	`IssueSubscriptionPayload!`	member	VE	`subscribers.py` `issue_subscribe` → `ActivityService.subscribe`
`issueUnsubscribe`	`input: IssueSubscriptionInput!`	`IssueSubscriptionPayload!`	member	No VE catch	`subscribers.py` `issue_unsubscribe` → `ActivityService.unsubscribe`

Templates and recurrence

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`issueTemplateCreate`	`input: IssueTemplateCreateInput!`	`IssueTemplateSavePayload!`	member	VE	`templates.py` `issue_template_create` → `TemplateService.create`
`issueTemplateUpdate`	`input: IssueTemplateUpdateInput!`	`IssueTemplateSavePayload!`	member	VE	`templates.py` `issue_template_update` → `TemplateService.update`
`issueTemplateDelete`	`input: IssueTemplateDeleteInput!`	`IssueTemplateDeletePayload!`	member	VE	`templates.py` `issue_template_delete` → `TemplateService.delete`
`issueTemplateRecurrenceSet`	`input: IssueRecurrenceSetInput!` (`frequency`, `startsOn`, `intervalCount`, `weekdays`, `dayOfMonth`, `dueInDays`, …)	`IssueRecurrenceSetPayload!`	member	VE	`templates.py` `issue_template_recurrence_set` → `TemplateService.set_recurrence`
`issueTemplateRecurrenceClear`	`input: IssueRecurrenceClearInput!`	`IssueRecurrenceClearPayload!`	member	No VE catch	`templates.py` `issue_template_recurrence_clear` → `TemplateService.clear_recurrence`
`issueCreateFromTemplate`	`input: IssueCreateFromTemplateInput!`	`IssueCreateFromTemplatePayload!`	member	VE	`templates.py` `issue_create_from_template` → `TemplateService.apply`

Triage

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`triageEnter`	`input: TriageEnterInput!`	`TriagePayload!`	member	VE	`triage.py` `triage_enter` → `TriageService.enter`
`triageAccept`	`input: TriageAcceptInput!`	`TriagePayload!`	member	VE	`triage.py` `triage_accept` → `TriageService.accept`
`triageDecline`	`input: TriageDeclineInput!`	`TriagePayload!`	member	VE (`NO_CANCELED_STATE`)	`triage.py` `triage_decline` → `TriageService.decline`
`triageMarkDuplicate`	`input: TriageMarkDuplicateInput!`	`TriagePayload!`	member	VE (`SELF_DUPLICATE`, `ALREADY_RELATED`)	`triage.py` `triage_mark_duplicate` → `TriageService.mark_duplicate`
`triageChangeTeam`	`input: TriageChangeTeamInput!`	`TriagePayload!`	member	VE	`triage.py` `triage_change_team` → `TriageService.change_team`

Releases and search maintenance

Field	Arguments	Returns	Auth	Notes / errors	Resolver → service
`environmentCreate`	`input: EnvironmentCreateInput!`	`EnvironmentPayload!`	member	VE	`releases.py environment_create` → `ReleaseService.create_environment`
`releaseCreate`	`input: ReleaseCreateInput!` (`repositoryId: ID!`, `commitSha`, …)	`ReleasePayload!`	member	VE. `repositoryId` must parse as a positive integer, else `INVALID`	`releases.py release_create` → `ReleaseService.create`
`releaseStatusSet`	`input: ReleaseStatusSetInput!`	`ReleasePayload!`	member	VE (`INVALID_TRANSITION`)	`releases.py release_status_set` → `ReleaseService.set_status`
`releaseDelete`	`input: ReleaseDeleteInput!`	`ReleaseDeletePayload!`	member	VE	`releases.py release_delete` → `ReleaseService.delete`
`embeddingsRefresh`	`workspaceSlug: String!, limit: Int! = 100`	`Int!` (rows written)	member	Top-level `BAD_USER_INPUT` when `limit` is outside 1..500	`search.py embeddings_refresh` → `SearchService.refresh_embeddings`

7.5 Key object types

All fields below are taken from `frontend/schema.graphql`. The Strawberry classes live in `app/graphql/types/*.py`.

Type	Fields
`Issue`	`id`, `teamId`, `identifier` (e.g. `ENG-42`), `number`, `title`, `description`, `priority: Int!`, `workflowStateId`, `assigneeId`, `creatorId`, `estimate`, `dueDate: Date`, `projectId`, `milestoneId`, `completedAt`, `archivedAt`, `createdAt`, `updatedAt`. Resolved fields: `cycle: Cycle`, `project: Project`, `parent: IssueSummary`, `children(first = 10, after)`, `relations(first = 10, after)`, `activity(first = 10, after)`, `development: GithubDevelopment!`, `labels: [Label!]!`, `comments(first = 20, after): CommentConnection!`
`IssueSummary`	Flat issue fields (adds `cycleId`) with no nested resolvers. Used in bulk payloads, notifications and the relation tree
`Team`	`id`, `key`, `name`, `estimateScale: EstimateScale!`, `createdAt`, `workflowStates: [WorkflowState!]!`
`WorkflowState`	`id`, `name`, `category: WorkflowStateCategory!` (`BACKLOG` / `UNSTARTED` / `STARTED` / `COMPLETED` / `CANCELED`), `position: Int!`, `color`
`Project`	`id`, `name`, `description`, `state: ProjectState!` (`PLANNED` / `STARTED` / `PAUSED` / `COMPLETED` / `CANCELED`), `health: Health`, `targetDate`, `leadId`, `teamIds`, `createdAt`, `updatedAt`, `milestones`, `updates`, `dependencies: ProjectDependencies!`
`Cycle`	`id`, `teamId`, `number`, `name`, `startsAt`, `endsAt`, `createdAt`, `updatedAt`
`Initiative`	`id`, `name`, `description`, `status: InitiativeStatus!`, `health`, `targetDate`, `ownerId`, `parentInitiativeId`, `projectIds`, `childInitiativeIds`, `createdAt`, `updatedAt`, `updates`
`Label` / `LabelGroup`	`id`, `name`, `color`, `groupId`, timestamps / `id`, `name`, `exclusive`, timestamps
`Comment`	`id`, `issueId`, `authorId`, `body`, `editedAt`, `createdAt`, `updatedAt`
`Document`	`id`, `title`, `projectId`, `initiativeId`, `creatorId`, `lastEditedBy`, timestamps, `content: JSON`, `revisions(first = 20, after)`, `comments(first = 20, after)`
`SavedView`	`id`, `name`, `teamId`, `filter: SavedViewFilter!`, `orderField`, `orderDirection`, `layout`, `grouping`, `subgrouping`, `visibility: SavedViewVisibility!`, `createdBy`, timestamps, `issues(first = 50, after): IssueConnection!`
`Notification`	`id`, `actorId`, `issueId`, `kind: NotificationKind!`, `readAt`, `createdAt`, `issue: IssueSummary`

Type	Fields
`Release` / `Environment`	`id`, `name`, `environmentId`, `repositoryId: ID!`, `commitSha`, `previousCommitSha`, `status: ReleaseStatus!`, `notes`, `deployedAt`, timestamps, `issueIds`, `pullRequestNumbers` / `id`, `name`, `kind`, `createdAt`
`User` / `WorkspaceMembership`	`id`, `email`, `name`, `createdAt`, `updatedAt` / `workspace: Workspace!`, `role: WorkspaceRole!` (`MEMBER` / `ADMIN` / `OWNER`), `createdAt`
`GithubIntegration` / `SlackIntegration`	`status`, `accountLogin`, `connectedAt`, `connectedById`, `repositories`, `automations` / `status`, `teamName`, `scopes`
`WorkspaceAnalytics`	`rangeStart`, `rangeEnd`, `days`, `throughput`, `totals`, `leadTime`, `cycleTime`, `issueAge`, `stateMix`, `priorityMix`, `workload`, `assigneeTotal`, `teams`, `teamTotal`, `projects`, `projectTotal`, `cycles`, `cycleTotal`, `overdue`

The schema defines 4 custom scalars: `UUID`, `DateTime`, `Date` and `JSON`.

7.6 REST and non-GraphQL routes

Routes are registered in this order (`app/main.py` → `create_app`):

1. Body-limit and security-header middleware
2. `/graphql`
3. The health router
4. The GitHub router, mounted twice: with no prefix and at `/integrations`
5. The Slack router, mounted twice: with no prefix and at `/integrations`
6. `add_spa(app)`, last

The integration routers are mounted twice because the provider-registered URLs use `/integrations/...` while older URLs remain live. Both mounts share the same handlers.

Method	Path	Auth	Request	Response / status codes	Purpose	Handler
GET	`/healthz`	none	—	200 `{"status":"ok"}`	Liveness. No database I/O	`app/rest/health.py` → `healthz`
GET	`/readyz`	none	—	200 `{"status":"ready"}`, or 503 `{"detail":"Service unavailable"}`	Readiness: `SELECT 1` through the pool. Answer bounded by `READINESS_TIMEOUT_SECONDS = 2.0`, connection acquire by 1.0 s. Concurrent callers share one probe	`app/rest/health.py` → `readyz`, `_reach_database`
GET	`/github/install`, `/integrations/github/install`	session cookie + admin	`?workspace=<slug>&return_to=<url>`	302 to `https://github.com/login/oauth/authorize?...`, setting cookie `vector_github_state` (HttpOnly, Lax, Max-Age 600). 302 to the callback origin when the host differs. 401 with no session; 404 when not an admin or when unconfigured; 422 when `workspace` is missing	Start the GitHub App OAuth flow	`app/rest/github.py` → `install`; `app/rest/oauth_origin.py` → `redirect_to_callback_origin`

Method	Path	Auth	Request	Response / status codes	Purpose	Handler
GET	`/github/callback`, `/github/oauth/callback` (and the `/integrations` variants)	state cookie bound to the same session + admin	`?state&installation_id&code`	302 to the allowlisted `return_to`, or 200 plain text "GitHub install received...". 400 "Invalid state" or an installation-resolution message. 401/404 auth. 409 when the installation is claimed elsewhere or its repositories are in use. The state cookie is always cleared	OAuth/installation callback that records a claim on the installation	`app/rest/github.py` → `callback`
POST	`/github/webhook`, `/integrations/github/webhook`	HMAC signature	Raw JSON body; headers `X-Hub-Signature-256`, `X-GitHub-Event`, `X-GitHub-Delivery`	204, including for unhandled events. 401 "Unauthorized" on a bad or missing signature or an unconfigured secret. 400 "Malformed payload". 413 above 256 KiB	Apply `installation`, `installation_repositories`, `pull_request` and `push` events; idempotent per delivery ID	`app/rest/github.py` → `webhook`; `app/services/github.py` → `verify_webhook_signature`, `GithubService.apply_webhook`
GET	`/slack/oauth/start`, `/integrations/slack/oauth/start`	session + admin	`?workspace=<slug>&return_to=<template>`	303 to Slack's authorize URL, setting cookie `vector_slack_oauth` (HttpOnly, Lax, Max-Age 600). 401 "Authentication required"; 404 "Workspace not found"; 503 "Slack is not configured"	Start the Slack OAuth flow. `return_to` is restricted to `/{slug}/settings` or `/{slug}/issues`	`app/rest/slack.py` → `slack_oauth_start`
GET	`/slack/oauth/callback`, `/integrations/slack/oauth/callback`	state cookie + same session + admin	`?state&code&error`	303 to the return path. 400 "Invalid OAuth state", or 400 when Slack rejects the code. 409 "That Slack workspace is already connected". 503 when unconfigured	Exchange the code and connect (`SlackService.connect`)	`app/rest/slack.py` → `slack_oauth_callback`
POST	`/slack/events`, `/integrations/slack/events`	Slack signing secret	Raw body; headers `X-Slack-Request-Timestamp`, `X-Slack-Signature`	200 `{"challenge": ...}` for `url_verification`, otherwise 200 `{"ok": true}`. 401 "Invalid request signature". 400 when malformed. 503 when no signing secret is set	Events API receiver. Verified events are de-duplicated and acknowledged; there is no event processing yet	`app/rest/slack.py` → `slack_events`, `verify_slack_signature`, `_handle_event_callback`
GET	`/assets/*`	none	—	Static files with `Cache-Control: public, max-age=31536000, immutable`	Hashed frontend bundle. Mounted only if `static/index.html` and `static/assets` both exist	`app/spa.py` → `add_spa` (`_HashedAssets`)
GET	`/{spa_path:path}`	none	—	200 `index.html`, `Cache-Control: no-store`, SPA CSP	Client-side routing fallback. Registered only if `static/index.html` exists; hidden from OpenAPI	`app/spa.py` → `add_spa`
GET	`/docs`, `/docs/oauth2-redirect`, `/redoc`, `/openapi.json`	none	—	200	FastAPI defaults. `FastAPI(title="Vector", lifespan=lifespan)` does not override `docs_url`, `redoc_url` or `openapi_url`, so these are served in every environment	`app/main.py` → `create_app`

Webhook signature verification

- **GitHub.**
  - The raw body is verified before it is parsed.
  - Expected header value: `"sha256=" + HMAC-SHA256(webhook_secret, raw_body).hexdigest()`, compared with `hmac.compare_digest`.
  - A missing secret, a missing header and a wrong digest all return the same 401.
  - (`app/services/github.py` → `verify_webhook_signature`; `app/rest/github.py` → `webhook`)
- **Slack.**
  - The timestamp must be an integer within `MAX_TIMESTAMP_AGE_SECONDS = 300` of the current time.
  - Expected signature: `"v0=" + HMAC-SHA256(signing_secret, b"v0:" + timestamp + b":" + raw_body).hexdigest()`, compared in constant time.
  - (`app/rest/slack.py` → `verify_slack_signature`)
- **OAuth state.** Both OAuth flows bind the state cookie to a digest of the session token, and reject a callback that arrives under a different session (`app/rest/github.py` → `callback`; `app/rest/slack.py` → `slack_oauth_callback`).

Security headers. These apply to every response (`app/http_headers.py` → `add_security_headers`):

- `Content-Security-Policy`: `default-src 'none'; frame-ancestors 'none'` for the API, with a separate policy for GraphiQL HTML.
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Permissions-Policy`
- `Strict-Transport-Security: max-age=31536000`, sent only over HTTPS.

7.7 Counts

Item	Count	Source
Query root fields	42	`frontend/schema.graphql` `type Query`
Mutation root fields	95	`frontend/schema.graphql` `type Mutation`
Contributing resolver classes	19 query / 21 mutation	`app/graphql/schema.py` `QUERY_TYPES` / `MUTATION_TYPES`
Object types (excluding `Query` / `Mutation`)	138	`^type` in SDL (140 − 2)
Input types / enums / custom scalars	89 / 24 / 4	SDL
Unions / interfaces / subscriptions	0 / 0 / 0	SDL
Connection types	15	`^type \w+Connection` in SDL
Per-request DataLoaders	10	`app/graphql/context.py`
REST handler functions	8, plus the SPA index handler	`app/rest/health.py`, `github.py`, `slack.py`; `app/spa.py`
Distinct REST method+path routes	16 (2 health + 4 GitHub paths × 2 prefixes + 3 Slack paths × 2 prefixes), plus the conditional `/assets` mount and SPA catch-all, plus 4 FastAPI docs routes	`app/main.py`

7.8 Gaps / to verify

Gaps / to verify:

- **API docs exposed in production.** `/docs`, `/redoc` and `/openapi.json` are not disabled in `create_app`. They list the REST routes only. Decide whether this is intended.
- **Inconsistent mutation result shapes.** `githubDisconnect` returns a bare `GithubIntegration!`. `slackDisconnect`, `slackChannelsSync` and `slackTestNotification` payloads have no `errors` field. `embeddingsRefresh` returns `Int!` and reports errors at the top level.
- **Resolvers with no `ValidationError` catch.** These would surface as "Internal server error" if their service ever raised one: `issueArchive`, `issueUnsubscribe`, `notificationMarkAllRead`, `issueTemplateRecurrenceClear`, the Slack disconnect/sync/test mutations, and several list queries (`teams`, `cycles`, `labelGroups`, `favorites`, `triageCount`, …). A textual scan found no direct raise in the corresponding service methods; helper calls were not traced.
- **Unbatched nested fields (N+1).** `Issue.children`, `.relations`, `.activity`, `.comments`, `.parent` and `.development`, `IssueConnection.totalCount`, `Document.revisions` and `.comments`, and `SavedView.issues` call a service once per parent. Fan-out is bounded only by `MAX_COMPLEXITY`.
- **Narrow role model.** Only membership, team and integration operations are gated to admins. Destructive operations — `projectDelete`, `labelDelete`, `initiativeDelete`, `releaseDelete`, `documentDelete`, `cycleDelete`, `issueBulkArchive`, `savedViewDelete` — and `embeddingsRefresh` are open to any member.
- **Saved-view visibility not verified.** Whether `savedView` and `savedViews` hide other users' `PERSONAL` views is decided in `SavedViewService` and the repository; this was not traced.
- **Slack events are a stub.** Verified events are claimed and acknowledged, but not processed (`app/rest/slack.py` → `_handle_event_callback`). There are no slash-command or interactivity routes.
- **Slack OAuth state cookie left in place on auth failure.** When the auth check in `slack_oauth_callback` returns 401/404, `vector_slack_oauth` is not cleared until its Max-Age expires. The GitHub callback always clears its cookie.
- **GitHub install target needs confirming.** `install` redirects to `https://github.com/login/oauth/authorize` (`GITHUB_AUTHORIZE_URL`) rather than to the App installation page. How `installation_id` arrives depends on GitHub App settings that are not in the repo.
- **SPA catch-all returns 200 for any unknown GET path** once a bundle is present. A mistyped probe path therefore looks healthy.
- **CSRF relies on two mechanisms only:** `SameSite=Lax` cookies, and GraphQL accepting only JSON `POST` bodies. There is no CSRF token (`app/http_cookies.py`).
- **Stale references in code.**
  - `app/graphql/schema.py` cites `tests/test_root_composition.py`, which does not exist.
  - The `github_scope` docstring mentions `info.context.tenant`, which does not exist on `VectorContext`.
  - `app/graphql/viewer.py` → `actor_user_id` has no callers.
- **Error-code examples are not exhaustive.** The codes in 7.1 and 7.4 come from a literal grep of `code="..."`, not a per-mutation trace. Codes built dynamically may be missing.

8. Authentication and authorization

Users sign in with an email and password. The server keeps sessions in the database and hands the browser an opaque token in an HttpOnly cookie. Authorization is per workspace: there are three membership roles, and every workspace-scoped GraphQL field and REST route checks them.

Authentication only establishes identity. In the words of the `AuthService` docstring (`app/services/auth.py`), "holding a UserEntity from this service grants access to nothing". Membership is a separate check.

8.1 Identity model

Concept	Storage	Key facts
User	`users` (`migrations/003_auth.sql`)	`id` defaults to `uuidv7()`. `email` must be stored lower-case (`users_email_lowercase CHECK (email = lower(email))`). Further constraints: `users_email_key UNIQUE`, `users_email_shape` (regex `^[^@[:space:]]+@[^@[:space:]]+$`) and `users_email_length` (3–320 characters). `password_hash` must match `LIKE '$argon2id$%'` (`users_password_hash_argon2id`).
Session	`sessions` (`003_auth.sql`)	`token_hash BYTEA`, which is `UNIQUE` and must be exactly 32 bytes (`CHECK (octet_length(token_hash) = 32)`). Other columns: `created_at`, `last_used_at`, `expires_at NOT NULL`. The `user_id` FK `sessions_user_fk` is `ON DELETE CASCADE`.
Workspace	`workspaces` (`002_tenancy.sql`)	Addressed by a unique `slug` (`workspaces_slug_key`).
Membership	`workspace_members` (`004_membership.sql`, `026_member_departure.sql`)	Primary key `(workspace_id, user_id)`. `role` is one of `member`, `admin` or `owner` (`workspace_members_role_check`). The FKs to `workspaces` and `users` are `ON DELETE RESTRICT`. Since migration 026, `removed_at TIMESTAMPTZ` marks a membership as ended without deleting it (`workspace_members_removed_after_created`).
Invitation	`workspace_invitations` (`004_membership.sql`)	Columns include `role` (same check as memberships), `token_hash` (`UNIQUE`, `CHECK ~ '^[a-f0-9]{64}$'`), `accepted_at` and `expires_at`. A check requires `expires_at > created_at`.

- **Roles in code** (`app/domain/tenancy.py`):
  - `WORKSPACE_ROLES = ("member", "admin", "owner")`
  - `WORKSPACE_ADMIN_ROLES = frozenset({"admin", "owner"})`
  - `WORKSPACE_OWNER_ROLE = "owner"`
- **GraphQL enum:** `WorkspaceRole {MEMBER, ADMIN, OWNER}` (`app/graphql/types/membership.py`).
- **No other roles.** There is no guest or viewer role.
- **Multiple workspaces.** A user can belong to many workspaces. The session does not carry a "current workspace" (see 8.5).
- **Password hash is never exposed:**
  - `UserType` exposes only `id, email, name, createdAt, updatedAt` (`app/graphql/types/auth.py`).
  - `UserEntity` has no `password_hash` field (`app/domain/auth.py`).
  - The only query that reads the hash is `UserRepository.find_credentials_by_email` (`app/repositories/users.py`).

8.2 Sign-up, login, logout and session lifecycle

GraphQL entry points (`app/graphql/mutations/auth.py` → `AuthMutation`; `app/graphql/queries/auth.py` → `AuthQuery`):

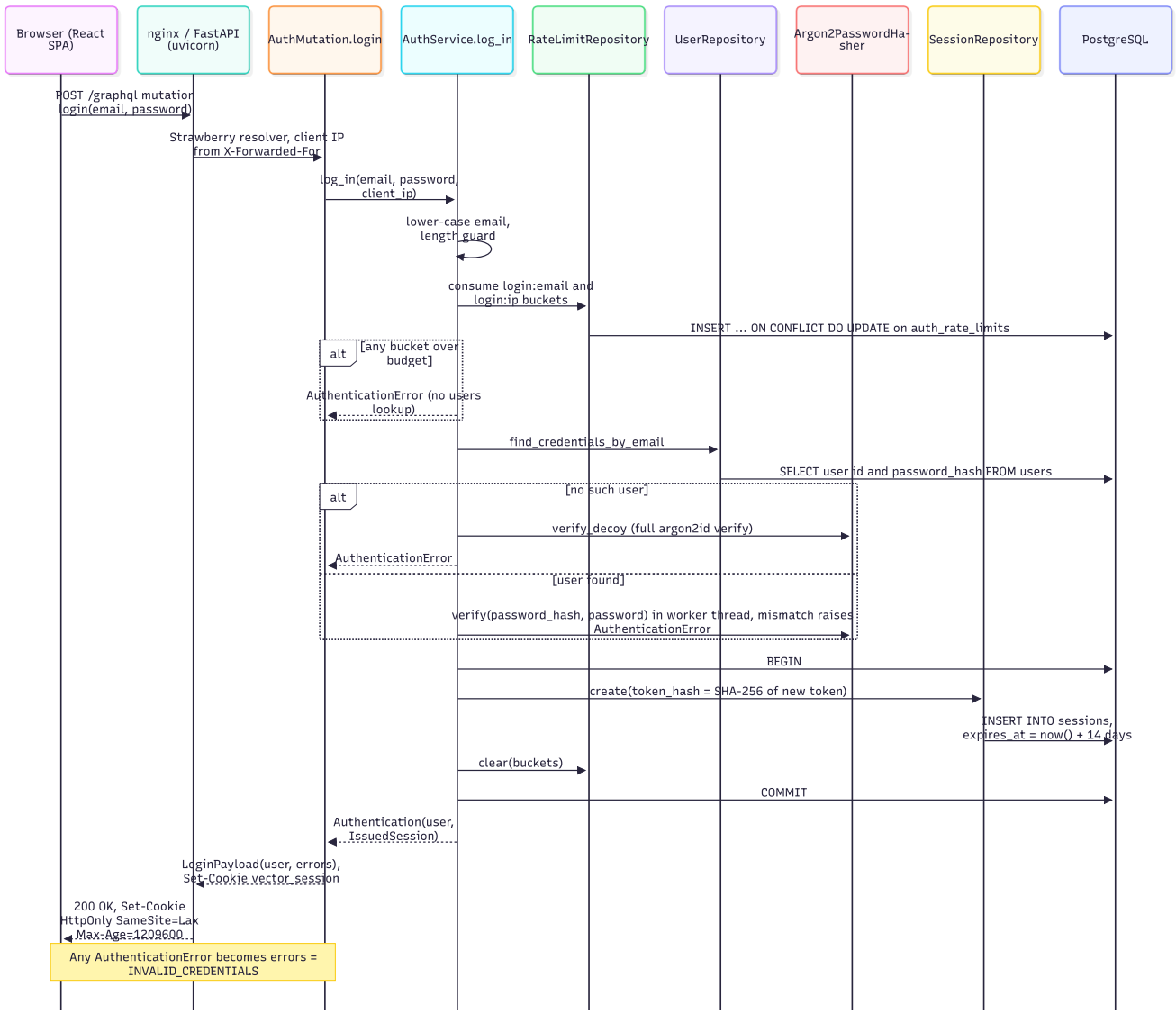
Operation	Service call	Result
`register(input: {email, password, name})`	`AuthService.register`	`RegisterPayload{user, errors}`, plus `Set-Cookie` on success

Operation	Service call	Result
`login(input: {email, password})`	`AuthService.log_in`	`LoginPayload{user, errors}`, plus `Set-Cookie` on success
`logout`	`AuthService.log_out`	`LogoutPayload{signedOut, errors}`, plus a cookie-clearing `Set-Cookie`
`me`	`VectorContext.viewer()` → `AuthService.authenticate`	`User`, or `null` when nobody is signed in (not an error)

Registration (`AuthService.register`)

1. **Normalise the email.** It is lower-cased with `email.lower()` and nothing else. Surrounding whitespace is deliberately left in place (`_normalized_email`).
2. **Validate.** `_validate_registration` collects every problem and raises them together. Validation runs before rate limiting, so typos do not use up the budget.
    - **email:** required, at most 320 characters (`EMAIL_MAX_LENGTH`), and must match `EMAIL_PATTERN = ^[^@\s]+@[^@\s]+$`.
    - **password:** 8–1024 characters (`PASSWORD_MIN_LENGTH`, `PASSWORD_MAX_LENGTH`), with no composition rules.
    - **name:** at most 200 characters (`NAME_MAX_LENGTH`).
3. **Rate-limit.** Consume the `register:ip` bucket, outside any transaction. If the caller is over budget, raise `ValidationError([TOO_MANY_ATTEMPTS])` (field `email`, message "Too many attempts. Try again later.").
4. **Hash the password** with argon2id. This happens after the rate-limit connection is released and before the connection for the write transaction is taken from the pool.
5. **Create the user and session** in one transaction: `UserRepository.create`, then `_issue_session`.
    - A `users_email_key` violation becomes `EMAIL_TAKEN`, raised `from None`.
    - Revealing that an address is already registered is an accepted way to discover accounts. The per-IP budget slows it down (`register` docstring).
6. **Sign the user in.** The resolver's `_sign_in` writes the session cookie straight away.

Login (`AuthService.log_in`)



1. **Normalise and guard.** The email is lower-cased. If the email is over 320 characters or the password over 1024, the service raises `AuthenticationError` at once, without hashing. This early exit depends only on input size, so it reveals nothing about accounts (docstring).
2. **Consume rate-limit buckets.** On one connection, outside a transaction, consume `login:email` (always) and `login:ip` (only when the client IP is known) (`_login_buckets`). If any bucket is over budget, raise `AuthenticationError` before touching `users`.
3. **Look up the account** with `find_credentials_by_email`.
    - **Unknown address:** run `verify_decoy` (a full argon2 verification against a cached decoy hash), then raise `AuthenticationError`.
    - **Wrong password:** raise `AuthenticationError`.
4. **Succeed in one transaction.** `get_by_id`, then `_issue_session`, then `RateLimitRepository.clear(buckets)`. A correct password resets the attempt counters.
5. **Report failure.** The resolver turns every `AuthenticationError` into one payload error, `INVALID_CREDENTIALS` (field `credentials`, message "Email or password is incorrect").
    - A rate-limited login, an unknown email and a wrong password therefore return identical content.
    - A rate-limited refusal is faster, because it skips argon2. The `log_in` docstring accepts this.
    - Login performs no email-format validation, by design.

Password hashing (`app/services/passwords.py`)

Aspect	Implementation
Algorithm and library	argon2id via `argon2.PasswordHasher()` from argon2-cffi (pinned to 25.1.0 in `requirements.txt`), constructed with no arguments (`Argon2PasswordHasher`)
Parameters	The library defaults, deliberately not restated in code. The module docstring describes them as RFC 9106 low-memory: 64 MiB memory, 3 passes, 4 lanes, 16-byte salt, 32-byte tag. Each encoded hash stores the parameters it was made with.
Off the event loop	Every `hash` and `verify` call runs in `asyncio.to_thread`
Concurrency cap	`MAX_CONCURRENT_HASHES = 4`, enforced by one `asyncio.Semaphore` per event loop (`_gate`, kept in the `_gates` `WeakKeyDictionary`). A comment derives the number from the memory settings in `k8s/30-api.yaml`: (512 MiB limit − 192 MiB request) / 64 MiB, rounded down to leave slack.
Mismatch handling	Only `VerifyMismatchError` counts as a wrong password. Any other argon2 error, such as a corrupt stored hash, propagates.
Rehash	`check_needs_rehash` is never called, so stored hashes are not upgraded at login
Decoy and equal timing	`_decoy_hash()` (cached with `functools.cache`) hashes a random `secrets.token_urlsafe(DECOY_PASSWORD_BYTES = 32)`. `verify_decoy` runs a real verification against it, so an unknown email costs as much time as a wrong password. `warm_password_hashing()` builds the decoy at startup (`app/main.py` → `lifespan`).
Database backstop	`users_password_hash_argon2id CHECK (password_hash LIKE '$argon2id$%')` (`003_auth.sql`)

Session tokens (`app/services/tokens.py`, `app/repositories/sessions.py`)

- **Generation:** `generate_session_token()` returns `secrets.token_urlsafe(TOKEN_ENTROPY_BYTES = 32)`: 256 bits, encoded as 43 URL-safe characters.
- **Storage:** only `hash_session_token(token)` is stored. It is a raw 32-byte, unsalted SHA-256 digest (`TOKEN_HASH_BYTES = 32`) in `sessions.token_hash`. Every `SessionRepository` method takes this hash, never the plaintext.
- **Opaque, not a JWT:** revoking a session means deleting its row.
- **Plaintext handling:** the plaintext token exists only in `IssuedSession.token` (`app/domain/auth.py`), on its way from `AuthService._issue_session` to `_sign_in`. It is written to the `Set-Cookie` header and nowhere else: not the GraphQL payload, not the logs (`_sign_in` docstring).

Session cookie (`app/http_cookies.py`)

Attribute	Value	Source
Name	`vector_session`	`SESSION_COOKIE_NAME`. It has no `__Host-` prefix, because that prefix requires `Secure`, so the cookie name would have to differ between HTTP development and HTTPS production (comment).
`HttpOnly`	always	`set_session_cookie`
`Secure`	only when `ENVIRONMENT == "production"`	`is_secure_environment`
`SameSite`	`lax`	`SESSION_COOKIE_SAMESITE`
`Path`	`/`	`SESSION_COOKIE_PATH`
`Domain`	not set (host-only)	`set_session_cookie`
`Max-Age`	1,209,600 s (14 days)	`_sign_in` in `mutations/auth.py`, computed as `int(session_lifetime.total_seconds())`
`Expires`	not used, so expiry does not depend on the browser's clock	`set_session_cookie` docstring

- **Reading:** `read_session_token` reads the cookie from any `HTTPConnection`.
- **Clearing:** `clear_session_cookie` calls `delete_cookie` with the same path and flags as the original, so the browser recognises it as the same cookie.
- **Missing response object:** if the GraphQL context has no response object, `_response` raises `RuntimeError` instead of reporting a sign-in that set no cookie. The error is masked before it reaches the client.

Lifetime, validation, expiry and cleanup

Item	Value or behaviour	Source
Absolute lifetime	`SESSION_LIFETIME = timedelta(days=14)`. The database computes `expires_at = now() + $3::interval`, so the database clock decides expiry.	`services/auth.py`, `SessionRepository.create`
Per-request check	`UPDATE sessions SET last_used_at = now() WHERE token_hash = $1 AND expires_at > now() RETURNING ...`, followed by `UserRepository.get_by_id`. A missing, unknown or expired token returns `None`, as does a token whose user has been deleted.	`SessionRepository.touch_valid`, `AuthService.authenticate`
Memoisation	The viewer is resolved lazily, at most once per request, and cached as an `asyncio.Future`. Anonymous operations never look up a session.	`app/graphql/context.py` → `VectorContext.viewer`
Idle timeout, sliding expiry, rotation	None. `last_used_at` is recorded but nothing reads it, and nothing extends `expires_at`.	`SESSION_LIFETIME` comment, `touch_valid`
Cleanup sweep	Runs every `SWEEP_INTERVAL_SECONDS = 900.0`, deleting in batches of `SWEEP_BATCH = 1000` with `DELETE ... WHERE id IN (SELECT id ... WHERE expires_at <= now() LIMIT $1)`. The same pass deletes stale rate-limit rows. It is started as a background task in `lifespan` and only reclaims space; correctness never depends on it.	`AuthService.sweep_once`, `run_sweep_loop`, `app/main.py`
Logout	Runs `DELETE FROM sessions WHERE token_hash = $1`, then clears the cookie, in that order. Returns `signedOut: true` whether or not a session existed.	`AuthService.log_out`, `AuthMutation.logout`

8.3 Workspace creation, invitations and joining

The resolvers are in `app/graphql/mutations/memberships.py` and `app/graphql/queries/memberships.py`. The logic is in `app/services/memberships.py` → `MembershipService`.

Operation	Who may call it	Behaviour
`workspaceCreate`	any authenticated user	Validates the name (required, at most 200 characters) and the slug (at most 63 characters, matching `SLUG_PATTERN = ^[a-z0-9]([a-z0-9-]*[a-z0-9])?$`). In one transaction it inserts the workspace and an `owner` membership for the caller. A `workspaces_slug_key` violation becomes `SLUG_TAKEN`. There is no limit on workspaces per user.
`invitationCreate`	admin or owner. Only an owner may invite someone as `owner` (`_validate_invitation`).	Validates the email with the registration rules and stores it as typed. Generates a `token_urlsafe(32)` token and stores only its SHA-256 lowercase hex digest. Invitations last `INVITATION_LIFETIME = timedelta(days=7)`. The raw token is returned once, in `InvitationCreatePayload.token`, because the product does not send email.
`invitations` (list)	admin or owner	Pending, unexpired invitations, at most `INVITATION_LIST_LIMIT = 200`
`invitationRevoke`	admin or owner	Deletes by `(workspace_id, id)`. A miss returns `INVITATION_NOT_FOUND`.

Operation	Who may call it	Behaviour
`invitationAccept(token)`	any authenticated user holding the token	Claims the invitation and creates the membership; steps below

`invitationAccept` runs in one transaction:

1. **Claim the invitation.** `InvitationRepository.claim` runs `UPDATE ... SET accepted_at = now() WHERE token_hash = $1 AND accepted_at IS NULL AND expires_at > now()`. The row lock means only one concurrent caller can claim it.
2. **Create the membership** with the invitation's role. If the user is a former member, `ON CONFLICT ON CONSTRAINT workspace_members_pkey DO UPDATE ... WHERE workspace_members.removed_at IS NOT NULL` revives their old row (`MembershipRepository.create`).
3. **Failures:**
    - An unknown, revoked, already-used or expired token all return the same `INVALID_INVITATION` issue (field `token`, code `INVALID`).
    - A caller who is already a current member gets `ALREADY_MEMBER`. The transaction rolls back, so the invitation stays usable.

The invitation's email address is not compared with the accepting account. Whoever holds the token can use it.

The frontend accepts invitations at `/invite/:token` (`frontend/src/features/onboarding/lib/progress.ts`, `AcceptInvitePage.tsx`).

8.4 Rate limiting

The code is in `app/services/auth.py` and `app/repositories/rate_limits.py`, and the table is created in `migrations/032_auth_hardening.sql`. Counters live in the database, so the limits hold across all API replicas.

Storage. Table `auth_rate_limits`:

- Columns: `scope TEXT`, `subject TEXT`, `window_started_at TIMESTAMPTZ`, `attempts INTEGER`.
- Primary key: `(scope, subject)`.
- Checks: `attempts > 0`, `length(scope)` 1–64, `length(subject)` 1–128.
- There is no tenant column.

Algorithm. A fixed window of `RATE_LIMIT_WINDOW = timedelta(minutes=15)`.

- `consume` is a single `INSERT ... SELECT ... ON CONFLICT (scope, subject) DO UPDATE` covering all buckets for the attempt.
- If `window_started_at <= now() - window`, the counter restarts at 1 with a new window start. Otherwise it increments.
- `_over_budget` refuses when `count > budget` after the increment, so a budget of N allows N attempts.
- Consumption happens outside any transaction, so failed attempts are never refunded.

Scope	Subject (key)	Budget per 15 min	Used by
`login:email`	SHA-256 hex of the lower-cased email (`_subject_for_email`)	10	`log_in`, always
`login:ip`	canonical client IP (`read_client_ip`)	100	`log_in`, only when the IP is known
`register:ip`	canonical client IP	20	`register`, only when the IP is known. With no IP, registration has no budget at all (`_register_buckets`).

- **Reset:** a successful login deletes its buckets (`RateLimitRepository.clear`). Stale rows are removed by `sweep_once`.
- **Accepted trade-offs** (documented in comments):
    - Anyone can use up a victim's `login:email` budget and lock that address out for the rest of the window.
    - There is no unlock path and no notification.
    - Fixed windows allow up to twice the budget across a window boundary.

- **Second line of defence:** the per-process argon2 semaphore (8.2).
- **Coverage:** nothing else consumes buckets, including `logout`, `invitationAccept`, `workspaceCreate` and all workspace-scoped operations. `consume(` is called only in `services/auth.py`.

8.5 Authorization

Resolving the viewer and workspace scope per request

- **No implicit workspace.** `VectorContext` (`app/graphql/context.py`) holds the request, the response and the lazily resolved viewer, but no workspace.
- **Explicit scope per field.** Every workspace-scoped field takes a `workspaceSlug`/`slug` argument and starts with `authorized_scope(info, slug)` (`app/graphql/scope.py`):
    1. `viewer_user_id(info)` (`app/graphql/viewer.py`) raises `GraphQLError("Authentication required", extensions={"code": "UNAUTHENTICATED"})` when there is no valid session. This happens before any workspace lookup, so anonymous callers cannot probe which slugs exist.
    2. `MembershipService.authorized_scope_for_slug` calls `MembershipRepository.find_membership`, which runs one statement: `workspace_members JOIN workspaces ... WHERE workspaces.slug = $1 AND workspace_members.user_id = $2 AND workspace_members.removed_at IS NULL`.
    3. A match yields `AuthorizedWorkspaceScope(workspace_id, user_id, role)` (`app/domain/tenancy.py`). This frozen dataclass is only ever built from a membership row. Services receive it, and `user_id` always comes from the session, never from client input.
- **Variants** that add a role check or translate errors differently:
    - `mutations/memberships.py` → `_scope`
    - `queries/github.py` → `github_scope`
    - `queries/slack.py` → `slack_admin_scope`
    - `search`, which calls `viewer_user_id` and then `authorized_scope_for_slug` directly
- **REST integration routes** (`app/rest/github.py`, `app/rest/slack.py`) authenticate with `AuthService.authenticate(read_session_token(request))` and then require admin.

Roles and permitted operations

Operation(s)	member	admin	owner	Enforced in
All workspace-scoped reads and writes: issues, labels, cycles, projects, initiatives, documents, comments, relations, releases, saved views, favourites, templates, triage, bulk, subscribers, notifications, search, analytics, teams list, `workspaceMembers`	✓	✓	✓	`authorized_scope`
`teamCreate`, `teamEstimateScaleSet`	—	✓	✓	`services/teams.py` → `require_workspace_admin`
`invitationCreate` (member/admin role), `invitations`, `invitationRevoke`	—	✓	✓	`MembershipService.create_invitation`, `list_invitations`, `revoke_invitation`
`invitationCreate` with role `owner`	—	—	✓	`_validate_invitation`
`memberRoleUpdate`, `memberRemove` on non-owners	—	✓	✓	`update_member_role`, `remove_member`
Promote to, demote or remove an owner	—	—	✓	`_require_owner_to_touch_owners`. The last owner cannot be demoted or removed (`LAST_OWNER`); owner rows are locked with `FOR UPDATE` in `lock_owner_ids`.
GitHub integration: status, connect, disconnect, tracked repos, issue automation; `GET /github/install`	—	✓	✓	`services/github.py` → `require_workspace_admin`

Operation(s)	member	admin	owner	Enforced in
Slack integration: status, connect, disconnect, channels, notification settings, test message; `GET /slack/oauth/start`	—	✓	✓	`services/slack.py` → `require_admin`
`workspaceCreate`, `invitationAccept`, `myWorkspaces` (`myWorkspace(slug)` also needs a current membership of that workspace, via `membership_for_slug`)	any authenticated user; no membership needed			`viewer_user_id`

Tenant isolation

- **Service level:**
  - Workspace scope only ever comes from `authorized_scope` / `AuthorizedWorkspaceScope`.
  - Repository SQL filters on `workspace_id = $1` (for example `repositories/memberships.py`, `repositories/invitations.py`).
  - Slugs are matched exactly.
  - `tests/test_repository_tenancy_gate.py` acts as a test gate on repository tenancy.
- **Database level: composite foreign keys.**
  - Child rows reference their parents by `(workspace_id, id)`, pointing at `UNIQUE (workspace_id, id)` targets. Example: `issues_team_fk (workspace_id, team_id) → teams (workspace_id, id)` (`002_tenancy.sql`). A row therefore cannot point into another tenant, even if a service bug tried to make it.
  - 17 composite FKs reference `workspace_members (workspace_id, user_id)`, spread across migrations 006, 007, 009, 012, 013, 014, 019, 020, 022 and 023. Any user that workspace data points at (assignee, comment author, project lead, document creator, integration connector, and so on) must be a member of the same workspace.

"Not found" and "forbidden" look the same

- **Missing workspace and non-member.** `find_membership` returns the same empty result for a slug that does not exist and for a workspace the caller is not a member of. `membership_for_slug` raises `WorkspaceAccessDeniedError` in both cases, never `WorkspaceNotFoundError`.
- **Insufficient role.** `require_workspace_admin`, the owner-only rules and the GitHub/Slack admin checks raise the same `WorkspaceAccessDeniedError`.
- **What the client sees.** Resolvers translate all of these into `GraphQLError("Workspace not found", extensions={"code": "NOT_FOUND"})` (`WORKSPACE_NOT_FOUND_MESSAGE`), raised `from None`.
- **Other places with one answer for several causes:**
  - An unknown user id and a non-member's user id both return `MEMBER_NOT_FOUND`.
  - A team in another workspace and a nonexistent team both return the same `teamId NOT_FOUND`.
  - All invitation-token failures return `INVALID_INVITATION`.
  - All session failures return `UNAUTHENTICATED`.

Removing a member (`migrations/026_member_departure.sql`, `MembershipService.remove_member`)

Removal stamps `removed_at` instead of deleting the row. Everything runs in one transaction:

1. **Check the caller and lock the owners.** `require_workspace_admin`, then the owner rows are locked with `FOR UPDATE`.
2. **Apply the owner rules:** only an owner may remove an owner, and the last owner cannot be removed.
3. **Check shared holdings.** `shared_holdings` probes six things the member might still hold. If any are found, the removal is refused with every matching code:

Holding	Error code
Project lead (`projects_lead_fk`)	`STILL_LEADS_PROJECT`

Holding	Error code
Initiative owner (`initiatives_owner_fk`)	`STILL_OWNS_INITIATIVE`
Creator of a shared view (`saved_views_creator_fk`)	`OWNS_SHARED_VIEW`
Template default assignee (`issue_templates_assignee_fk`)	`NAMED_IN_TEMPLATE`
Connected the GitHub integration (`github_installations_connected_by_fk`)	`CONNECTED_GITHUB`
Connected the Slack integration (`slack_installations_connected_by_fk`)	`CONNECTED_SLACK`

4. **Delete personal rows:** favourites, subscriptions, notifications and personal views.
5. **Unassign the member's issues.**
6. **Stamp** `removed_at = now()`.

Afterwards:

- Content the member wrote still shows them as its author.
- Their access ends on their next request, because `find_membership` requires `removed_at IS NULL`.
- Accepting a new invitation revives the same row.

8.6 Security measures

Area	Measure	Source
Security headers on all responses. Set with `setdefault`, so a route can override them.	`Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` on non-HTML responses	`app/http_headers.py` → `API_CONTENT_SECURITY_POLICY`, `add_security_headers`
	On GraphiQL (any `text/html` response the middleware sees): `default-src 'none'; frame-ancestors 'none'; script-src 'self' 'unsafe-inline' https://unpkg.com; style-src 'self' 'unsafe-inline' https://unpkg.com; img-src 'self' data:; font-src 'self' data:; connect-src 'self'`	`GRAPHIQL_CONTENT_SECURITY_POLICY`
	`X-Content-Type-Options: nosniff` and `Referrer-Policy: strict-origin-when-cross-origin`	`add_security_headers`, `REFERRER_POLICY`
	`Permissions-Policy: accelerometer=(), autoplay=(), camera=(), display-capture=(), encrypted-media=(), fullscreen=(self), geolocation=(), gyroscope=(), magnetometer=(), microphone=(), midi=(), payment=(), usb=()`	`PERMISSIONS_POLICY`
	`Strict-Transport-Security: max-age=31536000`, sent only when `request.url.scheme == "https"`, without `includeSubDomains` or `preload`	`add_security_headers`
	No `X-Frame-Options`; `frame-ancestors 'none'` covers framing instead	comment in `http_headers.py`
CSP for the SPA page (`index.html`)	`default-src 'none'; script-src 'self'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' data: https://fonts.gstatic.com; img-src 'self' data: https:; connect-src 'self'; frame-ancestors 'none'; base-uri 'none'; form-action 'self'`. `index.html` is sent with `Cache-Control: no-store`; hashed files under `/assets` get `public, max-age=31536000, immutable`.	`app/spa.py` → `SPA_CONTENT_SECURITY_POLICY`, `_HashedAssets`
Request body limit	`MAX_REQUEST_BODY_BYTES = 256 * 1024` (256 KiB), applied app-wide by Starlette's `RequestBodyLimitMiddleware`	`app/http_limits.py`
GraphQL operation limits	`MAX_DEPTH = 10`, `MAX_ALIASES = 15`, `MAX_SELECTIONS = 2000`, `MAX_COMPLEXITY = 1000`, and `ASSUMED_PAGE_SIZE = 100` when `first`/`last` is a variable. Checked during validation by `OperationLimitsRule`. A list field's cost is multiplied by its page size.	`app/graphql/limits.py`

Area	Measure	Source
GraphQL surface	Queries over GET are disabled (`allow_queries_via_get=False`). In production, GraphiQL is off and introspection is disabled (`DisableIntrospection`).	`app/graphql/router.py`, `app/graphql/schema.py`
Error masking	In every environment, `MaskErrors` and a schema wrapper replace each error with "Internal server error". Two kinds pass through: errors with no original exception (parse and validation errors), and errors whose `extensions.code` is in `PUBLIC_ERROR_CODES = {"BAD_USER_INPUT", "UNAUTHENTICATED", "NOT_FOUND"}`. Expected input errors are returned in `payload.errors[] {field, code, message}`. Constraint errors are re-raised `from None` so database detail is not attached.	`app/graphql/schema.py`, `app/graphql/types/errors.py`
Client IP and proxy trust	`read_client_ip` uses the first rule that applies: (1) if `TRUSTED_PROXY_HOPS > 0`, the Nth `X-Forwarded-For` entry counting from the right; (2) otherwise the TCP peer address, but only when `FORWARDED_ALLOW_IPS` contains no `*` (`peer_address_is_trustworthy`); (3) otherwise `None`. Results are normalised with `ipaddress`, and anything over 45 characters is rejected. `X-Real-IP` is never read. All deployment configs (render.yaml, docker-compose, k8s) set `TRUSTED_PROXY_HOPS=1`.	`app/http_client_ip.py`, `app/config.py`
CSRF	There is no CSRF token. The defences are the `SameSite=Lax` session cookie, POST-only GraphQL, the absence of CORS, and a same-origin deployment in which nginx proxies `/graphql`, `/healthz`, `/readyz`, `/integrations`, `/github` and `/slack` to the API.	`app/http_cookies.py`, `app/graphql/router.py`, `frontend/nginx.conf`
CORS	None. There is no `CORSMiddleware`, so browsers block cross-origin reads by default.	`app/main.py` (grep)
GitHub OAuth state	Cookie `vector_github_state`: HttpOnly, SameSite=Lax, Path=/, Max-Age 600, Secure in production. Its value is base64url JSON of `{state, session, workspace, return_to}`, where `state = token_urlsafe(32)` and `session` is the SHA-256 hex of the caller's session token. The callback compares both with `secrets.compare_digest`, clears the cookie, and checks the redirect against `GITHUB_REDIRECT_ALLOWLIST`.	`app/rest/github.py` → `STATE_COOKIE_NAME`, `STATE_COOKIE_MAX_AGE`, `_session_digest`, `_state_matches`
Slack OAuth state	Cookie `vector_slack_oauth`: HttpOnly, SameSite=Lax, Path=/, lifetime 600 s, Secure in production. Its value is `state:workspace_slug:session_digest:return_path`. The callback compares with `hmac.compare_digest`, re-checks that the caller is an admin, and re-validates the return path.	`app/rest/slack.py` → `OAUTH_STATE_COOKIE_NAME`, `OAUTH_STATE_TTL_SECONDS`
OAuth host binding	If the configured callback host differs from the request host, the start of the flow first redirects (302) to the callback's origin, so the state cookie is set on the host the callback will reach	`app/rest/oauth_origin.py` → `redirect_to_callback_origin`
Webhook signatures	GitHub: HMAC-SHA256 in `X-Hub-Signature-256`, compared with `hmac.compare_digest`; non-ASCII headers are rejected. Slack: v0 scheme, HMAC-SHA256 over `v0:{timestamp}:{body}` in `X-Slack-Signature`, compared with `hmac.compare_digest`; timestamps older than 300 s are rejected.	`app/services/github.py` → `verify_webhook_signature`; `app/rest/slack.py` (`MAX_TIMESTAMP_AGE_SECONDS`)
Secrets in configuration	The database URL and every GitHub and Slack secret are `pydantic.SecretStr`	`app/config.py`
Secret scanning	A CI job runs gitleaks 8.30.1, downloaded with a pinned sha256, on a full-history checkout (`fetch-depth: 0`). It first proves detection works on planted test secrets. It then runs `gitleaks dir .` and `gitleaks git .` with `--redact --exit-code 1`, and fails if no commits were scanned. `.gitleaks.toml` adds a custom `vector-postgres-dsn-password` rule to the defaults. The pre-commit hooks are ruff plus whitespace, end-of-file, YAML, TOML and merge-conflict checks; gitleaks is not among them.	`.github/workflows/ci.yml`, `.gitleaks.toml`, `.pre-commit-config.yaml`

Area	Measure	Source
Containers	The API image installs dependencies with `pip install --require-hashes` and runs as the non-root user `vector` (uid 10001, `USER vector`). The k8s API pod sets `runAsNonRoot`, `readOnlyRootFilesystem: true`, `allowPrivilegeEscalation: false`, drops all capabilities, and uses `seccompProfile: RuntimeDefault`. The web pod also sets `runAsNonRoot`.	`Dockerfile`, `k8s/30-api.yaml`, `k8s/40-web.yaml`
Database timeouts	The connection pool sets `command_timeout=10.0` and `statement_timeout=9500` ms	`app/db.py`

8.7 Gaps / to verify

Confirmed absent. A grep of `app/` and `migrations/` found no code for password reset, email verification, TOTP/MFA, CORS or CSRF tokens. Together with the list of auth mutations, that confirms the following are missing:

- **Password reset:** no forgot-password flow.
- **Email verification:** none. The `AuthService.register` docstring says "address verification is still owed".
- **Two-factor authentication:** no 2FA, TOTP or MFA.
- **Session management:** no change-password, list-sessions or "sign out everywhere" operation. The only auth mutations are `register`, `login` and `logout`.
- **Account deletion:** none.
- **Outbound email:** none. Invitation tokens are handed to the inviter, and nobody is notified of a lockout.
- **Session rotation:** none. Logging in or registering while already holding a session does not revoke the old one.
- **Session limits:** no idle timeout and no cap on concurrent sessions.
- **Password rehash:** stored hashes are not upgraded when parameters change.
- **CSRF token:** no CSRF token and no Origin/Referer check.
- **Other rate limits:** none on `invitationAccept`, `workspaceCreate` or any authenticated mutation.

Inconsistencies and caveats:

- **Registration can end up with no rate limit.** With no client IP, `register` has no budget at all. The `register` docstring says `read_client_ip` "falls back to the TCP peer rather than giving up". It does not: it returns `None` when `FORWARDED_ALLOW_IPS` contains `*` and no hop count is set (`app/http_client_ip.py`).
- **docker-compose lets a local caller pick their IP bucket.** The `api` service sets `TRUSTED_PROXY_HOPS: "1"` but also publishes port 8000 on `127.0.0.1`. A local caller who bypasses nginx can supply their own `X-Forwarded-For` and choose their IP bucket key. This only affects local setups.
- **k8s sign-in over plain HTTP.** k8s sets `ENVIRONMENT=production`, so cookies are `Secure`. But `k8s/40-web.yaml` is a plain-HTTP NodePort with no TLS or Ingress manifest, and the configmap comment acknowledges that sign-in will not persist over HTTP.
- **k8s may never send HSTS.** `FORWARDED_ALLOW_IPS` is not set in k8s, so the HSTS header may never be sent there. This is reasoned from the code, not tested.
- **Former members can still be assigned.** Assigning an issue to a former member is not rejected, because the old membership row still satisfies the composite FK (ponytail note on `MembershipRepository.unassign_issues`).
- **The GitHub state cookie is unsigned.** This is by design: every field in it is re-validated in the callback.
- **Tokens may appear in access logs.** There is no logging configuration or redaction (`app/logging.py` is a stub). Invitation tokens in `/invite/:token` URLs, and OAuth `code`/`state` query parameters, may therefore appear in uvicorn or nginx access logs. Not verified by running.
- **Stale docstrings and dead code:**
  - `AuthQuery.me` still says protected fields "must refuse" and "None exist yet".
  - `_scope` in `mutations/memberships.py` still calls itself a "stand-in until scope.py lands".
  - `actor_user_id` in `app/graphql/viewer.py` is never referenced.

> **Gaps / to verify:**

- **Argon2 parameters:** the values (64 MiB, t=3, p=4) come from the `passwords.py` docstring describing argon2-cffi's defaults. The code does not pin them.
- **Middleware order:** that the security-headers middleware wraps the body-limit middleware is inferred from Starlette's ordering rules, not asserted anywhere in the repo.
- **Unread tests:** `tests/test_repository_tenancy_gate.py` and `tests/test_security_headers.py` were not read, so exactly what they enforce is unconfirmed.
- **Sibling subdomains:** whether a same-site sibling subdomain could mount CSRF or cookie injection has not been assessed. `http_cookies.py` defers the `__Host-` prefix until there is a real domain.

9. Frontend

The frontend is a single-page React application in `frontend/`. It talks to the backend only through `POST /graphql`, plus browser navigations to the backend's OAuth start paths. All paths below are relative to `frontend/` unless they begin with `app/`, `design/` or a root file name.

9.1 Overview and build tooling

Runtime and tooling versions (resolved versions from `package-lock.json` `packages["node_modules/<name>"].version`):

Package	Version	Role
react / react-dom	19.2.8	UI; `createRoot` inside `<StrictMode>` (`src/main.tsx`)
react-router-dom	7.18.3	Data router (`src/app/App.tsx`)
@apollo/client	4.2.12	GraphQL client, normalized cache, React hooks (`src/lib/graphql/client.ts`)
graphql	16.14.2	Peer of Apollo and codegen
rxjs	7.8.2	Observable runtime required by Apollo Client 4
vite	8.2.2	Dev server and bundler (`vite.config.ts`)
@vitejs/plugin-react	6.1.1	Only Vite plugin (`vite.config.ts`, `vitest.config.ts`)
typescript	6.0.3	`tsc -b` typecheck
eslint / typescript-eslint	10.10.0 / 8.69.0	Lint (`eslint.config.js`)
vitest	5.0.0	Unit and component tests (jsdom)
@playwright/test	1.63.0	E2E (`playwright.config.ts`, `e2e/`)
@graphql-codegen/cli	7.4.0	Type and document generation (`codegen.ts`)

There are no other runtime dependencies. The app has no CSS framework, no state library, no chart library, no form library, no date library, no drag-and-drop library and no rich-text editor (`package.json` `dependencies`).

npm scripts (`package.json`): `dev` (`vite`), `build` (`tsc -b && vite build`), `typecheck`, `lint`, `test` (`vitest run`), `test:watch`, `preview`, `graphql:schema` / `graphql:schema:check` (`scripts/graphql-schema.mjs`), `graphql:codegen` / `graphql:check` (`scripts/graphql-codegen.mjs`). There is no npm script for Playwright. CI runs `npx playwright test` (`.github/workflows/ci.yml`, job `e2e`).

TypeScript (`tsconfig.app.json`, which covers `src`): `strict: true`, `noUncheckedIndexedAccess`, `noImplicitOverride`, `noUnusedLocals`, `noUnusedParameters`, `noFallthroughCasesInSwitch`, `noUncheckedSideEffectImports`, `erasableSyntaxOnly`, `verbatimModuleSyntax`, `moduleResolution: bundler` and `jsx: react-jsx`. `exactOptionalPropertyTypes` is deliberately left off (comment in the file). `tsconfig.node.json` covers `vite.config.ts` and `vitest.config.ts`. No tsconfig covers `codegen.ts`, `playwright.config.ts` or `e2e/**`.

ESLint (`eslint.config.js`, flat config): `src/**/*.{ts,tsx}` and the two Vite configs extend `tseslint.configs.recommendedTypeChecked` with `projectService: true`. `eslint-plugin-react-hooks` is not installed; a comment in the file calls this "the main gap in the current rule set".

GraphQL codegen pipeline (the frontend never introspects a live server):

1. `scripts/export_schema.py` calls `app.graphql.schema.build_schema("development").as_str()` and normalizes the result. `scripts/graphql-schema.mjs write|check` runs it, using `$PYTHON`, then `python`, then `python3`, and writes or diffs the checked-in `schema.graphql`.
2. `codegen.ts` uses `schema: './schema.graphql'` and `documents: ['src/**/*.graphql']`, and produces:

    - `src/generated/schema.ts` (plugin `typescript`)
    - `src/generated/operations.ts` (plugins `typescript-operations` and `typed-document-node`, with `documentNodeImport: '@apollo/client#TypedDocumentNode'`)

  Scalar mappings: `UUID`, `DateTime` and `Date` become `string`; `JSON` becomes `unknown`. Other settings: `strictScalars`, `enumsAsTypes`, `nonOptionalTypename`. The config does not use `client-preset`.

3. `scripts/graphql-codegen.mjs write|check` adds a banner and normalizes line endings. In `check` mode it also reports orphaned files in `src/generated/`. `.gitattributes` forces LF on `schema.graphql` and `src/generated/**`.

4. CI checks both kinds of drift (`.github/workflows/ci.yml`): the `graphql-contract` job checks the SDL, and the `frontend` job runs codegen and then a diff.

Document inventory: 27 `.graphql` files under `src/` contain 61 queries, 82 mutations, 31 fragments and 0 subscriptions (counted with `^\s*(query|mutation|fragment|subscription)\s`). `src/generated/operations.ts` exports 143 `…Document` constants (61 + 82) and 31 `…FragmentDoc` constants.

Dev proxy (`vite.config.ts`): both `server.proxy` and `preview.proxy` forward the following paths to `BACKEND_ORIGIN = 'http://127.0.0.1:8000'` with `changeOrigin: false`:

- `/graphql`
- the `PROVIDER_PATHS` `/integrations`, `/github` and `/slack`

`/healthz` and `/readyz` are not proxied. `allowedHosts` admits ngrok and Cloudflare tunnel hosts, which OAuth callbacks need. The backend installs no CORS middleware (comment in `vite.config.ts`), so the browser must reach the API same-origin. `src/lib/config/env.ts` reads `VITE_GRAPHQL_URL`, which defaults to `/graphql`. In development it warns if that URL is cross-origin.

Production serving. There are two paths in the repo:

Path	Mechanism
`frontend/Dockerfile` (compose service `web`)	`node:22-alpine` build stage, then `nginxinc/nginx-unprivileged:1.29-alpine` on port 8080. See the `nginx.conf` behavior below this table.
Root `Dockerfile` (API image)	Stage `frontend` runs `npm ci && npm run build`, and `COPY --from=frontend /build/dist ./static`. `app/spa.py` `add_spa` serves `static/index.html` for unmatched GETs (`no-store` plus a CSP) and `/assets` as immutable.

`nginx.conf` behavior in the `web` image:

- `/assets/` is cached for one year as `public, immutable`.
- `= /index.html` is served `no-store`.
- An allowlist regex `^/(graphql|healthz|readyz|integrations|github|slack)(/|$)` proxies to `http://api:8000`.
- `location /` falls back to `/index.html` (`try_files`).

`index.html` loads Space Grotesk (400–700) and IBM Plex Mono (400, 500) from Google Fonts.

9.2 Routing

The app uses react-router-dom 7 as a data router: `createBrowserRouter(routes)` and `<RouterProvider>` (`src/app/App.tsx`). The route table is data: `routes = [...authRoutes, ...onboardingRoutes, workspaceRoute]` (`src/app/routes/routes.tsx`). All routes are mounted eagerly: there is no `React.lazy`, no `<Suspense>`, and no loaders or actions. Path segments and URL builders live in `src/app/routes/paths.ts` (`ROUTE_SEGMENTS`, `paths`, `publicPaths`). Components navigate through `useAppPaths()` (`src/app/routes/useAppPaths.ts`).

Path	Page / component (file)	Guard
`/`	`LandingPage` (`src/features/landing/LandingPage.tsx`)	none

Path	Page / component (file)	Guard
`/login`	`LoginPage` (`src/features/auth/pages/LoginPage.tsx`)	`RequireNoAuth`
`/register`	`RegisterPage` (`src/features/auth/pages/RegisterPage.tsx`)	`RequireNoAuth`
`/onboarding` (index, `element: null`)	`OnboardingLayout` redirects via `resolveStep` (`src/features/onboarding/pages/OnboardingLayout.tsx`, `lib/progress.ts`)	**none**
`/onboarding/workspace`	`WorkspaceStep`	**none**
`/onboarding/team`	`TeamStep`	**none**
`/onboarding/invite`	`InviteStep`	**none**
`/onboarding/integrations`	`IntegrationsStep`	**none**
`/invite/:token`	`AcceptInvitePage` (outside `OnboardingLayout`)	**none**
`/:workspaceSlug` (index)	`IssueListPage` = `IssuesScreen` (`src/features/issues`)	`RequireAuth` → `AppLayout`
`/:workspaceSlug/issues`	`IssueListPage` = `IssuesScreen`	same
`/:workspaceSlug/issues/:issueId`	`IssueDetailPage` = `IssuesScreen` (the same component, with the split-pane inspector open)	same
`/:workspaceSlug/board`	`BoardScreen`	same
`/:workspaceSlug/my-issues`	`MyIssuesPage`	same
`/:workspaceSlug/inbox`	`InboxPage`	same
`/:workspaceSlug/team/:teamKey`	`TeamPage`	same
`/:workspaceSlug/team/:teamKey/issues`	`TeamIssuesPage`	same
`/:workspaceSlug/members`	`MembersPage`	same
`/:workspaceSlug/settings`	`SettingsPage`	same
`/:workspaceSlug/search`	`SearchPage`	same
`/:workspaceSlug/projects`	`ProjectListPage`	same
`/:workspaceSlug/projects/:projectId`	`ProjectDetailPage`	same
`/:workspaceSlug/cycles`	`CycleListPage`	same
`/:workspaceSlug/cycles/:cycleId`	`CycleDetailPage`	same
`/:workspaceSlug/triage`	`TriagePage`	same
`/:workspaceSlug/saved-views`	`SavedViewsPage`	same
`/:workspaceSlug/favorites`	`FavoritesPage`	same
`/:workspaceSlug/templates`	`TemplatesPage`	same
`/:workspaceSlug/initiatives`	`InitiativesPage`	same
`/:workspaceSlug/roadmap`	`RoadmapPage`	same

Path	Page / component (file)	Guard
`/:workspaceSlug/documents`	`DocumentsPage`	same
`/:workspaceSlug/releases`	`ReleasesPage`	same
`/:workspaceSlug/environments`	`EnvironmentsPage`	same
`/:workspaceSlug/label-groups`	`LabelGroupsPage`	same
`/:workspaceSlug/semantic-search`	`SemanticSearchPage`	same
`/:workspaceSlug/analytics`	`AnalyticsPage`	same
`/:workspaceSlug/*`	`NotFound`, rendered inside the shell	same

The workspace route also sets `errorElement: <RouteError/>`. The `SECOND_WAVE` placeholder table is spread into the children but is empty (`routes.tsx`). The app has no top-level `*` route, so an unknown single segment is treated as a workspace slug.

Guards (`src/features/auth/guards.tsx`, `src/app/layout/AppLayout.tsx`):

- **`RequireAuth`** reads `useViewer()` (the `Me` query).
  - While loading, it renders `SessionPending`.
  - On error, it renders `ErrorState` with a retry.
  - If `viewer === null`, it renders `<Navigate to="/login" replace state={{ from }}>`.
- **`RequireNoAuth`** redirects a signed-in user to one of:
  - a safe same-origin `state.from` (`returnPathFrom` rejects paths that do not start with `/`, and paths starting with `//` or `/\`)
  - the first membership's `/:slug/issues`
  - `/onboarding`

  The last two come from `signedInDestination` (`api/destination.ts`).
- **`AppLayout`** runs `WorkspaceShell`.
  - With zero memberships, it renders `<Navigate to="/onboarding">`.
  - If the slug is not among the viewer's memberships, it renders `WorkspaceNotFound` without querying that slug on the server.
- **Onboarding and `/invite/:token` are outside any auth guard.** `onboardingRoutes` is spread unwrapped (`src/features/onboarding/index.tsx`, `routes.tsx`). A signed-out visitor gets the server's `UNAUTHENTICATED` error rendered as an error state, not a redirect (comment in `OnboardingLayout.tsx`).

9.3 State management

Kind of state	Mechanism	Source
Server data	Apollo Client 4 `InMemoryCache`, the only state library (no Redux, Zustand, React Query or similar)	`src/lib/graphql/cache.ts`, `package.json`
Identity ("who am I")	The cached `Me` query. The session is an HttpOnly cookie, so no auth state is kept in JS.	`src/features/auth/api/useViewer.ts`

Kind of state	Mechanism	Source
Workspace scope	`WorkspaceContext` (`{ slug, workspace, role, memberships, viewer, paths }`), provided by `AppLayout`. `useWorkspace()` throws outside the shell.	`src/app/workspace/context.ts`
"New issue" action slot	`CreateIssueActionContext` and `RegisterCreateIssueActionContext`. A screen registers its composer opener; the rail's `CreateIssueButton` and the command palette read it.	`src/app/layout/createIssueAction.ts`, `CreateIssueActionProvider.tsx`
Toasts	`ToastContext` exists, but `ToastProvider` is not mounted anywhere	`src/components/Toast/Toast.tsx`, comment in `src/features/onboarding/pages/InviteStep.tsx`
URL state	The selected issue is the `:issueId` route param. Board filters, sort and grouping are query params (`team, group, sort, status, assignee, label, priority, project, cycle`). Search uses `?q=` with a 250 ms debounce and `replace`. Only `BoardScreen.tsx` and `SearchPage.tsx` call `useSearchParams`.	`src/features/board/lib/viewState.ts` (`PARAM`), `src/features/search/SearchPage.tsx`
Per-browser preferences	`localStorage` keys, each access wrapped in try/catch: `vector.sidebar.collapsed`, `vector.density` (sets `data-density` on `<html>`), `vector.list.group` (flat or grouped, via `useSyncExternalStore`), and `vector.theme` (whose hook `useTheme` has no callers)	`src/app/layout/preferences.ts`

Those four `createContext` call sites (in three files) are the only ones in non-test code (grep for `createContext`).

9.4 Data fetching and caching

Client (`src/lib/graphql/client.ts`, `createApolloClient()`, singleton `apolloClient`):

- **Link chain:** `ApolloLink.from([sessionExpiryLink, new HttpLink({ uri: graphqlUrl, credentials: 'same-origin' })])`.
- **Transport:** POST only. A comment explains that `useGETForQueries` is avoided because the backend router disables queries over GET.
- **Not configured:** custom headers, retry link, batching, persisted queries and `defaultOptions`.
- **Devtools:** enabled only in development.
- **Provider:** `AppProviders` wraps `ApolloProvider` (`src/app/providers/AppProviders.tsx`).

Cookies and CSRF. Auth relies only on the browser sending the session cookie same-origin. The SPA sends no CSRF token or header (grep of `src/` for csrf-related headers finds only comments about the OAuth `state` parameter). Protection depends on the cookie's `SameSite=Lax`, JSON POST, and the absence of CORS (see §9.10).

UNAUTHENTICATED handling. `sessionExpiryLink` (`src/features/auth/api/sessionExpiry.ts`) is an `ErrorLink`. When any GraphQL error carries `extensions.code === 'UNAUTHENTICATED'`, it runs `cache.writeQuery({ query: MeDocument, data: { me: null } })`. `RequireAuth` then re-renders into the `/login` redirect.

Errors. Expected validation failures arrive in payload `errors { field code message }` (fragment `ValidationFields`). A rejected promise means an outage or a bug. `src/lib/graphql/payload.ts` provides `readPayload()`, `partitionFieldErrors()` and `UNEXPECTED_RESPONSE`.

Login and logout:

- `useLogin` and `useRegister` write the returned user into the `Me` cache (`src/features/auth/api/useSignIn.ts`).
- `useLogout` calls the `Logout` mutation, navigates to `/`, then calls `client.clearStore()` (`src/features/auth/api/useLogout.ts`).

Type policies (`src/lib/graphql/cache.ts`, `createCache()`). Entities are normalized by the default `__typename:id`; there are no custom `keyFields`. `cursorConnectionPolicy(keyArgs)` merges as follows:

- A response with `after` appends `incoming.nodes` to the existing list, de-duplicating by `id`.
- A response without `after` replaces the list.

Field	keyArgs
`Query.issues`	`workspaceSlug, filter, orderBy`
`Query.labels`, `Query.projects`, `Query.initiatives`, `Query.releases`	`workspaceSlug`
`Query.notifications`	`workspaceSlug, unreadOnly`
`Query.documents`	`workspaceSlug, projectId, initiativeId`
`Issue.comments`, `Issue.children`, `Issue.relations`, `Document.revisions`, `Document.comments`	`false`

Pagination is keyset-only (`first`/`after`/`endCursor`), with `DEFAULT_PAGE_SIZE = 25` and `MAX_PAGE_SIZE = 100` (`src/lib/graphql/pagination.ts`).

Fetch policies. Production code uses Apollo's default `cache-first` everywhere except the two `cache-and-network` queries in `src/features/onboarding/api/useOnboardingProgress.ts`.

Invalidation:

- `refetchQueries` is used by the mutations of cycles, documents, environments, favorites, initiatives, labelGroups, members, onboarding, projects, releases, savedViews, semanticSearch, settings, templates and triage.
- **Manual cache edits:**
  - `src/features/issues/api/cache.ts` (for example `prependCreatedIssue`, `removeArchivedIssue`)
  - `src/features/collaboration/api/cache.ts`
  - `src/features/inbox/api.ts`, which writes the unread count into the sidebar's `ShellSidebar` entry

Optimistic updates (exactly two files use `optimisticResponse`):

- `src/features/board/api/useMoveIssue.ts`: the `BoardIssueMove` card move, with no refetch.
- `src/features/issues/api/useIssueMutations.ts`: `IssueUpdate`, built by `applyPatch`. `setProject`, `setCycle` and `archive` are not optimistic.

9.5 Real-time

None. The frontend has no subscriptions, polling, SSE or WebSockets:

- `schema.graphql` has no `type Subscription` (its only "Subscription" names are `IssueSubscriptionInput` and `IssueSubscriptionPayload`, which are for following an issue).
- The `.graphql` documents contain 0 `subscription` operations.
- A grep of non-test `src/` for `pollInterval|startPolling|WebSocket|EventSource|graphql-ws|useSubscription|setInterval|subscribeToMore` finds no matches.

The inbox badge and lists change only through cache writes, refetches or navigation.

9.6 Features

Each feature lives in `src/features/<name>/`. Larger features have `api/` (with `operations.graphql`), `components/`, `lib/` and `pages/`. Smaller screens use a flat `api.ts` plus `operations.graphql`. `src/features/screens.tsx` holds shared list helpers (`IssueRows`, `ListFooter`, `describeError`, `localToday`).

Operation names come from each feature's `operations.graphql`; "m:" marks a mutation.

Feature	UI provided	Key components	GraphQL operations
analytics	`/analytics`: throughput, cycle time and estimates, drawn as a hand-built SVG chart	`AnalyticsPage`, `ThroughputChart`, `BreakdownTable`, `lib/metrics.ts`	`WorkspaceAnalytics`
auth	Login and register pages, guards, viewer and logout hooks, session-expiry link	`LoginPage`, `RegisterPage`, `AuthScreen`, `guards.tsx`	`Me`, `MyWorkspaces`, m: `Register`, m: `Login`, m: `Logout`
board	`/board`: team Kanban whose columns are workflow states, with URL-encoded view state. Cards move by native HTML5 drag or a per-card `Menu`.	`BoardScreen`, `BoardColumn`, `BoardCard`, `BoardControls`, `lib/viewState.ts`	`BoardIssues`, `BoardLabels`, m: `BoardIssueMove` (optimistic)
collaboration	Panels inside the issue inspector: comments, labels, relations and sub-issues, plus an issue picker	`CommentsPanel`, `LabelsPanel`, `RelationsPanel`, `SubIssuesPanel`, `IssuePicker`, `api/cache.ts`	`IssueComments`, `CommentAuthors`, `IssueLabels`, `WorkspaceLabels`, `IssueRelations`, `IssueSubIssues`, `IssueSearch`; m: `CommentCreate`, `CommentDelete`, `IssueLabelAttach`, `IssueLabelDetach`, `LabelCreate`, `IssueRelationCreate`, `IssueRelationDelete`, `IssueSetParent`, `IssueClearParent`
command	Command palette (modal), navigation commands, search results, shortcut reference	`CommandPalette`, `CommandList`, `ShortcutsView`, `commands.tsx`, `lib/keyboard.ts`	`CommandSearch`
cycles	`/cycles`: team picker plus current, upcoming and past cycles. `/cycles/:id`: detail view; add unscheduled issues; create, edit and delete.	`CycleListPage`, `CycleDetailPage`, `CycleForm`, `lib/cycles.ts`	`CycleList`, `CycleDetail`, `CycleTeams`, `CycleIssues`, `CycleUnscheduledIssues`; m: `CycleCreate`, `CycleUpdate`, `CycleDelete`
documents	`/documents`: list plus panel. ProseMirror JSON is rendered by `DocumentBody`. The textarea editor appears only for plain-paragraph documents. Revisions, restore and comments.	`DocumentsPage`, `DocumentPanel`, `DocumentBody`, `DocumentForm`, `lib/content.ts`	`DocumentList`, `DocumentDetail`; m: `DocumentCreate`, `DocumentEdit`, `DocumentRestore`, `DocumentDelete`, `DocumentCommentCreate`, `DocumentCommentDelete`
environments	`/environments`: list and create; "deployed" status is derived from the first page of releases	`EnvironmentsPage`, `EnvironmentForm`	`EnvironmentList`, m: `EnvironmentCreate` (also uses releases' `ReleaseList`)
favorites	`/favorites`: the viewer's shortcuts, reordered with up and down buttons	`FavoritesPage`, `lib/favorites.ts`	`FavoriteList`, `FavoriteSavedViews`; m: `FavoriteAdd`, `FavoriteRemove`, `FavoriteReorder`
inbox	`/inbox`: notifications, mark read and mark all read; updates the sidebar badge through the cache	`InboxPage`, `api.ts`	`NotificationInbox`, `NotificationUnreadCount`; m: `NotificationMarkRead`, `NotificationMarkAllRead`
initiatives	`/initiatives`: tree list plus detail panel, health updates and project links	`InitiativesPage`, `InitiativeDetailPanel`, `InitiativeForm`	`InitiativeList`, `InitiativeDetail`; m: `InitiativeCreate`, `InitiativeUpdate`, `InitiativeDelete`, `InitiativeProjectAdd`, `InitiativeProjectRemove`, `InitiativeSetParent`, `InitiativeClearParent`, `InitiativeUpdatePost`
issues	List plus split-pane inspector (`IssuesScreen`), an inline composer form, inline field editing, archive, flat or grouped list, and "Load more" cursor paging	`IssuesScreen`, `IssueComposer`, `IssueInspector`, `IssueRow`, `EditableText`, `ListStates`	`IssueList`, `IssueDetail`, `IssueWorkspaceContext`, `TeamCycles`; m: `IssueCreate`, `IssueUpdate` (optimistic), `IssueSetProject`, `IssueSetCycle`, `IssueArchive`

Feature	UI provided	Key components	GraphQL operations
labelGroups	`/label-groups`: groups, optionally exclusive, and label assignment	`LabelGroupsPage`, `LabelGroupPanel`, `LabelGroupForm`	`LabelGroupList`, `GroupedLabelList`; m: `LabelGroupCreate`, `LabelGroupUpdate`, `LabelGroupDelete`, `LabelSetGroup`
landing	`/`: public page with a wordmark and sign-in and register calls to action	`LandingPage`	—
members	`/members`: members and roles, invitations (the token is shown once), role change, remove and revoke	`MembersPage`, `api.ts`	`WorkspaceMemberList`, `WorkspaceInvitationList`; m: `MemberInvitationCreate`, `MemberInvitationRevoke`, `MemberRoleUpdate`, `MemberRemove`
myIssues	`/my-issues`: issues filtered by `assigneeId: viewer.id`	`MyIssuesPage`	reuses issues' `IssueList`
navigation	Sidebar sections, expandable team rows (team, Issues, Cycles) and the Inbox badge	`SidebarNav`, `NavRow`, `navigationItems.tsx`	`ShellSidebar` (defined in `src/app/workspace/operations.graphql`)
onboarding	Workspace → Team → Invite → Integrations steps, plus invite acceptance	`OnboardingLayout`, the step pages, `AcceptInvitePage`, `lib/progress.ts`	`OnboardingWorkspaces`, `OnboardingTeams`, `OnboardingIntegrations`; m: `OnboardingWorkspaceCreate`, `OnboardingTeamCreate`, `OnboardingInvitationCreate`, `OnboardingInvitationAccept`
projects	`/projects`: cursor-paged list. `/projects/:id`: teams, milestones, issues, health ring and progress bar.	`ProjectListPage`, `ProjectDetailPage`, `HealthRing`, `ProgressBar`, `ProjectMilestones`	`ProjectList`, `ProjectDetail`, `ProjectTeams`, `ProjectMembers`, `ProjectIssues`, `ProjectUnfiledIssues`; m: `ProjectCreate`, `ProjectUpdate`, `ProjectDelete`, `ProjectTeamAdd`, `ProjectTeamRemove`, `ProjectMilestoneCreate`, `ProjectMilestoneUpdate`, `ProjectMilestoneDelete`
releases	`/releases`: list plus notes panel, create, set status and delete	`ReleasesPage`, `ReleaseDetailPanel`, `ReleaseForm`	`ReleaseList`, `ReleaseDetail`; m: `ReleaseCreate`, `ReleaseStatusSet`, `ReleaseDelete`
roadmap	`/roadmap`: initiatives and projects placed by `targetDate` month (markers only)	`RoadmapPage`, `lib/roadmap.ts`	`RoadmapInitiatives`, `RoadmapProjects`
savedViews	`/saved-views`: list plus a results panel that applies the stored filter on the server	`SavedViewsPage`, `SavedViewForm`	`SavedViewList`, `SavedViewResults`; m: `SavedViewCreate`, `SavedViewUpdate`, `SavedViewDelete`
search	`/search?q=`: hybrid issue and project search	`SearchPage`	`WorkspaceSearch`
semanticSearch	`/semantic-search`: embeddings-only duplicate search, index state and refresh	`SemanticSearchPage`, `IndexPanel`	`SemanticIssueMatches`, `EmbeddingIndexState`, m: `EmbeddingsRefresh`
settings	`/settings`: workspace identity, GitHub and Slack panels (connect links go to backend `/github/install` and `/slack/oauth/start`), disconnect, repository selection, automation	`SettingsPage` (`START_PATHS`), `src/lib/integrations.ts`	`WorkspaceIntegrations`; m: `WorkspaceGithubDisconnect`, `WorkspaceSlackDisconnect`, `WorkspaceGithubRepositoriesSet`, `WorkspaceGithubAutomationSet`
teams	`/team/:key`: overview of workflow states plus a preview. `/team/:key/issues`: flat list.	`TeamPage`, `TeamIssuesPage`, `TeamStates`	`TeamBoards`, `TeamIssueList`
templates	`/templates`: CRUD and "file from template"	`TemplatesPage`, `TemplateForm`	`IssueTemplateList`; m: `IssueTemplateCreate`, `IssueTemplateUpdate`, `IssueTemplateDelete`, `IssueCreateFromTemplate`

Feature	UI provided	Key components	GraphQL operations
triage	`/triage`: team picker plus queue; accept, decline, mark duplicate, change team and update	`TriagePage`, `TriageRow`	`TriageQueue`; m: `TriageEnter`, `TriageAccept`, `TriageDecline`, `TriageMarkDuplicate`, `TriageChangeTeam`, `TriageIssueUpdate`

Three operations sit at the shell level, outside any feature: `WorkspaceShell` and `ShellSidebar` (`src/app/workspace/operations.graphql`), and `WorkspaceEntry` (`src/app/routes/workspaceEntry.graphql`, which no route uses).

Schema mutations with no frontend operation include `issueSubscribe`, `issueBulkUpdate`, `projectUpdatePost`, `slackDefaultChannelSet` and `slackNotificationPreferenceSet` (`schema.graphql` compared against the `.graphql` documents).

Sidebar (`src/features/navigation/navigationItems.tsx`):

- Primary: My Issues, Inbox, Triage, All Issues, Board, Projects, Initiatives, Roadmap, Documents
- Views: Saved Views, Favorites
- Workspace: Members, Analytics, Releases, Environments, Label Groups, Templates, Settings

Cycles are reachable through each team's nested row in `SidebarNav`.

Keyboard shortcuts and command palette:

- **Palette chord:** ⌘K on Apple user agents, otherwise Ctrl+K (`COMMAND_CHORD`, `src/app/layout/ShellActions.tsx`).
- **Global keys,** ignored while typing (`src/features/command/CommandPalette.tsx`):
  - `/` opens the palette.
  - `?` opens the shortcut list.
  - `c` or `C` opens "New issue" when the current screen has registered a composer.
- **Inside the palette:** ↑ and ↓ move, Enter runs, Esc closes.
- **Palette commands** (`commands.tsx`): My Issues, Inbox, All Issues, Projects, Cycles, Search, Members and Settings, plus "New issue" and `CommandSearch` results.
- **Issue list** (`IssuesScreen.tsx`):
  - `c` opens the composer.
  - Esc closes the composer, or closes the inspector by navigating back to the list.
  - ↑ and ↓ move focus between rows.
  - Enter follows the row link.

`ShortcutsView.tsx` documents these shortcuts in the UI.

9.7 Shared components and composition

`src/components/` exports the following through one barrel (`src/components/index.ts`). Each component folder except `icons/` has its own `*.module.css`.

Group	Exports
Buttons and fields	`Button`, `IconButton`, `Input`, `Select`, `Textarea`, `Checkbox`
Overlays	`Menu`, `Dialog` (native `<dialog>`), `Tooltip`, `ToastProvider` and `useToast`
Navigation and controls	`Tabs`, `SegmentedControl`
Display	`Badge`, `Tag`, `Avatar`, `Kbd`, `Spinner`, `Skeleton`, `VisuallyHidden`
Lists	`List`, `ListRow`, `ListRowMain`, `ListRowMeta`, `IssueRow` (presentational), `GroupHeader`, `InspectorPanel`
States	`EmptyState`, `ErrorState`, `PermissionState`
Indicators and icons	`StatusIndicator`, `PriorityIndicator`, `ProgressIndicator`, `RelationIndicator`, SVG icons and `VectorMark`

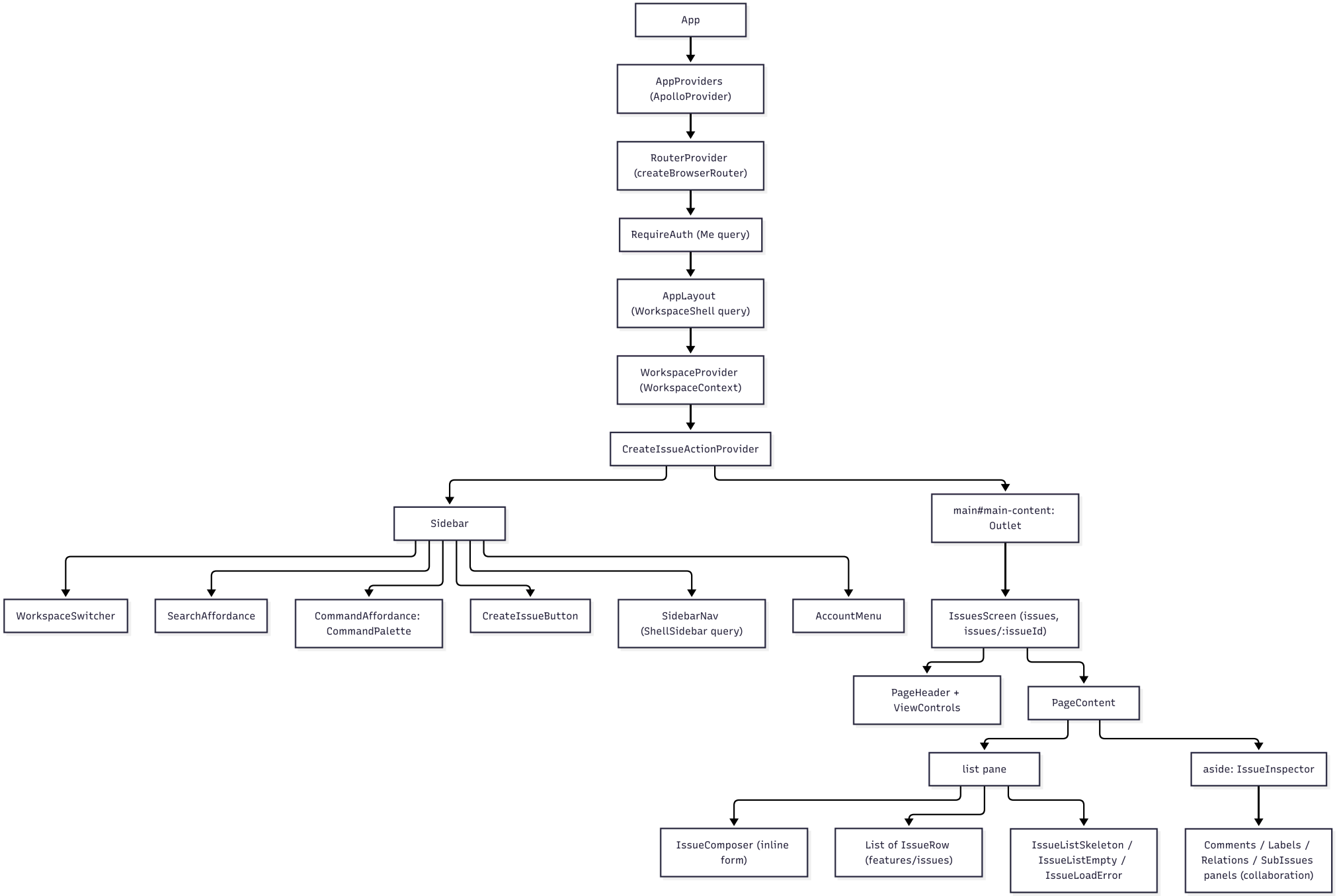
Group	Exports
Utility	`cx` (joins class names)

There are two `IssueRow` components:

- `src/components/IssueRow` is presentational and is used by `src/features/triage/components/TriageRow.tsx`.
- `src/features/issues/components/IssueRow.tsx` is data-aware and is used by `IssuesScreen` and `src/features/screens.tsx`.

Page composition (`src/app/layout/`). Every page renders `PageHeader` (the only `<h1>`, with an `actions` slot) and `PageContent`. List screens place `ViewControls` (density and grouping `SegmentedControl`s) in the header actions. `AppLayout` adds a "Skip to main content" link that targets `<main id="main-content" tabIndex={-1}>`.

The tree below shows the shell, with the issues page as the example:


App
AppProviders (ApolloProvider)
RouterProvider (createBrowserRouter)
RequireAuth (Me query)
AppLayout (WorkspaceShell query)
WorkspaceProvider (WorkspaceContext)
CreateIssueActionProvider
Sidebar
main#main-content: Outlet
WorkspaceSwitcher
SearchAffordance
CommandAffordance: CommandPalette
CreateIssueButton
SidebarNav (ShellSidebar query)
AccountMenu
IssuesScreen (issues, issues/:issueId)
PageHeader + ViewControls
PageContent
list pane
aside: IssueInspector
IssueComposer (inline form)
List of IssueRow (features/issues)
IssueListSkeleton / IssueListEmpty / IssueLoadError
Comments / Labels / Relations / SubIssues panels (collaboration)

Sources for the diagram: `src/app/App.tsx`, `src/app/routes/routes.tsx`, `src/app/layout/AppLayout.tsx`, `Sidebar.tsx`, `ShellActions.tsx`, `src/features/issues/pages/IssuesScreen.tsx` and `components/IssueInspector.tsx`.

9.8 Styling

- **Approach:** plain CSS with CSS Modules: 53 `*.module.css` files, placed next to their components. There is no Tailwind, CSS-in-JS or preprocessor. `src/styles/index.css` imports `tokens.css` (616 lines) and `base.css` (240 lines), and is imported once in `src/main.tsx`.
- **Tokens** (`src/styles/tokens.css`, 234 distinct custom-property names): two layers.
  - Primitives (`--vector-*`): navy, slate, silver, cyan, amber, green and red scales, plus fonts and shadows.
  - Semantics: `--color-*`, `--space-*`, `--radius-*`, `--text-*`, `--duration-*`, `--ease-*`, z-layers and `--row-height`.

  The file cites `design/Vector App.dc.html` as the source of its values. Hex values in module CSS appear only inside comments.
- **Theme:** one dark navy theme. `:root { color-scheme: dark }` (`tokens.css`). There is no `prefers-color-scheme` block and no light-theme rule. Density is dense by default; `:root[data-density='comfortable']` overrides it.
- **Fonts:** `--vector-font-sans` is Space Grotesk and `--vector-font-mono` is IBM Plex Mono, each with system fallbacks. Both load from Google Fonts in `index.html`.
- **Accessibility:** `base.css` contains `@media (forced-colors: active)` and `@media (prefers-reduced-motion: reduce)` blocks.

9.9 The `design/` folder

`design/` (19 files, about 297 KB of content; 329 KB on disk) contains Claude Design canvas files (`*.dc.html`) and their runtime, `design/support.js` (whose header marks it as generated):

- `Vector App.dc.html`, the token authority cited by `tokens.css`
- `Vector Landing.dc.html`
- `design/instrument/*.dc.html` (Board, Inbox, InstrumentIssues, InstrumentOnboarding, IssueDetail, Main, OnboardingInvite, Palette, Projects, Settings)
- `design/_ds/organic-2e12fdf2-…/`, an earlier "Organic" design-system bundle (`styles.css`, `_ds_bundle.js`, `_ds_manifest.json`, `readme.md`, `_adherence.oxlintrc.json`). `e2e/design-probe.mjs` checks that its colors do not leak into the app.

9.10 Gaps / to verify

Each item below was checked in code:

1. **Onboarding and invite routes have no client guard.** They are spread unwrapped (`src/features/onboarding/index.tsx`, `src/app/routes/routes.tsx`). The comment in `OnboardingLayout.tsx` says `<RequireAuth>` "does not exist yet", which is stale: it exists in `src/features/auth/guards.tsx`. The server still refuses with `UNAUTHENTICATED`.
2. **New issues always go to the first team.** `useCreateIssue` uses `teams[0]?.id` (`src/features/issues/api/useCreateIssue.ts:57–59`, marked with a `ponytail:` comment). The composer has no team picker.
3. **`/semantic-search` is reachable only by URL.** `paths.semanticSearch` is called only in a test (`src/app/routes/secondWave.test.tsx`), and the segment appears only in `paths.ts` and `routes.tsx`; there is no sidebar entry, palette command or page link.
4. **Dead or unused code:**
   - `WorkspaceEntry` and its query are exported from `src/app/routes/index.ts` but no route mounts them.

    - `useTheme` (`src/app/layout/preferences.ts`) has no callers.
    - `Tooltip` is not used in `src/features` or `src/app`.
    - `ToastProvider` is never mounted, and `useToast` throws without it.
    - `SECOND_WAVE` is an empty table (`routes.tsx`).
5. **Stale comments:**
    - `routes.tsx` still describes "the one second-wave route".
    - `AccountMenu.tsx` cites `app/layout/accountMenu.test.tsx`, which does not exist; the assertion is in `src/app/shell.test.tsx`.
    - `tsconfig.node.json` says `vitest.config.ts` "does not exist yet".
    - `src/features/onboarding/index.tsx` still proposes wiring `WorkspaceEntry`.
6. **Node engine mismatch:**
    - `package.json` declares `engines.node: ">=20.19"`.
    - The lockfile has stricter requirements: `vitest@5.0.0` needs `^22.12.0 || ^24.0.0 || >=26.0.0`, and `@testing-library/jest-dom@7.0.1` needs `>=22`.
    - The Dockerfiles use Node 22 (`ARG NODE_VERSION=22` in both `frontend/Dockerfile` and the root `Dockerfile`).
7. **`e2e/design-probe.mjs` is out of date.** It passes only if the landing page is LIGHT (`Landing is LIGHT … PASS/FAIL`), but `src/features/landing/landing.module.css` uses `var(--color-bg-app)`, and only a dark palette exists.
8. **`codegen.ts` declares `enumsAsTypes: true` twice** (lines 133 and 161) in the same config object. This is harmless, and the file is not type-checked.
9. **`/onboarding` is defined three times:** `ONBOARDING_PATH` in `src/app/routes/paths.ts` and in `src/features/auth/api/destination.ts`, and `ONBOARDING_ROOT` in `src/features/onboarding/lib/progress.ts`.
10. **No CSRF token or header** is sent by the SPA. The defense depends on `SameSite=Lax`, JSON-only POST, and no CORS.
11. **`nginx.conf` sets no security headers** (no CSP, `X-Frame-Options`, HSTS or `nosniff`) and no gzip. The FastAPI SPA path (`app/spa.py`) does send a CSP on `index.html`.
12. **No real-time updates:** inbox and unread counts refresh only through refetches or navigation.
13. **No `eslint-plugin-react-hooks`,** and `e2e/`, `codegen.ts` and `playwright.config.ts` are neither linted nor type-checked (`eslint.config.js` comments).

> **Gaps / to verify:**
>
> - **Unit-test flakiness.** In the research run, the full `vitest run` timed out on 5 tests and then 3, at the default 5000 ms, in different files each time (the issues accessibility test, releases, savedViews refusals, semanticSearch). This was not re-run for this section, and `vitest.config.ts` sets no `testTimeout`.
> - **Which production path is canonical.** It is unclear whether production uses nginx (`web`) or FastAPI serving `static/`. Both are built; the deploy target (see the infra section) decides.
> - **Older integration start paths.** `/github/install` and `/slack/oauth/start` are used by `SettingsPage.tsx` and `IntegrationsStep.tsx`, while `vite.config.ts` names `/integrations` as the configured prefix. Both prefixes are proxied and, per `app/main.py`, both are registered; whether the older paths are a deliberate permanent alias is unconfirmed.

10. Backend

Python 3.12, FastAPI 0.141.1, Strawberry GraphQL 0.326.0, asyncpg 0.31.0, PostgreSQL 18 (`requirements.txt`, `Dockerfile :: PYTHON_VERSION=3.12.14`). No ORM: every statement is hand-written, parameterized SQL.

10.1 Project layout

Path	Responsibility	Notes
`app/main.py`	Composition root: `create_app()` factory (middleware, routers, SPA) and `lifespan` (pool, argon2 warm-up, four background tasks)	Run as `uvicorn app.main:create_app --factory`; no module-level `app`
`app/config.py`	`Settings` (pydantic-settings), `get_settings()` with `@lru_cache(maxsize=1)`	`DATABASE_URL` and `ENVIRONMENT` (`development` | `test` | `production`) are required
`app/db.py`	Process-wide asyncpg pool: `connect`, `disconnect`, `get_pool`	`min_size=1, max_size=5, command_timeout=10.0`, `statement_timeout=9500` ms
`app/http_*.py`, `app/spa.py`	Edge concerns: body limit, security headers, session cookie, client-IP resolution, SPA static hosting	See §5.2
`app/domain/` (34 modules)	Frozen dataclasses (entities, scopes, filters, patches), enums, pure rules (recurrence arithmetic, cursor codecs, mention parsing, message formatting), domain exceptions	Imports no Strawberry, FastAPI or asyncpg (grep)
`app/repositories/` (33 modules)	One class per aggregate holding SQL; methods take `connection` and a keyword-only `scope`; map `asyncpg.Record` → domain entity	Never call `get_pool()` or `acquire()` (grep)
`app/services/` (29 modules)	Business rules, validation, transaction boundaries, external HTTP, background workers (`schedule.py`, `embedding_jobs.py`, `notifications.py`, `auth.py :: run_sweep_loop`)	Shared write-side helpers are module functions in `activity.py` and `events.py`
`app/graphql/`	`schema.py` (root merge, masking, introspection), `router.py`, `context.py` (per-request services and DataLoaders), `scope.py` / `viewer.py` (authn/z), `limits.py` (operation budgets), `errors.py`, and subpackages `queries/`, `mutations/`, `inputs/`, `types/`, `loaders/`	One `Query`/`Mutation` class per feature, merged in `schema.py`
`app/rest/`	`health.py` (`/healthz`, `/readyz`), `github.py` (install, callback, webhook), `slack.py` (OAuth, events), `oauth_origin.py` (callback-host hop)	REST only where protocol-specific
`migrations/`	`001`–`032` numbered SQL files, applied by `scripts/apply_migration.py` with a checksum ledger	Immutable once applied
`tests/`	117 entries: unit, GraphQL, structural gate tests, and `-m db` tests against a real PostgreSQL	§10.2

10.2 Layering rules and enforcement

Request path: resolver → service → repository → asyncpg (`CLAUDE.md`; `app/graphql/context.py :: get_context`).

- **Resolvers are thin.** A resolver calls `authorized_scope(info, slug)`, makes one service call, and maps the entity to a Strawberry type (example: `app/graphql/mutations/issues.py :: Mutation.issue_create`).

- **Services own transactions; repositories receive connections.** Repository methods take the connection first, and workspace-scoped ones a required keyword `scope`: `(self, connection: asyncpg.Connection, *, scope: WorkspaceScope, …)` (e.g. `app/repositories/issues.py :: IssueRepository.create`). Read paths acquire without a transaction (e.g. `IssueService.list`); `AnalyticsService.overview` is the noted exception that wraps reads in one READ COMMITTED transaction.
- **Tenant scope as a type.** `WorkspaceScope(workspace_id)` and its subclass `AuthorizedWorkspaceScope(workspace_id, user_id, role)`, which is only built from a `workspace_members` row (`app/domain/tenancy.py`; `app/graphql/scope.py :: authorized_scope`). Cross-tenant references are additionally blocked by composite foreign keys such as `issues_team_fk (workspace_id, team_id)`.
- **Shared write helpers.** `activity.record`, `record_changes`, `notify`, `auto_subscribe` and `events.record_issue_event` are module-level functions that take the caller's connection, so they join the caller's transaction (`app/services/activity.py`, `app/services/events.py`; the activity module docstring says this avoids a constructor argument costing fifteen call sites and four `__init__` signatures).

Concrete transaction example (`app/services/issues.py :: IssueService.create`): pure validation → `pool.acquire()` → estimate-scale read outside the transaction → `connection.transaction()` containing default-state lookup, `issue_counter` increment (row lock), `INSERT`, activity row, optional subscribe + notify → commit → map `TeamNotFoundError` / `ForeignKeyViolationError` to `ValidationError` after rollback. Full trace in §5.4.

Record → domain conversion: each repository has a `@staticmethod _to_entity(row: asyncpg.Record)` that returns a frozen, slotted dataclass (e.g. `IssueRepository._to_entity`, re-exported as `issue_from_row`). Scalars from `fetchval` are bound to annotated locals so mypy does not accept `Any` (`TeamRepository.allocate_issue_number`). A grep finds no `asyncpg.Record` in `app/services` or `app/graphql`.

How the rules are enforced:

Rule	Mechanism	Source
Every repository SQL statement is tenant-scoped	AST walk over `app/repositories/`: each statement string must mention `workspace_id` or touch only `TENANT_FREE_RELATIONS = {auth_rate_limits, github_deliveries, sessions, slack_event_deliveries, users, workspaces}`; a floor on the number of statements found stops the gate going blind	`tests/test_repository_tenancy_gate.py :: test_every_statement_is_scoped_to_a_workspace`, `test_the_collector_found_the_layer`, `test_the_tenant_free_list_is_still_earned`
Row mappers read every entity field	AST discovery of `_to_entity`-style mappers; each must read one key per dataclass field	`tests/test_integration_gates.py :: test_every_mapper_reads_every_field_of_its_entity`, `test_the_mappers_were_found`
Every feature's Query/Mutation reaches the root; roots defined once	Introspection of the merged schema	`tests/test_integration_gates.py :: test_every_query_type_reaches_the_root`, `test_every_mutation_type_reaches_the_root`, `test_the_root_types_are_defined_exactly_once`
Context builds every declared service; resolvers only read existing context attributes	Construction + AST scan	`tests/test_integration_gates.py :: test_get_context_constructs_every_service_it_declares`, `test_resolvers_only_read_context_attributes_that_exist`
Applied migrations are immutable	SHA comparisons against reviewed hashes; pre-commit hooks may not rewrite `migrations/`	`tests/test_phase1a2_gates.py :: test_the_applied_migration_still_hashes_to_what_the_ledger_recorded`, `test_pre_commit_keeps_file_rewriting_hooks_away_from_migrations`

Rule	Mechanism	Source
Static typing and lint strictness cannot silently loosen	Asserts on `pyproject.toml` (mypy strict flags, one override, exactly one `type: ignore`; ruff rule families)	`tests/test_phase1a2_gates.py :: test_mypy_keeps_every_strictness_flag_on`, `test_the_tree_carries_exactly_one_type_ignore`, `test_ruff_still_selects_every_agreed_rule_family`
Notification kinds stay consistent across domain, SQL CHECK and GraphQL	Cross-check	`tests/test_phase1a2_gates.py :: test_every_notification_kind_is_declared_in_all_three_places`
Cross-tenant behaviour end to end	Real-DB tests	`tests/test_issue_tenancy_db.py`, `tests/test_graphql_tenancy_surface_db.py`

Not enforced by a test, but true today by grep: `app/domain` imports none of Strawberry/FastAPI/asyncpg; no repository calls `get_pool()`/`acquire()`; no `OFFSET` appears in repository SQL (only in docstrings saying "No OFFSET").

10.3 Validation

Three layers:

1. **Transport parsing.** Strawberry types the input (`UUID`, `date`, `int`); malformed values fail GraphQL validation before any resolver (a public error because it has no `original_error`, `app/graphql/schema.py :: is_public_error`). Inputs omit server-owned fields (`IssueCreateInput` has no `creatorId`, `workflowStateId`, `completedAt`, `app/graphql/inputs/issue.py`).
2. **Service validation (pure).** Services collect every `ValidationIssue(field, code, message)` and raise one `ValidationError(issues)` (`app/domain/errors.py`). Example rules for issues: title 1–500 chars (`REQUIRED`/`TOO_LONG`), priority 0–4 (`OUT_OF_RANGE`), estimate ≥ 0 and on the team's scale, non-empty patch (`EMPTY`), no null for NOT NULL columns (`NOT_NULLABLE`), `first` 1–100, cursor decodes (`INVALID_CURSOR`) and matches the ordering (`ORDER_MISMATCH`) (`app/services/issues.py :: _title_issues`, `_priority_issues`, `_estimate_issues`, `_validate_patch`, `_validate_list`). Out-of-range values are rejected, not clamped (e.g. `SearchService._validate`).
3. **Database constraints as validation.** Constraint violations that a client can provoke are translated by exact `constraint_name` match; anything else is re-raised (`IssueService._validation_error_for` with `_EXPECTED_FOREIGN_KEYS`; `_raise_mapped` in projects/initiatives/documents/releases/saved views; `CycleService._overlapping_dates`).

Codes are UPPER_SNAKE strings (about 56 distinct; most frequent `NOT_FOUND`, `OUT_OF_RANGE`, `TOO_LONG`, `INVALID`, `REQUIRED`, `INVALID_CURSOR`), except the Slack service, which uses lowercase codes such as `not_connected`, `unknown_channel`, `channel_archived`, `unknown_event` (`app/services/slack.py`).

10.4 Error handling

Domain exceptions (`app/domain/errors.py`):

Class	Meaning
`ValidationIssue`, `ValidationError`	Expected input failures; message always "Validation failed", detail in `issues`
`AuthenticationError`	Log-in failed; unknown email and wrong password indistinguishable
`EmailAlreadyRegisteredError`	Unique violation translated by the user repository
`TeamNotFoundError`	Missing team and other-tenant team are the same
`WorkspaceNotFoundError`	Slug resolves to nothing

Class	Meaning
`WorkspaceAccessDeniedError`	No such workspace or not a member (one statement), or insufficient role (`app/domain/tenancy.py :: require_workspace_admin`)
`GithubNotConfiguredError`, `GithubInstallationClaimedError`, `GithubRepositoriesInUseError`	GitHub integration states

Others live beside their domain: `InvalidCursorError` (`pagination.py`), `UnreachableRecurrenceError` (`recurrence.py`), `RelatedIssueNotFoundError`, `DuplicateRelationError` (`relations.py`), `InvalidStoredFilterError` (`saved_views.py`), `DocumentContentError`, `InvalidStoredContentError` (`documents.py`), `SlackOAuthError`, `SlackTeamAlreadyConnectedError`, `SlackApiError` (`slack.py`).

GraphQL channels:

1. **Mutation payload errors.** `XPayload { <entity>, errors: [ValidationErrorType{field, code, message}] }`, HTTP 200 (`app/graphql/types/errors.py :: ValidationErrorType.from_domain`). Each mutation module has its own `_errors(exc)` helper (16 copies of `def _errors` under `app/graphql`).
2. **Query errors.** `app/graphql/errors.py :: bad_user_input` returns `GraphQLError(message, extensions={"code": "BAD_USER_INPUT", "issues": [...]})`, which callers raise `from None`.
3. **Auth/tenancy.** `UNAUTHENTICATED` (`app/graphql/viewer.py :: viewer_user_id`) and `NOT_FOUND` "Workspace not found" (`app/graphql/scope.py`); a forbidden role yields the same `NOT_FOUND`.
4. **Masking.** `PUBLIC_ERROR_CODES = {"BAD_USER_INPUT", "UNAUTHENTICATED", "NOT_FOUND"}`. Everything else becomes "Internal server error", applied twice: the `MaskErrors` extension and `_MaskedSchema.execute`/`execute_sync` → `_mask_result`, which also covers exceptions raised during validation (`app/graphql/schema.py`). Masking is on in every environment; `DisableIntrospection` only in production.

REST errors use `HTTPException` with fixed, non-revealing details: webhook signature failure → 401 "Unauthorized" for every cause; malformed webhook JSON → 400; unconfigured GitHub → 404, unconfigured Slack → 503; non-admin → 404 (`app/rest/github.py :: webhook, install`; `app/rest/slack.py :: slack_oauth_start, slack_events`). The GitHub OAuth callback converts `HTTPException` to `PlainTextResponse` so the state cookie can be cleared on every exit (`app/rest/github.py :: callback`).

Broad catches are limited to supervisor loops (`run_forever`/`run_delivery_loop`/`run_sweep_loop`), per-item embedding/recurrence failures, and `GithubService.installations_for_grant`, which fails closed to an empty grant (`app/services/github.py`, documented as deliberate).

10.5 Partial-update (patch) pattern

- **Three states at the transport.** `IssueUpdateInput` defaults every field to `strawberry.UNSET`; `to_patch()` maps each through `_patched` with an identity check, producing an `IssuePatch` whose fields are a value, `None`, or the domain `UNSET` (`app/graphql/inputs/issue.py`).
- **Validation.** `IssueService._validate_patch` rejects an all-UNSET patch (`EMPTY`) and an explicit null on `title`, `priority`, `workflowStateId` (`NOT_NULLABLE`).
- **One-statement SQL, no read-modify-write.** `IssueRepository.update` passes a flag/value parameter pair per field (via `_value`) and writes `title = CASE WHEN $3 THEN $4::TEXT ELSE issues.title END`, etc. `completed_at` is recomputed in the same statement from the target state's category (`COALESCE(issues.completed_at, now())` when terminal, else `NULL`). The same CASE pattern is used by `ProjectRepository.update`, `InitiativeRepository.update`, `DocumentRepository.update`, `SavedViewRepository.update` and `BulkRepository`.
- **History.** `IssueRepository.lock_snapshot` (`SELECT … FOR UPDATE`) captures an `IssueSnapshot` before the update; `activity.record_changes(before, after)` writes one `issue_activity` row per changed field and emits notifications and domain events (§10.8).
- **Two sentinels exist.** `app/domain/issues.py :: Unset` (single-member `Enum`, used by issues, bulk, saved-view filters) and `app/domain/patch.py :: UnsetType` (falsy `__slots__` class, used by projects, initiatives, documents, releases, saved views).

10.6 Keyset pagination and cursors

- **Encoding.** `_encode_payload` = URL-safe Base64 of `json.dumps(payload, separators=(",", ":"), sort_keys=True)`; deterministic, not signed or encrypted. The workspace is never inside a cursor; it always comes from the authorized scope (`app/domain/pagination.py`).
- **Codecs.**

Codec	Payload	Used by
`encode_issue_cursor` / `decode_issue_cursor` (aliases `encode_keyset_cursor` / `decode_keyset_cursor`)	`{"created_at": iso, "id": uuid}`, timezone-aware only	Projects, initiatives, documents, releases, notifications, activity, comments
`encode_issue_list_cursor` / `decode_issue_list_cursor`	`{"o": "<field>:<dir>", "k": key-or-null, "id": uuid}`; key type decided by the ordering field (`_parse_order_key`)	Issue list (`PRIORITY`, `CREATED_AT`, `UPDATED_AT`, `DUE_DATE`, ASC/DESC, `id` tie-break)
`encode_label_cursor` / `decode_label_cursor` (aliases `encode_name_cursor`)	`{"name": stored-name, "id": uuid}`	Labels, saved views

- **SQL.** `app/repositories/issues.py :: _add_keyset` builds a row-value predicate `(key, issues.id) < ($k, $id)` (or `>`), with explicit `IS NULL` branches for nullable keys. It relies on PostgreSQL defaults (ASC → NULLS LAST, DESC → NULLS FIRST) so one index serves both directions. Matching partial indexes exist, e.g. `issues_workspace_live_created_at_id_idx (workspace_id, created_at DESC, id DESC) WHERE archived_at IS NULL` (migrations 006, 015).
- **Page detection.** Fetch `first + 1` rows; `has_next_page = len(rows) > first` (`IssueService.list`).
- **Errors.** Any decode failure → `InvalidCursorError` → `after: INVALID_CURSOR`; a cursor minted under another ordering → `ORDER_MISMATCH`.
- **Unpaginated by design.** Search returns two capped lists (`first` ≤ 100, no cursor) (`app/domain/search.py :: SearchResults`); subscribers are capped at `SUBSCRIBERS_MAX = 500`; `IssueType.labels` is bounded by `LABELS_PER_ISSUE_MAX = 50` at write time.

10.7 Background jobs

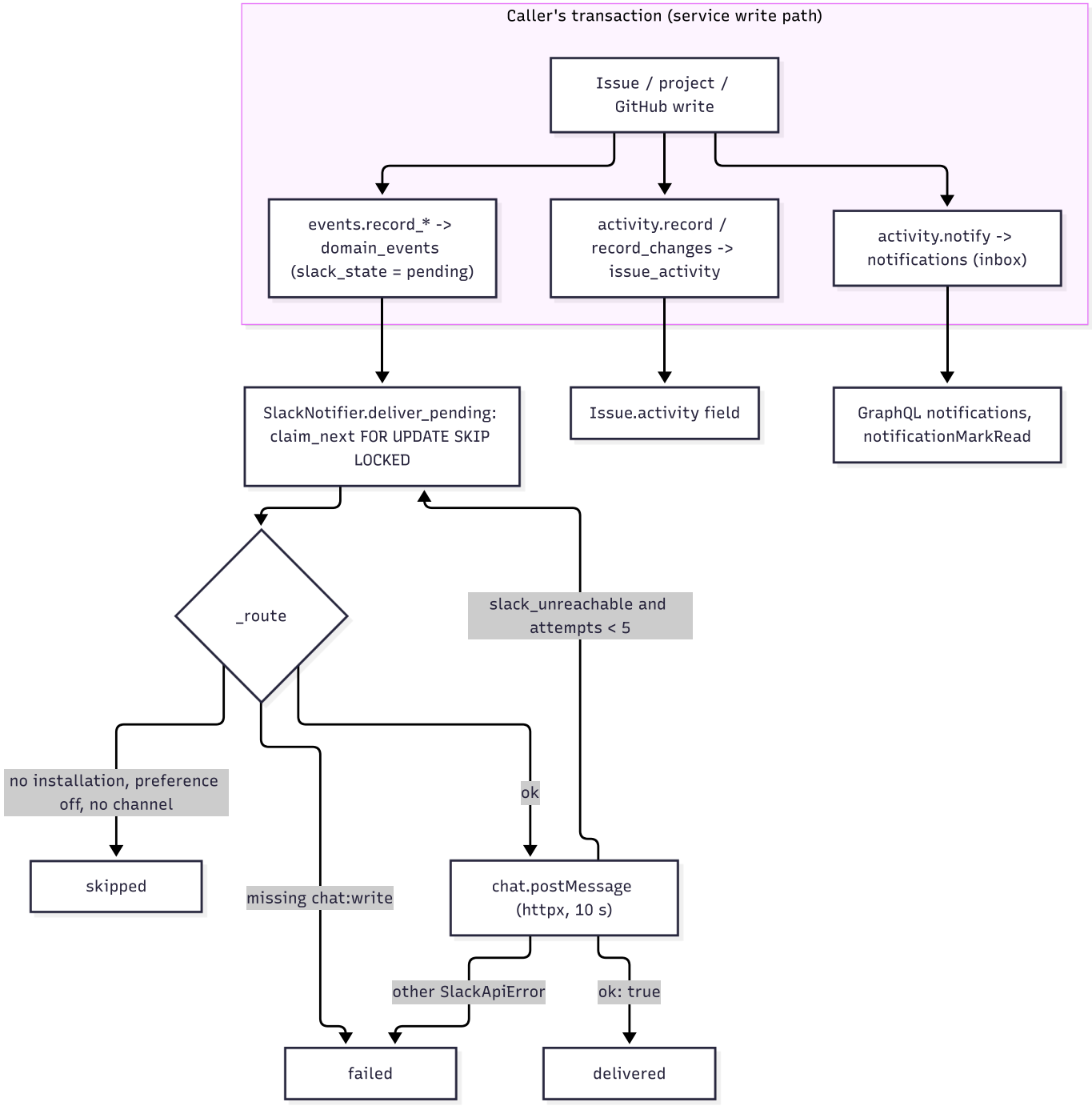
All four run as `asyncio` tasks in the API process, started in `app/main.py :: lifespan` after `connect()` and `warm_password_hashing()`. There is no Celery, cron or broker, and no advisory locks in the loops. On shutdown each is cancelled and awaited under `suppress(CancelledError)` in the order schedule → embedding → delivery → sweep, then `disconnect()`. Every loop catches `Exception` (logs and continues) and lets `CancelledError` propagate.

Task	Entry point	Enabled	Interval	Claiming	Retries / failure
Embedding worker	`app/services/embedding_jobs.py :: EmbeddingWorker.run_forever`	Only if `EMBEDDING_WORKER_ENABLED` (`Settings.embedding_worker_enabled`, default `False`; set `"true"` in compose, k8s and Render)	Immediately again if a pass wrote work; `IDLE_POLL_SECONDS = 30` when idle; `ERROR_BACKOFF_SECONDS = 60` after a failed pass	Enqueue by polling: `enqueue_stale` (≤ `ENQUEUE_BATCH = 500`, all workspaces, anti-join on model + digest). Claim: CTE `… WHERE next_attempt_at <= now() … LIMIT 50 FOR UPDATE SKIP LOCKED`, then `next_attempt_at = now() + 300 s` lease, committed before embedding	`MAX_ATTEMPTS = 5`; delay `min(30 · 2^(n-1), 3600)` s; exhausted → `next_attempt_at = 'infinity'` and `last_error` (≤ 500 chars). Batch embed falls back to per-item on failure (`app/domain/embedding_jobs.py`; fallback in `app/services/embedding_jobs.py :: EmbeddingWorker._embed`)

Task	Entry point	Enabled	Interval	Claiming	Retries / failure
Slack delivery	`app/services/notifications.py :: run_delivery_loop(SlackNotifier)`	Always, even without Slack (closes rows as `skipped`)	`POLL_INTERVAL_SECONDS = 5.0` after every pass; ≤ `DRAIN_BATCH = 20` events per pass	`EventRepository.claim_next`: one row, `slack_state = 'pending' AND slack_next_attempt_at <= now()`, `FOR UPDATE SKIP LOCKED`, attempts incremented at claim, own committed transaction	Linear backoff `RETRY_DELAY (1 min) × LEAST(attempts, BACKOFF_STEPS = 5)`; only `slack_unreachable` is retried (up to `MAX_ATTEMPTS = 5`); any other failure → terminal `failed` with a reason
Schedule worker	`app/services/schedule.py :: ScheduleWorker.run_forever` → `run_once` = `_remind` then `_generate`	Always	`POLL_INTERVAL_SECONDS = 900` (15 min), sleep after every pass outside the `try`	Reminders: `DueReminderRepository.claim_due` — `INSERT … SELECT … ON CONFLICT DO NOTHING RETURNING` into the `issue_due_reminders` ledger (PK `workspace_id, issue_id, due_date`), batches of `REMINDER_BATCH = 200`, `REMINDER_LEAD_DAYS = 1`. Recurrences: `RecurrenceRepository.lock_due(limit=25)` with `FOR UPDATE SKIP LOCKED`, `advance` in the same transaction	Reminder claim + `DUE_SOON` fan-out share one transaction (exactly-once). Recurrence issue filing (`TemplateService.apply`) runs after the advance commits; a failure is logged (`schedule.recurrence_failed`) and that occurrence is lost (at-most-once). One recurrence batch of 25 per pass
Auth sweep	`app/services/auth.py :: run_sweep_loop(AuthService)` → `sweep_once`	Always, every replica	`SWEEP_INTERVAL_SECONDS = 900.0`	None needed: idempotent `DELETE`s of expired sessions and stale rate-limit rows in batches of `SWEEP_BATCH = 1000`, looping until a batch comes back short	Logs `auth.sweep.pass_failed`; logs collected counts only when non-zero

Multi-replica safety is the design argument in each builder's docstring (`app/main.py :: _start_embedding_worker`, `_build_schedule_worker`, `_build_notifier`); k8s runs two API replicas, so two copies of every loop.

10.8 Domain events (outbox), activity log, notifications



Activity log (`issue_activity`, migration 012; `caused_by` added in 030). Kinds: `created`, `title_changed`, `state_changed`, `priority_changed`, `assignee_changed`, `archived`, `commented`, `label_attached`, `label_detached`, `relation_added`, `project_changed`, `cycle_changed` (`app/domain/activity.py :: ActivityKind`). Values are two opaque TEXT columns (`from_value`, `to_value`), not JSONB. `changes(before, after)` diffs `IssueSnapshot` fields title, workflow_state_id, priority, assignee_id, project_id, cycle_id. Read via `ActivityService.list_for_issue` (keyset, newest first).

Inbox notifications (`notifications`, migration 012). Kinds `assigned`, `commented`, `blocked`, `status_changed`, `due_soon` (`app/domain/notifications.py :: NotificationKind`; no mentions). Generated by one statement, `NotificationRepository.notify_about_issue`: recipients = assignee (+ creator when `include_creator`) ∪ `issue_subscribers`, active members only, actor excluded, `SELECT DISTINCT`. Call sites: assignment and state change (`activity.record_changes`), create-with-assignee (`IssueService.create`), comments (`include_creator=True`), blocking relations, GitHub automation (actor `None`), due-soon reminders (`include_creator=True`). Subscribers are added automatically on assignment and on commenting (`activity.auto_subscribe`). Inbox operations: list (keyset, `unread_only`), unread count, mark read (idempotent

`COALESCE(read_at, now())`), mark all read. No archive, snooze, delete, or cross-event aggregation; not delivered by email/push/Slack.

Domain events / transactional outbox (`domain_events`, migration 027; PK `(workspace_id, kind, dedupe_key)`). Kinds: `issue_assigned`, `issue_completed`, `issue_priority_urgent`, `project_health_changed`, `project_update_published`, `pull_request_merged` (`app/domain/events.py :: DomainEventKind`). Emitters run on the caller's connection, so the event commits or rolls back with the change (`app/services/events.py`):

Emitter	Trigger	Dedupe key
`record_issue_event` from `activity.record_changes`	Assignee set to non-null; workflow-state move (row inserted only if the new state's type is `completed`); priority moved to 1 (urgent)	`uuid4().hex`
`record_issue_event` from `GithubService` automation	PR-driven state move	`uuid4().hex`
`record_pull_request_merged`	Merged PR webhook, one row per linked issue	`"{repository_id}/{number}/{issue_id}"` (idempotent across redeliveries)
`record_project_update`	Project update posted (`project_update_published`); plus `project_health_changed` only if health `IS DISTINCT FROM` the previous value	`uuid4().hex`

The only consumer is `SlackNotifier` (§10.7). Message text: `"{HEADLINE} · {subject} — {summary}"` plus `\n{PUBLIC_BASE_URL}{path}` when configured (`app/domain/events.py :: message_for`). `IssueService.create` emits no event; triage state moves (`app/services/triage.py`) record activity but emit neither events nor notifications.

10.9 Search

Full-text. PostgreSQL FTS only (`migrations/011_search.sql` rejects Elasticsearch/Typesense and pg_trgm). `issues.search_vector` and `projects.search_vector` are generated STORED `tsvector`s: `setweight(to_tsvector('english', title), 'A') || setweight(to_tsvector('english', description), 'B')` (projects use `name`). Indexed by composite GIN `(workspace_id, search_vector)` via `btree_gin` (issues partial on `archived_at IS NULL`). Queries use `websearch_to_tsquery('english', $2)` ranked by `ts_rank DESC, id DESC` (`IssueRepository.search`, `ProjectRepository.search`). An exact identifier such as `ENG-12` is matched first by `parse_issue_identifier` and `IssueRepository.get_by_identifier` (`app/domain/search.py`, `app/services/search.py :: SearchService.search`). Limits: query ≤ 200 chars, `first` 1–100; empty query returns empty results without a DB round trip.

"Semantic" search, precisely.

- **Embedder.** `load_embedder()` (`@functools.cache`) returns `SentenceTransformerEmbedder("sentence-transformers/all-MiniLM-L6-v2")` only if the `sentence_transformers` package is importable; it is not in `requirements*.txt` or `pyproject.toml`, so deployed images use `HashingEmbedder`, named `hashing-v1` (`app/services/embeddings.py`).
- **`hashing-v1`** is signed feature hashing, a bag of words: lowercase tokens matching `[a-z0-9]{2,}` plus 6-character prefixes of longer tokens, each hashed with `blake2b(digest_size=5)` into one of 384 buckets with a ±1 sign, then L2-normalised. Its own docstring says it is not a semantic model; it captures lexical overlap, not meaning.
- **Storage.** `issue_embeddings.embedding vector(384)` (pgvector), PK `(workspace_id, issue_id)`, with `model` and `source_digest` (`migrations/025_semantic_search.sql`; `EMBEDDING_DIMENSIONS = 384` in `app/domain/semantic_search.py`). Staleness: generated `issues.embedding_source_digest = md5(title) || md5(description)`; rows whose digest or model differ are ignored at read time.
- **No ANN index.** No HNSW or IVFFlat; only B-tree `issue_embeddings_workspace_model_digest_idx`. Similarity is an exact per-workspace scan with cosine distance `<=>` (migration 025 "THE INDEX DECISION": a global ANN index filtered by tenant after `LIMIT` would be a cross-tenant correctness defect).
- **Hybrid ranking.** `EmbeddingRepository.search_hybrid` is one statement: lexical CTE and vector CTE (each `LIMIT FUSION_DEPTH = 100`), `FULL OUTER JOIN`, Reciprocal Rank Fusion `1/(60 + rank_lex) + 1/(60 + rank_vec)` (`RRF_K = 60`). Always used for issues in the current wiring because `get_context` always passes an embedder; projects stay lexical-only.

- **Duplicate suggestions.** `SearchService.suggest_duplicates` → `EmbeddingRepository.search_similar`, vector-only, `1 - distance >= MIN_DUPLICATE_SIMILARITY = 0.6`.
- **Indexing.** Background worker (§10.7) or manual `embeddingsRefresh` (any member, ≤ `REFRESH_MAX = 500` issues, `SearchService.refresh_embeddings`). Query embedding runs synchronously on the event loop before acquiring a connection.

10.10 Integrations

Outbound HTTP. `httpx==0.28.1`, a new `AsyncClient` per call or crawl, no in-call retries:

Caller	Library, timeout	Endpoint
`GithubUserInstallations.installations_for_user` (`app/services/github.py`)	httpx, `VERIFY_TIMEOUT_SECONDS = 10.0`	`POST github.com/login/oauth/access_token`; `GET api.github.com/user/installations?per_page=100`; `GET …/user/installations/{id}/repositories?per_page=100` (one page)
`SlackOAuthClient.exchange` → `_post_form` (`app/services/slack.py`)	`urllib.request` in `asyncio.to_thread`, `OAUTH_EXCHANGE_TIMEOUT_SECONDS = 10.0`	`POST slack.com/api/oauth.v2.access`
`SlackWebClient.conversations_list`	httpx, `WEB_API_TIMEOUT_SECONDS = 10.0`	`GET slack.com/api/conversations.list` (≤ 10 pages)
`SlackWebClient.post_message`	httpx, 10 s	`POST slack.com/api/chat.postMessage`

No GitHub App JWT or installation access tokens are minted; `app_id`/`private_key` only decide whether GitHub is "configured" (`app/services/github.py`).

GitHub App (`app/rest/github.py`, `app/services/github.py`, migrations 013, 016, 030):

- Install: admin-only `GET /github/install` sets an unsigned base64url state cookie `vector_github_state` (`{state, session digest, workspace, return_to}`, 600 s) and redirects to GitHub. Callback verifies state and session digest with `secrets.compare_digest`, exchanges the OAuth `code` to confirm the user can see the installation, then records a claim (`connect`) and confirms it (`confirm_claim`, 15-minute `CLAIM_TTL`).
- Webhook: `POST /github/webhook` verifies `X-Hub-Signature-256` (HMAC-SHA256, `hmac.compare_digest`) over the raw body; handles `installation`, `installation_repositories`, `pull_request`, `push`; one transaction per delivery deduped by `X-GitHub-Delivery` in `github_deliveries`; 204 for every verified, well-formed delivery (malformed JSON after verification → 400).
- Linking: PR title/body/branch and commit messages are scanned for identifiers like `ENG-12` (no magic words, ≤ 20 per source) and linked only to unarchived issues in the owning workspace (`app/domain/github.py :: issue_identifiers`).
- Automations: per-team config moves issues forward only — PR opened/reopened/ready → `started`, merged → `completed` — via one `UPDATE` (`GithubRepository.move_issues_for_automation`), writing activity with `caused_by`, a `status_changed` notification, and an `issue_completed` event.

Slack (`app/rest/slack.py`, `app/services/slack.py`, migrations 014, 018):

- OAuth v2 with scopes `channels:read,chat:write`; colon-joined state cookie `vector_slack_oauth`; bot token stored **plaintext** in `slack_installations.bot_token_reference` via `DatabaseTokenStore` (documented, with `SlackTokenStore` as the swap seam).
- One default channel per workspace plus per-event opt-in preferences (default off).
- Inbound `POST /slack/events`: v0 signature with a 300 s timestamp window, `url_verification` challenge, bot-loop prevention, dedupe in `slack_event_deliveries`; no event ingestion is implemented.
- Outbound: only the outbox loop and a "send test notification" mutation post messages.

10.11 Caching

Layer	What	Source
Process	`get_settings` `@lru_cache(maxsize=1)`; `load_embedder` `@functools.cache`; argon2 decoy hash `@cache`, warmed at startup	`app/config.py`, `app/services/embeddings.py`, `app/services/passwords.py :: _decoy_hash`
Request	Strawberry DataLoaders on `VectorContext`, some built lazily via `functools.cached_property`: `issue_labels`, `issue_summaries`, `cycle_loader`, `project_loader`, project milestones/updates/dependencies loaders, `initiative_updates_loader`, `document_content_loader`, `revision_content_loader`. Keys include the workspace	`app/graphql/context.py`, `app/graphql/loaders/*`
HTTP	SPA only: hashed `/assets/*` → `public, max-age=31536000, immutable`; `index.html` → `no-store`. GraphQL is POST-only and uncached	`app/spa.py :: ASSET_CACHE_CONTROL`, `INDEX_CACHE_CONTROL`; `app/graphql/router.py`
Database	asyncpg's default prepared-statement cache; one cached statement per filter combination for the issue list	`app/db.py`, `app/repositories/issues.py :: _add_filters` docstring

No result cache exists for search, analytics, or unread counts. Not batched (one query per parent row, bounded only by `MAX_COMPLEXITY`): `IssueType.parent`, `children`, `relations`, `activity`, `comments`, `development` (`app/graphql/types/issue.py`).

10.12 Notable design patterns

- **Gapless per-team numbering** with a counter column incremented inside the insert transaction instead of a sequence (`TeamRepository.allocate_issue_number`).
- **Database-enforced tenancy**: `workspace_id` predicate in every statement plus composite FKs `(workspace_id, x)`, checked structurally by a test (§10.2).
- **Existence hiding**: non-member, nonexistent workspace and insufficient role all return the same `NOT_FOUND` (`app/graphql/scope.py`).
- **Transactional outbox** in `domain_events` with delivery state on the row, `FOR UPDATE SKIP LOCKED` claims and a pending partial index (§10.8).
- **Ledger-as-claim** for exactly-once reminders (`INSERT … ON CONFLICT DO NOTHING RETURNING`).
- **Lease-based job queue** with polling enqueue for embeddings, keyed by content digest (§10.7).
- **CASE-per-field single-statement updates** that avoid lost updates (§10.5).
- **Generated columns** for derived data: `search_vector`, `embedding_source_digest`; `completed_at` derived in SQL from state category.
- **Double error masking** with an allowlist of public codes (§10.4).
- **Validation-time query cost limits** instead of runtime timeouts alone (`app/graphql/limits.py :: OperationLimitsRule`); statement timeout 9.5 s below the 10 s client timeout so PostgreSQL cancels first (`app/db.py`).
- **Background work in-process** behind flags, replacing cron/brokers (`app/main.py`).
- `ponytail:` **comments** mark deliberate shortcuts with their ceiling, e.g. bulk activity round trips (`app/services/bulk.py`), Slack `DRAIN_BATCH` pacing, non-atomic `TemplateService.apply`, no pruning of `github_deliveries`/terminal `domain_events` (migrations 017, 027; `slack_event_deliveries` non-pruning is a plain comment in migration 014).

10.13 Gaps / to verify

> **Gaps / to verify:**
>
> - **Semantic search quality.** Deployed embedder is effectively `hashing-v1` (bag of words) because `sentence-transformers` is not a dependency; confirm against the running image. A future real model would run

synchronously on the event loop (no `to_thread`), including first load inside `get_context`.

- **Embedding freshness.** No enqueue on any write path; with the worker disabled (the code default) new or edited issues get no vectors until `embeddingsRefresh`. A job retired to `infinity` is not reset by a model change (`embedding_jobs` has no `model` column).
- **No domain event on create.** `IssueService.create` with an assignee sends no `issue_assigned` Slack event, unlike an update that assigns.
- **Slack delivery edge case.** `SlackNotifier._deliver` catches only `SlackApiError`; any other exception from `_route`/`fetch_token` aborts the batch with the attempt consumed and the row left `pending`, and `claim_next` has no attempts cap, so it can retry indefinitely.
- **Recurrence** is at-most-once, files only one catch-up occurrence after downtime, and handles at most 25 schedules per 15-minute pass.
- **Timezone.** Reminders and recurrence use `CURRENT_DATE`, documented as UTC, but `app/db.py` sets no `TimeZone` server setting.
- **Archived issues.** `IssueRepository.set_cycle`, `set_project` and `lock_snapshot` lack `archived_at IS NULL`; there is no unarchive path although the archive mutation's docstring calls it reversible.
- **Two UNSET sentinels** (`app/domain/issues.py :: Unset`, `app/domain/patch.py :: UnsetType`) and 16 duplicated `_errors` helpers.
- **No pruning** of `github_deliveries`, `slack_event_deliveries` or terminal `domain_events`.
- **Slack OAuth** network errors from `urllib` are not caught in `slack_oauth_callback` (likely 500, state cookie not cleared); bot token stored in plaintext.
- **Stale docstrings**: `app/rest/github.py :: callback` says the OAuth code is never exchanged (it is); `SlackOAuthClient` says no async client exists (httpx is a dependency); `IssueService.create` mentions unauthenticated callers.
- **`embeddingsRefresh`** has no role check (any member, up to 500 embeds per call).
- Layering rules other than tenancy and mapper completeness (domain purity, no `acquire` in repositories, no `Record` leakage, no OFFSET) are verified here by grep only; no test enforces them.

11. Database and infrastructure

11.1 Database choice and configuration

- **PostgreSQL 18 is a hard requirement.** `migrations/001_issues.sql` declares `id UUID PRIMARY KEY DEFAULT uuidv7()`, a function native to PG 18 that does not exist in 17 (`DEPLOYMENT_ENV_VARS.md` "Neon").
- **Local / CI / compose / k8s** use the `pgvector/pgvector:pg18` image (the official `postgres:18` image plus pgvector), because migration 025 needs the `vector` extension (`tests/conftest.py` comment, `docker-compose.yml`, `k8s/20-postgres.yaml`, `run-vector-local.ps1`, `.github/workflows/ci.yml`).
- **Staging** uses a hand-provisioned Neon database (`neondb`, PostgreSQL 18.6); Render's managed Postgres is not used (`DEPLOYMENT_ENV_VARS.md` "Neon"; `render.yaml` declares no database). The DSN must end in `?sslmode=verify-full&sslrootcert=/etc/ssl/certs/ca-certificates.crt`; `channel_binding=require` from Neon's connection string is to be removed (`DEPLOYMENT_ENV_VARS.md` "TLS — and the classic asyncpg failure"). TLS is configured only through the DSN; `app/db.py` passes no SSL arguments.
- **Extensions** are created by migrations, each `IF NOT EXISTS`, none version-pinned:

Extension	Created in	Used for
`btree_gist`	`migrations/008_cycles.sql`	UUID equality inside the GiST exclusion constraint `cycles_no_overlap`.
`btree_gin`	`migrations/011_search.sql`	UUID `workspace_id` as a key column in GIN full-text indexes.
`vector`	`migrations/025_semantic_search.sql`	`issue_embeddings.embedding vector(384)`.

- **Full-text search** uses stored generated columns: `issues.search_vector TSVECTOR GENERATED ALWAYS AS (setweight(to_tsvector('english', coalesce(title, '')), 'A') \|\| setweight(to_tsvector('english', coalesce(description, '')), 'B')) STORED` (`migrations/011_search.sql`), likewise for `projects`.

11.2 Connection handling

Aspect	Value / behaviour	Source
Driver	asyncpg 0.31.0, one process-wide pool	`app/db.py`
Pool size	`min_size=1`, `max_size=5`	`app/db.py::connect`
Client timeout	`command_timeout=10.0` s (`COMMAND_TIMEOUT_SECONDS`)	`app/db.py`
Server timeout	`statement_timeout = "9500"` ms sent as `server_settings` in the startup packet (survives asyncpg's `RESET ALL` on release); kept 500 ms below the client timeout so PostgreSQL cancels first instead of asyncpg opening a second connection for a `CancelRequest`	`app/db.py` `STATEMENT_TIMEOUT_MS` comment
Init	`connect()` in the FastAPI lifespan, then argon2 warm-up, then the `startup.ready` log with `pool_connect_seconds`, `argon2_warm_seconds`, `total_seconds`, `embedding_worker`	`app/main.py::lifespan`
Shutdown	the four background tasks are cancelled and awaited, then `disconnect()` closes the pool	`app/main.py::lifespan`
Access	`get_pool()` raises `RuntimeError` if the pool is uninitialised; nothing else creates a pool	`app/db.py::get_pool`
Request path	`get_context()` borrows `get_pool()` and injects it into per-request services; REST routers do the same	`app/graphql/context.py::get_context`, `app/rest/github.py`, `app/rest/slack.py`
Transactions	services do `async with self._pool.acquire() as connection:` and, for writes, `async with connection.transaction():`, passing `connection` to repositories	e.g. `app/services/issues.py`
Readiness	`/readyz` acquires with `timeout=1.0` s (`ACQUIRE_TIMEOUT_SECONDS = READINESS_TIMEOUT_SECONDS / 2`) and runs `SELECT 1`; `/healthz` never touches the DB	`app/rest/health.py`

Aspect	Value / behaviour	Source
Migration runner	separate single `asyncpg.connect()`, not the pool	`scripts/apply_migration.py::_connect`

Pool shared with background loops. The same 5-connection pool serves requests and four lifespan tasks (`app/main.py::lifespan`):

Task	Enabled	Poll interval	Claim mechanism
Embedding worker	only if `EMBEDDING_WORKER_ENABLED`	idle poll 30 s, error backoff 60 s (`app/services/embedding_jobs.py`)	`FOR UPDATE SKIP LOCKED` (`app/repositories/embedding_jobs.py`)
Slack delivery loop	always	5 s (`app/services/notifications.py` `POLL_INTERVAL_SECONDS`)	`FOR UPDATE SKIP LOCKED` (`app/repositories/events.py`)
Schedule worker (due reminders, recurrences)	always	900 s (`app/services/schedule.py`)	PK collision for reminders; `FOR UPDATE SKIP LOCKED` for recurrences (`app/repositories/recurrences.py`)
Auth sweep (expired sessions, rate-limit rows)	always	900 s (`app/services/auth.py` `SWEEP_INTERVAL_SECONDS`)	idempotent `DELETE`s, no claim

The embedding worker releases its connection during model work so sharing the request pool is safe (`app/main.py::_start_embedding_worker` docstring). `DEPLOYMENT_ENV_VARS.md` "Pool sizing" argues 5 is enough for one Render free instance (0.1 CPU); Neon autosuspend-dropped connections are replaced on acquire.

11.3 Indexes

92 `CREATE [UNIQUE] INDEX` statements across `migrations/` plus inline `UNIQUE` constraints. Nearly every index leads with `workspace_id` (tenant-scoped). Notable ones:

Index / constraint	Definition	Kind	Migration
`issues_created_at_id_idx`	`issues (created_at DESC, id DESC)`	keyset; dropped and superseded in 002	001
`issues_workspace_created_at_id_idx`	`(workspace_id, created_at DESC, id DESC)`	tenant keyset	002
`issues_workspace_live_created_at_id_idx`	same, `WHERE archived_at IS NULL`	partial keyset	006
`issues_workspace_live_{assignee_created_at,updated_at,priority,due_date}_id_idx`	e.g. `(workspace_id, NULLIF(priority, 0) DESC, id DESC) WHERE archived_at IS NULL`	partial keyset per sort order; priority one is an expression index	015
`issues_workspace_parent_created_at_id_idx`	`... WHERE parent_id IS NOT NULL`	partial keyset (sub-issues)	010
`issues_workspace_team_cycle_idx`	`(workspace_id, team_id, cycle_id) WHERE cycle_id IS NOT NULL`	partial	008
`issues_workspace_team_triage_idx`	`... WHERE triage_entered_at IS NOT NULL`	partial	021
`issues_workspace_completed_at_idx`	`(workspace_id, completed_at) WHERE completed_at IS NOT NULL`	partial (analytics)	031
`issues_workspace_live_search_idx`	`issues USING GIN (workspace_id, search_vector) WHERE archived_at IS NULL`	GIN tsvector, partial	011

Index / constraint	Definition	Kind	Migration
`projects_workspace_search_idx`	`projects USING GIN (workspace_id, search_vector)`	GIN tsvector	011
`cycles_no_overlap`	`EXCLUDE USING gist (workspace_id WITH =, team_id WITH =, tstzrange(starts_at, ends_at, '[)') WITH &&)`	GiST exclusion	008
`notifications_workspace_user_unread_idx`	`(workspace_id, user_id, created_at DESC, id DESC) WHERE read_at IS NULL`	partial keyset	012
`domain_events_pending_idx`	`(slack_next_attempt_at) WHERE slack_state = 'pending'`	partial queue index	027
`embedding_jobs_due_idx` / `issue_recurrences_due_idx`	`(next_attempt_at, ...)` / `(next_run_on, ...)`	queue scan	028 / 029
`issue_embeddings_workspace_model_digest_idx`	`(workspace_id, model, source_digest)`	btree; **no HNSW/IVFFlat** on purpose (pgvector has no composite opclass with `workspace_id`, so ANN would post-filter tenants)	025
`labels_workspace_name_key`, `label_groups_workspace_name_key`	`UNIQUE (workspace_id, lower(name))`	unique expression	007, 021
`issue_labels_exclusive_group_key`	`UNIQUE (workspace_id, issue_id, exclusivity_key) WHERE exclusivity_key <> label_id`	partial unique	021
`issues_team_number_key`	`UNIQUE (team_id, number)`	unique (issue identifiers)	005
`*_workspace_id_key` (issues, teams, labels, projects, saved views, ...)	`UNIQUE (workspace_id, id)`	composite-FK targets for tenant-safe references	002, 007, 009, 019, ...
`users_email_key`, `sessions_token_hash_key`, `workspace_invitations_token_hash_key`	`UNIQUE`	unique	003, 004

11.4 Migration runner (`scripts/apply_migration.py`)

Feature	Behaviour
CLI	`python -m scripts.apply_migration <file>` | `--status` | `--all` (`main`; exactly one argument). `USAGE` lists only `<file>` and `--status`.
Config	`DATABASE_URL` via `app.config.get_settings()`, so `ENVIRONMENT` must also be set; `.env` is read as fallback (pydantic-settings `env_file=".env"`). No environment guard.
Ledger	`schema_migrations (version TEXT PRIMARY KEY, applied_at TIMESTAMPTZ NOT NULL DEFAULT now(), checksum TEXT NOT NULL)`, created `IF NOT EXISTS`.
Version	numeric filename prefix `^(\d+)_` with zero padding kept; duplicates rejected.
Checksums	sha256 of the file read as text (`read_text` normalises CRLF/LF) (`compute_checksum`, `read_migration`). Every applied version on disk is re-verified before any apply; a `MISMATCH` aborts ("restore the file and add a new numbered migration"); an applied version with no file is reported `no file`.
Locking	`SELECT pg_advisory_xact_lock($1)` with `ADVISORY_LOCK_KEY = 0x56454354` as the first statement of each ledger transaction (`_prepare_ledger`), so concurrent replicas serialise.

Feature	Behaviour
Adoption of 001	if the ledger is empty and `public.issues` exists, the live table's columns, PK `(id)`, index `issues_created_at_id_idx` and check `issues_priority_range` are fingerprinted against `001_issues.sql`; an exact match inserts the `001` row, otherwise it refuses (`_adopt_initial_migration`).
Apply	one transaction per file; whole file sent via the simple query protocol (files must not contain `BEGIN`/`COMMIT`); ledger row inserted; "Applied migration NNN ..." printed after commit. Already applied → "nothing to do".
`--status`	prints `VERSION / APPLIED AT / CHECKSUM` and `Pending: none` or the pending list; non-zero exit on mismatch.
`--all`	one connection, one up-front verification pass (each pending apply still re-verifies inside its own transaction), then each pending file in its own transaction; prints timing. Added in `577b868` to replace 32 interpreter starts at container boot (`Dockerfile` CMD).

Related: `scripts/db_gate.py` (fails CI if a JUnit report has 0 tests, any skip, failure or error), `scripts/inspect_deployment_db.py` (read-only deployment DB inspection from `--env-file`, default `.env`).

11.5 Docker images

`Dockerfile` (API + bundled SPA; used by compose `migrate`/`api`, k8s `api`, Render) — `# syntax=docker/dockerfile:1`, args `PYTHON_VERSION=3.12.14`, `NODE_VERSION=22`:

Stage	Base	Steps
`frontend`	`node:22-alpine`	`WORKDIR /build`; copy `frontend/package.json` + `package-lock.json`; `ENV PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1`; `npm ci`; copy `frontend/`; `npm run build` (`tsc -b && vite build`, so type errors fail the image).
`builder`	`python:3.12.14-slim-bookworm`	`python -m venv /opt/venv`; copy `requirements.txt`; `pip install --no-cache-dir --require-hashes --requirement requirements.txt` (dev lock excluded).
`runtime`	`python:3.12.14-slim-bookworm`	`PYTHONDONTWRITEBYTECODE=1`, `PYTHONUNBUFFERED=1`, venv on `PATH`; system user `vector` uid 10001, nologin; copy venv; `WORKDIR /srv/vector`; copy `app`, `migrations`, `scripts`; copy `/build/dist` → `./static` (served by `app/spa.py::add_spa` only if `static/index.html` exists); `USER vector`; `EXPOSE 8000`.

CMD (shell form): `set -eu; python -m scripts.apply_migration --all; exec uvicorn "app.main:create_app" "--factory" --host 0.0.0.0 --port "${PORT:-8000}"`. No `HEALTHCHECK`, no default `DATABASE_URL`/`ENVIRONMENT`.

`frontend/Dockerfile` (nginx web image; compose `web`, k8s `web`) — args `NODE_VERSION=22`, `NGINX_VERSION=1.29-alpine`:

Stage	Base	Steps
`builder`	`node:22-alpine`	`npm ci` (Playwright download skipped), `COPY . .`, `npm run build`.
`runtime`	`nginxinc/nginx-unprivileged:1.29-alpine`	`nginx.conf` → `/etc/nginx/conf.d/default.conf`; `dist` → `/usr/share/nginx/html`; `USER 101`; `EXPOSE 8080`.

`frontend/nginx.conf`: `listen 8080`; `/assets/` cached 1 year `immutable`; `= /index.html` `no-store`; `~ ^/(graphql|healthz|readyz|integrations|github|slack)(/|$)` → `proxy_pass http://api:8000` with `X-Forwarded-*` headers; `/` SPA fallback `try_files $uri $uri/ /index.html`.

`.dockerignore` (root, allowlist): `*`, then `!app`, `!requirements.txt`, `!migrations`, `!scripts`, `!frontend`; re-excluded: `__pycache__`, `*.pyc`, `frontend/node_modules`, `frontend/dist`, `frontend/e2e`, `frontend/playwright-report`, `frontend/test-results`, `.env`, `.env.*`, `**/.env`, `**/.env.*`, `**/*.pem`, `.venv`, `.git`, `.pytest_cache`. Asserted by `tests/test_phase1a2_gates.py::test_the_build_context_excludes_the_local_environment_file`. `frontend/.dockerignore` excludes `node_modules`, `dist`, `.env`, `.env.*`, `.git`, `e2e`.

11.6 docker-compose services (`docker-compose.yml`, project `name: vector`)

Service	Image / build	Host port	Environment (names)	Depends on	Healthcheck
`db`	`pgvector/pgvector:pg18`	`127.0.0.1:5433:5432`	`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB` (literal throwaway `vector`)	—	`pg_isready -h 127.0.0.1 -U vector -d vector`, 3s/3s, 30 retries, start 10s
`migrate`	`vector-api:local` from `Dockerfile`	—	`DATABASE_URL` (→ `db:5432/vector`), `ENVIRONMENT=development`	`db` healthy	— (`restart: "no"`); entrypoint `/bin/sh -euc`: per-file `apply_migration` loop, then `--status`
`api`	`vector-api:local` from `Dockerfile`	`127.0.0.1:${VECTOR_API_PORT:-8000}:8000`	`DATABASE_URL`, `ENVIRONMENT=development`, `TRUSTED_PROXY_HOPS=1`, `EMBEDDING_WORKER_ENABLED=true`, `PUBLIC_BASE_URL=http://localhost:5173`	`migrate` completed successfully	Python `urlopen("http://127.0.0.1:8000/readyz")`, 5s/5s, 20 retries, start 15s
`web`	`vector-web:local` from `frontend/Dockerfile`	`127.0.0.1:${VECTOR_WEB_PORT:-5173}:8080`	—	`api` healthy	`wget http://127.0.0.1:8080/`, 5s/3s, 10 retries

Volume `db-data` → `/var/lib/postgresql` (PG 18 layout). No `env_file`. `api` also runs `apply_migration --all` from the image CMD, so migrations are checked twice per `up` (second pass is a no-op). `--scale api=3` is described as safe because queues use `FOR UPDATE SKIP LOCKED` (comment in `docker-compose.yml`).

11.7 Kubernetes objects (`k8s/`, plain manifests, namespace `vector`)

File	Kind / name	Key settings
`00-namespace.yaml`	Namespace `vector`	label `app.kubernetes.io/name: vector`
`10-configmap.yaml`	ConfigMap `vector-config`	`ENVIRONMENT: production`, `EMBEDDING_WORKER_ENABLED: "true"`, `PUBLIC_BASE_URL: http://vector.local`, `TRUSTED_PROXY_HOPS: "1"`
`20-postgres.yaml`	PVC `postgres-data`	RWO, 2Gi, default storage class (dev clusters only)
`20-postgres.yaml`	Deployment `postgres`	1 replica, `Recreate`; `pgvector/pgvector:pg18`; `POSTGRES_PASSWORD` from Secret; `PGDATA=/var/lib/postgresql/18/docker`; `pg_isready` readiness (5s) and liveness (15s, threshold 4); requests 100m/128Mi, limit 1Gi
`20-postgres.yaml`	Service `postgres`	ClusterIP 5432
`30-api.yaml`	Deployment `api`	**2 replicas**; `vector-api:local`, `IfNotPresent`, port 8000; `DATABASE_URL` from Secret `vector-secrets`, `envFrom` ConfigMap; startup `/healthz` (2s × 30), liveness `/healthz` (10s), readiness `/readyz` (5s, timeout 3s); requests 100m/192Mi, limit 512Mi; non-root uid/gid 10001, seccomp `RuntimeDefault`, `readOnlyRootFilesystem`, no privilege escalation, drop ALL

File	Kind / name	Key settings
`30-api.yaml`	Service `api`	ClusterIP 8000 (name is load-bearing: nginx proxies to `api:8000`)
`40-web.yaml`	Deployment `web`	1 replica; `vector-web:local`, port 8080; uid/gid 101, seccomp, drop ALL (writable root FS for nginx); probes `GET /`; requests 10m/32Mi, limit 128Mi
`40-web.yaml`	Service `web`	**NodePort**, port 80 → 8080
(none)	Secret `vector-secrets`	keys `POSTGRES_PASSWORD`, `DATABASE_URL`; created by hand with `kubectl create secret generic` (`README.md`)

No Ingress, TLS, HPA, NetworkPolicy, PodDisruptionBudget or CPU limits. Each `api` pod runs `apply_migration --all` at start (image CMD); the advisory lock serialises the two replicas.

11.8 Render deployment (`render.yaml`)

Field	Value
Service	`type: web`, `name: vector-staging`, `runtime: docker`, `dockerfilePath: ./Dockerfile`, `dockerContext: .`
Plan / region	`free`, `oregon`
Deploy trigger	`branch: main`, `autoDeploy: true`
Health check	`healthCheckPath: /healthz` (no DB, so a Neon resume from autosuspend does not cause restarts; `DEPLOYMENT_ENV_VARS.md` "Health and readiness")
Start	image CMD (no `startCommand`, no `preDeployCommand`): migrations `--all`, then uvicorn on `$PORT`
Env (dashboard, `sync: false`)	`DATABASE_URL`, `ENVIRONMENT` (must be `production`; `staging` fails `Literal` validation in `app/config.py`), `PUBLIC_BASE_URL`
Env (committed)	`EMBEDDING_WORKER_ENABLED="true"`, `FORWARDED_ALLOW_IPS="*"`, `TRUSTED_PROXY_HOPS="1"`
Database	none declared; external Neon PG 18

One origin: FastAPI serves the SPA from `static/` (`app/spa.py`), unlike compose/k8s which put nginx in front. `GITHUB_*` / `SLACK_*` are deliberately omitted (`DEPLOYMENT_ENV_VARS.md`).

11.9 Environment variables (names only)

Backend settings come from `app/config.py::Settings` (`extra="forbid"`, `env_file=".env"`). `.env.example` defines only `DATABASE_URL` and `ENVIRONMENT`.

Name	Required	Default	Purpose	Read by
`DATABASE_URL`	yes	—	asyncpg DSN (`SecretStr`)	`app/db.py::connect`, `scripts/apply_migration.py::_connect`
`ENVIRONMENT`	yes	—	`development` | `test` | `production`; production disables GraphiQL/introspection and sets `Secure` cookies	`app/main.py`, `app/graphql/context.py`, `app/graphql/mutations/auth.py`, `app/rest/github.py`, `app/rest/slack.py`
`EMBEDDING_WORKER_ENABLED`	no	`False`	Start the embedding worker in this process	`app/main.py::_start_embedding_worker`, `app/services/embedding_jobs.py`

Name	Required	Default	Purpose	Read by
`PUBLIC_BASE_URL`	no	`None`	Origin for Slack deep links	`app/main.py::_build_notifier`
`TRUSTED_PROXY_HOPS`	no	`0`	`X-Forwarded-For` hops trusted for client IP (rate limiting)	`app/http_client_ip.py`, `app/graphql/mutations/auth.py`
`FORWARDED_ALLOW_IPS`	no	`"127.0.0.1"`	Read by uvicorn itself; mirrored so the app knows `*` makes `request.client` untrustworthy	uvicorn; `app/config.py::Settings.peer_address_is_trustworthy` → `app/graphql/mutations/auth.py`, `app/http_client_ip.py`
`GITHUB_APP_ID`	no	`None`	GitHub App id	`app/services/github.py`
`GITHUB_APP_PRIVATE_KEY`	no	`None`	App private key PEM (`SecretStr`)	`app/services/github.py`
`GITHUB_PRIVATE_KEY_PATH`	no	`None`	Key from a file; used only if `GITHUB_APP_PRIVATE_KEY` is unset	`app/services/github.py`
`GITHUB_WEBHOOK_SECRET`	no	`None`	Webhook HMAC secret	`app/services/github.py`
`GITHUB_CLIENT_ID`	no	`None`	OAuth client id	`app/rest/github.py`, `app/services/github.py`
`GITHUB_CLIENT_SECRET`	no	`None`	OAuth client secret	`app/rest/github.py`, `app/services/github.py`
`GITHUB_OAUTH_CALLBACK_URL`	no	`None`	Registered OAuth callback	`app/rest/github.py`
`GITHUB_REDIRECT_ALLOWLIST`	no	`""`	Comma-separated post-install redirect origins	`app/services/github.py`
`SLACK_CLIENT_ID`	no	`None`	Slack OAuth client id	`app/rest/slack.py`
`SLACK_CLIENT_SECRET`	no	`None`	Slack OAuth client secret	`app/rest/slack.py`
`SLACK_SIGNING_SECRET`	no	`None`	Slack request signature verification	`app/rest/slack.py`
`SLACK_OAUTH_CALLBACK_URL`	no	`None`	Slack `redirect_uri`	`app/rest/slack.py`
`PORT`	no	`8000`	Listen port; set by Render	`Dockerfile` CMD
`VECTOR_API_PORT` / `VECTOR_WEB_PORT`	no	`8000` / `5173`	Host-port overrides	`docker-compose.yml`
`VITE_GRAPHQL_URL`	no (build time)	`/graphql`	GraphQL endpoint baked into the bundle	`frontend/src/lib/config/env.ts`
`VECTOR_E2E_BASE_URL`	no	`http://127.0.0.1:5173`	Playwright target	`frontend/playwright.config.ts`
`STAGING_DATABASE_URL`	no	—	Fallback key read from the `--env-file` only	`scripts/inspect_deployment_db.py`
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB` / `PGDATA`	container	—	Postgres image initialisation	`docker-compose.yml`, `k8s/20-postgres.yaml`

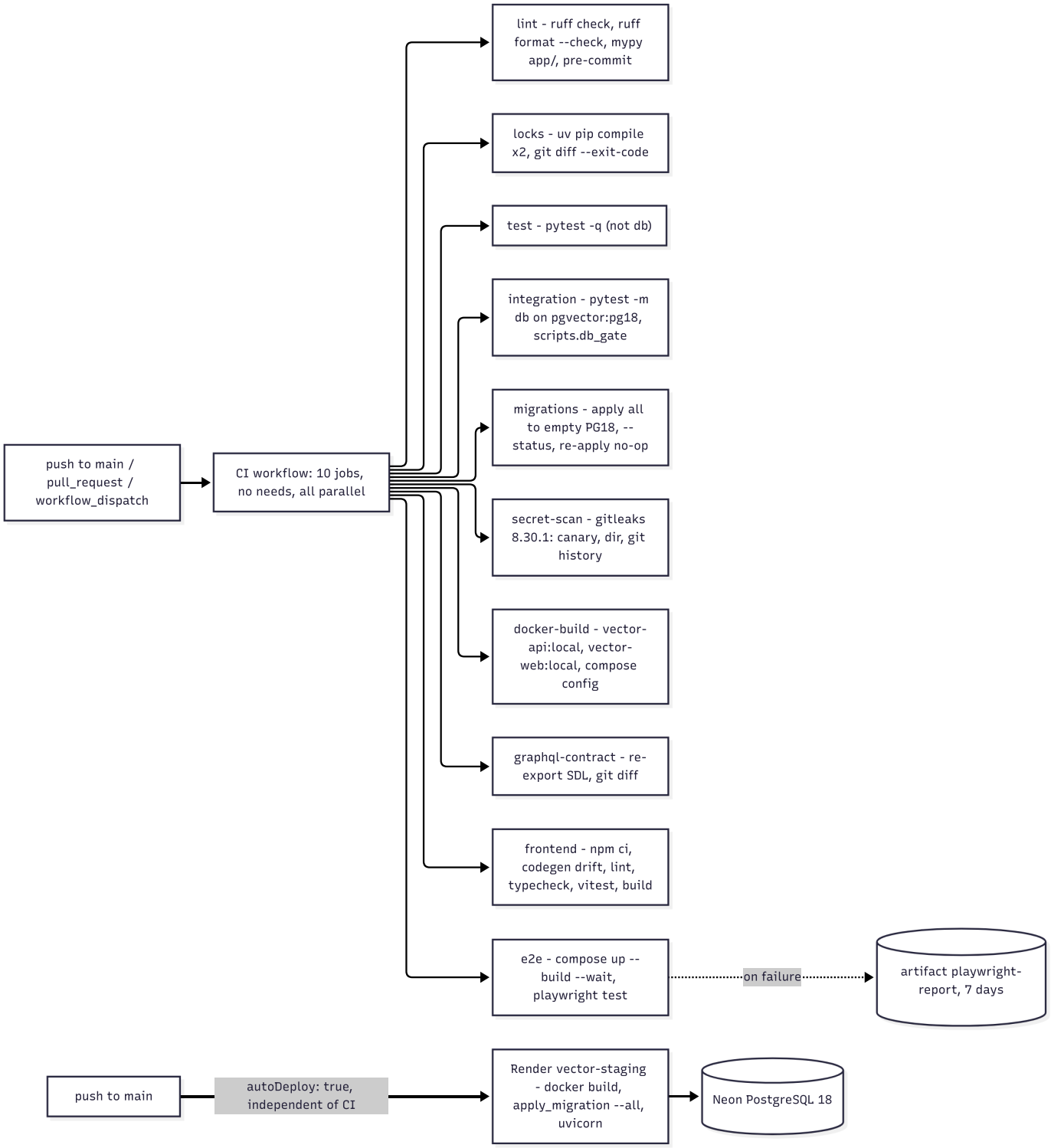
11.10 CI/CD pipeline (`.github/workflows/ci.yml`)

Workflow `CI`; triggers `push` to `main`, every `pull_request`, `workflow_dispatch`. `permissions: contents: read`; `concurrency: ci-${{ github.ref }}`, cancel-in-progress only for PRs; `shell: bash`; workflow env `ENVIRONMENT: test`. All 10 jobs run on

`ubuntu-24.04` with no `needs:` (parallel), no `continue-on-error`, no `timeout-minutes`. Every checkout uses `persist-credentials: false`. Python jobs: `actions/setup-python@v7` 3.12, pip cache keyed on `requirements-dev.txt`, then `python -m pip install --require-hashes -r requirements-dev.txt`.

#	Job (name)	Steps in order
1	`lint` (Lint, format and types)	checkout → setup-python → install dev lock → `ruff check --output-format github .` → `ruff format --check --diff .` → `mypy app/` → `actions/cache@v6` for `~/.cache/pre-commit` (key on `.pre-commit-config.yaml` hash) → `pre-commit run --all-files --show-diff-on-failure`
2	`locks` (Dependency locks are up to date)	checkout → setup-python → install → `uv --version`; `uv pip compile` both `.in` files (`--universal --python-version 3.12 --generate-hashes`) → `git diff --exit-code -- requirements.txt requirements-dev.txt`
3	`test` (Unit tests)	checkout → setup-python → install → `python -m pytest -q` (default `-m 'not db'`)
4	`integration` (PostgreSQL 18 integration tests)	checkout → setup-python → install → `docker pull pgvector/pgvector:pg18` → `python -m pytest -q -m db --junitxml="${RUNNER_TEMP}/db-tests.xml"` (fixture in `tests/conftest.py` starts the container) → `python -m scripts.db_gate "${RUNNER_TEMP}/db-tests.xml"`
5	`migrations` (Migrations apply to an empty database)	checkout → setup-python → install → `docker run pgvector/pgvector:pg18` as `vector-migrations` on `127.0.0.1:5432`, poll `pg_isready` up to 60 s → per-file `python -m scripts.apply_migration "$file"` → `--status` must contain the line `Pending: none` → second per-file pass must print no `^Applied migration` line
6	`secret-scan` (Secret scan)	env `GITLEAKS_VERSION=8.30.1`, `GITLEAKS_SHA256` → checkout `fetch-depth: 0` → download gitleaks tarball, `sha256sum --check --strict` → canary: plant a random Postgres DSN and a `ghp_` token, require rules `vector-postgres-dsn-password` and `github-pat` to fire → `gitleaks dir .` (`--exit-code 1`) → `gitleaks git .` and require `N commits scanned` with N ≥ 1
7	`docker-build` (Docker image build)	checkout → `docker build --pull --file Dockerfile --tag vector-api:local .` → `docker build --pull --file frontend/Dockerfile --tag vector-web:local frontend` → `docker compose config --quiet` (no push)
8	`graphql-contract` (GraphQL schema contract)	checkout → setup-python → install → require `frontend/schema.graphql` tracked → `rm` it and `PYTHONPATH=. python frontend/scripts/export_schema.py > frontend/schema.graphql` → `git diff --exit-code -- frontend/schema.graphql`
9	`frontend` (Frontend; `working-directory: frontend`)	checkout → `actions/setup-node@v7` Node 22, npm cache on `frontend/package-lock.json` → require `package-lock.json`, `schema.graphql`, `src/generated/schema.ts`, `src/generated/operations.ts` tracked → `npm ci` → delete generated files, `npm run graphql:codegen` → fail on `git status --porcelain` drift in `frontend/src/generated` → `npm run lint` → `npm run typecheck` → `npm test` → `npm run build`
10	`e2e` (End-to-end journeys in a browser)	checkout → setup-node 22 → `npm ci` with `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1` → `npx playwright install --with-deps chromium` → `docker compose up --build --wait` → `npx playwright test` → on failure: `docker compose logs` + `ps --all` into `frontend/playwright-report/compose.log` → on failure: `actions/upload-artifact@v6` `playwright-report` (`frontend/playwright-report`, `frontend/test-results`, 7-day retention)

There is no deploy job. Render deploys every push to `main` via `autoDeploy: true` (`render.yaml`), independently of the CI result; nothing in the repo links the two.



11.11 Gaps / to verify

Gaps / to verify:

- Branch protection / required checks are not in the repo; whether CI actually gates merges, and whether Render waits for CI, must be checked on GitHub/Render.
- CI `migrations` job, compose `migrate` service, the launcher and `README.md` still use the per-file loop; `--all` (the production path) is exercised in CI only by `tests/test_migration_apply_all_db.py` and, as a no-op after `migrate`, by the `api` image CMD in the `e2e` job's `docker compose up`.
- `scripts/apply_migration.py` has no environment guard and falls back to `.env` (which `DEPLOYMENT_ENV_VARS.md` implies holds a real Neon DSN) when `DATABASE_URL` is unset in the shell.

- `DEPLOYMENT_ENV_VARS.md` says exactly one of `GITHUB_APP_PRIVATE_KEY` / `GITHUB_PRIVATE_KEY_PATH` may be set; code does not enforce it — the inline key silently wins (`app/services/github.py` key resolver).
- Stale comments: `k8s/20-postgres.yaml` Service comment says migrations are applied through it via port-forward (README procedure), but pods self-migrate; `.pre-commit-config.yaml` calls that file "a Service and a StatefulSet" (it is PVC + Deployment + Service); `docker-compose.yml` refers to `k8s/configmap.yaml` (actual `k8s/10-configmap.yaml`). Compose/k8s comments describe two background tasks; `app/main.py` runs four.
- k8s `ENVIRONMENT: production` makes cookies `Secure`, but no TLS/Ingress is provided (`README.md` warns).
- Images `vector-api:local` / `vector-web:local` are never pushed to a registry; Docker base images are tag- not digest-pinned.
- Neon state (PG 18.6, 001 adopted, 002–032 applied) is as documented in `DEPLOYMENT_ENV_VARS.md`, not re-verified here.

12. Testing

12.1 Strategy and layout

- **Backend:** pytest under `tests/` — 114 `test_*.py` files plus `conftest.py` and `__init__.py`. 2,155 `def test_*` functions, collected as 3,799 items after parametrisation (counted with `pytest --collect-only` on 2026-10-20).
- **Two tiers, split by a marker** (`pyproject.toml`, `[tool.pytest.ini_options]`):
  - `asyncio_mode = "auto"`, `testpaths = ["tests"]`
  - `markers = ["db: requires a PostgreSQL container (deselected by default)"]`
  - `addopts = "-m 'not db'"`. Plain `python -m pytest -q` therefore runs only the default tier; `python -m pytest -m db` runs the PostgreSQL-backed tier, because the explicit `-m` overrides the one in `addopts`. A gate test pins this default (`tests/test_phase1a2_gates.py::test_the_database_suite_is_deselected_by_default`).

Tier	Test functions	Collected items	What backs it
Default (`not db`)	1,264	2,676	Fakes: `FakeConnection` / `FakePool` / `FakeIssueRepository`, `ExplodingPool`, fake auth/membership services, the real ASGI app over HTTP (`tests/conftest.py`)
`db`	891	1,123	A throwaway PostgreSQL 18 + pgvector container
Total	**2,155**	**3,799**	

- **How db tests are marked:** most `*_db.py` files set a module-level `pytestmark = pytest.mark.db`. Five other files contain both kinds: `test_db_runtime_safety.py` (2 of 6 db), `test_health_readiness.py` (2/13), `test_membership_scope.py` (13/40), `test_phase1a1_regression.py` (4/12), `test_workspace_resolution.py` (6/22).
- **How db tests get PostgreSQL.** They use the `docker` CLI through `subprocess`, not testcontainers (`tests/conftest.py`):
  - `docker_available()` runs `docker info` (`DOCKER_PROBE_TIMEOUT = 10.0`). If the daemon is unreachable, every db test calls `pytest.skip`.
  - The session-scoped fixture `postgres_dsn` runs `docker run --detach --name vector-test-<hex> … --publish 127.0.0.1::5432 pgvector/pgvector:pg18`. This gives an ephemeral host port on loopback (`PUBLISH_SPEC = "127.0.0.1::5432"`). The fixture finds that port by polling `docker inspect` (`PORT_PUBLISH_TIMEOUT = 30.0`) and waits until a real `asyncpg.connect` succeeds (`CONTAINER_READY_TIMEOUT = 60.0`; `CONTAINER_START_TIMEOUT = 300.0`). Its `finally` block always removes the container with `docker rm --force`.
  - **Image `pgvector/pgvector:pg18`** (`POSTGRES_IMAGE`, `tests/conftest.py:88`). PostgreSQL 18 is required because `uuidv7()` in `migrations/001_issues.sql` exists natively only in PG18. pgvector is required because `migrations/025_semantic_search.sql` creates the `vector` extension. The alternative of keeping stock `postgres:18` and skipping 025 was rejected: it "would have meant a green db run that never created the extension" (commit `eddb267`; comment in `.github/workflows/ci.yml` `integration` job).
  - **Schema per test:** `reset_schema()` drops every table in `public` and then every extension except plpgsql (extensions are dropped since `e5a1a6e`). `apply_all_migrations()` then applies every `migrations/*.sql` in filename order through the real runner `scripts.apply_migration`. The migration-specific suites (`test_migration_0NN_db.py`) apply only the migrations up to the one they test.
  - `tests/test_postgres_container.py` unit-tests this Docker plumbing against a `FakeDocker`, so it does not need Docker.
- **Fixture conventions** (`tests/conftest.py`):
  - The fixed tenant ids are deliberately not the bootstrap ids that `002_tenancy.sql` seeds.
  - `ExplodingPool` is a pool whose `acquire()` fails. Tests use it to prove that validation runs before any database access.
  - `graphql_context(**services)` fills unused context slots with a service that fails loudly rather than with `None` (`8297995`).
  - The autouse fixture `_no_ambient_provider_credentials` clears 12 GitHub/Slack environment variables, so a developer's real credentials cannot change test results (`4f33e8d`).

12.2 Inventory by area

"DB" column: **no** = default tier only. **yes** = every test is db-marked. **mixed** = the file contains both. Descriptions are condensed from each module's docstring.

Area	Files	What they cover	DB
Issues: validation, repository SQL, service	`test_issues.py`, `test_issue_list_repository.py`, `test_issue_repository_writes.py`, `test_issue_list_service.py`, `test_pagination_cursor.py`	issueCreate contract and validation; repository SQL against a fake asyncpg connection; service-level pagination; cursor encode/decode	no
Issues against PostgreSQL	`test_issue_pagination_db.py`, `test_issue_pagination_db_adversarial.py`, `test_issue_filters_db.py`, `test_issue_fields_db.py`, `test_issue_tenancy_db.py`, `test_issue_number_concurrency_db.py`, `test_issue_create_full_chain_db.py`	Keyset page walks, including cross-tenant timestamp ties and replayed cursors; every filter and ordering, walked at page size 3 and page size 1 (`3508b70`); fields from migration 006; workspace isolation; 32 concurrent creations plus a control using the forbidden `max(number)+1` that must fail (`8297995`); creation over the full migration chain	yes
Teams, estimates, dates, recurrence	`test_teams.py`, `test_estimates.py`, `test_due_dates.py`, `test_recurrence.py`, `test_schedule.py`	Team keys, workflow states and number allocation; estimate scales; due-date filters; recurrence arithmetic; the date sweep	no
Relations, labels, comments, bulk, triage	`test_relations.py`, `test_labels_comments.py`, `test_bulk_actions.py`, `test_triage_label_groups.py`, `test_templates.py`, `test_subscribers.py`, `test_saved_views.py`	Domain rules, validation and SQL shape for each feature	no
(same, against PostgreSQL)	`test_relations_db.py`, `test_labels_comments_db.py`, `test_bulk_actions_db.py`	Real constraints; bulk all-or-nothing as a rollback	yes
Projects, initiatives, documents, releases	`test_projects.py`, `test_initiatives.py`, `test_documents.py`, `test_releases.py`	APIs, validation, the document content parser, the release-note renderer	no
(same, against PostgreSQL)	`test_projects_db.py`, `test_initiatives_db.py`	`ProjectService` / `InitiativeService` against PG18	yes
Search	`test_search.py`, `test_semantic_search.py`, `test_embedding_jobs.py`	Query parsing and validation; hashing embedder, wiring and bounds; embedding lifecycle decisions	no
(same, against PostgreSQL)	`test_search_db.py`	`SearchService` on PG18	yes
Activity, notifications, analytics	`test_activity_notifications.py`, `test_notification_pipeline.py`, `test_analytics.py`	Pipeline properties that need no database; analytics decisions made before any SQL is issued	no
(same, against PostgreSQL)	`test_activity_notifications_db.py`, `test_notification_pipeline_db.py`, `test_analytics_db.py`	The whole chain end to end; every analytics statement checked over two tenants	yes

Area	Files	What they cover	DB
Auth and sessions	`test_auth_service.py`, `test_auth_graphql.py`, `test_password_hashing.py`, `test_session_tokens.py`, `test_client_ip.py`, `test_oauth_redirect_contract.py`	AuthService rules and their ordering; auth over real HTTP with a fake service; argon2 hasher contract; token mint and digest; what the rate limiter trusts as the client IP; OAuth redirect and cookie contract	no
(same, against PostgreSQL)	`test_auth_db.py`, `test_members_invites_db.py`	Auth with real argon2; onboarding, membership and invitations	yes
Tenancy and membership	`test_workspace_resolution.py`, `test_membership_scope.py`	Slug → `WorkspaceScope` → `AuthorizedWorkspaceScope`, each "at four levels of the same question"	mixed
(GraphQL isolation)	`test_graphql_workspace_scope_db.py`, `test_graphql_tenancy_surface_db.py`	What a member of one workspace can reach in another ("Nothing"); the same rules applied to six later entities	yes
GraphQL transport and hardening	`test_graphql_issue_create.py`, `test_graphql_issue_mutations.py`, `test_graphql_issues_query.py`, `test_graphql_memberships.py`, `test_graphql_error_masking.py`, `test_graphql_limits.py`, `test_graphql_router.py`	Payload contracts; error masking; operation limits and complexity pricing; production router hardening	no
HTTP edge and app composition	`test_security_headers.py`, `test_request_body_limit.py`, `test_spa.py`, `test_app_composition.py`, `test_settings.py`, `test_health_readiness.py`, `test_db_runtime_safety.py`	Security headers, including on failure responses; body-size limit; SPA serving without overriding API routes; fail-fast composition root; mandatory settings; `/healthz` and `/readyz`; bounds on every database operation	no / mixed
GitHub integration	`test_github.py`, `test_github_rest.py`, `test_github_development.py`, `test_github_automations.py`	Configuration and rules; install flow and webhook receiver over real HTTP with no network; `pull_request` / `push` handling; PR → issue status automation	no
(same, against PostgreSQL)	`test_github_development_db.py`, `test_github_automations_db.py`	Development statements and the automation, against a real schema	yes
Slack integration	`test_slack_service.py`, `test_slack_rest.py`, `test_slack_graphql.py`, `test_slack_channels.py`, `test_slack_signature.py`, `test_slack_composition.py`	Service seams; OAuth state and event feed; channels and preferences; v0 request signature as a pure function; that Slack is optional	no
Migrations	`test_apply_migration.py`, `test_migration_lint.py`	The runner against a fake connection; text lint over `migrations/*.sql` (§12.3)	no
(same, against PostgreSQL)	`test_migration_0NN_db.py` for 002, 004, 005, 006, 008, 009, 013, 014, 016–029 and 032 (23 files); `test_migration_apply_db.py`; `test_migration_apply_all_db.py`	Each migration applied by the real runner, then interrogated (constraints, backfills, refusals); `--all` applies the whole directory in one process while each file keeps its own transaction (`577b868`)	yes
Product journey and regression	`test_product_journey_db.py`, `test_phase1a1_regression.py`	One account and one browser session driven through the real ASGI app (`3509635`); adversarial regression cover for Phase 1a-1	yes / mixed
Gates (§12.3)	`test_phase1a2_gates.py`, `test_repository_tenancy_gate.py`, `test_integration_gates.py`, `test_postgres_container.py`	Configuration and structural invariants	no

12.3 Architecture and gate tests

All four files run in the default tier and need no database.

File (tests)	Invariants enforced	Origin
`tests/test_phase1a2_gates.py` (46 defs)	These tests check configuration, not behaviour. They cover: • **Migration immutability:** applied migrations still hash to their reviewed values, and pre-commit's file-rewriting hooks exclude `migrations/`. • **Ruff:** the agreed rule families are selected and nothing is disabled. • **mypy:** strict flags are on; there is exactly one library override and exactly one `type: ignore` in the tree. • **Locks:** every requirement is pinned with hashes, and the dev lock is a superset of the runtime lock. • **Docker image:** `.env` is excluded, the image runs as non-root, and it starts through the factory. • **CI:** the workflow token is read-only; there is no `continue-on-error` or `|| true`; the scanner is pinned by digest; the integration job pins PG18. • **`scripts/db_gate.py`:** rejects all-skipped, nothing-collected and failure/error runs, and a missing shard. • **Secrets:** gitleaks allowlist scope, `.env` untracked. • **Cross-file:** every notification kind is declared in all three places, and the db tier is deselected by default.	`e4f1073`, `9206b29`
`tests/test_repository_tenancy_gate.py` (3)	Collects every SQL statement in `app/repositories/` statically. Each statement must either mention `workspace_id` or reference only relations on an explicit tenant-free exemption list. Further tests check that the collector actually found the layer and that each exemption is still needed. The rationale given: tenancy is "a `workspace_id` equality in a WHERE clause and a composite foreign key … no row-level security, no session GUC and no ORM filter" (module docstring).	`90361a1`; regex truncation bug fixed in `a09c03a`
`tests/test_migration_lint.py` (38)	Text lint over `migrations/*.sql`: (1) no transaction control; (2) no `IF NOT EXISTS` / bare `IF EXISTS`, except `CREATE EXTENSION` (`5ce9b74`); (3) no `ON CONFLICT`; (4) every `SET NOT NULL` must follow a backfill of the same table-qualified column, and `ADD COLUMN … NOT NULL` without a default is rejected; (5) destructive statements are reported, though dropping a named index is allowed. The scanner handles comments, string literals and dollar-quoted bodies.	`cd06e14`
`tests/test_integration_gates.py` (9)	Three failure modes that "each reached `main` more than once" from parallel branches (`aaaa1dc`): (1) every module under `app/` imports — a method named `list` breaks later `list[...]` annotations; (2) every Query/Mutation type reaches the merged root, and `Query`/`Mutation` are each bound exactly once; (3) every row mapper reads every field of its entity. Also checks that `get_context` constructs every service it declares and that resolvers read only context attributes that exist. Each gate was proved by deliberately re-introducing the bug and confirming it fails.	`aaaa1dc`, `b26fa0b`

Other structural checks live in feature suites. For example, `5ac26f6` adds a test that walks the real import graph and proves the write paths cannot import Slack.

Two deliberate strict xfails in the default tier:

- `tests/test_migration_lint.py::test_a_plpgsql_function_body_does_not_trip_rule_1` — "Hole #4, deferred 2026-09-02": rule 1 misreads the `BEGIN` of a PL/pgSQL body as transaction control.
- `tests/test_phase1a2_gates.py::test_mypy_also_checks_the_migration_runner` — mypy is scoped to `app/`, so `scripts/` is not type-checked.

12.4 Frontend tests

Vitest unit and component tests.

- 68 `*.test.ts(x)` files under `frontend/src`, 678 tests (`npm test` = `vitest run`, `frontend/package.json`).
- Stack: vitest 5.0.0, @testing-library/react, user-event, jest-dom 7.0.1 and the jsdom environment.
- `frontend/vitest.config.ts` sets `isolate: true` and `restoreMocks: true` and sets no custom `testTimeout`, so the default of 5,000 ms applies.
- Components are rendered against a mocked Apollo link (`frontend/playwright.config.ts` header comment).

- Distribution (from `git ls-files frontend/src`): `app/` 6 files, including `shell.test.tsx` (landmarks, skip link, workspace resolution, switcher, rail, account menu, palette); `components/` 8; `lib/` 2; `features/` 52. Every feature directory except `navigation` has at least one test file, and `features/issues` has 13, covering accessibility, cache, create, editing, keyboard, list, navigation and pagination.
- Refusal and double-submit edge cases: `features/auth/doubleSubmit.test.tsx`, `features/formRefusals.test.tsx`, `features/savedViews/refusals.test.tsx`, `features/collaboration/labelHalfFailure.test.tsx`.

Playwright end-to-end journeys (`frontend/e2e/journeys.spec.ts`, `describe('Vector, end to end')`). The suite runs Chromium only, with 1 worker, `retries: 0`, a 60 s test timeout, `forbidOnly` in CI and no `webServer`. It assumes the stack is already running via `docker compose up --build --wait` (`frontend/playwright.config.ts`).

#	Test (line)	Property checked
1	"a session survives sign-up, sign-out and sign-in through nginx" (L243)	Session cookie round-trips through nginx with the attributes `app/http_cookies.py` intends
2	"an issue created in the UI is in PostgreSQL, not just in the cache" (L300)	A created issue survives a reload
3	"a second account cannot see the first workspace or its issues" (L354)	Tenant isolation through the UI and directly against the API. A foreign issue id under the user's own slug must answer byte-identically to a nonexistent id (`dab3467`).
4	"a signed-out visitor on a protected URL gets a refusal, not data" (L455)	Redirect in the UI, `UNAUTHENTICATED` from the API

The spec's own rules (file header): no `waitForTimeout`; every locator is a role plus an accessible name; each test creates its own account and workspace. `frontend/e2e/design-probe.mjs` is a separate script and is not part of the Playwright run.

12.5 CI test jobs

`.github/workflows/ci.yml` runs on push to `main`, on pull requests and on `workflow_dispatch`. Header: "Every gate in this file is blocking, and every gate fails closed. There is no `continue-on-error`, no `\|\| true`". All jobs run under `bash -eo pipefail` with `permissions: contents: read` and `ENVIRONMENT: test`, and no repository secret is referenced.

Job (name)	What it runs
`lint` (Lint, format and types)	`ruff check`, `ruff format --check --diff`, `mypy app/`, `pre-commit run --all-files`
`locks` (Dependency locks are up to date)	Recompiles the hash locks with pinned `uv==0.12.9` and diffs them
`test` (Unit tests)	`python -m pytest -q` (default tier)
`integration` (PostgreSQL 18 integration tests)	`docker pull pgvector/pgvector:pg18` (fails here if the daemon is unusable, before a skip can look like a pass), then `python -m pytest -q -m db --junitxml=$RUNNER_TEMP/db-tests.xml`, then `python -m scripts.db_gate <report>`. It deliberately has no `services:` block, so no second database sits on port 5432.
`migrations` (Migrations apply to an empty database)	Starts PG18 + pgvector and applies every migration with `python -m scripts.apply_migration`, one file at a time, which is the operator path that no test touches. It then asserts that the ledger shows every migration applied and unmodified (`--status`) and that a re-apply is a no-op.
`secret-scan`	gitleaks 8.30.1, with the release tarball checked against a pinned SHA-256, plus a step that proves the scanner detects planted canary secrets
`docker-build`	Builds the API and web images; `docker compose config --quiet`
`graphql-contract` (GraphQL schema contract)	Re-exports the SDL from Strawberry and fails on `git diff` against `frontend/schema.graphql`
`frontend`	`npm ci`, codegen drift check on `frontend/src/generated`, `npm run lint`, `npm run typecheck`, `npm test`, `npm run build`
`e2e` (End-to-end journeys in a browser)	`npx playwright install --with-deps chromium`, `docker compose up --build --wait`, `npx playwright test`. On failure it uploads the compose logs, the HTML report and the traces (7-day retention).

`scripts/db_gate.py` **(fail-closed logic).**

- It reads one or more JUnit XML reports, handling both a bare `<testsuite>` and a `<testsuites>` wrapper, and never trusts a process exit code.
- A run is accepted only if `total > 0 and failed == 0 and errors == 0 and skipped == 0`, both per report and in aggregate. Any skip fails the gate, because on CI a skip means Docker was unreachable.
- A **missing report file is a failure** (exit 1), because it means a shard did not finish, for example after being OOM-killed. It is not treated as an empty pass.
- Several reports can be passed in to aggregate a sharded run.
- It prints passed, failed, errors and skipped every time.
- Motivation: a CI step piped through `| tail -8` once reported "16 failures and 998 errors as green", and the earlier inline heredoc had checked only `skipped` and `tests == 0` (`9206b29`; docstring of `scripts/db_gate.py`).

12.6 Measured results

These results were measured on 2026-10-20 at HEAD `577b868` on the local Windows 11 development machine (Python venv; Node v24.19.0), not on CI.

Suite	Command	Result
Backend default tier	`python -m pytest -q`	**2674 passed, 1123 deselected, 2 xfailed in 151.48 s** (exit 0). The 2 xfails are the strict known-gap markers listed in §12.3.
Backend db tier	`python -m pytest -q -m db`	**Not run.** `docker info` did not succeed on this machine, so the fixture would have skipped all 1,123 items, which is exactly the run `db_gate.py` rejects.
Frontend Vitest	`npm test` / `npx vitest run`	**3 local runs, 678 tests each: 2, 5 and 3 failures.** See the notes below the table.
Playwright e2e	`npx playwright test`	**Not run.** It needs the Docker compose stack.
Coverage	—	**Not measured.** No coverage tool is configured (`pytest-cov` is not installed; no Vitest coverage configuration was found).

Notes on the Vitest failures:

- The failures were mostly `Test timed out in 5000ms` and hit different files on different runs: `features/issues/accessibility.test.tsx`, `features/releases/releases.test.tsx`, `features/savedViews/refusals.test.tsx` and a `semanticSearch` test (local Vitest output, 2026-10-20).
- One failure in the 2-failure run was an assertion, not a timeout: in the accessibility test "moves focus through the composer in the order it is read", focus was on the description `<textarea>` instead of the `Title` input.
- Taken together, the failures are non-deterministic and timing-driven on this machine, not a stable set of broken tests.

Historical test counts quoted in commit messages:

Commit	Date	Statement
`cd06e14`	2026-09-02	"254 passing without a database (from 51), 4 against a real PostgreSQL 18 container, and 1 documented xfail"
`d62dbfe`	2026-09-06	"pytest 626 passed, pytest -m db 280 passed"
`4f33e8d`	2026-09-07	"1275 passed both ways" (with and without `.env`)
`a5e7052`	2026-09-08	"2088 passed non-db, mypy clean over 204 files"
`86dce4e`	2026-09-09	"2334 passed non-db, mypy clean over 217 files"

12.7 How to run each suite

Suite	Prerequisites	Command (from repo root unless noted)
Backend default	Python venv with `requirements-dev.txt`	`python -m pytest -q`
Backend db	Above + a reachable Docker daemon (image `pgvector/pgvector:pg18` is pulled on first use)	`python -m pytest -q -m db --junitxml=db.xml` then `python -m scripts.db_gate db.xml`
Frontend unit	`cd frontend && npm ci`	`npm test` (watch mode: `npm run test:watch`)
Frontend static	same	`npm run lint`, `npm run typecheck`, `npm run build`
E2E	Docker; `npx playwright install chromium` in `frontend/`	`docker compose up --build --wait` (repo root), then `cd frontend && npx playwright test`. Base URL defaults to `http://127.0.0.1:5173`; override with `VECTOR_E2E_BASE_URL`.
Lint/types	dev requirements	`ruff check .`, `ruff format --check .`, `mypy app/`, `pre-commit run --all-files`

Gaps / to verify:

- The current status of the 1,123 db-tier items and of the 4 Playwright journeys is unverified. Neither could run without Docker on this machine. Re-run where Docker is available, or check the latest CI run (`gh run list`).
- The Vitest timeouts: whether they also occur on CI hardware is unknown. `frontend/vitest.config.ts` sets no `testTimeout`.
- Node engine mismatch: `frontend/package.json` declares `engines.node ">=20.19"`, but the lockfile's `vitest@5.0.0` requires `^22.12.0 || ^24.0.0 || >=26.0.0` and `@testing-library/jest-dom@7.0.1` requires `>=22` (`frontend/package-lock.json`). On Node 20 the frontend tests may not install or run cleanly. This has not been tested.
- No code coverage figure exists for either backend or frontend.
- The "covers" column in §12.2 is condensed from module docstrings, not from an audit of each test body.
- `scripts/` (including the ~900-line migration runner) is outside mypy's scope; a strict xfail tracks this.

13. Performance and scalability

This section covers what has been done for performance, the few measured numbers in the repository, the bottlenecks you can see in the code, and how the system would scale. Section 13.4 is analysis, not measurement.

13.1 What has been done

Technique	Where	Notes
Keyset (cursor) pagination	`app/repositories/activity.py`, `comments.py`, `documents.py`, `issues.py`	Cursors are row-value comparisons, for example `(created_at, id)`. The repository docstrings say "No OFFSET". A cursor carries its own ordering, and a cursor built for a different ordering is refused (commit `b397542`).
One index per issue ordering, partial on live rows	`migrations/006_issue_fields.sql` (`issues_workspace_live_created_at_id_idx`), `migrations/015_issue_list_indexes.sql` (`..._assignee_created_at_id_idx`, `..._updated_at_id_idx`, `..._priority_id_idx`, `..._due_date_id_idx`)	All are `WHERE archived_at IS NULL`. Each relies on a btree being scannable in both directions, so the id tie-break shares the sort direction (commit `3508b70`).
Generated `tsvector` columns with composite GIN indexes	`migrations/011_search.sql` (`issues.search_vector`, `projects.search_vector` `GENERATED ALWAYS AS ... STORED`; `issues_workspace_live_search_idx`, `projects_workspace_search_idx`)	`btree_gin` lets `(workspace_id, search_vector)` share one GIN index, so the tenant is a scan key rather than a post-filter. Queries are parsed with `websearch_to_tsquery`.
Plan-shape tests	`tests/test_migration_002_db.py` (`EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`), `tests/test_search_db.py`, `tests/test_issue_filters_db.py`, `tests/test_migration_029_db.py`	These DB-marked tests assert which index a query uses. They do not measure latency.
DataLoaders	`app/graphql/loaders/` (cycles, documents, initiatives, issues, labels, projects); wired in `app/graphql/context.py` (`project_loader`, `cycle_loader`, `issue_labels`, `issue_summaries`, ...)	Batches `Issue.project`, `Issue.cycle`, `Issue.labels`, issue summaries, project milestones/updates/dependencies, initiative updates, and document/revision content.
Capped child lists	`app/services/projects.py` (ponytail note at line 123), `app/services/initiatives.py:53`	Project and initiative update histories are capped at 200 per parent. This is a cap, not pagination.
GraphQL cost limits	`app/graphql/limits.py`: `MAX_DEPTH = 10`, `MAX_ALIASES = 15`, `MAX_SELECTIONS = 2000`, `MAX_COMPLEXITY = 1000`	Complexity is `page_size * inner_complexity` per connection. `__typename` is priced: before commits `765d5f3`/`58465e4` a 4 KB document scored 0 and "would have run 6,030 statements".
Statement timeout below client timeout	`app/db.py`: `STATEMENT_TIMEOUT_MS = "9500"`, `COMMAND_TIMEOUT_SECONDS = 10.0`	The server cancels the statement before the client gives up.
Startup migration in one process	`Dockerfile` `CMD` runs `python -m scripts.apply_migration --all`; commit `577b868`	Replaced 32 interpreter starts. Each migration still runs in its own transaction. See 13.2.
Argon2 warm-up and concurrency gate	`app/main.py :: lifespan` calls `warm_password_hashing()`; `app/services/passwords.py`: `MAX_CONCURRENT_HASHES = 4`	The warm-up builds the decoy hash before the process accepts traffic. The gate bounds memory: argon2id uses 64 MiB per hash, so unbounded decoy verifications could reach about 1.6 GiB against a 512Mi pod limit (commit `a5e7052`).
`FOR UPDATE SKIP LOCKED` work queues	`app/repositories/events.py :: claim_next` (Slack outbox), `app/repositories/embedding_jobs.py` (embedding jobs), `app/repositories/recurrences.py` (recurring issues)	Concurrent drainers on different replicas take disjoint rows. The session sweep has no lease; it uses idempotent DELETEs (`app/repositories/sessions.py` docstring at line 178).

Technique	Where	Notes
Two API replicas	`k8s/30-api.yaml`: `replicas: 2`; the header comment ties this to the SKIP LOCKED claims	Requests per pod: `cpu: 100m`, `memory: 192Mi`. Limit: `memory: 512Mi`. No CPU limit.
Startup timing	`app/main.py :: lifespan` logs `startup.ready` with `pool_connect_seconds`, `argon2_warm_seconds`, `total_seconds`	Added in `577b868`.

13.2 Measured numbers

The repository contains no load tests, benchmarks or latency percentiles. `git ls-files` shows no locust, k6, `pytest-benchmark` or similar tooling, and nothing in the tracked files mentions p95 or p99. Every number below comes from a commit message or from running the test suite locally.

Source	Measurement
`577b868` perf(deploy)	Per-file boot migration loop: **14.6 s**, of which **13.8 s = 432 ms × 32** interpreter starts and imports. One process that still re-verified every checksum on each file (32 × 32 = 1024 file reads and digests): **10.5 s**. `--all`: **14.6 s → 1.3 s**. Measured "against the real database from a fast development machine". The same commit says the Render demo took **~15 minutes** to become usable, compared with Render's documented ~1 minute free-tier cold start, on an instance with **0.1 CPU**.
`b8ef95d` search	Composite GIN `(workspace_id, search_vector)` against a bare `GIN (search_vector)` on **20,000 rows**: cost **292/209 buffers vs 399/215**.
`a5e7052` auth hardening	argon2id decoy verify: **64 MiB, ~100 ms** per hash. With the gate of 4 per process, that is about 40 hashes/s per process (`app/services/passwords.py` comment at line 76).
`765d5f3` limits	4 KB `__typename` document: complexity score **0** and **6,030** statements before the fix; **600,000** (refused) after.
Local backend test run (measured 2026-10-20, Windows, see §12.6)	`python -m pytest -q`: **2674 passed, 1123 deselected, 2 xfailed in 151.48 s**. The 1123 deselected tests are DB tests, which were not run because Docker was unavailable.
Local frontend test run (measured 2026-10-20, Windows, see §12.6)	`vitest run`: 68 files, 678 tests, **duration 184.50 s** (3 m 28 s wall), **2 failed** on the first run. Two further runs failed 5 and 3 tests, mostly 5000 ms timeouts in varying files (§12.6, §15.1).

13.3 Visible bottlenecks

Bottleneck	Evidence	Effect
Small connection pool shared with background loops	`app/db.py :: connect`: `min_size=1, max_size=5`, hard-coded rather than read from settings. `app/main.py :: lifespan` starts four tasks on the same pool: embedding worker (opt-in), Slack delivery loop, schedule worker and auth sweep.	Five concurrent connections per process. Background passes compete with requests.
Unbatched per-issue fields (N+1)	`app/graphql/types/issue.py`: `parent` (`relation_service.find_parent`), `children`, `relations`, `activity`, `development`, `comments`. Each makes one service call per issue. `IssueConnection.total_count` runs a separate `issue_service.count` only when that field is selected. Document `revisions` / `comments` are also per parent (`app/graphql/types/document.py`).	Only `MAX_COMPLEXITY = 1000` bounds the fan-out.
Exact vector scan	`migrations/025_semantic_search.sql` line 308, "why there is no HNSW or IVFFlat index here"	Semantic search is O(n) per workspace. This is deliberate: neither ANN index type can use the tenant as a scan key (commit `eddb267`).

Bottleneck	Evidence	Effect
Embedding on the event loop	`app/services/search.py` lines 162, 257, 328 and `app/services/embedding_jobs.py` lines 249, 261 call `embedder.embed(...)` directly, with no `to_thread` or executor.	Negligible with the default hashing embedder (`sentence-transformers` is not in `requirements.txt`). A real model would block the loop.
Per-team issue-number lock	`app/repositories/teams.py :: allocate_issue_number`, called from `IssueService.create`	Issue creates on the same team are serialised until commit.
Slack delivery throughput	`app/services/notifications.py`: `DRAIN_BATCH = 20`, `POLL_INTERVAL_SECONDS = 5.0` (ponytail note at line 98)	About 4 events/s per process.
Recurrence throughput	`app/services/schedule.py :: _generate` claims one batch of `RECURRENCE_BATCH = 25` per pass, with `POLL_INTERVAL_SECONDS = 900.0`. Unlike `_remind`, it does not loop.	A backlog of more than 25 due schedules spills into later 15-minute passes.
Bulk actions	`app/domain/bulk.py`: `BULK_MAX = 100`. `app/services/bulk.py:441` ponytail note: "N round trips per batch".	Up to 100 `FOR UPDATE` row locks, plus activity writes one round trip at a time.
Unbounded lists	`app/repositories/cycles.py :: list_for_team` and `app/repositories/releases.py :: list_environments` have `ORDER BY` with no `LIMIT`.	Fine for small sets; nothing in the code bounds them.
Demo hosting	`render.yaml`: `plan: free` (512 MB, 0.1 CPU), one service; the free instance sleeps after 15 minutes idle. The database is a free Neon project that autosuspends.	Cold start, then DB resume, on the first request after idle. CPU-bound boot work becomes minutes (see `577b868`).
No pruning	No app-side DELETE for `github_deliveries`, `slack_event_deliveries` or terminal `domain_events` (comments in migrations 014, 017 and 027; see 15.4).	These tables grow without bound.

13.4 How it would scale (analysis)

This is reasoning from the code, not measurement.

- **What already scales horizontally.**
  - The API process holds no per-user state. Sessions are rows in PostgreSQL, looked up by SHA-256 token digest (commit `e64ad5f`).
  - Rate-limit counters are in PostgreSQL (`auth_rate_limits`), not process memory. Commit `a5e7052` gives the reason: with two replicas, an in-process counter would reset on every rollout.
  - All three queue drainers claim with `FOR UPDATE SKIP LOCKED`, so adding replicas divides the backlog rather than duplicating work. `k8s/30-api.yaml` says `replicas` must return to 1 if that ever changes.
  - Number allocation and cycle overlap are enforced by row locks and an `EXCLUDE` constraint, so their correctness does not depend on how many instances run.
- **What limits it.**
  - Every replica adds up to 5 connections, so total connections grow at 5 × replicas. On Neon this needs a pooled endpoint, or the pool size moved into settings.
  - Per-tenant work is bounded by indexes, but semantic search and the `list_stale` anti-join scale linearly with workspace size.
  - Background work runs inside API processes, so it competes for the same CPU and pool. `k8s/10-configmap.yaml` suggests a second Deployment with `EMBEDDING_WORKER_ENABLED=true` and no Service if embedding moves off the request path.
- **What would need to change at larger scale.**
  1. Configurable pool size, or an external pooler.
  2. DataLoaders for the per-issue fields listed in 13.3.
  3. An ANN index with a tenant strategy, such as per-tenant partial indexes or partitioning, if workspaces grow large.
  4. `to_thread` (or a separate worker) for real embedding models.

5. Pruning jobs for the delivery and outbox tables.
6. Looping recurrence claims.
7. Real-time updates. The API has no subscriptions and the frontend does no polling (`frontend/schema.graphql` has no `type Subscription`).

14. Design decisions and trade-offs

Every entry below is backed by a code comment, a migration comment or a commit message that states the rationale. Quoted phrases come from the cited source. "Not stated" means the source gives a reason but names no alternative.

14.1 Architecture and data access

#	Decision	Alternatives considered	Rationale	Evidence
1	GraphQL is the first-party product API; REST is used only for protocol-shaped endpoints (health, webhooks, OAuth callbacks, uploads)	Not stated	Recorded as a project rule, not argued	`CLAUDE.md` ("Backend Stack")
2	Raw asyncpg with parameterised SQL and a hand-written WHERE builder; no ORM	SQLAlchemy Core (which `CLAUDE.md` would have allowed for this file)	The builder is "thirty lines". An expression-language dependency "would stand between every reader and the only question that matters about this file -- whether the tenant predicate is ANDed onto every statement". Filter values are bound as `$n` and never reach the statement text.	`b397542`; `CLAUDE.md`
3	Services own connection acquisition and transaction boundaries; repositories receive a connection	Not stated	Project rule. Services also take the workspace as an argument on every call instead of holding it on the instance, because once anything caches or reuses a service "an instance attribute becomes an ambient" tenant.	`CLAUDE.md`; `app/services/issues.py` (class docstring, ~L157)
4	`WorkspaceScope` is a required keyword argument on every tenant-owned call	Instance state	"`scope=` appears at every call site where a cross-tenant read would be introduced"	`d119c76`
5	Tenancy = a `workspace_id` equality plus composite foreign keys such as `(workspace_id, team_id) → teams(workspace_id, id)`	A pair of single-column FKs; row-level security, session GUC or ORM filter (none are used)	Single-column FKs would still permit "the half-owned shape … where an issue claims one workspace while pointing at a team belonging to another". Because there is no RLS, an omitted predicate is "unscoped", so a static gate checks every repository statement (§12.3).	`migrations/002_tenancy.sql` header; `tests/test_repository_tenancy_gate.py` docstring; `8297995`, `e5a1a6e`
6	"Absent" and "unauthorized" share one answer (`NOT_FOUND`), computed in a single joined statement	Separate existence and permission checks	The server "never computes the difference between a workspace that does not exist and one the caller may not see"	`2fd6063`, `4d58058`, `13a5285`
7	Every workspace field takes `workspaceSlug`; nested resolvers carry the authorised scope as a Private field	A single hard-coded bootstrap tenant (the previous state); an ambient tenant on the context	"a document may now name two" workspaces; the server no longer picks a team for the client	`4d58058`
8	Keyset (cursor) pagination; every ordering ends in `id`; a cursor carries the ordering it was minted under	OFFSET; adding `workspace_id` into the row comparison	With keyset, "the cost of page N is the cost of page 1". Widening the comparison "would put workspaces into the ordering", letting a page walk cross tenants. Resuming a cursor in a different sort "returns a plausible page of wrong rows", so a mismatched cursor is refused.	`app/repositories/issues.py` (~L995–1006); `d119c76`; `b397542`

#	Decision	Alternatives considered	Rationale	Evidence
9	One btree index per ordering key; no `NULLS` clause is written	`DESC NULLS LAST` plus a second index	A btree scans both ways when the `id` tie-break shares the key's direction; "spelling DESC NULLS LAST would read better and need a second index"	`3508b70` (migration 015)
10	`create_app()` factory, run as `uvicorn app.main:create_app --factory`; `get_settings()` is lazy	Module-level `app = create_app()` and a settings singleton	A module-level app made "a test or a linter that merely imported it" fail for want of a runtime environment. Settings are still resolved eagerly in the factory, so a misconfigured deployment fails at startup.	`app/main.py` `create_app` docstring; `cd06e14`
11	`ENVIRONMENT` is required and has no default (`Literal["development", "test","production"]`)	Defaulting to `"development"`	A forgotten variable silently gave production "GraphiQL served, introspection answered". Requiring it inverts the failure: the deployment "cannot start".	`app/config.py` `Settings` docstring

14.2 Persistence, migrations and background work

#	Decision	Alternatives considered	Rationale	Evidence
12	Numbered SQL migrations applied by an in-repo runner, with a `schema_migrations` ledger of SHA-256 checksums computed over decoded text	Byte digests	Turns "never modify an applied migration" "from convention into a check". Text digests let CRLF and LF checkouts agree ("469 bytes against the 451 stored").	`cd06e14`; CI `migrations` job runs `--status`
13	Migration lint bans `IF NOT EXISTS`, but exempts `CREATE EXTENSION`	Keep a blanket ban; use the bare `CREATE EXTENSION`	The ban exists because guards hide schema drift, and "an extension has no shape". The bare form fails permanently where the extension already exists, and the guarded form skips the privilege check under a managed-Postgres role.	`5ce9b74`
14	Each migration runs in its own transaction, even with `--all`	One transaction for the whole directory ("faster still")	A failure part-way leaves earlier migrations committed and rolls back the failing one whole	`577b868`
15	Issue numbers come from a transactional counter (`UPDATE teams SET issue_counter = issue_counter + 1 … RETURNING`)	A sequence; `SELECT max(number)+1`	The row lock serialises allocation, and a rolled-back create "returns its number instead of burning it". A test shows `max+1` fails under 32 concurrent creates.	`8297995`
16	Roles enforced by `CHECK`, not an enum type	PostgreSQL enum	PostgreSQL "will not use an enum value in the transaction that added it", and every migration is one transaction	`2fd6063`
17	Cycle overlap enforced with `EXCLUDE USING gist` over a half-open `tstzrange`	Application-level SELECT-then-INSERT	"two statements with a gap", i.e. a race	`e5a1a6e`
18	Members are removed by tombstone (`removed_at`), not by deleting the row	`DELETE`	16 of 17 FKs to `workspace_members` are `RESTRICT` in immutable migrations, and authorship rows cannot be deleted or reassigned	`0f7d9cf` (migration 026)
19	Domain events go through an outbox table written in the same transaction as the change; PK `(workspace_id, kind, dedupe_key)` with `ON CONFLICT DO NOTHING`	Calling Slack directly from write paths	Prevents "no window in which Slack has been told about an assignment that rolled back" and stops double posts. A direct call would make "an HTTP call inside an open transaction holding a pool connection".	`5ac26f6` (migration 027)

#	Decision	Alternatives considered	Rationale	Evidence
20	Background work runs as asyncio tasks on the app lifespan, claiming rows with `FOR UPDATE SKIP LOCKED`	Cron container, Celery or another queue broker	"infrastructure a deployment has to operate, bought to call a coroutine every thirty seconds"	`4eada47`; `app/config.py` (L33); `app/services/embedding_jobs.py` (L189)
21	Auth rate limits stored in PostgreSQL (fixed window)	In-process counter; Redis	With `replicas: 2`, an in-process counter "resets on every rollout". Redis "is rejected on cost rather than on fit": a new dependency, container and failure mode on the log-in path.	`a5e7052`; `migrations/032_auth_hardening.sql` (L30–45)
22	Full-text search uses a GIN index keyed `(workspace_id, search_vector)` and `websearch_to_tsquery`	A bare `GIN(search_vector)`; `to_tsquery`; pg_trgm	The tenant equality becomes an index scan key (measured on 20,000 rows: cost 292/209 buffers vs 399/215). User input never becomes tsquery syntax.	`b8ef95d` (migration 011)
23	pgvector hybrid search without an HNSW or IVFFlat index; an in-tree hashing embedder, with `sentence-transformers` used only if installed	ANN index; a mandatory model dependency	Neither ANN index "can carry the tenant as a scan key", so the tenant filter becomes a post-filter that "under-fills exactly for the smallest tenants". Embeddings are "free, local and optional".	`eddb267` (migration 025)

14.3 Authentication and security

#	Decision	Alternatives considered	Rationale	Evidence
24	Opaque random 256-bit session tokens in an HttpOnly cookie; only the SHA-256 digest is stored	JWT; argon2 for the token digest	A JWT puts session contents in the client and makes revocation a deny list, whereas with an opaque token "revocation is a DELETE and expiry is a column". There is "no guessable preimage space for a slow hash to defend, and a salted hash could not be looked up by equality".	`app/services/tokens.py` docstring; `e64ad5f`
25	argon2id passwords at argon2-cffi defaults (RFC 9106 low-memory profile), run in worker threads, with a decoy verify for unknown emails and at most 4 concurrent hashes per process (`MAX_CONCURRENT_HASHES = 4`)	Not stated (algorithm choice); an unbounded executor (concurrency)	The decoy makes the unknown-email and wrong-password paths indistinguishable by timing. An unbounded executor reaches "1.6 GiB of live arenas" against a 512Mi pod limit.	`app/services/passwords.py` (L3, L86); `e64ad5f`; `a5e7052`
26	Emails stored as lowercased `TEXT`	`CITEXT`	No extension is needed. Also, CITEXT stores whichever capitalisation registered first.	`e64ad5f`; `5ce9b74`
27	CSRF defence = `SameSite=Lax` + no CORS + refusing CORS-simple content types (multipart off) + batching off	—	Each control is "one ordinary-looking edit from being gone", so each is pinned by a test proved against its regression	`2a2dfdd`
28	Security headers set by application middleware	nginx	nginx is absent from "three of the four ways this application is reached". `Referrer-Policy: strict-origin-when-cross-origin` is used rather than `no-referrer` so OAuth providers can check the origin. HSTS is sent only over TLS.	`95db8d7`
29	Client IP is taken from `Settings.trusted_proxy_hops` counted from the right of `X-Forwarded-For`; `X-Real-IP` is never read; default 0	Trust `X-Real-IP`; trust uvicorn's rewritten peer	On Render both were forgeable. "a deployment that says nothing gets the safe answer".	`d4457bd`, `f55a99a`
30	A GitHub installation id from the callback is only a claim, confirmed by a signed webhook	Recording the callback id directly	Under first-come uniqueness, an admin could claim another organisation's installation	`9f70ba9` (migration 016)

#	Decision	Alternatives considered	Rationale	Evidence
31	GitHub App installation tokens (RS256 JWT); the dependencies were added, but token minting is not implemented (§15.2)	User-to-server OAuth token	A user token "ties what Vector can see to one person's account, breaks when they leave"	`3f71962`
32	Webhook idempotency row claimed as the first statement inside the delivery transaction; payload shapes clamped	—	"A signed payload must never be able to abort its own transaction" (a 500 causes infinite redelivery)	`05471a7`

14.4 Deployment and quality gates

#	Decision	Alternatives considered	Rationale	Evidence
33	One image, one origin: FastAPI serves the built SPA (`app/spa.py`, registered last) on Render. The compose/Kubernetes path keeps a separate nginx web image.	Splitting the page and API across origins	The session and OAuth state cookies need one origin: "no CORS, no SameSite=None". Render cannot bind-mount, so `migrations/` and `scripts/` are baked into the image.	`de021b8`; `1a90a92`
34	Migrations run in the container `CMD` under `set -eu` before uvicorn binds, via one `--all` process	A shell loop with 32 runner processes	A failed migration exits the container. The loop cost 14.6 s (13.8 s of interpreter starts), and `--all` brought it to 1.3 s (both measured on a fast development machine). On the 0.1-CPU free instance, boot had taken "~15 minutes".	`de021b8`; `577b868`; `Dockerfile` (CMD, ~L129–166)
35	Render health check on `/healthz` (liveness), not `/readyz`	`/readyz`	Prevents "a Neon compute resuming from autosuspend" from restarting a working app	`de021b8`
36	CI gates fail closed: no `continue-on-error`/`|| true`, `pipefail`, read-only token, no secrets	—	"a missing tool is a broken gate, and a broken gate has to look exactly like a failing one"	`.github/workflows/ci.yml` header
37	The db-test gate reads the JUnit report, not exit codes; a missing shard is a failure	An inline heredoc checking only `skipped`/`tests==0`	A piped `| tail -8` once reported "16 failures and 998 errors as green"	`9206b29`; `scripts/db_gate.py`
38	DB tests run on `pgvector/pgvector:pg18` via the Docker CLI on an ephemeral loopback port; no CI `services:` block	`postgres:18` with 025 skipped; a CI service container	Skipping would give "a green db run that never created the extension". A service container on 5432 "would let a test appear to pass against the wrong server".	`eddb267`; `ci.yml` `integration` job
39	Integration gates for failure modes caused by parallel merges	Rely on per-feature tests	Each failure "reached `main` more than once" while the suite stayed green	`aaaa1dc`
40	E2E: Chromium only, `retries: 0`, 1 worker, no sleeps	Retries, multiple browser engines	"A retry does not fix a flaky test; it hides one". The other engines cost ~400 MB "to answer a question this suite is not asking".	`frontend/playwright.config.ts`

14.5 Decisions to document

These are significant choices visible in the code for which no recorded rationale or rejected alternative was found:

- **In-repo migration runner instead of Alembic or another migration tool.** The ledger and checksum design is argued in `cd06e14`, but why no existing tool was used is not recorded.
- **Strawberry as the GraphQL library**, and FastAPI as the outer ASGI framework. Both are stated as project rules in `CLAUDE.md` with no reasons.
- **argon2id over bcrypt or scrypt.** `app/services/passwords.py` names the parameters but no alternative.
- **Apollo Client and React on the frontend.** No commit message found by `git log --grep=apollo` argues the choice.
- **Render + Neon as the hosting target**, versus the Kubernetes manifests that also ship (`1a90a92`). The commits record how each works but not why Render was chosen.

- `uuidv7()` **primary keys**, which force PostgreSQL 18. The consequence is recorded (`cd06e14`, `tests/conftest.py`); the reason for choosing UUIDv7 is not.

> **Gaps / to verify:**
>
> - Rows 1 and 3 rest on project rules in `CLAUDE.md`, not on an argued trade-off.
> - Commit `4eada47` itself records "NOT VERIFIED HERE: tests/test_migration_028_db.py could not run". Whether later db runs covered it was not checked.

15. Known limitations, technical debt and bugs

Sources: the inconsistencies recorded while reading the code for the other sections, deduplicated and re-checked against the code. "Verified" means the cited code was read and shows the behaviour. "Plausible" means the code suggests the behaviour, but it was not executed.

How to read these tables:

- **Status "verified"** means the cited code was read and behaves as described. It does not mean a failing test was run.
- **Status "plausible, unverified"** means the code points that way but the outcome was not reproduced.
- **Severity** is the author's judgement (low / med / high), not a triage outcome.
- Many of these items are marked in the code with `ponytail:` comments, the codebase's marker for a deliberate shortcut. Those are noted where they apply.

15.1 Probable bugs

Severity (judgement)	Description	Evidence	Status
med	Archived issues can be moved into a cycle or project, and the mutation returns them. This contradicts the class docstring "Every read and every write carries `archived_at IS NULL`".	`app/repositories/issues.py :: IssueRepository.set_cycle`, `set_project` (`WHERE workspace_id = $1 AND id = $2`). `lock_snapshot` docstring: "`update` excludes them, `set_cycle` and `set_project` do not". Reached by `issueSetCycle` / `issueSetProject` and `TemplateService.apply`.	verified (code)
med	Deleting a project or initiative that still has documents should fail on an `ON DELETE RESTRICT` FK. The service neither detaches documents nor catches `ForeignKeyViolationError`, so the client would get the masked "Internal server error".	`migrations/023_documents.sql:280-290` (`documents_project_fk`, `documents_initiative_fk`, RESTRICT). `app/services/projects.py :: ProjectService.delete` (lines 595-672) and `app/services/initiatives.py :: InitiativeService.delete` (line 449) clear issues, milestones, teams, updates, dependencies and links, but not documents. No test covers it.	plausible, unverified (not executed)
low	A Slack delivery that fails with something other than `SlackApiError` (for example a bug in `_route`) is never finalised. The exception propagates to the loop, and the event stays `pending`, retried every ≤5 minutes forever.	`app/services/notifications.py :: _deliver` catches only `SlackApiError`. `run_delivery_loop` logs and continues. `app/repositories/events.py :: claim_next` selects `slack_state = 'pending'` with no attempt cap. `_record_failure` is the only path that finalises, and it is not reached here.	verified (code path); not reproduced
med	Registration has no rate-limit budget at all when no client IP can be determined, for example with `TRUSTED_PROXY_HOPS=0` behind `FORWARDED_ALLOW_IPS="*"`, or with a short `X-Forwarded-For`. The `register` docstring claims a fallback to the TCP peer that does not happen when the peer is untrusted.	`app/services/auth.py :: _register_buckets` (`if client_ip is None: return []`). `app/http_client_ip.py :: read_client_ip` returns None when `not peer_is_trustworthy` (line 172). Docstring at `app/services/auth.py:281-282`.	verified (code)
low	Slack OAuth network failures are not handled. `_post_form` uses `urllib.request.urlopen` and `json.loads`, which can raise `URLError`, `HTTPError`, `TimeoutError` or `JSONDecodeError`. The callback catches only `SlackOAuthError`, so the result is likely a 500 with the state cookie left in place.	`app/services/slack.py :: _post_form` (line 236). `app/rest/slack.py :: slack_oauth_callback` (`except SlackOAuthError`, line 737).	verified (code); not reproduced

Severity (judgement)	Description	Evidence	Status
low	The Slack OAuth state cookie is not cleared when the admin check fails in the callback. `_authorized_admin_scope` raises before any `consumed(...)` response is built, so the cookie lives until its `Max-Age`.	`app/rest/slack.py :: slack_oauth_callback` (line 718, before `consumed` at line 721)	verified (code)
low	The milestone read path writes. `_get_milestone` calls `update_milestone` with every `set_*` flag false, but the SQL sets `updated_at = now()` unconditionally, so a read bumps the timestamp.	`app/services/projects.py :: _get_milestone` (line 883). `app/repositories/projects.py :: update_milestone` (`updated_at = now()`, line 728).	verified (code)
low	Concurrent milestone creates can get the same `position`, because `MAX(position)+1` is computed without a lock and there is no unique constraint on position. The docstring acknowledges this.	`app/repositories/projects.py :: create_milestone` (docstring lines 641-646)	verified (documented)
low	Bulk label attach bypasses the 50-labels-per-issue cap.	`app/services/labels.py` enforces `LABELS_PER_ISSUE_MAX = 50` (line 660). `app/services/bulk.py` never references the constant (`IssueLabelRepository.attach_many`).	verified (code)
low	Triage `change_team` updates `team_id`, `number` and `workflow_state_id` but not `cycle_id`. An issue in triage that holds a cycle of its old team would violate the composite `issues_cycle_fk`, which is not mapped there.	`app/repositories/triage.py` (`SET team_id = $3 ...`, line 333). `app/services/triage.py` maps only `issues_workflow_state_fk` (line 283).	plausible, unverified
low	`CURRENT_DATE` is assumed to be UTC (migration 029 comment, `app/domain/recurrence.py:12`), but the session TimeZone is never set. Due-date and recurrence day boundaries therefore follow the database or role default.	`app/db.py :: connect`: `server_settings={"statement_timeout": ...}` only (line 50)	verified (code)
low	Invitation tokens and OAuth `code`/`state` values may be written to the access log. uvicorn's access log is on by default, and the SPA serves `GET /invite/<token>`.	`Dockerfile` `CMD` (`exec uvicorn ...` with no `--no-access-log`). `app/logging.py` is a stub (see 15.4).	plausible, unverified (not executed)
low	Frontend unit tests time out intermittently at Vitest's default 5000 ms; there is no `testTimeout` in `frontend/vitest.config.ts`. Local runs recorded 5, then 3, timeouts in different files. One run had 2 failures in `frontend/src/features/issues/accessibility.test.tsx`: a timeout and a focus-order assertion.	`frontend/vitest.config.ts` has no timeout setting (grep). Run results are from local runs on 2026-10-20 (§12.6); CI status is unknown.	plausible (environment-dependent)
low	`frontend/e2e/design-probe.mjs` exits 1 unless the landing page is light (`landingLight = luminance > 0.5`), but `features/landing/landing.module.css` paints it with `--color-bg-app`, which is `--vector-navy-975` (`frontend/src/styles/tokens.css`)	`frontend/e2e/design-probe.mjs` lines 114–124	verified by reading; script not run

15.2 Missing features and functional limitations

Area	Limitation	Evidence
Issues	No unarchive or restore and no archived-issues query, even though `issue_archive` is described as reversible	No statement clears `archived_at` (grep `app/`). `app/graphql/mutations/issues.py :: issue_archive`.

Area	Limitation	Evidence
Archive	No other entity can be archived: projects, initiatives, cycles, documents and releases have no archive	Only `issue_archive` and `issue_bulk_archive` exist (`app/graphql/mutations/`)
Comments	Issue comments cannot be edited; `edited_at` is exposed but never written	No update path in `app/services/comments.py` or `app/graphql/mutations/comments.py`
Teams	No team rename or delete, and no workflow-state create, rename, reorder or delete. States are seeded only by `TeamService.create`.	`app/graphql/mutations/memberships.py` exposes only `team_create` and `team_estimate_scale_set` for teams
Auth	No password reset, email verification, 2FA, password change, "sign out everywhere", session listing or account deletion	`app/graphql/mutations/auth.py` exposes only `register`, `login` and `logout`. The `AuthService.register` docstring says "address verification is still owed".
Auth	Password hashes are never upgraded	`app/services/passwords.py:165`: "`check_needs_rehash` is deliberately not consulted"
Email	No outbound mail. Invitation tokens are returned to the inviter in `InvitationCreatePayload.token`. Account lockout has no unlock path and no notification.	`app/services/auth.py:116` (ponytail note)
Slack	Inbound events are verified and claimed but not processed. There are no slash commands or interactivity.	`app/rest/slack.py:880`: "Deliberately empty"
Search	"Semantic" search runs on the in-tree `hashing-v1` feature-hash embedder unless `sentence-transformers` is installed, and it is not in `requirements.txt`	`k8s/10-configmap.yaml` comment. `app/services/embeddings.py :: load_embedder`.
Search	New or edited issues are not embedded unless the opt-in worker is on (`EMBEDDING_WORKER_ENABLED`, default false) or someone runs `embeddingsRefresh`	`app/config.py`. `app/main.py :: _start_embedding_worker`.
Relations	Blocking cycles (A blocks B, B blocks A) are allowed; only parent cycles are guarded	`migrations/010_issue_relations.sql` comment ("nothing prevents it at any layer")
Recurrence	At-most-once generation: a crash between the advance and `TemplateService.apply` loses that occurrence	`app/services/schedule.py :: _generate`. `RECURRENCE_BATCH` comment: "a crash forfeits at most twenty-five occurrences".
Templates	Applying a template is not atomic. It can leave an issue without its project, cycle or labels.	`app/services/templates.py:648`. `app/graphql/types/template.py:254` (ponytail notes).
Roles	Only membership, team-admin, GitHub and Slack operations are admin-gated. Any member can delete projects, labels, initiatives, releases, documents and cycles, bulk-archive, or trigger `embeddingsRefresh` (up to `REFRESH_MAX = 500`). Admins cannot delete other members' comments or saved views.	`app/graphql/mutations/search.py :: embeddings_refresh` (only `authorized_scope`). `app/services/search.py:39`.
Members	Since migration 026, issues can still be newly assigned to a former member	`app/repositories/memberships.py:644` (ponytail note)
Front end	The issue composer always files into the first team	`frontend/src/features/issues/api/useCreateIssue.ts:59` (`teams[0]?.id`)
Front end	No real-time updates: no GraphQL subscriptions, polling, SSE or WebSockets	`frontend/schema.graphql` has no `type Subscription`. Grep of `frontend/src` found none.
Front end	`/semantic-search` is reachable only by URL	`ROUTE_SEGMENTS.semanticSearch` is used only by the route table (`frontend/src/app/routes/routes.tsx`); no nav item or command references `paths.semanticSearch`

Area	Limitation	Evidence
Lists	Cycle and environment lists are unpaginated and unbounded	`app/repositories/cycles.py :: list_for_team`. `app/repositories/releases.py :: list_environments`.
GitHub	No installation tokens (RS256 app JWT), so no app-to-server API calls. Repository lists beyond one page rely on later webhooks.	Comments at `app/services/github.py:739` and `app/rest/github.py:607` ("needs RS256 JWT signing ... which this application deliberately does not carry"). `cryptography` is unused (15.4).

15.3 Security gaps

Severity (judgement)	Gap	Evidence	Status
med	The Slack bot token is stored in plaintext in the database	`app/services/slack.py:140` docstring: "The token, in the database column, in plaintext."	verified (documented)
low	`/docs`, `/redoc` and `/openapi.json` are not disabled in production. GraphQL introspection and GraphiQL are.	`app/main.py :: create_app`: `FastAPI(title="Vector", lifespan=lifespan)` with no `docs_url=None`	verified (code)
low	CSRF protection rests on `SameSite=Lax`, JSON-only POST and no CORS. There is no CSRF token and no Origin check, and the `__Host-` cookie prefix is noted as future work.	`app/http_cookies.py` module docstring. No `CORSMiddleware` in `app/`. Commit `2a2dfdd`.	verified (design)
low	Rate limits are fixed-window, so a caller can get up to 2× the budget across a window boundary. `invitationAccept`, `workspaceCreate` and authenticated mutations are not rate-limited.	`app/services/auth.py` comment	verified (documented)
low	Login issues a fresh session but never revokes the user's other sessions. There is no logout-everywhere, no idle timeout (lifetime is absolute) and no cap on concurrent sessions.	`app/services/auth.py` (`log_in`, comment at line 62); `app/repositories/sessions.py` has only `create`, `touch_valid`, `delete_by_token_hash`, `delete_expired`	verified
low	In compose and k8s, nginx serves the SPA's static files with no security headers (only `Cache-Control`). The app adds CSP, HSTS and the other headers only to responses it serves itself.	`frontend/nginx.conf` lines 32 and 40. `app/http_headers.py`. Commit `95db8d7`.	verified (code)
low	k8s sets `ENVIRONMENT: production` (Secure cookies) but exposes the web tier as a plain-HTTP NodePort (`k8s/40-web.yaml:103`) with no Ingress or TLS manifest. `FORWARDED_ALLOW_IPS` is unset in k8s, so HSTS and scheme detection may not work behind nginx.	`k8s/10-configmap.yaml` (comment acknowledges this). `k8s/40-web.yaml`.	verified (config); HSTS effect unverified
low	In local compose, `TRUSTED_PROXY_HOPS: "1"` is set while the API is also published directly on `127.0.0.1:8000`, so a local caller can choose its own IP bucket	`docker-compose.yml:155`	verified (local only)
low	A broad `except Exception` in `GithubService.installations_for_grant` contradicts the CLAUDE.md rule to catch specific exceptions. It is deliberate and fails closed to PENDING.	`app/services/github.py:1038`	verified
low	Supply chain: base images are pinned by tag, not digest. GitHub Actions are pinned by major tag, not SHA. Render `autoDeploy: true` on `main` deploys on every push regardless of CI.	`Dockerfile`, `.github/workflows/ci.yml`, `render.yaml`	verified (config)

Severity (judgement)	Gap	Evidence	Status
low	`scripts/apply_migration.py` falls back to `.env` through pydantic-settings when `DATABASE_URL` is unset, so it can target whatever database `.env` names	`app/config.py` (`env_file=".env"`, line 258); the runner loads settings via `get_settings()`. `.env` contents were not read.	verified (mechanism)

15.4 Technical debt and dead code

Item	Evidence
`actor_user_id` is dead code, and its docstring ("do not yet require an identity") is stale	`app/graphql/viewer.py:56`; no caller elsewhere in `app/` (grep)
Two "unset" sentinels	`app/domain/patch.py :: UnsetType` and `app/domain/issues.py :: Unset(Enum)`
16 near-identical `_errors` helpers, one per mutation module	`grep -n '^def _errors' app/graphql/mutations/*.py` → 16
Inconsistent mutation result shapes: `githubDisconnect` returns a bare `GithubIntegration!`, Slack payloads lack `errors`, `embeddingsRefresh` returns `Int!`. Slack error codes are lowercase (`not_connected`); every other module uses UPPER_SNAKE.	`frontend/schema.graphql`; `app/graphql/mutations/github.py`, `slack.py`, `search.py`
`cryptography` is a direct dependency with no import in `app/`, `tests/` or `scripts/`	`requirements.in:57`; grep for `import cryptography` / `from cryptography` finds nothing
`certifi` is imported directly but only installed transitively	`scripts/inspect_deployment_db.py` line 32; absent from `requirements.in`
`uv.lock` is tracked but empty (`version`, `revision`, `requires-python` only), and `pyproject.toml` has no `[project]` table	`uv.lock`
`app/logging.py` contains only `# TODO: structlog`. Nothing calls `structlog.configure`, `basicConfig` or `dictConfig` in `app/`, even though `structlog.get_logger` is used.	`app/logging.py:3`
`check_db.py` calls itself "Temporary" (Phase 0) but is still tracked at the repo root	`check_db.py:1`
Migration 017 declares UNIQUE constraints over exactly the primary-key columns, which PostgreSQL backs with a second index. The comment says the PK is "named here for the reader rather than declared twice".	`migrations/017_github_development.sql:203` (`github_pull_requests_workspace_repository_number_key`), `:309` (`github_commits_workspace_repository_sha_key`). The migration is immutable, so a fix would need a new migration.
No pruning of `github_deliveries`, `slack_event_deliveries` or terminal `domain_events`	`-- ponytail: no pruning job` comments in `migrations/017_github_development.sql` and `027_domain_events.sql`, and a "nothing here prunes it" comment in `014_slack_integration.sql`; no `DELETE FROM` these tables in `app/`
`frontend/codegen.ts` declares `enumsAsTypes: true` twice in one object	`grep -c enumsAsTypes frontend/codegen.ts` → 2
Unused frontend code: `WorkspaceEntry` (exported, never routed), `useTheme`, `Tooltip`, `ToastProvider`/`useToast` (never mounted; `useToast` throws without a provider)	`frontend/src/app/routes/index.ts:33`. `frontend/src/components/index.ts`; `useTheme` is in `frontend/src/app/layout/preferences.ts`. The only references outside the definitions are comments (e.g. `InviteStep.tsx:134`, `AccountMenu.tsx:33`, `features/collaboration/index.ts:34`).

Item	Evidence
`'/onboarding'` is defined in three places (twice as `ONBOARDING_PATH`, once as `ONBOARDING_ROOT`), and `localToday()` has three copies	`app/routes/paths.ts`, `features/auth/api/destination.ts`, `features/onboarding/lib/progress.ts`. `features/issues/pages/IssuesScreen.tsx`, `features/board/pages/BoardScreen.tsx` and `features/screens.tsx` (the last acknowledged by a ponytail comment).
`engines.node >=20.19`, but `vitest@5.0.0` requires `^22.12 || ^24 || >=26` and `@testing-library/jest-dom@7.0.1` requires `>=22`	`frontend/package.json:7`; the installed packages' `engines` fields
e2e and config files are unlinted and untyped, and there is no `eslint-plugin-react-hooks`	`frontend/eslint.config.js:42` (ponytail note)
The compose `migrate` service and the CI `migrations` job still use the per-file migration loop instead of `--all`	`docker-compose.yml` lines 118–122; `.github/workflows/ci.yml` lines 373, 410
Unused domain defaults `DEFAULT_PROJECT_STATE` and `DEFAULT_INITIATIVE_STATUS`; the real defaults live in the GraphQL inputs	`app/domain/projects.py:30`, `app/domain/initiatives.py:30`

15.5 Documentation drift

Stale statement	Where	Reality
"the runtime image deliberately ships `app/` and nothing else"	`README.md:98`	`Dockerfile` lines 102-110 copy `app`, `migrations`, `scripts` and the built frontend into `static`.
Manual `kubectl port-forward` plus a per-file migration loop for k8s	`README.md:103` and the `k8s/20-postgres.yaml` Service comment	Each API pod runs `apply_migration --all` on start (`Dockerfile CMD`).
"Two background tasks run inside every API container"	`README.md:56`, `k8s/30-api.yaml:6`, `docker-compose.yml:169`	Four tasks: embedding worker, Slack delivery, schedule worker, auth sweep (`app/main.py :: lifespan`).
The launcher "installs both dependency sets"	`README.md:8`	`run-vector-local.ps1` never references `requirements-dev.txt`.
No Render section; the README describes a two-origin nginx + API topology	`README.md`	Render serves the SPA from FastAPI on one origin (`render.yaml`, `app/spa.py`).
`USAGE` lists only `<migration-file>` and `--status`	`scripts/apply_migration.py:195`	`--all` exists and is what the image runs.
Comment points at `tests/test_root_composition.py`	`app/graphql/schema.py:76`	That file does not exist.
"exactly two" published error messages; NOT_FOUND message location	`app/graphql/schema.py:139`	NOT_FOUND now lives in `app/graphql/scope.py :: WORKSPACE_NOT_FOUND_MESSAGE`, and BAD_USER_INPUT is published from many resolvers.
`code` is "deliberately ignored and never exchanged"; no HTTP client in the runtime	`app/rest/github.py:591`, `:608`	`_complete_install` exchanges the code through `installations_for_grant` (httpx).
"no async client in the runtime dependency set"	`app/services/slack.py:196`	httpx is pinned and used by `SlackWebClient` in the same file.
`info.context.tenant`	`app/graphql/queries/github.py:30`	`VectorContext` has no `tenant` attribute.
"None exist yet" (protected fields)	`app/graphql/queries/auth.py:22` (`AuthQuery.me`)	Protected fields exist throughout the schema.
`_scope` is "a stand-in until `app/graphql/scope.py` lands"	`app/graphql/mutations/memberships.py:348`	`scope.py` exists.

Stale statement	Where	Reality
"Exactly one of this and `github_app_private_key` may be set; see `github_private_key` below"	`app/config.py:172`, `DEPLOYMENT_ENV_VARS.md`	`Settings` has no `github_private_key`, and nothing enforces exclusivity.
`"no due date" is DueWindow.NONE`	`app/domain/issues.py:262`, `app/domain/saved_views.py:345`	The member is `NO_DUE_DATE` (`app/domain/issues.py`).
`k8s/20-postgres.yaml` "carries a Service and a StatefulSet"	`.pre-commit-config.yaml:41`	It holds a PersistentVolumeClaim, a Deployment and a Service.
`k8s/configmap.yaml`	`docker-compose.yml:141`	The file is `k8s/10-configmap.yaml`.
Workspace and team "deliberately NOT created here"; "a workspace called `vector`"	`scripts/seed_local_demo.py:13`, `:557`	`bootstrap_workspace` creates slug `demo` and team `ENG` (lines 66, 73).
"The three promises this script makes"	`run-vector-local.ps1:17`	Four are listed.
`RequireAuth` / `features/auth` "does not exist yet"	`frontend/src/features/onboarding/pages/OnboardingLayout.tsx:80`	`RequireAuth` exists in `frontend/src/features/auth/guards.tsx:81`.
`vitest.config.ts` "does not exist yet"	`frontend/tsconfig.node.json:33`	It exists.
References `app/layout/accountMenu.test.tsx`	`frontend/src/app/layout/AccountMenu.tsx:42`	The file does not exist; the assertion is in `src/app/shell.test.tsx`.
"The one second-wave route at the foot of the table"	`frontend/src/app/routes/routes.tsx:91`	`SECOND_WAVE` is declared as `[]` (line 59).
`CLAUDE.md` says it is "checked into the codebase"	`CLAUDE.md` header	`.gitignore:27` ignores it, and `git ls-files CLAUDE.md` is empty. Its Testing section also ends in an unclosed code fence.
Draft `schema_drafts/full_schema.sql` header names itself `002_full_schema.sql`	`schema_drafts/full_schema.sql`	The real 002 is `002_tenancy.sql`. The draft is a design document only.

16. Development history

All figures were produced on 2026-10-20 by read-only `git` commands against HEAD `577b868` on `main`. Dates are author dates in +0530.

16.1 Summary

Metric	Value	Source
First commit	`42e83e7` 2026-09-02 02:10, "feat: establish Vector backend foundation"	`git log --reverse`
Last commit	`577b868` 2026-09-10 15:36, "perf(deploy): stop paying 32 interpreter starts before uvicorn is launched"	`git log -1`
Span	9 calendar days (09-02 → 09-10)	
Commits	**289** in total: **77 merges**, **212 non-merge**	`git log --oneline | wc -l`; `git log --merges`
Authors	One name (Aditi Vadmal) under two email addresses: 287 commits and 2 commits (`2fd6063`, `05471a7`)	`git shortlog -sne HEAD`
`Co-Authored-By` trailers	0	`git log --format=%B | grep -ci co-authored-by`
Branches	42 (35 `feature/*`, `main`, `vercel-proxy-prep`, 2 `worktree-agent-*`, 3 under `remotes/`)	`git branch -a`
Migrations added	`001`–`032` in `migrations/`; `schema_drafts/033_google_identity.draft.sql` is untracked	`git status`

16.2 Cadence

Date	Commits	of which merges
2026-09-02	4	0
2026-09-03	1	0
2026-09-04	1	0
2026-09-05	2	0
2026-09-06	43	13
2026-09-07	124	30
2026-09-08	51	17
2026-09-09	45	14
2026-09-10	18	3
Total	**289**	**77**

The first four days (8 commits) are large single-phase commits. 263 of the 289 commits (91%) land between 09-06 and 09-09.

16.3 Commit-message conventions

Subject-line prefixes of the 212 non-merge commits:

Prefix	Count
`feat`	94

Prefix	Count
`fix`	42
`test`	34
`chore`	10
none	10
`wip`	9
`docs`	7
`refactor`	2
`build`	2
`style`	1
`perf`	1

The 10 unprefixed non-merge commits are:

- the nine "Phase …" commits from 09-02 to 09-06: `cd06e14`, `f155ed1`, `e4f1073`, `dd4aa52`, `52417f9`, `df8ecf2`, `d119c76`, `b341255` (FE-1), `e64ad5f` (1c);
- `72c5d5a` ("Merge feature/issue-filters: …"), which is a single-parent commit despite its subject.

One merge commit carries a `feat(auth)` subject (`6a173c1`). The most frequent scopes are `github`, `issues`, `security`, `graphql`, `frontend` and `board` (`git log --format=%s`).

Commit bodies typically record the reasoning, rejected alternatives and verification (for example `cd06e14`, `8297995`, `a5e7052`, `577b868`). §14 draws on them.

16.4 Parallel agent branches

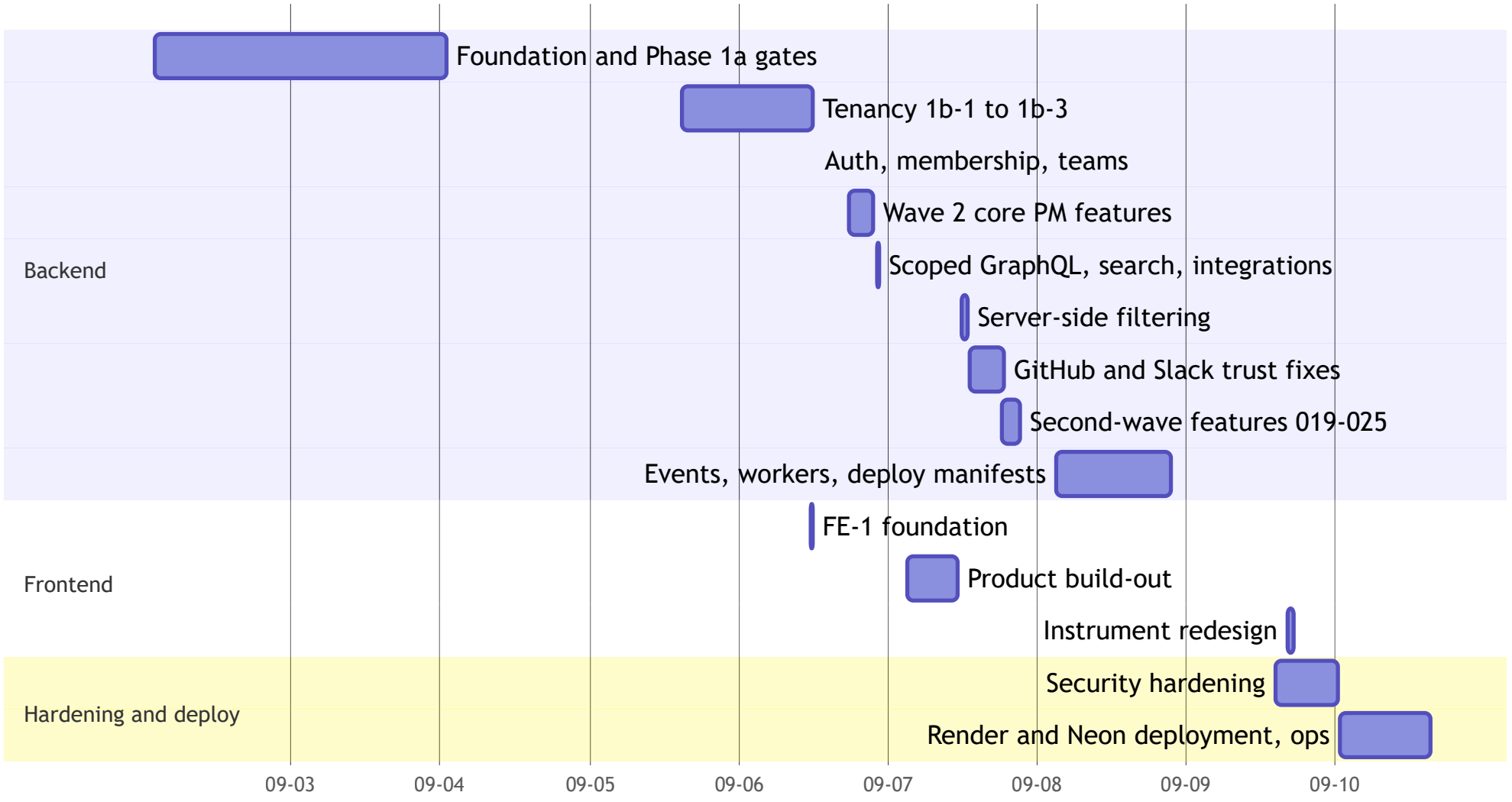
The history records that features were built on parallel branches and merged into `main`:

- **44 merge commits** name a `worktree-agent-<hex>` branch, covering 41 distinct branch names (`git log --merges --format=%s`). For example: `f37f891` "Merge branch 'worktree-agent-a31b05b5b193e3a3c'".
- **31 merge subjects** name a `feature/*` branch, e.g. "Merge feature/teams: …".
- `6bfa4de` and `df1cb55` are titled "wip(…): work in progress, agent interrupted by session limit".
- `aaaa1dc`: failure modes "made independently by people who could not see each other's work … Two agents hit this".
- `a09c03a`: "Merged from the stopped security agent's branch". `2d987ef`: "adopt A3's onboarding routes".
- Several commits' author dates are out of topological order because they were merged from separate worktrees. For example, `944a64e`, `5884e89`, `6bfa4de` and `df1cb55` all carry 09-06 17:55.

16.5 Phase timeline

#	Dates	Phase	Representative commits
1	09-02	Backend foundation	`42e83e7` issues API foundation (migration 001). `cd06e14` Phase 1a: migration ledger with SHA-256 checksums, lazy settings, migration lint, PG18 container fixture.
2	09-02 → 09-04	Engineering gates	`f155ed1` 1a-1 runtime safety. `e4f1073` 1a-2 lint, types, locking, CI and packaging gates. `dd4aa52` 1a-3 keyset pagination proved on PostgreSQL.
3	09-05 → 09-06	Tenancy	`52417f9` 1b-1 tenancy migration (002). `df8ecf2` 1b-2 slug → `WorkspaceScope`. `d119c76` 1b-3 scoped issue reads and writes.
4	09-06 11:39	Frontend start	`b341255` Phase FE-1 frontend foundation. `74e3152` one-command local launcher.
5	09-06 12:25–12:29	Auth, membership, teams	`e64ad5f` users, argon2id, DB sessions (003). `2fd6063` membership + `AuthorizedWorkspaceScope` (004). `8297995` team keys, workflow states, concurrency-safe issue numbers (005).

#	Dates	Phase	Representative commits
6	09-06 17:55–21:41	Core PM features ("Wave 2")	`5884e89` sub-issues and relations. `944a64e` labels and comments. `f859d2f` issue fields (006). `e5a1a6e` cycles (008). `d62dbfe` projects and milestones (009). `aaaa1dc` integration gates.
7	09-06 22:32–22:58	Workspace-scoped GraphQL, search, integration foundations	`4d58058` `workspaceSlug` on every field. `b8ef95d` full-text search (011). `fef54de` GitHub App foundation. `377b9b4` Slack foundation. `e3bf99c` activity and notifications.
8	09-07 03:27–11:27	Frontend product build-out	`6a173c1` landing, sign-in, register. `3931854` onboarding. `ee6446b` app shell. `2400655` split-pane issue screen. `1dd4199` inbox. `a639d51` board.
9	09-07 12:12–13:11	Server-side filtering	`b397542` `IssueFilterInput`, ordering, `totalCount`. `3508b70` indexes (015). `3509635` product-journey test over the real ASGI app.
10	09-07 13:19–18:51	GitHub/Slack trust	`765d5f3` `__typename` pricing and Slack hardening. `9f70ba9` installation claim must be confirmed (016). `3f71962` RS256 dependencies. `805d668` PRs and commits (017). `7e24222` Slack channels (018). `05471a7` idempotent deliveries.
11	09-07 18:36–21:36	Second-wave features	`3de022e` saved views (019). `f89890e` initiatives (022). `9410129` subscribers and templates (020). `7c1f11d` triage, label groups, bulk (021). `269f6cf` documents (023). `beb0a78` releases (024). `eddb267` pgvector hybrid search (025).
12	09-08	Events, workers, deploy manifests	`5ac26f6` domain-event outbox (027). `4eada47` embedding worker (028). `0f7d9cf` member tombstone (026). `1a90a92` compose + Kubernetes + CI. `a5e7052` auth rate limiting (032). `a54fb31` estimates and dates. `ea57ff5` analytics. `dab3467` Playwright e2e. `0ce2d4c` GitHub automations.
13	09-09 14:34 → 09-10 00:32	Security hardening	`9206b29` db gate rejects every false green. `13a5285` foreign `teamId` → field error. `95db8d7` security headers. `58465e4` tenancy tests for six more entities. `2a2dfdd` CSRF surface. `90361a1` / `a09c03a` repository tenancy gate.
14	09-09 16:40–17:31	"Instrument" visual redesign (11 subjects name it)	`0b5351b` token layer. `12b537a` shell. `62bcad1` settings panels. `ba29f72` list screens.
15	09-10 00:57–15:36	Deployment (Render + Neon) and ops	`de021b8` one origin, one image, Render. `ae531d4` HSTS behind proxy. `5d8b2d3` Neon TLS notes. `499fb8f` first CI run fixes. `5c7c4c3` read-only checksum/boot-migration inspector. `d4457bd` / `f55a99a` rate-limit identity spoofing. `bc24df3` GitHub install return. `577b868` `--all` boot migration (14.6 s → 1.3 s).


Vector development phases (author dates, +0530)
Backend
Frontend
Hardening and deploy
Foundation and Phase 1a gates
Tenancy 1b-1 to 1b-3
Auth, membership, teams
Wave 2 core PM features
Scoped GraphQL, search, integrations
Server-side filtering
GitHub and Slack trust fixes
Second-wave features 019-025
Events, workers, deploy manifests
FE-1 foundation
Product build-out
Instrument redesign
Security hardening
Render and Neon deployment, ops
09-03
09-04
09-05
09-06
09-07
09-08
09-09
09-10

The phase boundaries are editorial groupings of commits by date and subject. After "Phase 1c" (`e64ad5f`), the history contains no phase markers. The FE-1 bar is drawn with a nominal 21-minute width so it is visible.

16.6 Test-suite growth recorded in commit messages

Commit	Date	Statement
`cd06e14`	09-02	"254 passing without a database (from 51), 4 against a real PostgreSQL 18 container"
`d62dbfe`	09-06	"pytest 626 passed, pytest -m db 280 passed"
`4f33e8d`	09-07	"1275 passed both ways"
`a5e7052`	09-08	"2088 passed non-db, mypy clean over 204 files"
`86dce4e`	09-09	"2334 passed non-db, mypy clean over 217 files"
(local run, §12.6)	10-20	2674 passed, 2 xfailed (non-db)

> **Gaps / to verify:**
>
> - The bodies of the initial commit and of Phases 1a-1, 1a-2, 1a-3, 1b-1 and 1b-2 are empty. Their scope is inferred from subjects only (`git log --format=%B`).
> - Why a second email address appears on two commits is not recorded.
> - Author dates were preserved or rewritten across worktree merges, so the phase boundaries are approximate.
> - No commits are recorded after 2026-09-10, but this document is dated 2026-10-20. Whether work continued elsewhere, or in the untracked `docs/` and `schema_drafts/033_google_identity.draft.sql`, is unknown.

17. How to run locally and deploy

Six procedures, ordered from the least to the most setup. All ports bind to loopback unless stated. Prerequisites for A–D: Docker, Node.js ≥ 20.19 (`frontend/package.json` `engines`), Python 3.12+.

17.A Windows one-command launcher (`run-vector-local.ps1`)

Windows PowerShell 5.1+ (`#Requires -Version 5.1`). Needs `docker`, `node`, `npm` on `PATH` and a Python 3.12+ found as `py -3.12`, `py -3`, `python` or `python3` (`run-vector-local.ps1` `$Prerequisites`, `$MinimumPythonMinor = 12`).

```
.\run-vector-local.ps1              # start everything
.\run-vector-local.ps1 -Fresh       # alias -ResetDb: delete the DB container first, re-apply 001..032
.\run-vector-local.ps1 -Preview     # serve the production bundle (npm run build + npm run preview)
.\run-vector-local.ps1 -NoBrowser   # do not open the browser
.\run-vector-local.ps1 -Stop        # stop what a previous run started
```

What a start does, in order (`Invoke-StartMode`):

1. Preflight: repository layout, tools on `PATH`, `docker info` (`Assert-Prerequisites`).
2. Create `.venv` and `pip install --requirement requirements.txt` (runtime lock only) (`Install-BackendVirtualenv`).
3. `-Fresh`: `docker rm --force vector-ui-dev`; otherwise refuse if a launcher stack is already running.
4. Port checks: 5433 (only if not held by its own container), 8000, 5173.
5. Start or create container `vector-ui-dev` from `pgvector/pgvector:pg18`, published on `127.0.0.1:5433`, database `vector_ui`, throwaway credentials built into the script; wait up to 60 s.
6. Apply each `migrations/*.sql` through `python -m scripts.apply_migration <file>`, then verify with `--status` (`Initialize-Schema`).
7. `npm ci` in `frontend/` if `node_modules` is missing.
8. GraphQL contract check: `npm run graphql:schema:check`, `npm run graphql:check` (never regenerates).
9. `-Preview` only: `npm run build`.
10. Backend window: `python -m uvicorn app.main:create_app --factory --reload --host 127.0.0.1 --port 8000` with `DATABASE_URL` and `ENVIRONMENT=development` set; poll `http://127.0.0.1:8000/healthz` (30 s).
11. Seed demo data via GraphQL: `python -m scripts.seed_local_demo --api-url http://127.0.0.1:8000/graphql` (refuses non-loopback hosts). Account email `demo@vector.local`; the password is printed by the launcher's seeding step (`scripts/seed_local_demo.py` `_print_credentials`). Creates workspace `demo`, team `ENG`, 9 issues; idempotent.
12. Frontend window: `npm run dev` (or `npm run preview -- --port 5173 --strictPort` with `-Preview`); poll `http://localhost:5173/` (60 s).
13. POST a `{ __typename }` query to `http://localhost:5173/graphql` to prove the Vite proxy works.
14. Open `http://localhost:5173/issues` (unless `-NoBrowser`), print a banner, wait for Ctrl+C.

Stop (Ctrl+C or `-Stop`): kills only the recorded PIDs (state file `%TEMP%\vector-local-dev-state.json`, `taskkill /T`), `docker stop -t 10 vector-ui-dev` (container and data kept). The launcher never reads `.env`.

17.B Docker Compose (deployment shape)

```
docker compose up --build          # app at http://localhost:5173
VECTOR_API_PORT=8001 VECTOR_WEB_PORT=5174 docker compose up --build   # if 8000/5173 are taken
docker compose down                # stop, keep the database volume
docker compose down -v             # stop and delete the database volume
```

Start order (`docker-compose.yml`): `db` healthy → `migrate` runs every migration per file and `--status`, exits 0 → `api` (image CMD re-runs `apply_migration --all`, a no-op, then uvicorn) healthy on `/readyz` → `web` (nginx, proxies `/graphql`, `/healthz`, `/readyz`, `/integrations`, `/github`, `/slack` to `api:8000`). Endpoints: app `http://localhost:5173` (GraphiQL at `/graphql`, since `ENVIRONMENT=development`), API `127.0.0.1:8000`, PostgreSQL `127.0.0.1:5433` (user/password/database all the literal

throwaway `vector`). No demo data is seeded; sign up in the UI. The compose `db` and the launcher's `vector-ui-dev` both publish host port 5433, so run one at a time.

17.C Manual backend + frontend dev server against your own database

Backend (bash; on Windows activate with `.venv\Scripts\activate`):

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt          # superset of requirements.txt; use requirements.txt for runtime only
export DATABASE_URL='postgresql://USER:PASSWORD@HOST:PORT/DB'   # PostgreSQL 18 required
export ENVIRONMENT=development                # development | test | production
python -m scripts.apply_migration --all      # apply every pending migration
python -m scripts.apply_migration --status   # expect "Pending: none"
uvicorn app.main:create_app --factory --reload   # --factory is mandatory; binds 127.0.0.1:8000
```

Alternatively copy `.env.example` to `.env` and fill both variables; `app/config.py` reads `.env` as a fallback. A throwaway database matching the compose `db` service:

```
docker run --detach --name vector-pg -e POSTGRES_USER=vector -e POSTGRES_PASSWORD=vector -e POSTGRES_DB=vector -p
         127.0.0.1:5433:5432 pgvector/pgvector:pg18
```

Frontend (second terminal):

```
cd frontend
npm ci
npm run dev        # Vite on http://localhost:5173
```

The Vite dev server (and `vite preview`) proxies `/graphql`, `/integrations`, `/github` and `/slack` to `http://127.0.0.1:8000` with `changeOrigin: false` (`frontend/vite.config.ts` `proxy`). The proxy is load-bearing: the backend installs no CORS middleware, so the app must call the same-origin relative path `/graphql` (default of `VITE_GRAPHQL_URL`, `frontend/src/lib/config/env.ts`). `/healthz` and `/readyz` are not proxied.

17.D Tests, lint and type checks

Backend (from the repository root, with `requirements-dev.txt` installed):

```
python -m pytest -q                    # unit suite; -m 'not db' by default (pyproject.toml)
python -m pytest -q -m db              # PostgreSQL 18 suite; the fixture starts pgvector/pgvector:pg18 via Docker
ruff check .
ruff format --check --diff .
mypy app/
pre-commit run --all-files
```

In CI, `-m db` runs with `--junitxml` and `python -m scripts.db_gate <report.xml>` fails the job if anything was skipped or nothing ran.

Frontend (`frontend/`):

```
npm run lint                 # eslint .
npm run typecheck            # tsc -b
npm test                     # vitest run (jsdom)
npm run build                # tsc -b && vite build
npm run graphql:schema       # re-export frontend/schema.graphql from the Strawberry schema
npm run graphql:codegen      # regenerate src/generated/schema.ts and operations.ts
npm run graphql:schema:check # fail on drift without writing
npm run graphql:check
```

End-to-end (Playwright, against a running stack on `http://127.0.0.1:5173`, override with `VECTOR_E2E_BASE_URL`):

```
docker compose up --build --wait
cd frontend
npx playwright install --with-deps chromium
npx playwright test
```

17.E Render staging deploy (Neon)

1. Create (or reuse) a Neon project on **PostgreSQL 18**. Build the DSN ending in `?sslmode=verify-full&sslrootcert=/etc/ssl/certs/ca-certificates.crt`; drop `channel_binding=require` (`DEPLOYMENT_ENV_VARS.md` "Neon", "TLS"). No extension needs enabling by hand; migrations 008, 011, 025 create them.
2. Optional pre-check from a workstation (read-only; reads `DATABASE_URL` or `STAGING_DATABASE_URL` from the given file only):

   ```
   python -m scripts.inspect_deployment_db --env-file PATH_TO_ENV_FILE
   ```

3. Create the Render service from the `render.yaml` blueprint (`vector-staging`, Docker, free plan, Oregon, branch `main`).
4. In the Render dashboard set `DATABASE_URL`, `ENVIRONMENT=production` (not `staging`, which fails validation) and, after the first deploy, `PUBLIC_BASE_URL`. `EMBEDDING_WORKER_ENABLED`, `FORWARDED_ALLOW_IPS`, `TRUSTED_PROXY_HOPS` are committed in `render.yaml`.
5. Optionally add complete `GITHUB_*` / `SLACK_*` sets (`DEPLOYMENT_ENV_VARS.md`).
6. Deploy = push to `main` (`autoDeploy: true`). Render builds the root `Dockerfile`; the container runs `apply_migration --all`, then uvicorn on `$PORT`; Render health-checks `/healthz`. The SPA is served by FastAPI from `static/` on the same origin.

17.F Kubernetes (kind / k3d / minikube)

```
docker build -t vector-api:local .
docker build -t vector-web:local frontend
minikube image load vector-api:local          # kind: kind load docker-image vector-api:local
minikube image load vector-web:local
# Windows 11 workaround when minikube image load fails on missing wmic (README.md):
# docker save vector-api:local vector-web:local | docker exec -i minikube docker load
# docker save pgvector/pgvector:pg18 | docker exec -i minikube docker load

kubectl apply -f k8s/00-namespace.yaml
kubectl create secret generic vector-secrets -n vector \
  --from-literal=POSTGRES_PASSWORD='CHOOSE_ONE' \
  --from-literal=DATABASE_URL='postgresql://vector:CHOOSE_ONE@postgres:5432/vector'
kubectl apply -f k8s/
kubectl rollout status -n vector deployment/api

minikube service web -n vector --url          # or: kubectl port-forward -n vector svc/web 5173:80
```

Migrations run automatically: each `api` pod's image CMD runs `apply_migration --all`, serialised across the two replicas by the advisory lock. `k8s/10-configmap.yaml` sets `ENVIRONMENT: production`, so session cookies are `Secure`; over plain http (NodePort, port-forward) sign-in will not stick — put TLS in front or set `development` on a throwaway cluster (`README.md`). `k8s/20-postgres.yaml` is for development clusters only; production points the Secret's `DATABASE_URL` at managed PostgreSQL.

17.G README staleness

`README.md` says	Code says
Launcher "installs both dependency sets"	Installs `requirements.txt` only (`run-vector-local.ps1 Install-BackendVirtualenv`); `requirements-dev.txt` never referenced.

`README.md` says	Code says
k8s: the runtime image "ships `app/` and nothing else"; migrations must be applied via `kubectl port-forward` and a per-file loop	`Dockerfile` copies `migrations/` and `scripts/` and its CMD runs `apply_migration --all`; `.dockerignore` allowlists both; the test now requires them.
"Two background tasks run inside every API container"	Four: embedding worker, Slack delivery loop, schedule worker, auth sweep (`app/main.py::lifespan`).
Setup section: venv, `pip install -r requirements.txt`, `.env`, then "Running"	No migration step is listed; the database must be migrated first (17.C).
No Render section	Render is documented only in `render.yaml` and `DEPLOYMENT_ENV_VARS.md`.
"A Linear-style issue tracker, built as a backend learning project"	`CLAUDE.md` describes it as a production-oriented application.

> **Gaps / to verify:**
>
> - Procedures were checked against files, not executed in this pass.
> - Whether `pip install -r requirements-dev.txt` on Windows resolves cleanly (it pins `uvloop` only for non-Windows platforms via markers) is assumed, not tested.
> - Render blueprint creation steps (dashboard flow) are inferred from `render.yaml` and `DEPLOYMENT_ENV_VARS.md`; the live service configuration was not inspected.
> - The kind/k3d paths are mentioned in `README.md` but only the minikube commands are spelled out.

18. Glossary

Project-specific terms as the code uses them. General terms (GraphQL, ASGI, keyset pagination) are defined only where Vector gives them a specific meaning.

Term	Meaning in Vector	Where defined
Workspace	The tenant. Every tenant-owned row carries `workspace_id`. It is addressed in URLs and GraphQL arguments by `slug`, which must match `^[a-z0-9]([a-z0-9-]*[a-z0-9])?$`.	(`migrations/002_tenancy.sql`, `workspaces_slug_format`)
Workspace member / role	A row in `workspace_members` linking a user to a workspace with role `member`, `admin` or `owner`. Setting `removed_at` marks a member as departed (a tombstone).	(`migrations/004_membership.sql`, `migrations/026_member_departure.sql`)
Team	A unit inside a workspace that owns issues, workflow states and cycles. Each team has a `key`.	(`migrations/005_team_workflows.sql`)
Team key	An uppercase key matching `^[A-Z][A-Z0-9]{0,9}$`. It contains no hyphen, so `KEY-123` parses unambiguously.	(`migrations/005_team_workflows.sql`, `teams_key_format`)
Issue identifier	`<TEAM KEY>-<number>`, e.g. `ENG-12`. It is computed and never stored. `number` comes from incrementing `teams.issue_counter` and is unique per team.	(`app/domain/issues.py`, `IssueEntity.identifier`; `issues_team_number_key`)
Workflow state	A per-team status with a `type` of `backlog`, `unstarted`, `started`, `completed` or `canceled`. Board columns are workflow states.	(`migrations/005_team_workflows.sql`, `workflow_states_type_check`)
Priority	An integer from 0 to 4 on an issue.	(`migrations/001_issues.sql`, `issues_priority_range`)
Estimate scale	A per-team scale: `none`, `points`, `hours` or `tshirt`.	(`migrations/029_estimates_dates.sql`, `teams_estimate_scale_known`)
Archived issue	An issue whose `archived_at` is set. Vector has no hard delete for issues and no `deleted_at` column anywhere.	(`migrations/001_issues.sql` onward; see §6)
Triage	A queue of incoming issues, marked by `issues.triage_entered_at`. An issue can be accepted, declined, marked duplicate or moved to another team. Archived issues cannot be in triage.	(`migrations/021_triage_label_groups.sql`, `issues_triage_is_not_archived`; `app/services/triage.py`)
Label group	A named group of labels. An exclusive group allows at most one of its labels per issue.	(`migrations/021_triage_label_groups.sql`)
Relation	A link between two issues of type `blocks`, `related` or `duplicate`. Symmetric types are stored in canonical order (source id < target id).	(`migrations/010_issue_relations.sql`)
Sub-issue	An issue whose `parent_id` points at another issue.	(`migrations/010_issue_relations.sql`, `issues_parent_not_self`)
Cycle	A time-boxed iteration of a team, with an ordered `starts_at` < `ends_at`.	(`migrations/008_cycles.sql`)
Project / milestone	A cross-team body of work with state (`planned` … `canceled`), optional health and milestones. An issue can have a milestone only if it has a project.	(`migrations/009_projects.sql`, `issues_milestone_requires_project`)
Initiative	A group of projects above the project level. It has a status, a health value, update posts and an optional parent initiative.	(`migrations/022_initiatives.sql`)
Document	Rich-text content (ProseMirror JSON) with revisions and comments.	(`migrations/023_documents.sql`; `frontend/src/features/documents`)

Term	Meaning in Vector	Where defined
Release / environment	A release of a repository to an environment (`development`, `staging`, `production` or `custom`), with status `pending`, `deployed`, `failed` or `rolled_back`.	(`migrations/024_releases.sql`)
Saved view	A stored issue filter with layout, grouping and ordering, either `personal` or `shared`.	(`migrations/019_saved_views.sql`)
Favorite	A per-user shortcut to exactly one team, project or saved view.	(`migrations/019_saved_views.sql`, `favorites_one_target`)
Subscriber	A user who receives notifications for an issue.	(`migrations/020_subscribers_templates.sql`)
Template	A stored issue blueprint used by "create from template" and by recurrences.	(`migrations/020_subscribers_templates.sql`; `app/services/templates.py`)
Recurrence	A rule (`daily`, `weekly` or `monthly`, with an interval of 1 to 52) that files an issue from a template when `next_run_on` arrives.	(`migrations/029_estimates_dates.sql`; `app/services/schedule.py`)
Due reminder	A notification of kind `due_soon`, recorded once per (workspace, issue, due date) in a ledger table.	(`app/services/schedule.py`, `ScheduleWorker._remind`)
Schedule worker	The in-process loop that sweeps reminders and recurrences every 900 s (`POLL_INTERVAL_SECONDS`). There is no external scheduler.	(`app/services/schedule.py`)
Domain event	A row in `domain_events`, the outbox, with primary key `(workspace_id, kind, dedupe_key)`. The key makes emitting an event idempotent. The Slack delivery loop claims these rows.	(`migrations/027_domain_events.sql`; `app/repositories/events.py`, `EventRepository.claim_next`; `app/services/notifications.py`)
Embedding job / embedding worker	`embedding_jobs` is an attempt log. The optional worker (`EMBEDDING_WORKER_ENABLED`) refreshes issue embeddings and claims work with `FOR UPDATE SKIP LOCKED`.	(`migrations/028_embedding_jobs.sql`; `app/services/embedding_jobs.py`)
`hashing-v1`	The name of the built-in embedder: a deterministic hashing bag-of-words vector of 384 dimensions, not a learned model.	(`app/services/embeddings.py`, `HASHING_EMBEDDER_NAME`)
Semantic search / duplicate suggestions	Cosine-distance search over `vector(384)` issue embeddings, without an ANN index.	(`migrations/025_semantic_search.sql`; `<=>` operator in `app/repositories/embeddings.py`)
Viewer	The authenticated user making the current GraphQL request, resolved from the session cookie.	(`app/graphql/viewer.py`, `viewer_user_id`)
Scope / `AuthorizedWorkspaceScope`	Proof that the viewer holds a role in a workspace. Only a lookup against `workspace_members` produces one. Resolvers obtain it from a slug.	(`app/domain/tenancy.py`; `app/graphql/scope.py`, `authorized_scope`)
Session	An opaque random token in the `vector_session` cookie. Only its SHA-256 digest is stored, in `sessions.token_hash`.	(`migrations/003_auth.sql`; `app/services/tokens.py`)
`UNSET`	The sentinel in partial-update inputs that tells "field not supplied" apart from "set to null".	(`app/domain/patch.py`, `UnsetType`)
Cursor	An opaque URL-safe Base64 string wrapping canonical JSON (sorted keys) that holds the keyset position. A malformed cursor raises `InvalidCursorError`.	(`app/domain/pagination.py`, `_encode_payload`)
`schema_migrations` ledger	The table the migration runner creates. It records the version, apply time and SHA-256 checksum of each applied file. A checksum mismatch blocks further applies.	(`scripts/apply_migration.py`)
Adoption (of 001)	A one-time step in the runner: if `issues` already exists with no ledger, it fingerprints the table against `001_issues.sql` and records 001 as applied instead of re-running it.	(`scripts/apply_migration.py`, `_adopt_initial_migration`)
`--all`	The runner mode used at container boot. It applies every pending migration over one connection, each in its own transaction.	(`scripts/apply_migration.py`; commit `577b868`)
DB gate	`scripts/db_gate.py`, which fails CI unless the `-m db` JUnit report has more than 0 tests and 0 failures, errors and skips.	(`scripts/db_gate.py`)

Term	Meaning in Vector	Where defined
Gate tests	Tests that check architecture and supply-chain invariants rather than behaviour, such as the expected migration list and the Docker build-context allowlist.	(`tests/test_phase1a2_gates.py`, `tests/test_repository_tenancy_gate.py`)
GraphQL contract	The CI check that `frontend/schema.graphql` is byte-identical to the schema exported from code, and that generated TypeScript matches it.	(`.github/workflows/ci.yml`, jobs `graphql-contract` and `frontend`)
Launcher	`run-vector-local.ps1`, the Windows script that sets up the whole local development stack.	(`run-vector-local.ps1`)

19. Appendix: directory tree

This tree comes from `git ls-files` at HEAD `577b868`, with 944 tracked files. Untracked local items are listed at the end. Generated, cache and dependency folders (`.venv/`, `node_modules/`, `.mypy_cache/`, `.pytest_cache/`, `.ruff_cache/`) are ignored by git (`.mypy_cache/` and `.ruff_cache/` through their own generated `.gitignore`, the rest through the root `.gitignore`) and are not part of the project.

19.1 Top level

```
vector/
├── .github/workflows/ci.yml   CI: 10 parallel blocking jobs (lint, locks, test, integration,
│                              migrations, secret-scan, docker-build, graphql-contract, frontend, e2e)
├── app/                       Backend Python package (FastAPI + Strawberry GraphQL + asyncpg), 218 files
├── design/                    Design canvases (.dc.html) and a bundled design system; not used by the build
├── frontend/                  React 19 + Vite + Apollo Client SPA, 527 files
├── k8s/                       Plain Kubernetes manifests (namespace, configmap, dev postgres, api, web)
├── migrations/                Numbered, append-only SQL migrations 001..032 (+ .gitkeep)
├── schema_drafts/             full_schema.sql design document (not executed)
├── scripts/                   Migration runner, CI DB gate, deployment DB inspector, demo seeder
├── tests/                     pytest suite, 116 files (default unit suite + `-m db` PostgreSQL suite)
├── .dockerignore              Allowlist build context: app, requirements.txt, migrations, scripts, frontend
├── .env.example               Names the two required variables (DATABASE_URL, ENVIRONMENT)
├── .gitattributes             Forces LF endings on requirements*.in/.txt
├── .gitignore                 Ignores venv, caches, .env*, CLAUDE.md, node_modules, dist
├── .gitleaks.toml             gitleaks config: default rules + custom Postgres-DSN-password rule
├── .pre-commit-config.yaml    ruff check/format + generic hygiene hooks
├── DEPLOYMENT_ENV_VARS.md     Every deployment environment variable (names, purpose), Neon/Render notes
├── Dockerfile                 3-stage API image: build SPA, build venv, runtime (migrate then uvicorn)
├── README.md                  Run/deploy instructions (partly stale; see §15)
├── check_db.py                Self-described "temporary" pool connectivity check (Phase 0)
├── docker-compose.yml         db (pgvector pg18) → migrate → api → web (nginx) stack
├── pyproject.toml             pytest, ruff and mypy configuration (no [project] table)
├── render.yaml                Render blueprint: one free Docker web service "vector-staging"
├── requirements.in / .txt     Direct runtime deps / uv-compiled hash-pinned lock
├── requirements-dev.in / .txt Dev deps (+ runtime) / lock
├── run-vector-local.ps1       Windows one-command local dev launcher
└── uv.lock                    Empty lock (no packages); unused
```

19.2 `app/` (backend)

```
app/
├── main.py              create_app() factory: settings, pool, middleware, routers, lifespan background tasks
├── config.py            pydantic-settings Settings (DATABASE_URL, ENVIRONMENT, GitHub/Slack, proxy trust)
├── db.py                asyncpg pool lifecycle (connect/disconnect, min 1 / max 5)
├── spa.py               Serves the built SPA (static/) on the API origin with an SPA fallback
├── logging.py           Placeholder ("TODO: structlog")
├── http_client_ip.py    Client IP resolution for rate limiting (trusted proxy hops)
├── http_cookies.py      Session cookie policy (name, flags, set/clear)
├── http_headers.py      Security response headers middleware
├── http_limits.py       Request body size limit middleware
├── domain/              Framework-free entities, value objects, validation and errors (34 modules)
├── repositories/        SQL only; receive a connection; map asyncpg.Record → domain (33 modules)
├── services/            Business rules and transaction boundaries (29 modules)
├── graphql/
│   ├── schema.py        Strawberry schema assembly (Query/Mutation roots)
│   ├── router.py        GraphQL router (POST-only, GraphiQL/introspection gating)
│   ├── context.py       Per-request context (pool, request, response, loaders)
│   ├── errors.py        Error masking / structured error extensions
│   ├── limits.py        Depth / alias / complexity limits
│   ├── scope.py         authorized_scope(): slug → AuthorizedWorkspaceScope
│   ├── viewer.py        viewer_user_id(): session → user id
│   ├── queries/         Query field resolvers by domain (19 modules)
│   ├── mutations/       Mutation field resolvers by domain (20 modules)
│   ├── types/           Strawberry output types (27 modules)
│   ├── inputs/          Strawberry input types (19 modules)
│   └── loaders/         DataLoaders (cycles, documents, initiatives, issues, labels, projects)
└── rest/                (GitHub and Slack routers are also mounted under /integrations)
    ├── health.py        /healthz (liveness), /readyz (DB readiness)
    ├── github.py        GET /github/install, GET /github/callback + /github/oauth/callback, POST /github/webhook
    ├── slack.py         GET /slack/oauth/start, GET /slack/oauth/callback, POST /slack/events
    └── oauth_origin.py  Moves the OAuth start leg onto the callback host so the state cookie survives
```

19.3 `frontend/`

```
frontend/
├── package.json / package-lock.json   npm manifest and lock (engines: node >=20.19)
├── schema.graphql                     SDL exported from the backend; CI checks it for drift
├── codegen.ts                         GraphQL Code Generator config → src/generated/
├── scripts/                           export_schema.py, graphql-schema.mjs, graphql-codegen.mjs
├── vite.config.ts / vitest.config.ts  Dev server (+ API proxy) and unit test config
├── playwright.config.ts, e2e/         Browser journeys (journeys.spec.ts) and a design probe script
├── eslint.config.js, tsconfig*.json   Lint and TypeScript configuration
├── Dockerfile, nginx.conf             Web image: nginx-unprivileged serving dist/ and proxying API paths
├── index.html                         Vite entry HTML
└── src/
    ├── main.tsx                       React root
    ├── app/                           App.tsx, routes/ (route table, guards' wiring), layout/ (shell, sidebar,
    │                                  header, create-issue action), providers/, workspace/ (context)
    ├── components/                    Shared UI primitives (Avatar, Button, Dialog, Menu, IssueRow, Toast, ...)
    ├── features/                      28 feature folders (see §9): analytics, auth, board, collaboration,
    │                                  command, cycles, documents, environments, favorites, inbox,
    │                                  initiatives, issues, labelGroups, landing, members, myIssues,
    │                                  navigation, onboarding, projects, releases, roadmap, savedViews,
    │                                  search, semanticSearch, settings, teams, templates, triage
    ├── generated/                     schema.ts, operations.ts (codegen output, checked in)
    ├── lib/                           config/ (env), graphql/ (Apollo client, cache, pagination, payload),
    │                                  integrations.ts
    ├── styles/                        tokens.css, base.css, index.css
    └── test/                          Vitest setup, render helpers, factories, a11y helpers
```

19.4 Other folders

Path	Contents
`migrations/`	`001_issues.sql` … `032_auth_hardening.sql`. See §6.5 for the history.

Path	Contents
`scripts/`	`apply_migration.py` (runner: ledger, checksums, advisory lock, `--status`, `--all`), `db_gate.py` (fail-closed JUnit gate), `inspect_deployment_db.py` (read-only deployment DB report), `seed_local_demo.py` (seeds a demo workspace through the GraphQL API, loopback only), `__init__.py`
`k8s/`	`00-namespace.yaml`, `10-configmap.yaml`, `20-postgres.yaml` (dev only), `30-api.yaml`, `40-web.yaml`
`tests/`	`conftest.py` (Docker CLI PostgreSQL fixture) and 114 `test_*.py` files. See §12.
`design/`	`Vector App.dc.html`, `Vector Landing.dc.html`, `instrument/*.dc.html` (10 screens), `_ds/` design-system bundle, `support.js`, `.thumbnail`
`schema_drafts/`	`full_schema.sql` (tracked design document)

19.5 Untracked or ignored local files

Path	Status
`schema_drafts/033_google_identity.draft.sql`	Untracked draft migration for Google sign-in. It has not been applied (see §6.5).
`docs/`	Untracked local directory (`git status`). Its contents were not reviewed for this appendix.
`CLAUDE.md`	Exists locally, but `.gitignore` excludes it, so it is untracked.
`.env`, `.env.staging`	Local environment files, ignored by git. They were not read for this document.
`.claude/`, `.venv/`, caches	Local tooling state, ignored by git.